NeuroVAT User Tutorial

Neuromorphic Verilog-A test bench and spiking-network trainer — version 1.3.1

Md Mahmudul Kabir Peyal, Boise State University

https://github.com/mpeyal/neurovat-verilog-tester

November 2026

How to use this tutorial

This tutorial is for graduate students and researchers who work on neuromorphic devices. It assumes you know what a synapse, spike-timing-dependent plasticity (STDP) and a Verilog-A model are, but have never used NeuroVAT. Every task is written as numbered steps that say what to click or set, and why.

Where to start. Chapters 1 and 2 are for everyone: they install the app and show you around the window. After that, pick the chapter that matches your work:

- **Device Lab (Chapter 3)** for testing a single device: transients, pulse response, STDP, noise and retention.
- **Network Studio (Chapter 4)** for building and training a spiking network in supervised or unsupervised epochs.
- **Stream mode (Chapter 5)** for unsupervised STDP on one continuous spike stream.

Chapters 6 to 9 cover the AI assistant and automation, validation and adding your own device, the measured results, and troubleshooting. Read them as you need them. The Appendix lists the control names (widget tags) that scripts and the control bridge use.

How to read the figures. Each screenshot marks the controls you need with **red boxes** and numbered red badges. The numbered list under each figure matches those badges: item 1 explains badge 1, and so on.

> **Tip.** Boxes like this one give a shortcut or good practice. Blue **note** boxes add background, and amber **warning** boxes describe things that can go wrong.

Contents

Chapter 1. Installing and starting NeuroVAT 5

1.1 Option A: the Windows release (v1.3.1) 5

1.2 Option B: from source 6

1.3 The workspace folders 8

1.4 Optional extras 8

Chapter 2. A tour of the window 10

2.1 The whole window 11

2.2 What each tab is for 12

2.3 The Network Studio side panel 12

2.4 The menus 13

2.5 Keyboard and mouse 19

Chapter 3. Device Lab: testing a single synapse 20

3.1 Step 1. Pick the synapse and set a parameter 21

3.2 Step 2. Design the stimulus 23

3.3 Step 3. Run and read the transient 24

3.4 Step 4. Pulse response: G versus pulse number 27

3.5 Step 5. STDP: the timing experiment 28

3.6 Step 6. Noise & retention 31

3.7 Step 7. Source: view or edit the .va 32

3.8 Step 8. Compute: the device as a compute element 33

3.9 Step 9. Export CSV and PNG 35

3.10 Step 10. Tools menu 37

Chapter 4. Network Studio: building, training and testing a spiking network 43

4.1 Step 1. Open Network Studio and pick the synapse 43

4.2 Step 2. Choose the learning mode 46

4.3 Step 3. Set the architecture, neuron and learning controls 47

4.4 Step 4. Choose the data and the encoding (Data tab) 50

4.5 Step 5. Build and train 57

4.6 Step 6. Test, and read the Results tab 59

4.7 Step 7. The inspection tabs 63

4.8 Step 8. The Power tab and power profiles 70

4.9 Step 9. Save and load a network 73

4.10 Summary 76

Chapter 5. Unsupervised stream mode: finding a hidden pattern with STDP 77

5.1 The idea in plain words 77

5.2 Set up the stream 77

5.3 Test, Train (60 s), Test at the defaults 81

5.4 The other views 87

5.5 Image encodings 90

5.6 Robustness tab 93

5.7 Poisson-pattern input (Masquelier 2008) 99
5.8 Device synapses in stream mode 100

Chapter 6. The AI assistant and automation 102
6.1 Open the Assistant panel 102
6.2 Choose the provider, the model and the account 103
6.3 File edits, shell commands and Autonomous mode 108
6.4 Ask a question 110
6.5 What the agent can do in the app 112
6.6 The control bridge: drive the app from a script 113
6.7 The Virtuoso skillbridge (describe only) 118

Chapter 7. Validating results and adding your own device 120
7.1 Run the test suites 120
7.2 Twin and .va consistency (Tools menu) 123
7.3 Validate against Cadence 124
7.4 Add your own device 125

Chapter 8. Measured results and benchmarks 132
8.1 Device model vs paper (ECFET v2) 132
8.2 Core vs legacy engine: speed and equivalence 133
8.3 Unsupervised pattern finding (Masquelier et al. 2008) 134
8.4 Stream encodings: untrained → trained 134
8.5 Robustness sweep: recall and retrain 135
8.6 The frozen-Poisson fix 137
8.7 Known limits of the measured results 137

Chapter 9. Limits, troubleshooting and FAQ 139
9.1 Known limits 139
9.2 Common problems and fixes 140
9.3 FAQ 142
9.4 Where to get help 142
9.5 How to cite 143

Appendix. Control names for scripting 144
A.1 How to use the tags 144
A.2 Device Lab 144
A.3 Tabs 147
A.4 Network Studio: network, data and epoch training 147
A.5 Stream mode 149
A.6 Robustness tab 151
A.7 Buttons and their bridge commands 151
A.8 Scripting a whole stream run 151

CHAPTER 1

Installing and starting NeuroVAT

NeuroVAT runs on your own computer. It does not need a web browser, a server or a Cadence licence. There are two ways to install it:

- **Option A, the Windows release.** Download one zip file, unzip it and double-click `NeuroVAT.exe`. You do not need Python. Use this if you only want to use the app.
- **Option B, from source.** Install Python and run the app from the source folder. Use this on macOS or Linux, if you want to change the code, or if you need the command-line tools (for example `tools/nvat_ctl.py`).

Both options open the same window and use the same workspace folders.

1.1 Option A: the Windows release (v1.3.1)

1. Download `NeuroVAT-win64.zip` from the v1.3.1 release on the project's GitHub page (repository `mpeyal/neurovat-verilog-tester`, page **Releases**). With the GitHub command-line tool you can do this in one line:

   ```
   gh release download v1.3.1 -R mpeyal/neurovat-verilog-tester -p NeuroVAT-win64.zip
   ```

   The zip is about 60 MB.
2. Unzip it into a folder that you can write to, for example `Documents\NeuroVAT`. Do not run the exe from inside the zip. The app writes its results into the folder that holds the exe, so the folder must be writable. Avoid `C:\Program Files`.
3. Check that the folder holds `NeuroVAT.exe`, `README.txt`, a `_internal` folder (the bundled Python and libraries) and the three workspace folders `verilog`, `twins` and `patterns`.
4. Double-click `NeuroVAT.exe`. The first start takes a few seconds while the bundled libraries load. If Windows SmartScreen warns about an unknown publisher, choose **More info** and then **Run anyway**. The exe is not code-signed.

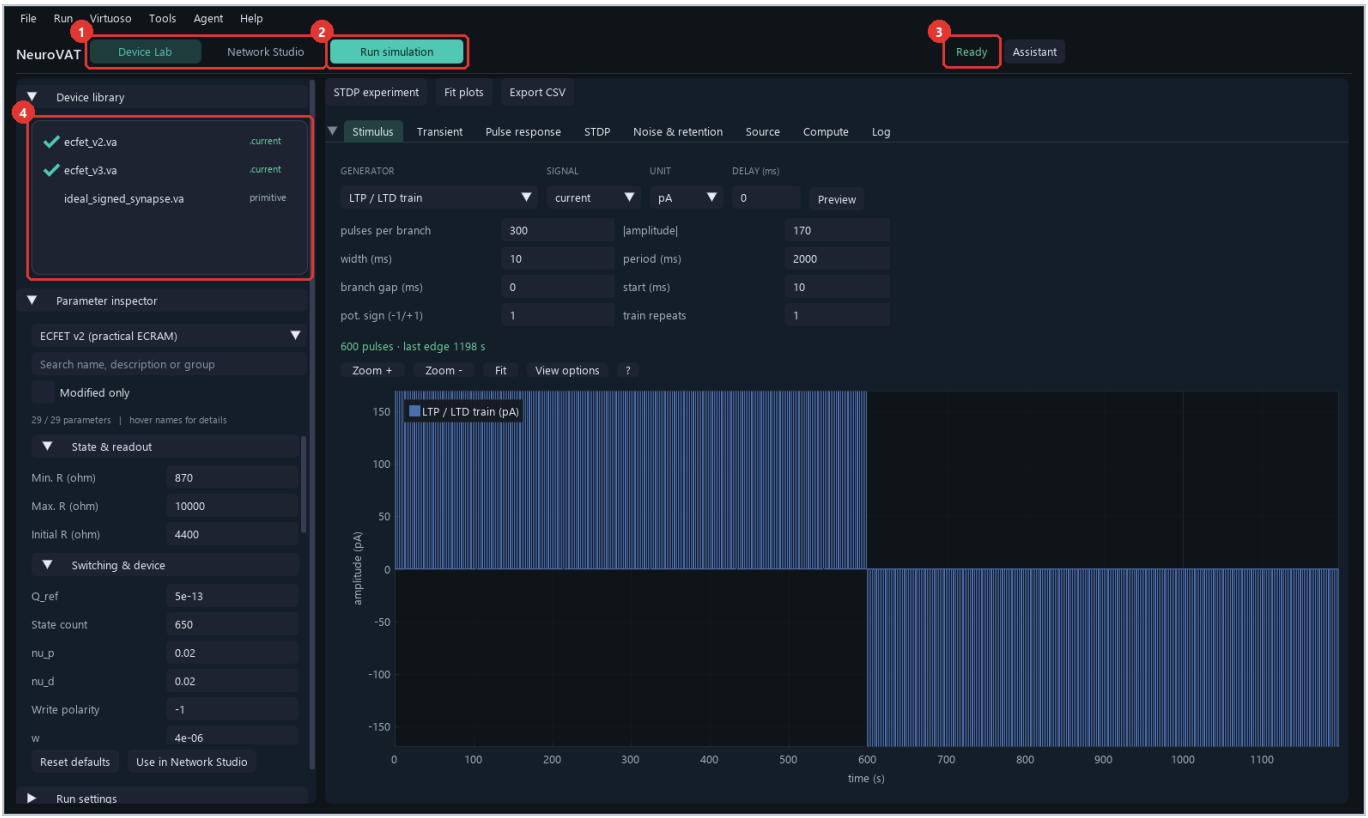


Figure: The first window of the v1.3.1 Windows release (NeuroVAT.exe, unzipped and started once). It opens in Device Lab with the three public Verilog-A models found.

1. **Device Lab / Network Studio:** the two workspaces. The app starts in Device Lab, where you test one synapse. Chapter 3 covers Device Lab.
2. **Run simulation:** runs whatever the active page shows. F5 does the same.
3. **Ready:** the status line. It turns amber and shows what is running while a simulation is busy.
4. **Device library:** the `.va` files found in the `verilog` folder. If you see `ecfet_v2.va`, `ecfet_v3.va` and `ideal_signed_synapse.va`, the app found its workspace and is ready to use.

> **Note.** We tested this release on Windows 11 for this tutorial. We downloaded `NeuroVAT-win64.zip` for v1.3.1, unzipped it, and started `NeuroVAT.exe` once. The window opened at its default size (1584 x 941 pixels inside the frame). It found the three `.va` files and printed no errors. It also created two new folders next to the exe: `results` and `control`.

The release includes the AI assistant. It is optional, and the device tester and the network trainer work fully offline without it. `README.txt` in the zip lists the supported back ends: the Claude Code CLI or `ANTHROPIC_API_KEY`, the codex CLI or `OPENAI_API_KEY`, and the gemini CLI or `GEMINI_API_KEY`. The chapter on the assistant explains how to set one up.

1.2 Option B: from source

You need **Python 3.9 or newer** from `python.org`. On Windows, tick **Add python.exe to PATH** when you install it. Then get the source, either by cloning the repository or by downloading the source zip of the v1.3.1 release.

One-click setup scripts

The source folder has two setup scripts. Each one does the same three things:

1. It checks for Python 3.9 or newer. On Windows it tries `py -3` first and then `python`. On macOS it uses `python3`. If no suitable Python is found, it tells you where to download one and stops.
2. On the first run, it creates a private virtual environment in a `.venv` folder inside the source folder. It then upgrades pip and installs `requirements.txt` into that environment. Nothing is installed into your global Python. The first run takes a few minutes.
3. It starts the app with `run_gui.py`. Later runs skip the install unless `requirements.txt` has changed. The script detects a change by comparing the file with the copy it saved in `.venv\requirements.installed`.

- **Windows:** double-click `setup_windows.bat`. Any extra arguments are passed to `run_gui.py`, for example `setup_windows.bat --no-bridge`.
- **macOS:** double-click `setup_mac.command`. If macOS blocks the file, right-click it and choose **Open**, or run `chmod +x setup_mac.command` once in Terminal. On macOS the app uses DearPyGui's built-in font in place of Segoe UI (README).

Manual setup

If you prefer to manage your own environment, run these commands from the source folder:

```
python -m venv .venv
<activate the environment: see the table below>
pip install -r requirements.txt
python run_gui.py
```

The activation command depends on your shell:

Shell	Activate with
Windows PowerShell	`.venv\Scripts\Activate.ps1`. If the execution policy blocks the script, run `Set-ExecutionPolicy -Scope Process Bypass` first. It applies to this PowerShell window only.
Windows cmd	`.venv\Scripts\activate.bat`
Git Bash (Windows)	`source .venv/Scripts/activate`
macOS / Linux	`source .venv/bin/activate`

When the environment is active, the prompt starts with `(.venv)`.

`requirements.txt` installs `numpy`, `matplotlib` and `dearpygui` (1.11 or newer), which the app itself needs. It also installs `uharfbuzz` and `freetype-py`, which shape complex scripts such as Bangla in the chat. Without them the chat falls back to plain text. Last, it installs `anthropic`, which lets an `ANTHROPIC_API_KEY` alone power the chat. The OpenAI package is commented out, so install `openai` yourself if you want that provider.

Command-line options of run_gui.py

Option	What it does
`--workspace DIR`	Sets the folder to scan for `.va` files and to write `results` into. The default is the folder that holds `run_gui.py`, or the folder that holds the exe in the Windows release. Use it to keep each project's models and results apart, for example `python run_gui.py --workspace D:\my_models`.

Option	What it does
`--no-bridge`	Starts with the **control bridge** switched off. The bridge is on by default. It is a file mailbox in `<workspace>\control\` that lets an outside script (`tools/nvat_ctl.py`) or the assistant drive the running app. You can also switch it on and off at any time under **Tools > Control bridge**.
`--smoke N`	Renders N frames and then exits. This is a quick self-test to check that the window can open. The bridge is always off during a smoke test.
`-h`, `--help`	Prints these options.

Tip. To check an installation without clicking anything, run `python run_gui.py --smoke 60`. The window should appear briefly and close on its own.

Note. The Windows exe reads the same options. For example, `NeuroVAT.exe --workspace D:\my_models --no-bridge` starts the release on another folder with the bridge off.

1.3 The workspace folders

The **workspace** is the folder the app scans at start-up. Use **File > Change workspace...** to switch to another one, or `--workspace` to start in one. Inside it, these folders matter:

Folder	What goes in it
`verilog\`	Your Verilog-A models (`.va`). Every `.va` here appears in the Device library. A file is simulated by the Python twin whose keyword matches its name. The release includes `ecfet_v2.va`, `ecfet_v3.va` and `ideal_signed_synapse.va`. Older workspaces that keep `.va` files in the top folder still work.
`twins\`	Python "twins" of your own devices. Each twin is one `.py` file with a model class, a parameter dataclass and a `TWIN_SPEC` dict (see `twins\README.md`). The app loads every `*.py` here at start-up. The built-in ECFET twins live in `simulation_core\` instead, so this folder starts with only a README.
`patterns\`	Python files that define custom training patterns for Network Studio. The release includes `example_corners.py`, `example_xnor.py` and `xor_aug.py`.
`datasets\`	Downloaded datasets, for example `mnist.npz`. This folder does not come in the zip. Network Studio creates it the first time you download MNIST or a dataset from a URL.
`results\`	Everything the app writes: CSV and PNG exports, the STDP and compute exports, and saved logs. It is created when the app starts.
`control\`	The control-bridge mailbox (only while the bridge is on).

Warning. The files in `twins\` and `patterns\` are Python code, and the app **runs them** when it loads them, without any sandbox. It also reloads them when they change. Only put files there that you wrote or trust.

1.4 Optional extras

Two more requirement files add optional features. Neither one is needed for Device Lab or for the built-in network trainer.

- `requirements-snn.txt` installs `torch==2.14.1`, the tensor engine for the native **SNN Model Builder** (convolutional and dense spiking networks trained with surrogate gradients). The file notes that snnTorch is not needed by the builder or the device trainer. Install it with `pip install -r requirements-snn.txt`.
- `requirements-validation.txt` installs `torch==2.14.1` and `snntorch==1.0.0`. These are for the **independent comparison**. `tools/validate_snntorch.py` reruns the app's own training engine next to snnTorch neurons and PyTorch autograd, with the same data and settings. It writes every result to `results/snntorch_validation.json` (README, section "Trainer integrity and independent comparison"). Install it only if you want to reproduce that check.

> **Note.** PyTorch is a large download. The Windows release does not bundle it, so the SNN Model Builder needs a source install with `requirements-snn.txt`.

One more optional tool is **OpenVAF**, a free Verilog-A compiler. NeuroVAT does not install it. If you put it on your PATH, **Tools > Compile .va (OpenVAF)** checks your `.va` files for errors before you take them to Cadence. Chapter 3 shows this.

CHAPTER 2

A tour of the window

NeuroVAT has one main window. This chapter names every part of it, so that the later chapters can say "click X" and you know where X is. You do not need to change anything yet.

2.1 The whole window

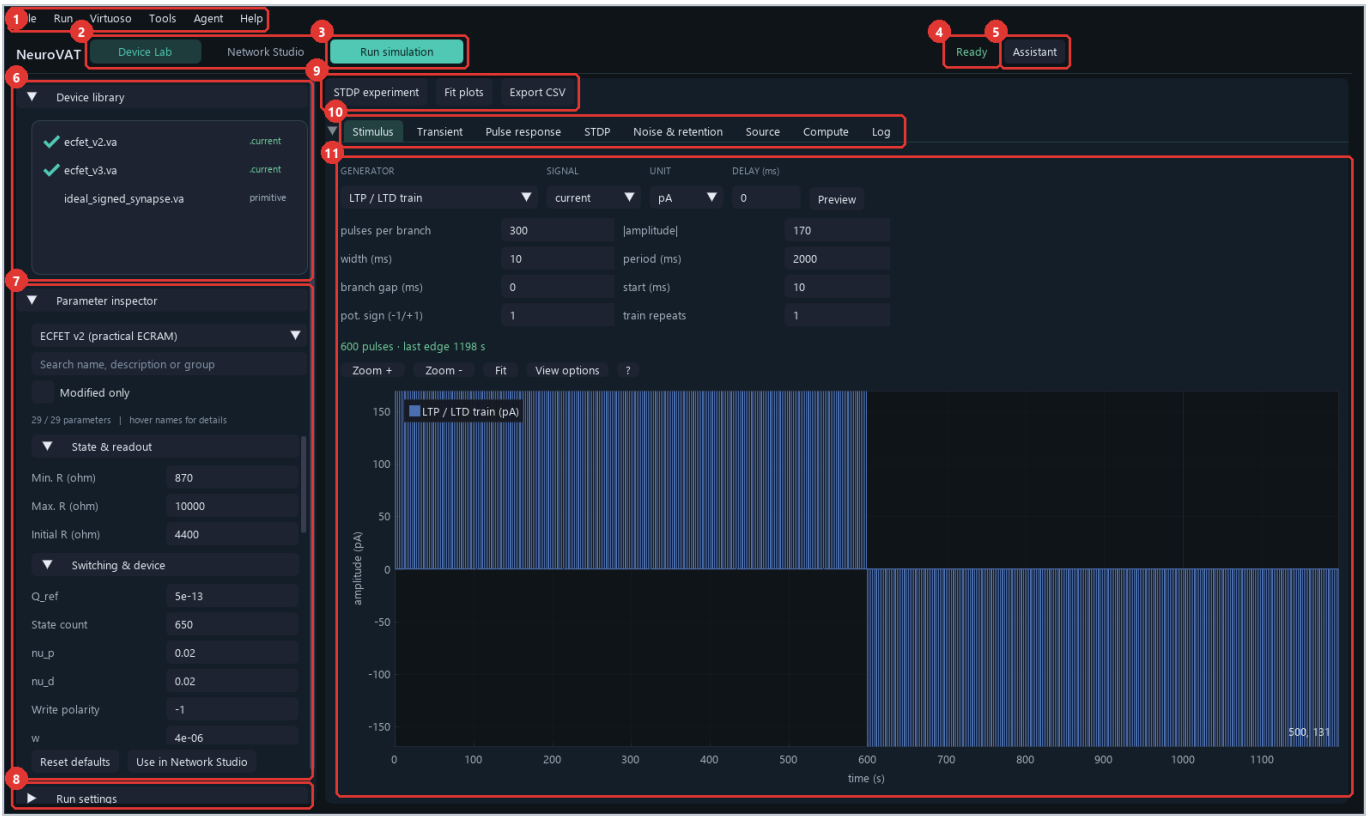


Figure: The NeuroVAT window just after start-up, in Device Lab on the Stimulus tab (default size, 1584 x 941 pixels).

1. **Menu bar:** File, Run, Virtuoso, Tools, Agent and Help. Each menu is shown below.
2. **Workspace switch: Device Lab** tests one synapse model. **Network Studio** builds and trains a network of those synapses. The active workspace is highlighted. You can switch at any time, and a network you built is kept.
3. **Run button:** runs the experiment on the page you are looking at, and its label changes to match. It reads **Run simulation** on the Stimulus, Transient and Pulse response tabs, **Run STDP** on the STDP tab, **Run noise analysis**, **Run compute**, and **Run training** in Network Studio. **F5** does the same thing.
4. **Status:** shows **Ready** when the app is idle. While something runs, it turns amber and says what is running. When the run finishes it shows a short result, for example "done · 1 result(s)".
5. **Assistant:** opens or closes the optional AI chat drawer on the right. The assistant has its own chapter.
6. **Device library:** lists every `.va` file found in the workspace's `verilog` folder. A tick means the model is simulated on the next run, a click loads that file's parameter values into the inspector, and a double-click opens the source. The small tag on the right shows the drive type (`.current` or `.voltage`). A file marked `primitive` is an ideal source asset, which cannot be simulated in Device Lab. Only one device class can be ticked at a time. If you tick a FeFET model, for example, the ECFET models are unticked.
7. **Parameter inspector:** the parameters of the model chosen in the drop-down. Type a word in the search box to filter the list. **Modified only** hides the values you have not changed. Hover over a name to see its symbol, its description from the model source, and its default value. **Reset defaults** restores the model's defaults. **Use in Network Studio** makes this device the synapse of the network.
8. **Run settings** (collapsed): the post-stimulus tail (60 s by default), a manual stop time, and "auto-save CSV after run". Below it, scroll down the left panel to find **Cadence Virtuoso**, which has Connect and

Disconnect buttons for the SSH skillbridge to a Virtuoso host.

9. **Device actions: STDP experiment** opens the STDP tab. **Fit plots** rescales every plot to its data. **Export CSV** writes the last results to `results\`. This row is hidden on pages that have their own action buttons (STDP, Noise, Compute).
10. **Centre tabs:** the pages of Device Lab, described in the table below.
11. **Page area:** the content of the active tab. Here it shows the stimulus designer and its waveform preview.

2.2 What each tab is for

The tab names below are the labels in `vatester/app.py` (each `dpg.tab(label=...)` in the centre tab bar).

Tab	What it is for
Stimulus	Design the input waveform: a single spike, pulse trains, LTP/LTD trains, paired pulses, Poisson trains, bursts, STDP pairs, staircases or a custom pulse list. You also choose current or voltage drive and the units here.
Transient	Shows the result of a run against time: the stimulus, the device resistance R(t) and the conductance G(t). All three plots share one time axis.
Pulse response	The retained conductance (or resistance) after each pulse, plotted against pulse number. This is the LTP/LTD curve, and the tab reports the mean ΔG per pulse.
STDP	A timing experiment: pre and post pulses at many time offsets, with the resulting state change plotted against Δt.
Network Studio	The network workspace. Its tab is shown only while Network Studio is active (switch with the workspace buttons, number 2 in the figure).
Polarization	Polarization measurements (P-V loop, PUND and switching kinetics) for ferroelectric devices. It is hidden unless the ticked model declares a polarization analysis. None of the public ECFET models does, so you will not see this tab with them.
Noise & retention	The noise spectrum and the dynamic range of a model that defines a noise budget. None of the three public models defines one, so the tab shows only an error message (see Chapter 3).
Source	View and edit a `.va` file, save it, or ask the assistant about it. It can also load a source from, or write one back to, a connected Virtuoso library.
Compute	Use the device as a compute element: matrix-vector multiply or logic/ALU on a crossbar of the ticked device, with energy and latency estimates. It can also compute terminal energy from a supplied V/I trace.
Log	Every message the app prints: run summaries, export paths and tool reports. **Copy log**, **Save to file** (writes `results\session_log.txt`) and **Clear** are at the top.

> **Tip.** If the window is narrow, some tabs may not fit. The small arrow at the left end of the tab bar opens a list of all tabs.

2.3 The Network Studio side panel

When you switch to Network Studio, the left panel changes to show the network settings. The top of that panel is where you pick the device that every synapse in the network will use.

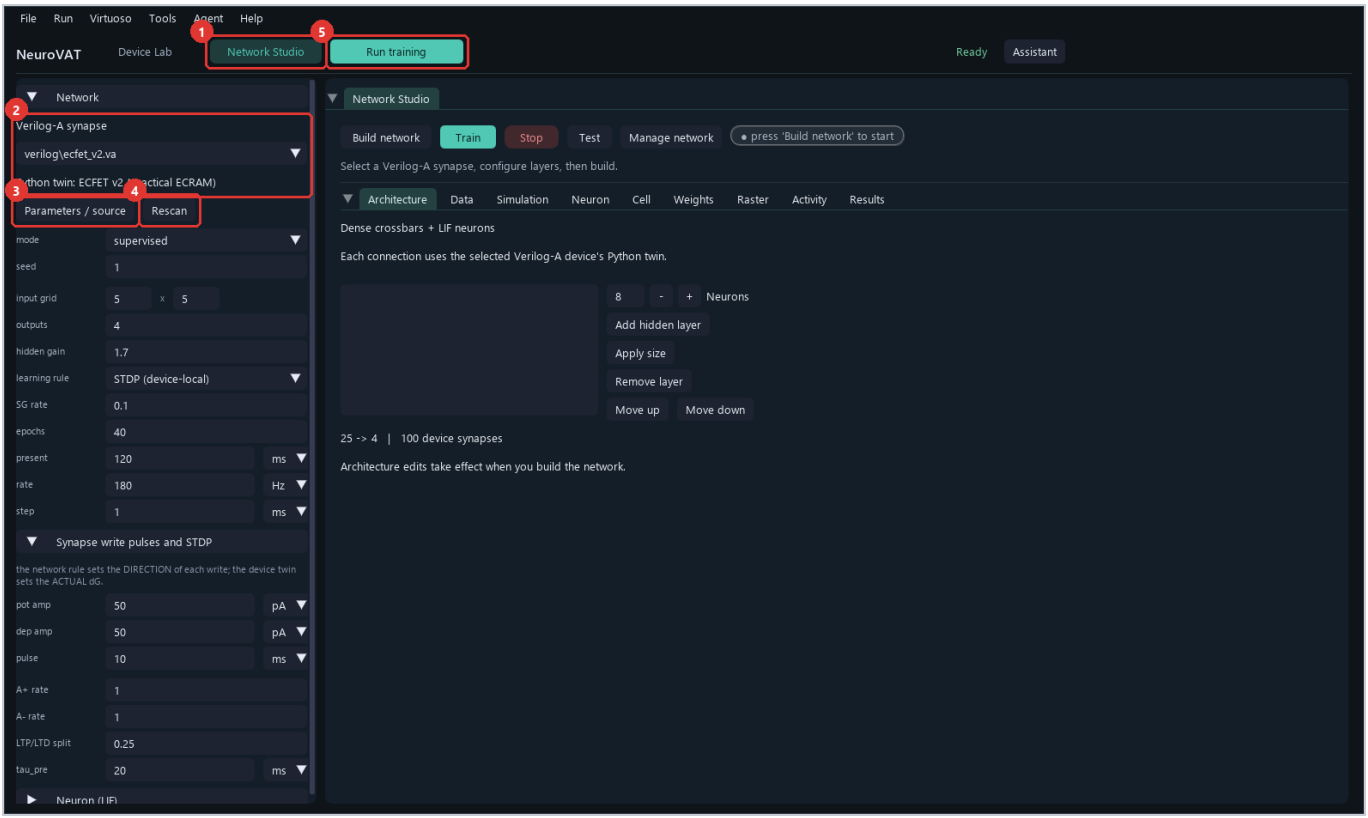


Figure: Network Studio. The left panel now holds the network settings, starting with the Verilog-A synapse picker.

1. **Network Studio:** the workspace button. It is highlighted while this workspace is active.
2. **Verilog-A synapse:** the `.va` source the network's synapses use, here `verilog\ecfet_v2.va`. The line below it names the Python twin that is actually run ("Python twin: ECFET v2 (practical ECRAM)"). Only sources mapped to a conductance twin are offered.
3. **Parameters / source:** shows the inspector parameters and the source of the chosen synapse, so you can check them before you build.
4. **Rescan:** scans the `verilog` folder again, for example after you have added a `.va` file. It does the same as **File > Rescan Verilog-A files**.
5. **Run training:** in Network Studio the Run button (and F5) trains the network.

The chapter on Network Studio covers the rest of this panel.

2.4 The menus

We opened each menu once and took a screenshot. The box marks the menu's items, and the list explains each one.

File

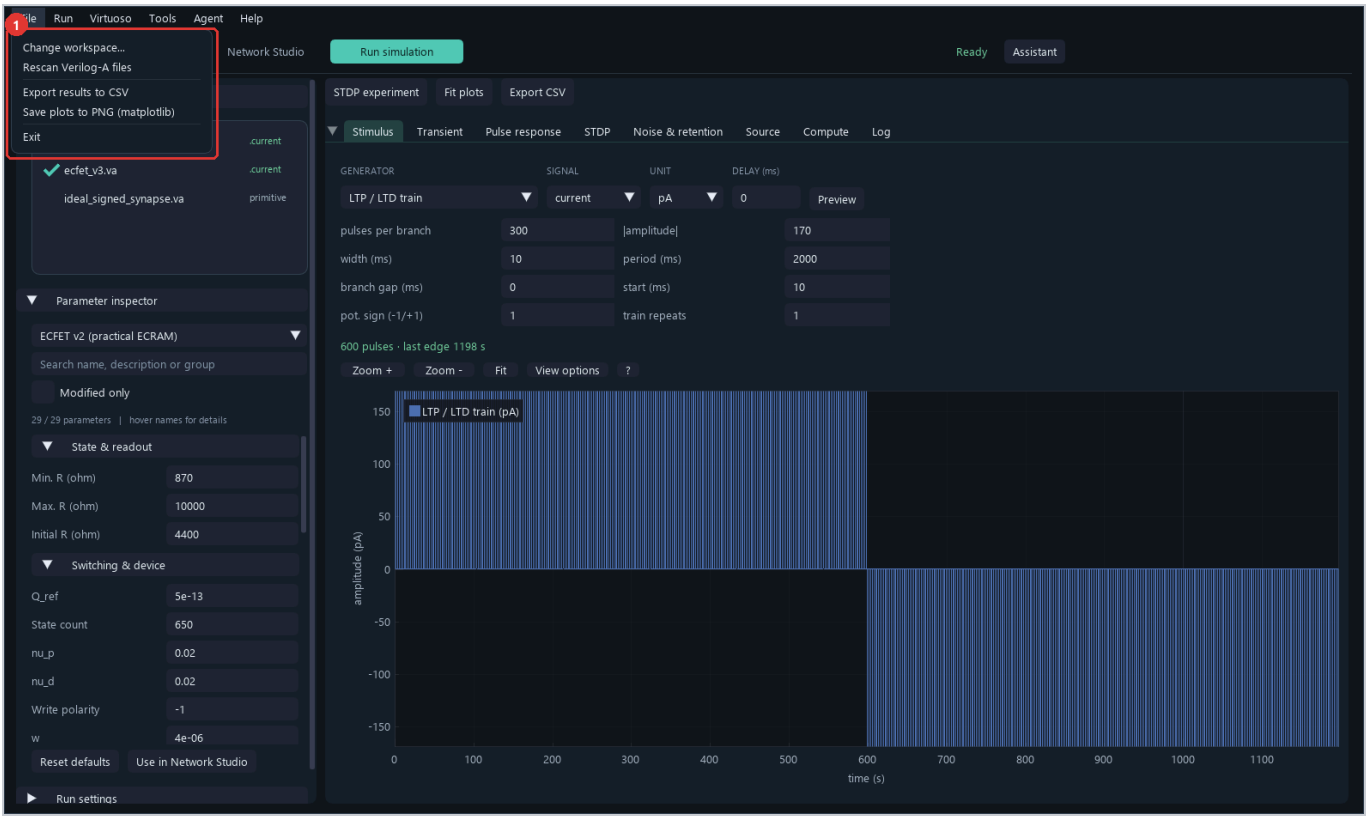


Figure: The File menu.

1. The File menu:
    - **Change workspace...**: opens a folder picker to switch to another workspace folder.
    - **Rescan Verilog-A files**: scans `verilog\` again.
    - **Export results to CSV**: writes each result of the last run to `results\gui_<model>.csv`.
    - **Save plots to PNG (matplotlib)**: writes `results\gui_plot.png`.
    - **Exit**: closes the app.

> **Warning.** In our test of the source install (DearPyGui 2.3.1), clicking **File > Change workspace...** closed the whole app. It crashed with `KeyError: 'rect_size'` in `desktop_layout.py`. To work in another folder, start the app with `--workspace` instead (Chapter 1). Chapter 3 lists the other file dialogs that crash in the same way.

Run

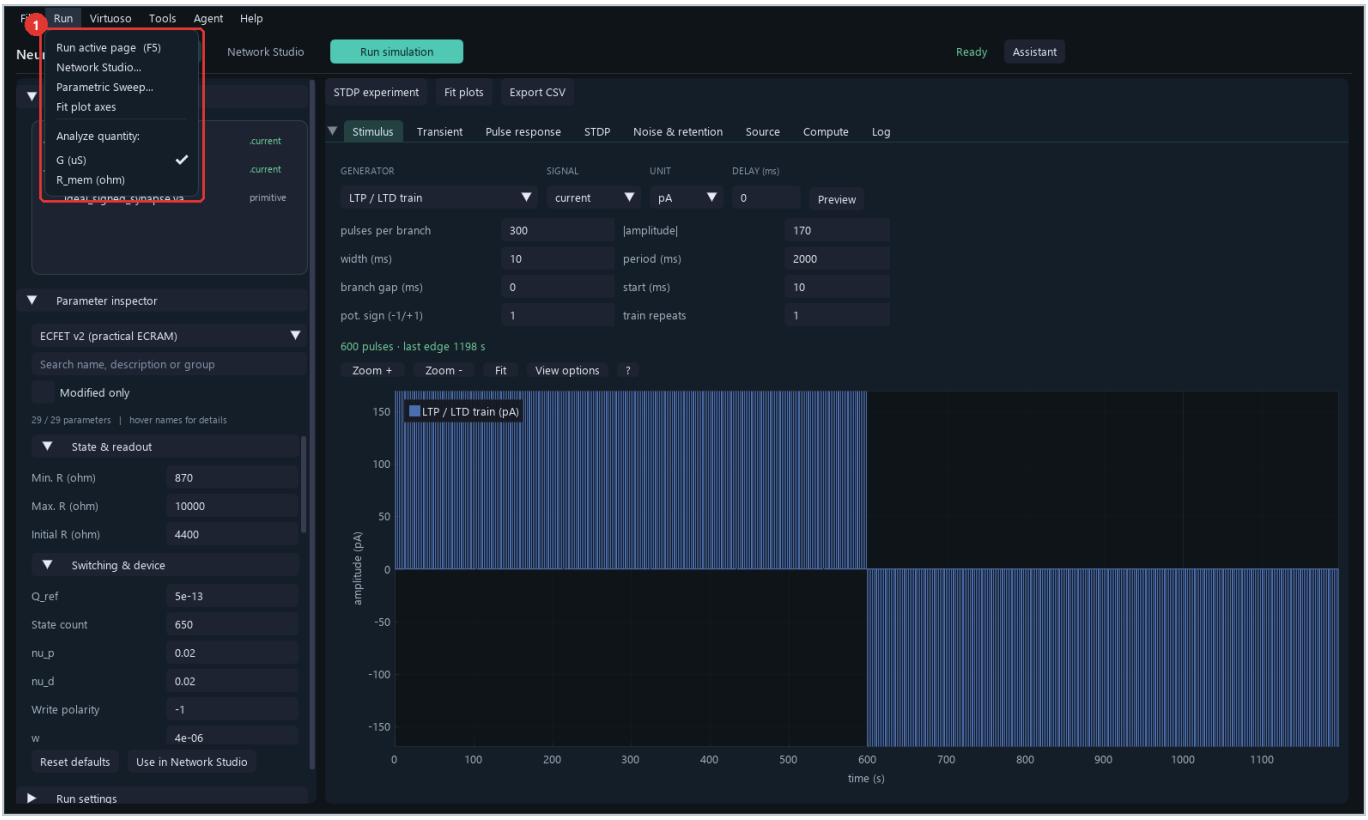


Figure: The Run menu.

1. The Run menu:
    - **Run active page (F5)**: the same as the Run button.
    - **Network Studio...**: switches to Network Studio.
    - **Parametric Sweep...**: opens a dialog that sweeps one or more model parameters. Each value becomes its own curve on the next run.
    - **Fit plot axes**: rescales the plots to their data.
    - **Analyze quantity**: chooses whether the Pulse response tab plots **G (uS)** or **R_mem (ohm)**.

Virtuoso

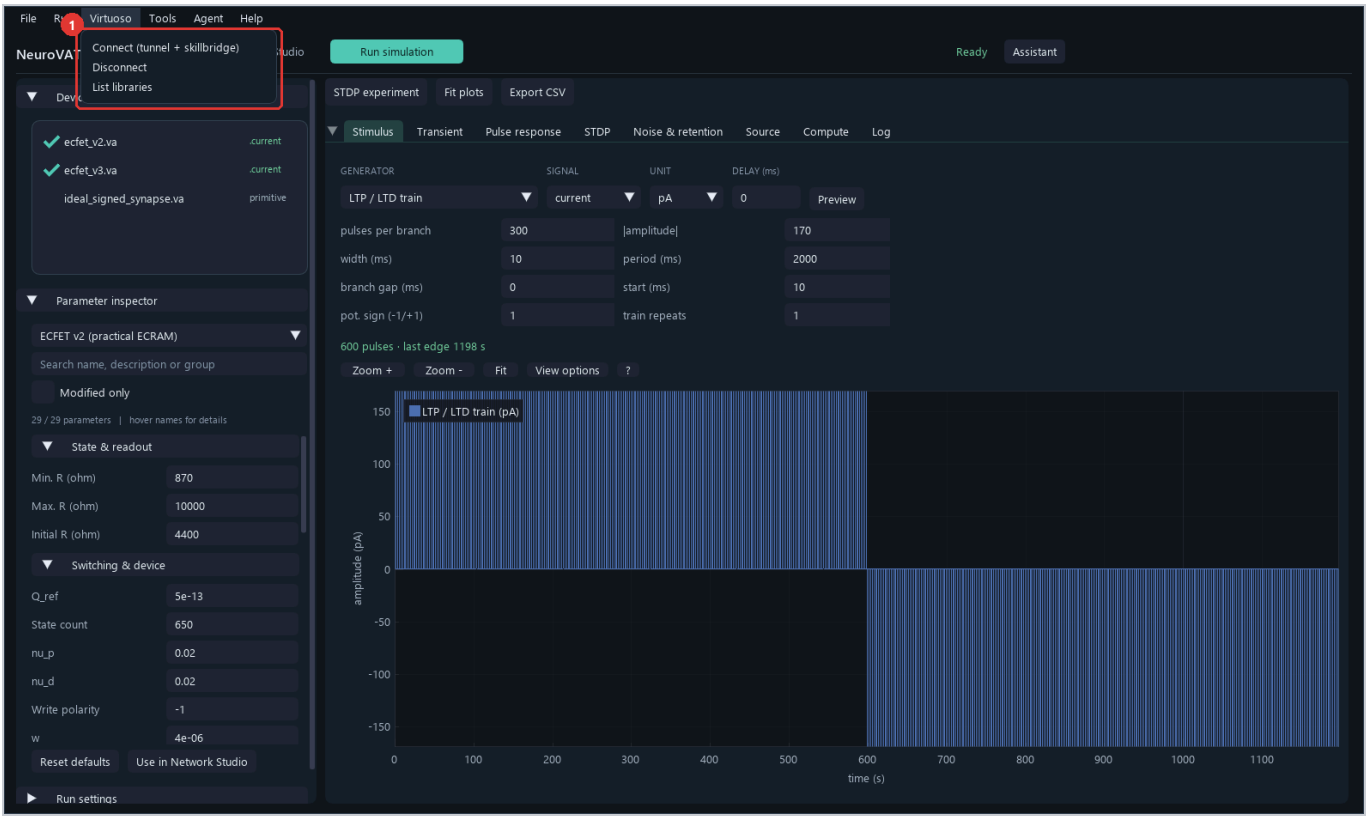


Figure: The Virtuoso menu.

1. The Virtuoso menu: **Connect (tunnel + skillbridge)**, **Disconnect** and **List libraries**. These work only with a Cadence Virtuoso host that has the skillbridge server running. The host login is read from the `NVAT_VIRTUOSO_HOST` environment variable or from a local `virtuoso.local` file. It is never stored in the app.

Tools

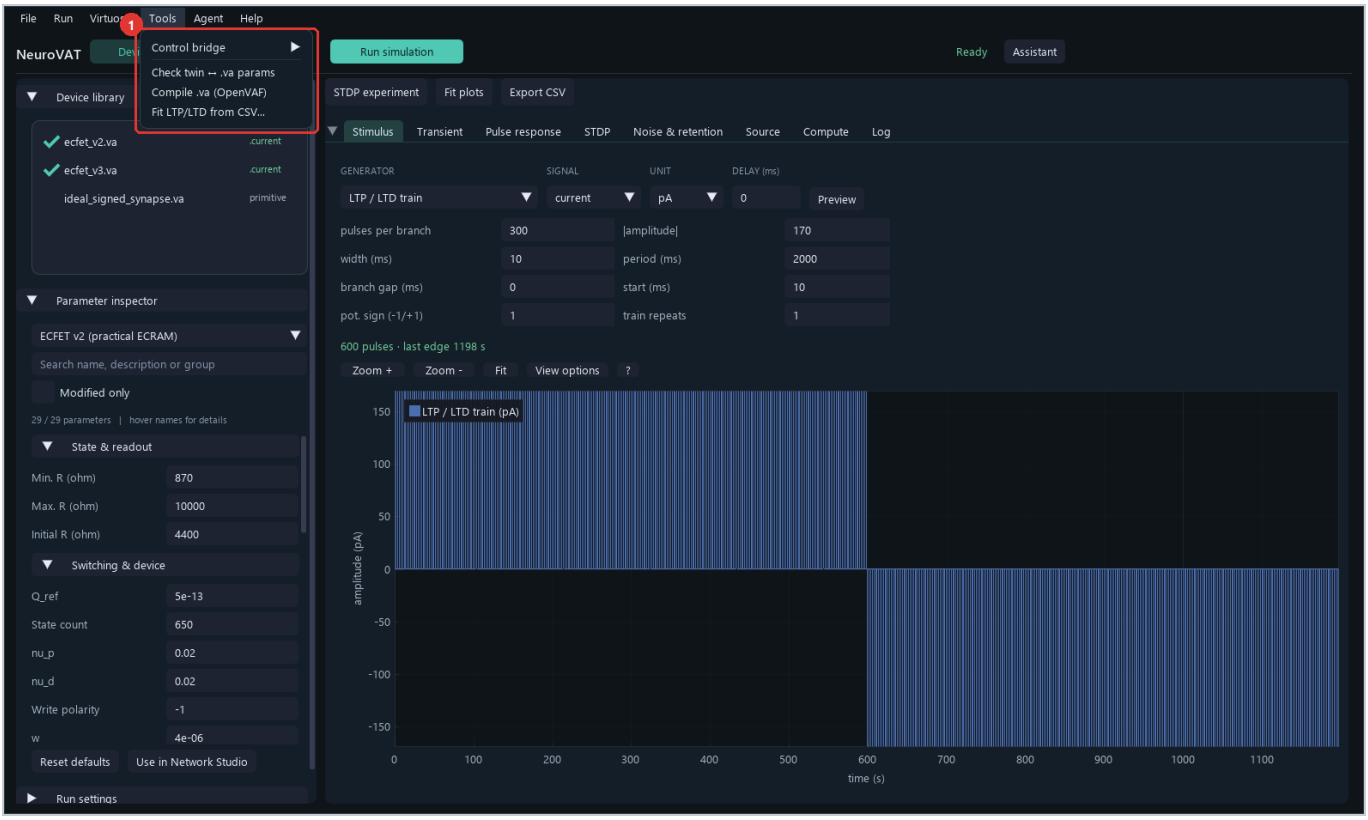


Figure: The Tools menu.

1. The Tools menu:
   - **Control bridge**: a submenu that shows the bridge status and connects or disconnects it.
   - **Check twin ↔ .va params**: compares every `.va` file's parameters with its Python twin.
   - **Compile .va (OpenVAF)**: compiles the ticked `.va` files with OpenVAF.
   - **Fit LTP/LTD from CSV...**: fits the nonlinearity of a measured conductance-per-pulse curve.

   Chapter 3 shows all three tools in use.

Agent

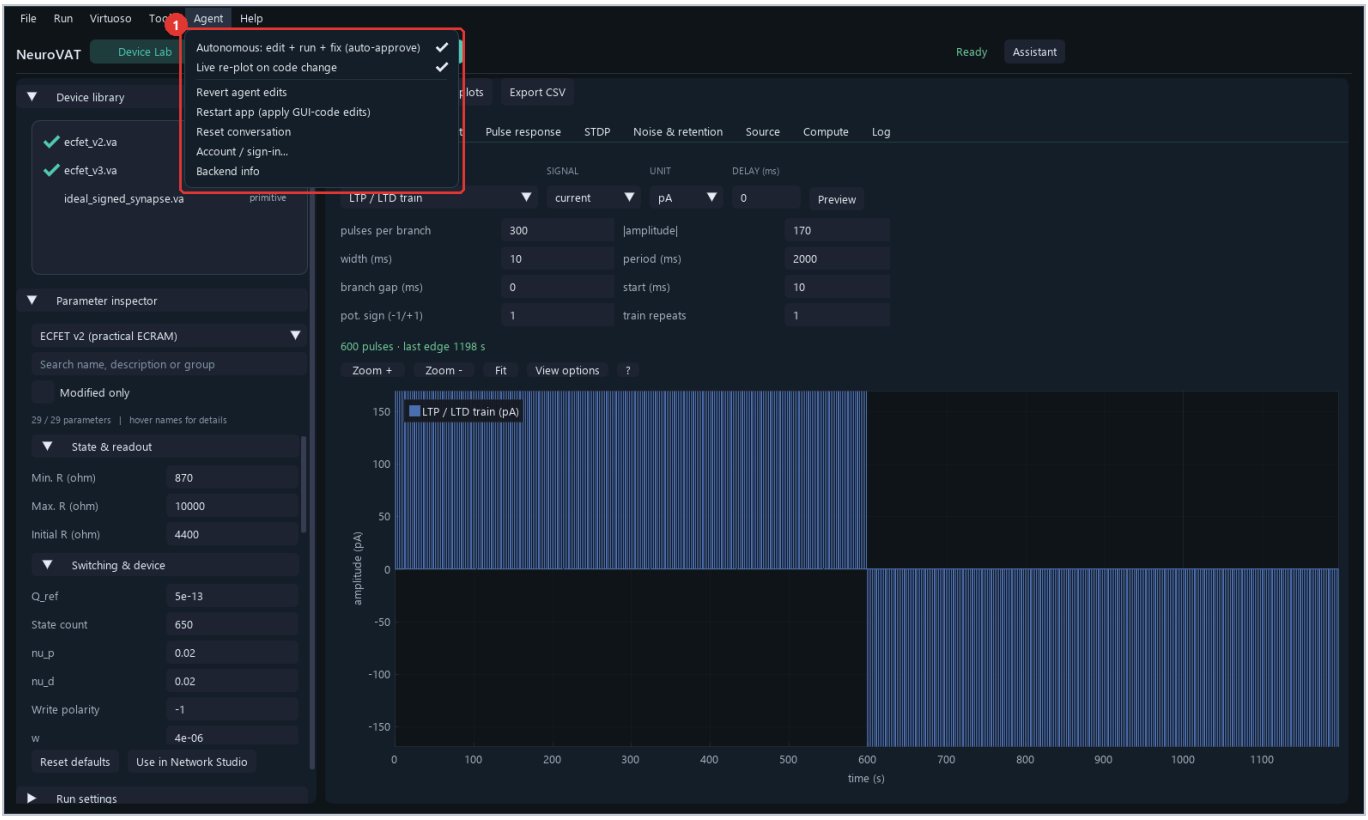


Figure: The Agent menu (the AI assistant's settings).

1. The Agent menu:
    - **Autonomous: edit + run + fix (auto-approve)**: on by default. When it is on, the assistant may edit files and run shell commands without asking you first. When it is off, the assistant can only read.
    - **Live re-plot on code change**: reruns the plot when a model twin's code changes.
    - **Revert agent edits**: undoes the assistant's file edits.
    - **Restart app (apply GUI-code edits)**: restarts the app so that edits to the app's own code take effect.
    - **Reset conversation**, **Account / sign-in...** and **Backend info**: manage the chat session and its login.

> **Warning.** Autonomous mode is **on** by default. In this mode the assistant can change your `.va` and twin files and run commands without asking. Turn it off if you only want answers. The assistant chapter covers this in detail.

Help

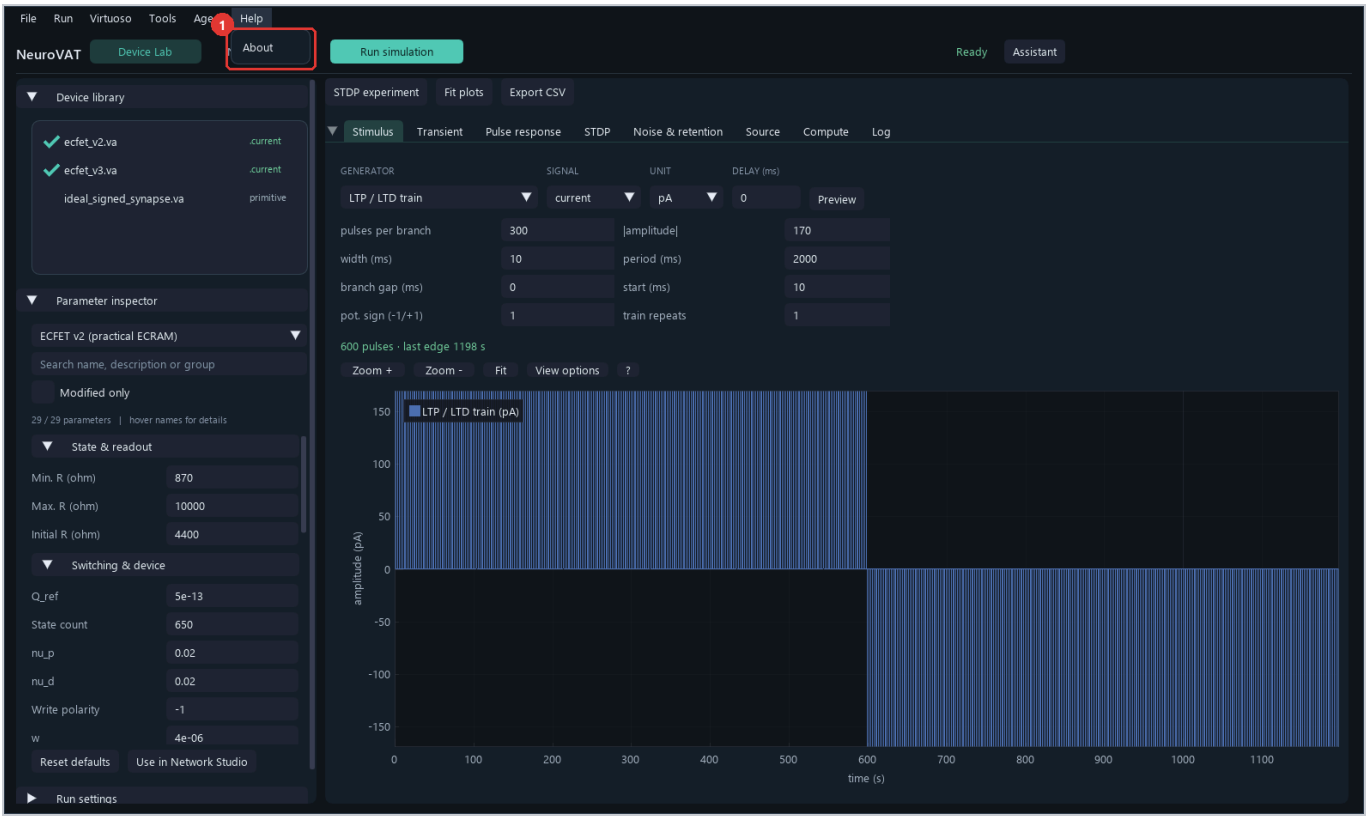


Figure: The Help menu.

1. The Help menu has one item, **About**. It prints a one-line description of the app in the Log tab and switches to that tab.

2.5 Keyboard and mouse

- **F5**: run the active page.
- In a plot: the **mouse wheel** zooms around the cursor, **dragging** pans, and a **double-click** fits the plot to its data.
- Press **A** or **B** and then click a curve to place probe A or B. A readout above the plot then shows both points and the difference between them. Press **C** over a plot to clear its probes, and **Esc** to cancel a probe you have not placed yet.
- **Right-click** a plot for its menu.

CHAPTER 3

Device Lab: testing a single synapse

This chapter is a complete session on one device: the public **ECFET v2** model (`verilog\ecfet_v2.va`, simulated by its Python twin `simulation_core\model_v2.py`). You will pick the model, change one parameter, design an LTP/LTD pulse train, run it, read the transient and the pulse response, run an STDP experiment, look at the remaining tabs, export the data, and use the three Tools-menu utilities.

Every number in this chapter was measured in one session with NeuroVAT v1.3.1 from source (Python 3.13, DearPyGui 2.3.1, Windows 11), using the settings given in each step. Your numbers will match if you use the same settings. The simulations are deterministic, and the model's cycle-to-cycle noise `sigma_c2c` is 0 by default.

3.1 Step 1. Pick the synapse and set a parameter

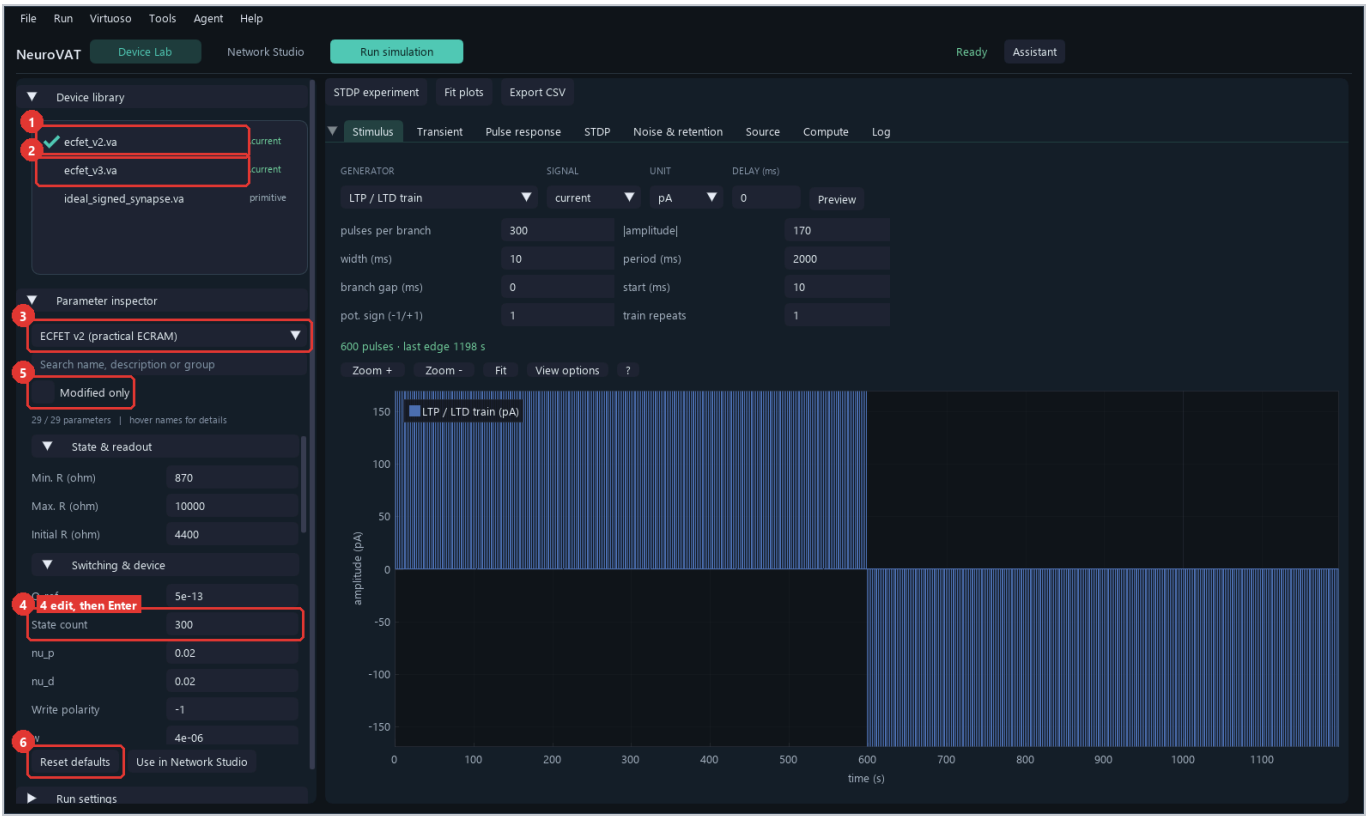


Figure: Device library and Parameter inspector. Only ecfet_v2.va is ticked, and State count has been changed from 650 to 300.

1. **ecfet_v2.va:** tick it and click its name. The tick means "simulate this model on the next run". The click loads the parameter values written in the `.va` file into the inspector. The log then reports something like "ecfet_v2.va -> v2: applied N param(s)".
2. **ecfet_v3.va:** untick it. Both ECFET models are ticked when the app starts, and every ticked model is simulated and plotted together. Unticking v3 keeps the plots to one device.
3. **Model drop-down:** check that it reads **ECFET v2 (practical ECRAM)**. The inspector edits the parameters of the model chosen here.
4. **State count** (`n_states`): type `300` (default 650) and press Enter. The source says that one reference pulse (`Q_ref` = 50 pA x 10 ms) moves G by about `(Gmax - Gmin) / n_states` times a soft-bound window. Halving the state count therefore roughly doubles the conductance step per pulse, which makes the steps easy to see in the plots.
5. **Modified only:** tick it to list only the values you changed (next figure).
6. **Reset defaults:** restores every parameter of this model to its default value.

> **Note. There is no Apply button.** A value takes effect as soon as you leave the field or press Enter. It is stored for this model and used by the next Run, STDP or Compute experiment. The values are kept only for this session and are never written back into the `.va` file. To make a change permanent, edit the source (Step 7).

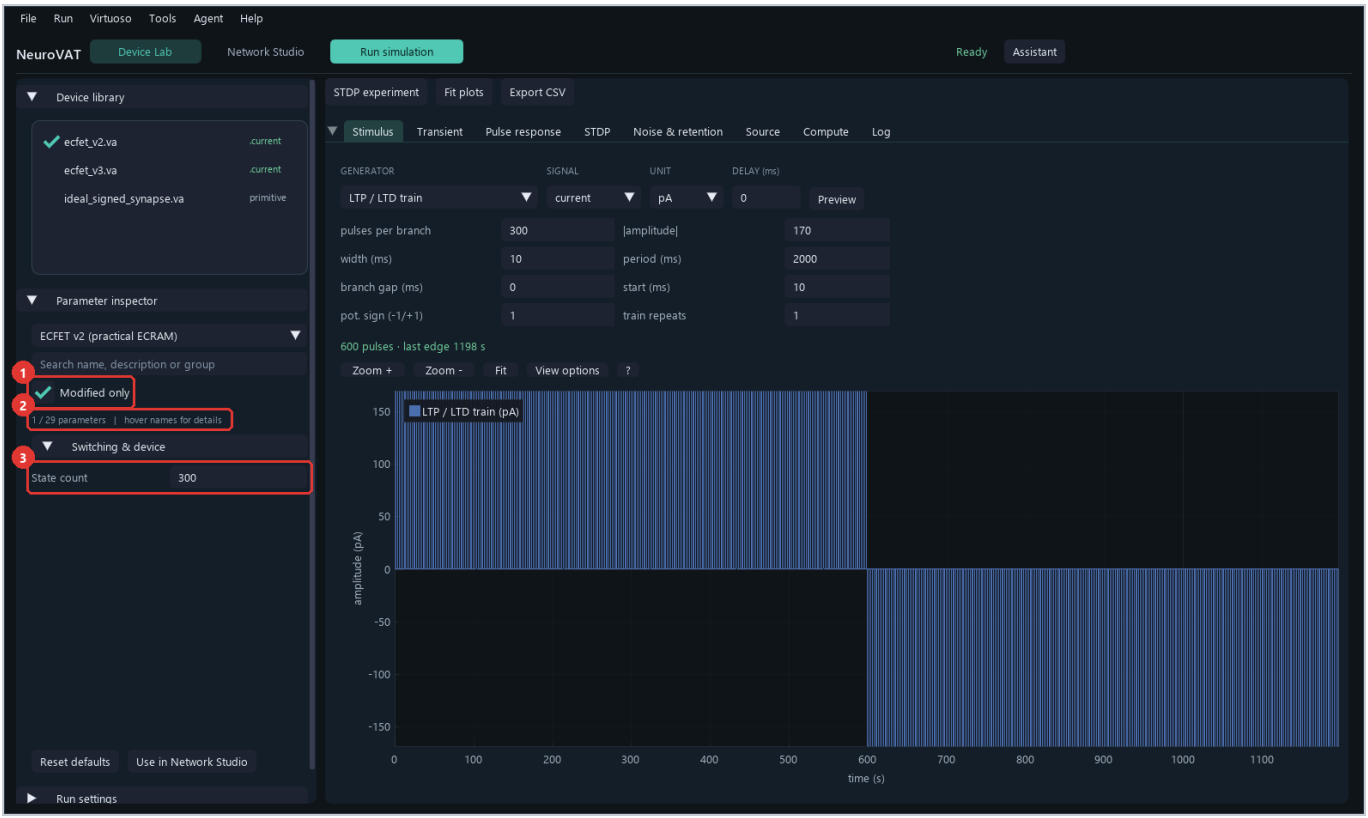


Figure: With "Modified only" ticked, the inspector lists only the changed parameter.

1. **Modified only:** ticked.
2. **Count:** "1 / 29 parameters". ECFET v2 has 29 parameters, and you changed one.
3. **State count = 300:** the changed value. Hover over a parameter name to see its default and the description from the model source.

3.2 Step 2. Design the stimulus

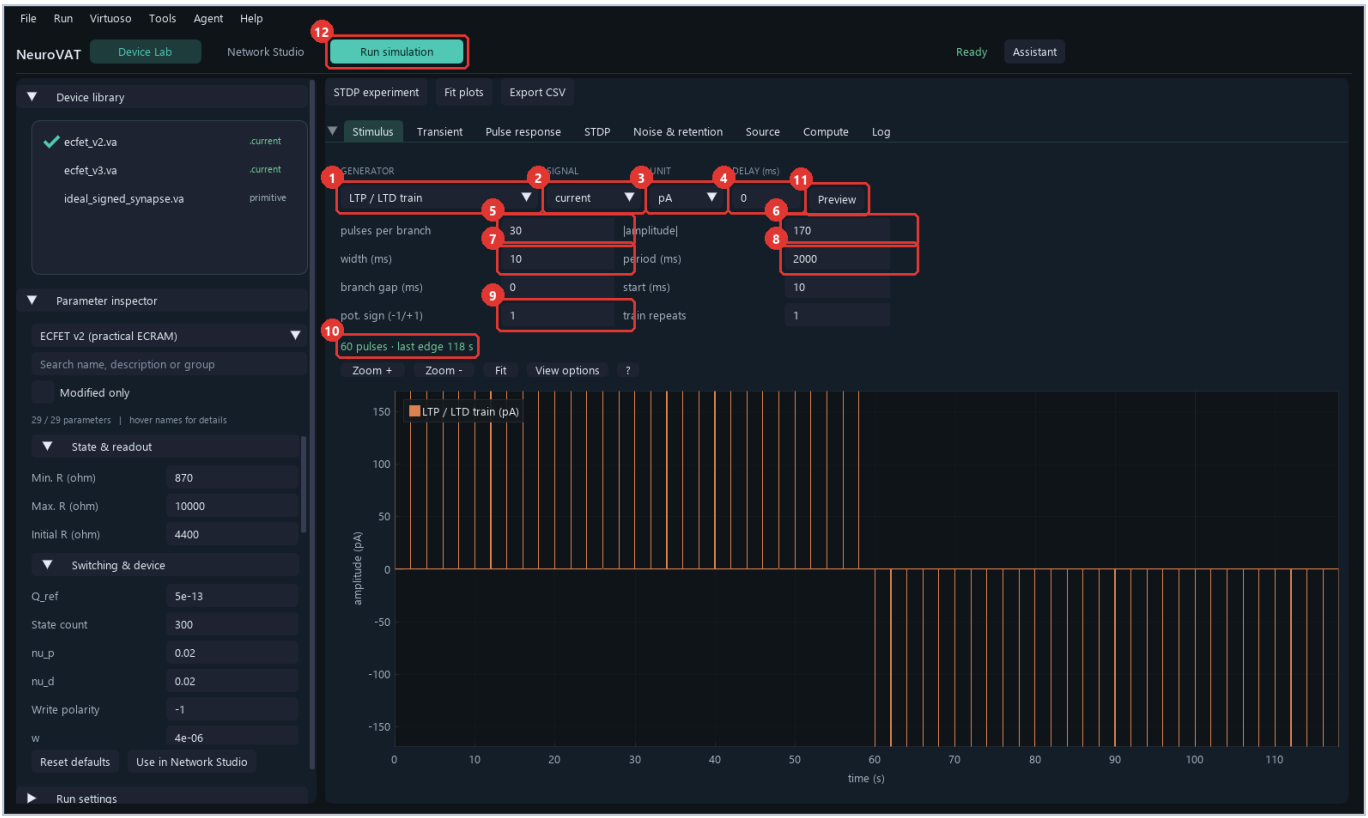


Figure: The Stimulus tab set up for a 30 + 30 pulse LTP/LTD train at 170 pA.

1. **Generator: LTP / LTD train**. It sends n potentiating pulses and then n depressing pulses, which gives the classic synaptic LTP/LTD curve. The other generators are Single spike, Pulse train, Paired pulses (PPF), Poisson spike train, Burst pattern, STDP pair (pre/post), Staircase and Custom pattern. Custom pattern takes one row per pulse: `t_start_ms width_ms amplitude`. Hover over the drop-down to read a description of the selected generator.
2. **Signal: current**. The ECFET is a current-driven (gate-current) device; the library shows it as `.current`. Choose **voltage** only for voltage-driven twins such as a FeFET. If the drive does not match the model, the log warns that the amplitudes are being treated as the model's own drive type.
3. **Unit: pA**. Every amplitude on this page is in this unit. Current drive offers pA, nA, uA, mA and A; voltage drive offers mV and V.
4. **Delay (ms):** 0. This shifts the whole waveform later in time.
5. **Pulses per branch:** `30` (default 300). That gives 30 LTP and 30 LTD pulses. It keeps the run short and the individual steps visible.
6. **|amplitude|:** 170 pA (the default). The sign is set by the pot. sign field.
7. **Width (ms):** 10. Each pulse carries 170 pA x 10 ms = 1.7 pC, which is 3.4 times `Q_ref`.
8. **Period (ms):** 2000. The generator's description says the default spacing is chosen so the slow volatile pool (tau3 = 19 s) relaxes between pulses. The retained curve is then a clean ramp, and the transient does not saturate.
9. **Pot. sign:** `+1`. For ECFET v2, a positive current potentiates (raises G). Use -1 for a model with the opposite sign convention.

10. **Summary:** "60 pulses · last edge 118 s". Check this line before you run.
11. **Preview:** redraws the waveform plot. The preview also updates by itself a moment after any edit.
12. **Run simulation:** starts the run (Step 3).

> **Tip.** The simulation stops at the last pulse edge plus the **post-stimulus tail**. The tail is set under **Run settings** in the left panel and is 60 s by default. Here the run ends at about 118 + 60 = 178 s. A long tail shows the relaxation after the train. Use a shorter tail if you only want the pulses.

3.3 Step 3. Run and read the transient

Press **Run simulation** or **F5**. The app switches to the Transient tab when the run finishes. With the settings above, the run produced 7559 time points over 178 s. It took 0.4 s of wall time in our session and 0.1 s in a second test on a fresh copy. Wall times depend on your machine and on what else is running (our session had other app instances open at the same time).



Figure: Transient tab after the run: stimulus, R_mem(t) and G(t) on one linked time axis. Probe A (start) and probe B (end of the LTP branch) are placed on the G plot.

1. **Run simulation (F5):** runs every ticked model with the current stimulus and parameters.
2. **Status:** "done · 1 result(s)". There is one result because only v2 is ticked.
3. **Plot toolbar: Zoom +** and **Zoom -** zoom all three plots, **Fit** rescales them to the data, **View options** fits one axis or shows and hides legends, and **?** lists the mouse gestures.
4. **Probe readout:** shows A and B and their difference. Press **A** and click the curve, then press **B** and click again. Here A is at 0.005 s (G = 227.3 µS) and B is at 58.02 s (G = 383.2 µS), so dY = +155.9 µS over the LTP branch. This is the *total* conductance, so it includes the volatile part that relaxes after each pulse.
5. **Stimulus** (I_gate in pA): 30 positive pulses and then 30 negative pulses.
6. **R_mem (ohm):** starts at 4400 Ω, falls to a minimum of 2610 Ω during LTP, rises to a maximum of 5319 Ω at the end of LTD, and relaxes to 4416 Ω by the end of the tail. The log line "[v2 (practical ECFET)] R: start 4400 ohm, end 4416 ohm, min 2610, max 5319" gives these numbers.
7. **G (uS):** each pulse makes a sharp step, and part of each step then relaxes (the sawtooth). The underlying level ramps up during LTP and down during LTD.

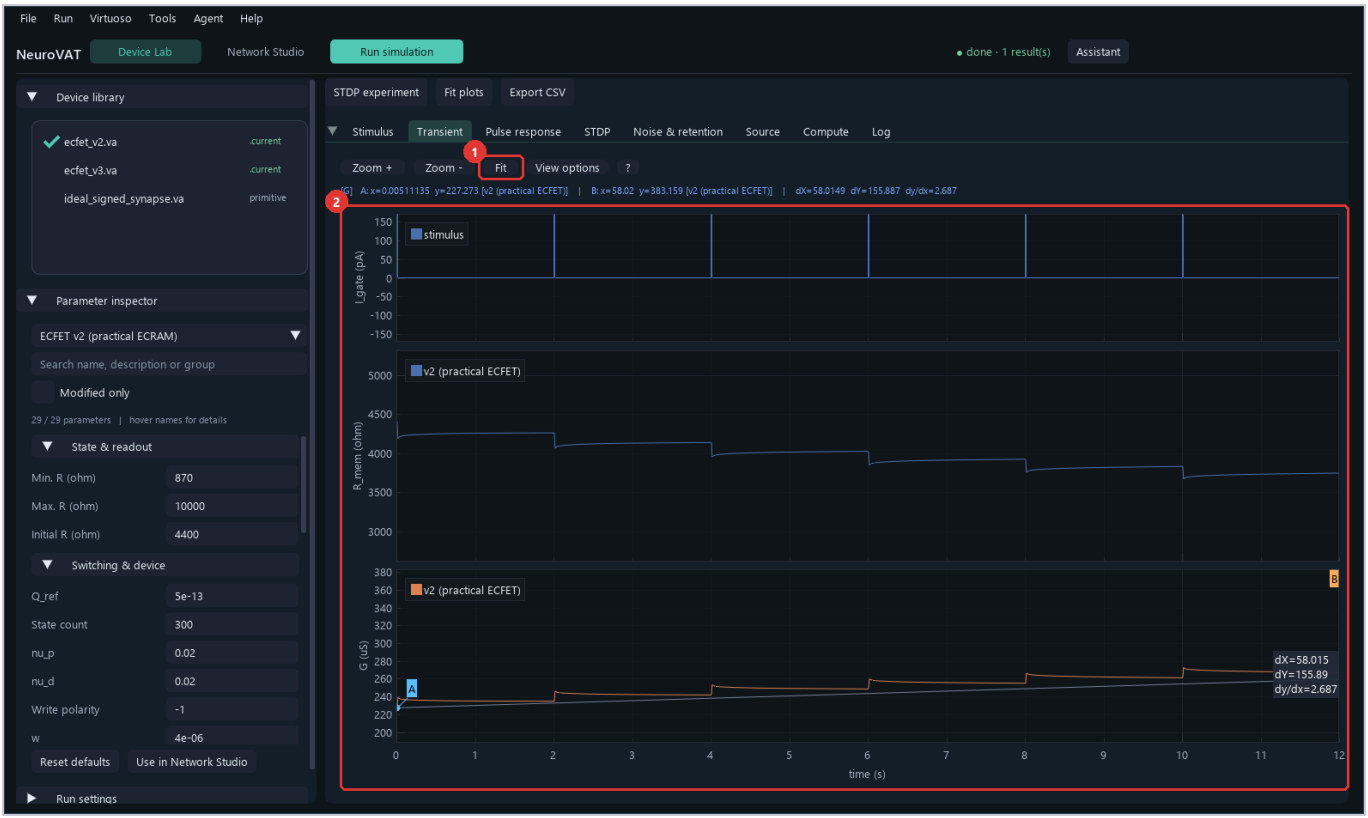


Figure: The same run zoomed to 0-12 s. The three plots share the time axis, so zooming one zooms all of them.

1. **Fit:** returns to the full view after zooming. A double-click on a plot does the same.
2. **Linked plots:** zoomed to the first five pulses. Each pulse gives an immediate step in R and G and then a slower partial relaxation, which is the volatile part of the write. Zoom with the mouse wheel over a plot; the zoom centres on the cursor.

3.4 Step 4. Pulse response: G versus pulse number

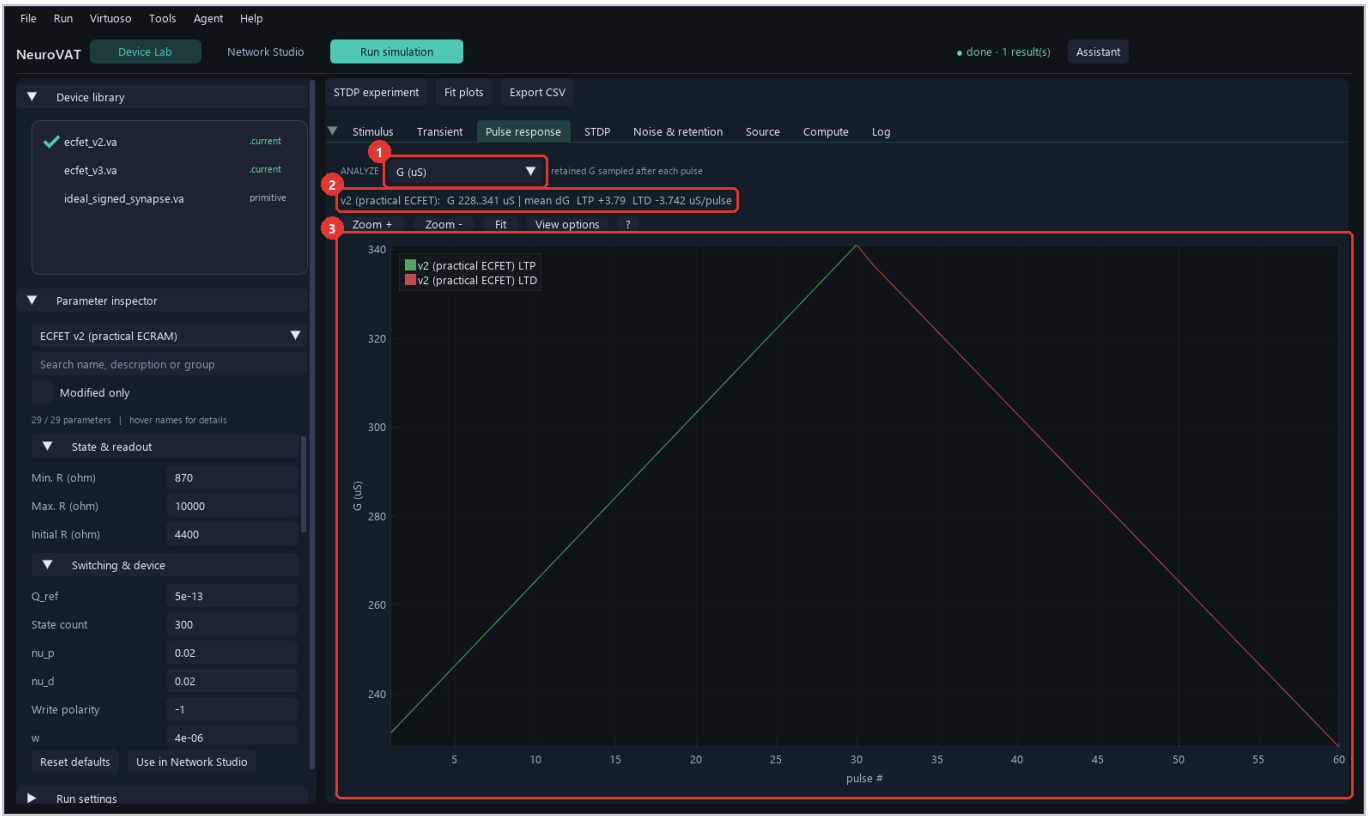


Figure: Pulse response tab: retained conductance after each of the 60 pulses, drawn as a line. In this screenshot the LTP branch is green and the LTD branch is red.

1. **Analyze: G (uS)** or **R_mem (ohm)**. The same choice is in the **Run** menu.
2. **Summary line:** "G 228..341 uS | mean dG LTP +3.79 LTD -3.742 uS/pulse".
3. **Plot:** a line through the per-pulse values (one value per pulse, no markers). The LTP branch rises and the LTD branch falls.

> **Note.** Plot colours can differ from the screenshots. In a later test on a fresh copy, the app drew the LTP branch blue and the LTD branch orange here, and the Transient tab's G trace blue (it is orange in the Transient figure above). Use the legend in the top-left corner of each plot to tell the series apart.

What the numbers mean. This tab does not use the total G from the Transient tab. For ECFET v2 it plots the **retained**, nonvolatile weight `G_nv`, sampled shortly after each pulse, which is the weight a network would keep. The mean ΔG is the average step per pulse on each branch. Here it is +3.79 µS per potentiating pulse and -3.742 µS per depressing pulse, so the response is almost symmetric. The range of 228 to 341 µS (a ratio of about 1.5) is how far 30 pulses moved the weight. Both branches are nearly straight lines, as you would expect for `nu_p` = `nu_d` = 0.02 (near-linear windows). Step 10 measures that linearity with the fit tool.

> **Note.** Why the two tabs differ: the Transient probe gave +155.9 µS over the LTP branch, but the retained weight rose by only about 113 µS (228 to 341). The difference is the volatile conductance, which is still present right after the last pulse and relaxes away later.

3.5 Step 5. STDP: the timing experiment

Open the **STDP** tab, either with its tab or with the **STDP experiment** button. The Run button now reads **Run STDP**.

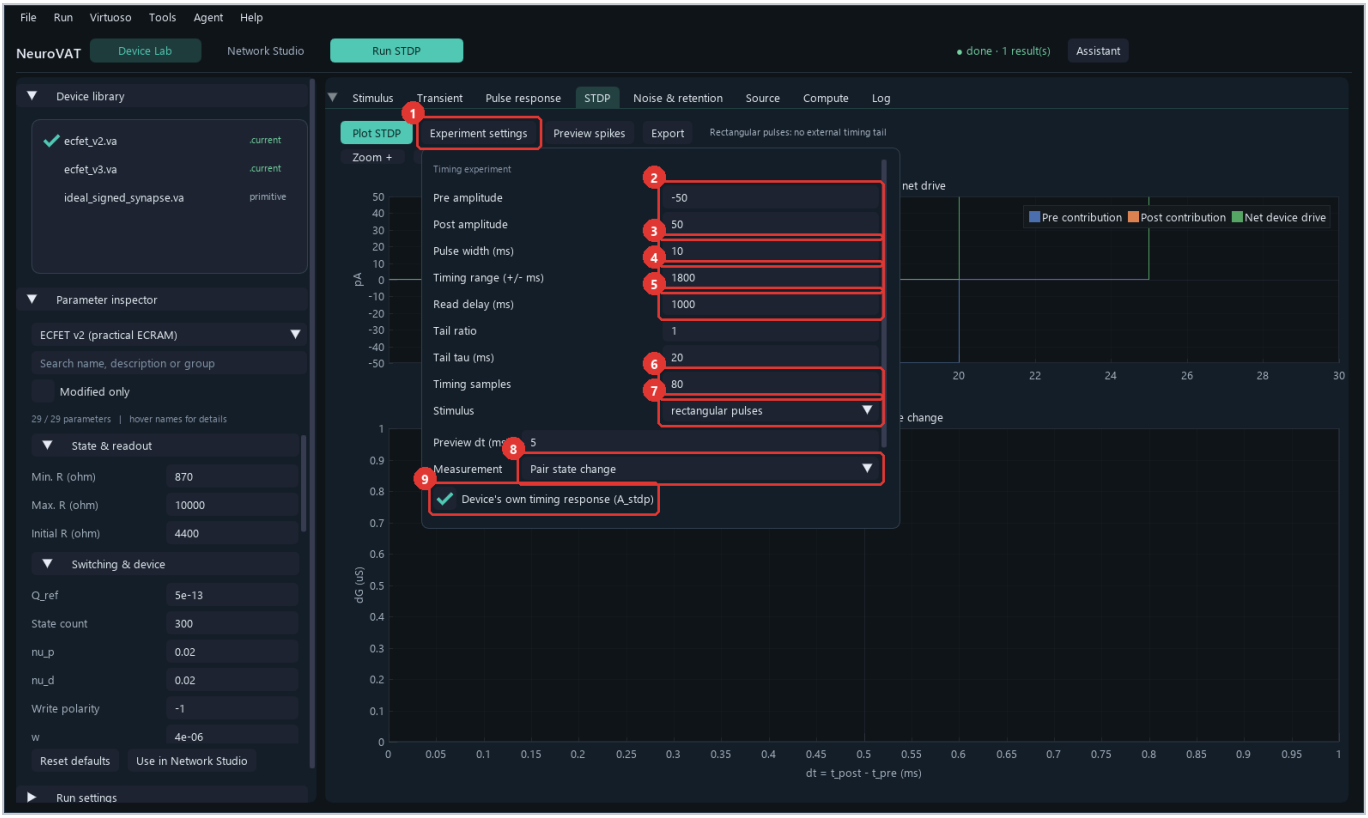


Figure: STDP tab with the Experiment settings panel open (default values).

1. **Experiment settings:** opens this panel.
2. **Pre / post amplitude:** -50 and +50, in the stimulus unit (pA here). These are signed contributions, and the device sees their sum. For two-terminal voltage programming, enter -Vpre and +Vpost.
3. **Pulse width (ms):** 10.
4. **Timing range (+/- ms):** 1800. Δt = t_post - t_pre is swept from -1800 to +1800 ms.
5. **Read delay (ms):** 1000. The state is read this long after the pair, so part of the volatile response has settled.
6. **Timing samples:** 80. More samples give a smoother curve but take longer.
7. **Stimulus: rectangular pulses**. The other option, overlapping spikes, enables the spike shaper (exponential tail, linear tail or RC high-pass) for waveform-overlap STDP.
8. **Measurement: Pair state change** is the actual modelled change at the read delay. **Interaction** (pair - pre - post + idle) isolates the non-additive part.
9. **Device's own timing response (A_stdp):** ticked by default. When it is ticked, the model's added phenomenological timing rule (`A_stdp`) is included. Untick it to see the device's response to the waveform alone.

Press **Run STDP**. With 80 samples, the run took 15.6 s.



Figure: STDP result with the default settings (the A_stdp rule included).

1. **Run STDP:** runs the experiment (F5 or Plot STDP do the same).
2. **Plot STDP:** the same action, inside the tab.
3. **Spike contributions and net drive:** the pre pulse, the post pulse and their sum for one example Δt. Check this first, because it shows exactly what the device receives.
4. **Timing-dependent state change:** ΔG (μS) against Δt (ms). Here the window is antisymmetric. It runs from -3.76 to +3.83 μS. Near Δt = ±20 ms it is about +3.39 μS (post after pre) and -3.32 μS (post before pre). It decays to about +0.86 / -0.79 μS at ±1800 ms.
5. **Export:** writes `results\stdp_experiment.json` and `.csv`, with the parameters, waveform settings, timing grid and all four control responses.



Figure: The same experiment with "Device's own timing response (A_stdp)" unticked.

1. **Experiment settings:** untick **Device's own timing response (A_stdp)** and run again.
2. **Result:** the window is gone. ΔG stays between -0.092 and +0.157 µS and has no peak near Δt = 0. It is about +0.03 µS at both Δt = +20 ms and -20 ms.

> **Warning. How to read the STDP window.** With non-overlapping rectangular pulses, the large antisymmetric window in the first figure comes from the model's *added* timing rule (`A_stdp`). It does not come from the pulse waveforms. With the rule off, the ECFET v2 twin shows almost no timing dependence. The README says these rules are "phenomenological rules, not proof of intrinsic device timing memory". Report which setting you used. To study waveform-driven STDP, choose overlapping spikes with a shaper.

3.6 Step 6. Noise & retention

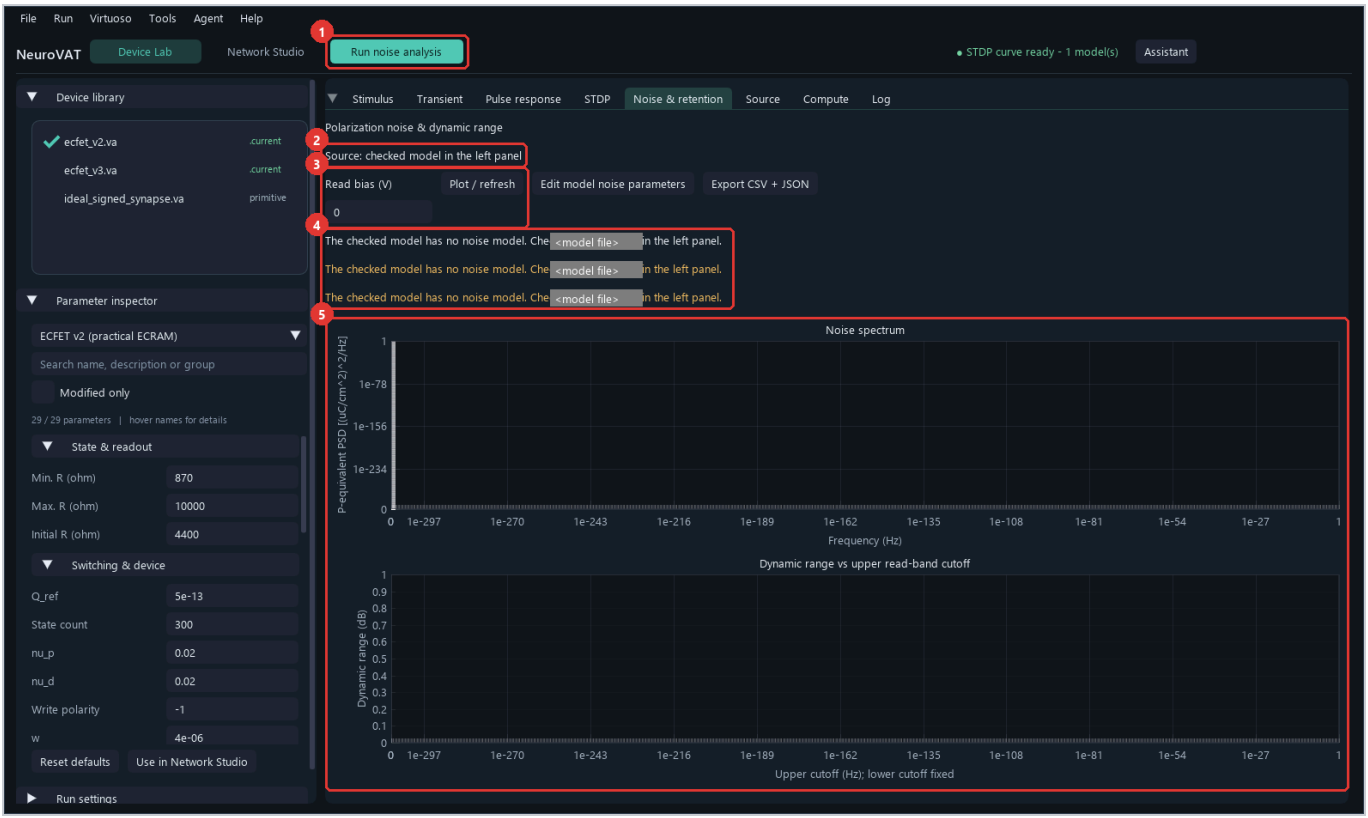


Figure: The Noise & retention tab with ECFET v2 ticked. The message names a model file that is not part of the public release (covered here).

1. **Run noise analysis:** the Run button's label on this tab.
2. **Source:** the tab uses the ticked model in the left panel.
3. **Read bias (V)** and **Plot / refresh:** the read condition, and the button that recalculates. **Edit model noise parameters** and **Export CSV + JSON** sit next to them.
4. **Status:** "The checked model has no noise model. Check <model file> in the left panel." The same text appears three times.
5. **Plots:** the noise spectrum and the dynamic range against the upper cutoff. They stay empty, and their axes show meaningless ticks (1e-297 and so on).

> **Warning. This tab does nothing with the public models.** It draws polarization-noise spectra and dynamic range, and only for a twin that defines `noise_psd` and `noise_budget`. None of the public models (ECFET v1/v2/v3) defines them, so every model gives the error above. The model file that the message names is not shipped with the app. We checked this inside our own driver and again on the live app (bridge command `noise_plot`), and both returned the same error. The tab also does not compute retention. For retention, run a long post-stimulus tail on the Transient tab (Run settings > post-stimulus tail), or run `python run_ecfet.py retention --hold-s 60` from a source install.

3.7 Step 7. Source: view or edit the .va

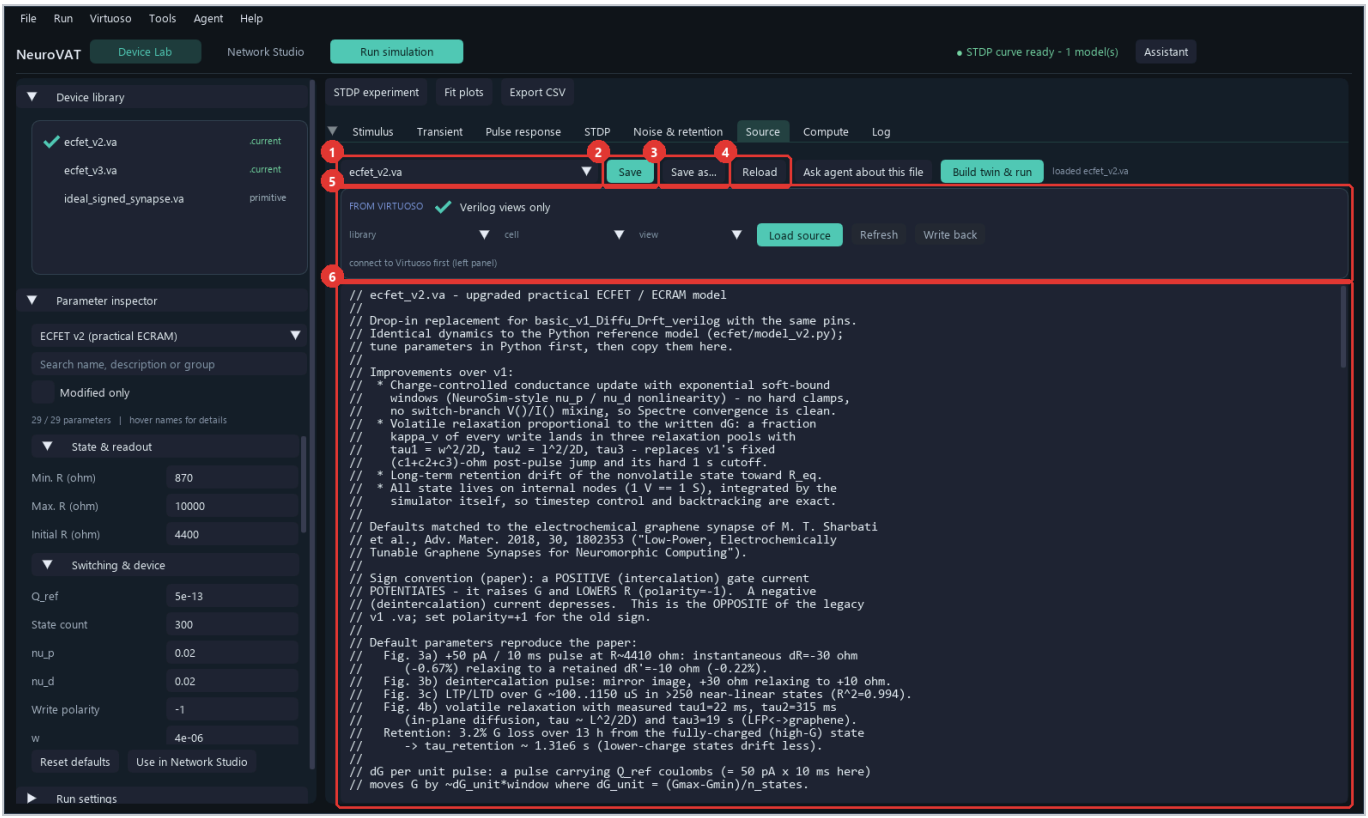


Figure: The Source tab with ecfet_v2.va loaded.

1. **File:** the `.va` file to show. Double-clicking a file in the Device library also opens it here.
2. **Save:** writes the editor text back to the file and rescans the library.
3. **Save as...:** saves a copy under a new name in `verilog\`. See the warning below.
4. **Reload:** discards your unsaved edits and reloads the file from disk.
5. **From Virtuoso:** loads a Verilog-A view from a connected Virtuoso library, or writes the editor text back to one. You must connect first (left panel, Cadence Virtuoso). **Write back** overwrites the cellview on the remote host, and there is no undo.
6. **Editor:** the full source. The header of `ecfet_v2.va` explains the model, the sign convention (a positive gate current potentiates) and which published figures the defaults reproduce.

> **Note.** Editing the `.va` file does not change the simulation. Device Lab runs the Python twin (`simulation_core\model_v2.py`). Keep the two in step by hand, and use **Tools > Check twin ↔ .va params** (Step 10) to catch parameter mismatches. Two other buttons, **Ask agent about this file** and **Build twin & run**, use the AI assistant.

> **Warning. Save as...** opens a file dialog. In our source install (DearPyGui 2.3.1), showing that dialog raised `KeyError: 'rect_size'` in the layout code. In the normal app this error closes the window. Use **Save**, or copy the file in Explorer, instead.

3.8 Step 8. Compute: the device as a compute element

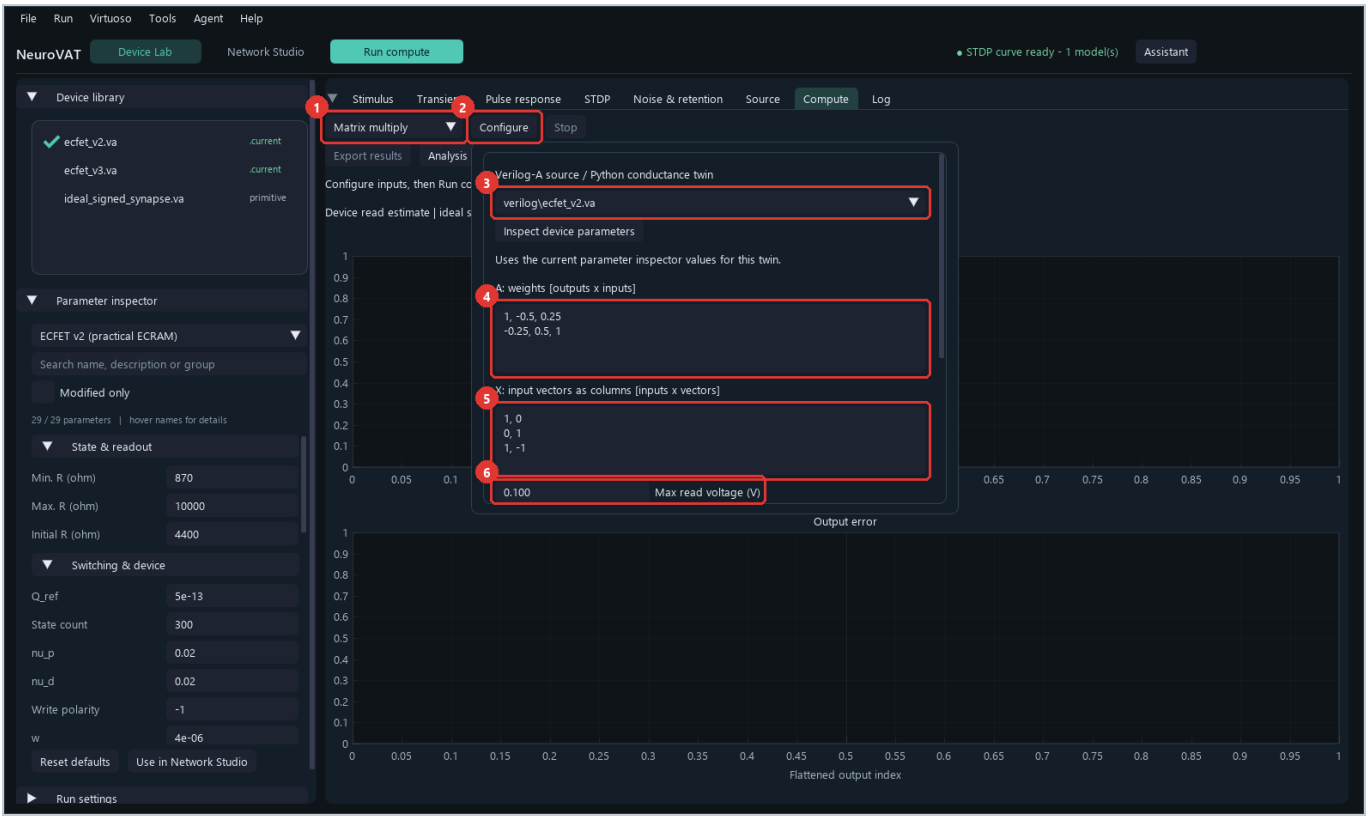


Figure: The Compute tab with its Configure panel open (default matrix-multiply inputs).

1. **Mode: Matrix multiply**, **Logic / ALU** or **Terminal energy**.
2. **Configure:** opens this panel.
3. **Verilog-A source:** `verilog\ecfet_v2.va`. The calculation uses its twin with the inspector's current values, so `n_states` = 300 here.
4. **A:** the weights, as outputs x inputs. The default is a 2 x 3 matrix with negative entries. Signed weights use a differential pair of devices.
5. **X:** the input vectors as columns (3 x 2 by default).
6. **Read settings:** scroll down for the maximum read voltage (0.1 V), read duration (100 ns), conductance levels (256), mismatch (0), seed (17), parallel banks (1), per-read overhead and peripheral energy.

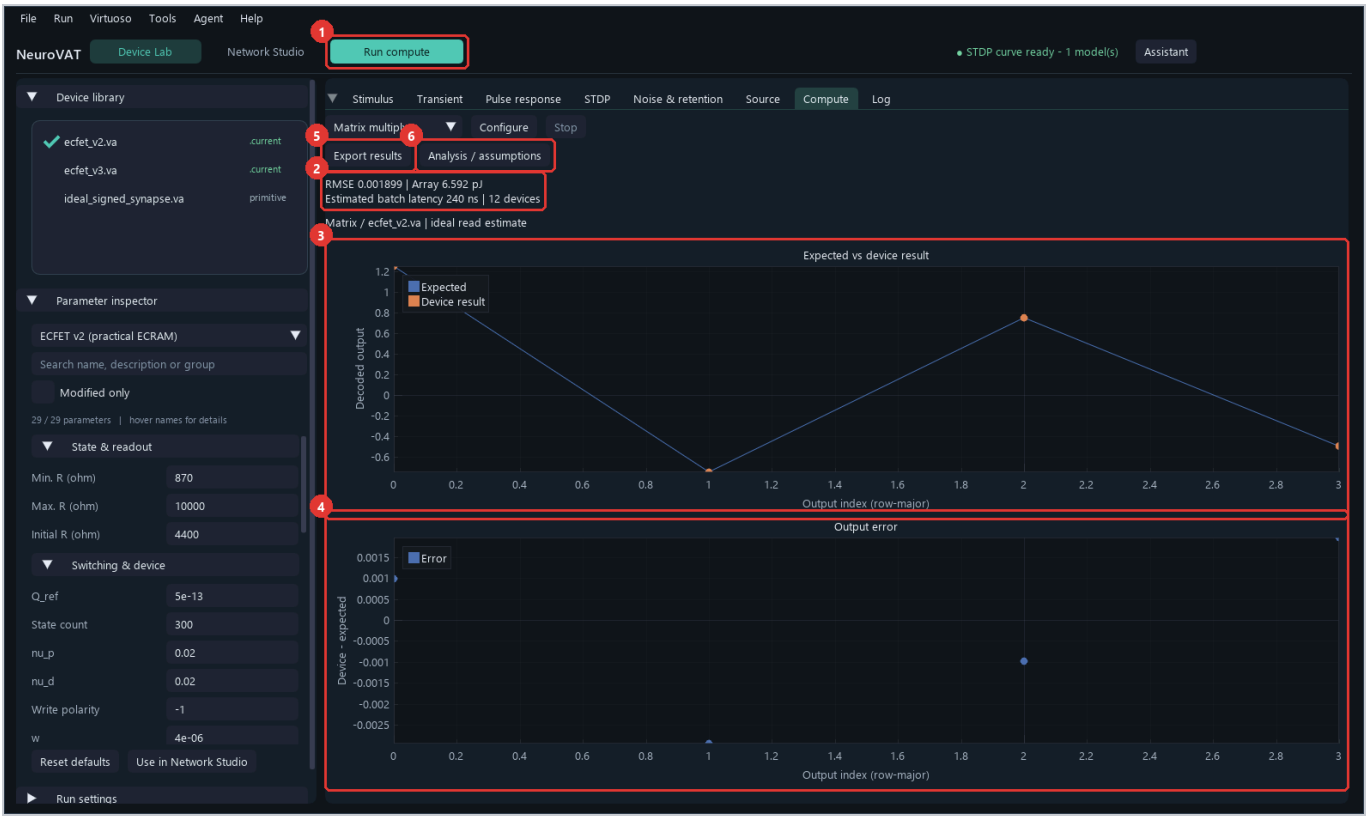


Figure: The Compute result for the default matrix multiply on ECFET v2.

1. **Run compute (F5):** runs the experiment.
2. **Summary:** "RMSE 0.001899 | Array 6.592 pJ | Estimated batch latency 240 ns | 12 devices".
3. **Expected vs device result:** NumPy's `A @ X` next to the decoded crossbar output, for each of the 4 outputs.
4. **Output error:** device minus expected. The largest absolute error was 0.00294. With mismatch set to 0, this comes from quantizing to 256 conductance levels.
5. **Export results:** writes JSON and CSV to `results\compute\`. The JSON records the settings, the parameters and the SHA-256 of the `.va` source.
6. **Analysis / assumptions:** energy, power, throughput and the stated assumptions.

What it computes. The app sets ideal conductance states for the weights, quantizes them and adds optional seeded mismatch. It then reads the crossbar with the read voltage and decodes the output currents. Energy is `sum(V² · G) · read_duration` over both branches of each signed pair. The README says this does *not* simulate programming pulses, programming energy, wire parasitics or a nonlinear read bias. Logic / ALU checks every input case of NAND, AND, OR, XOR, NOT or a 1-4 bit adder. Terminal energy integrates V·I from a CSV with columns `t_s,v_1,i_1`.

3.9 Step 9. Export CSV and PNG

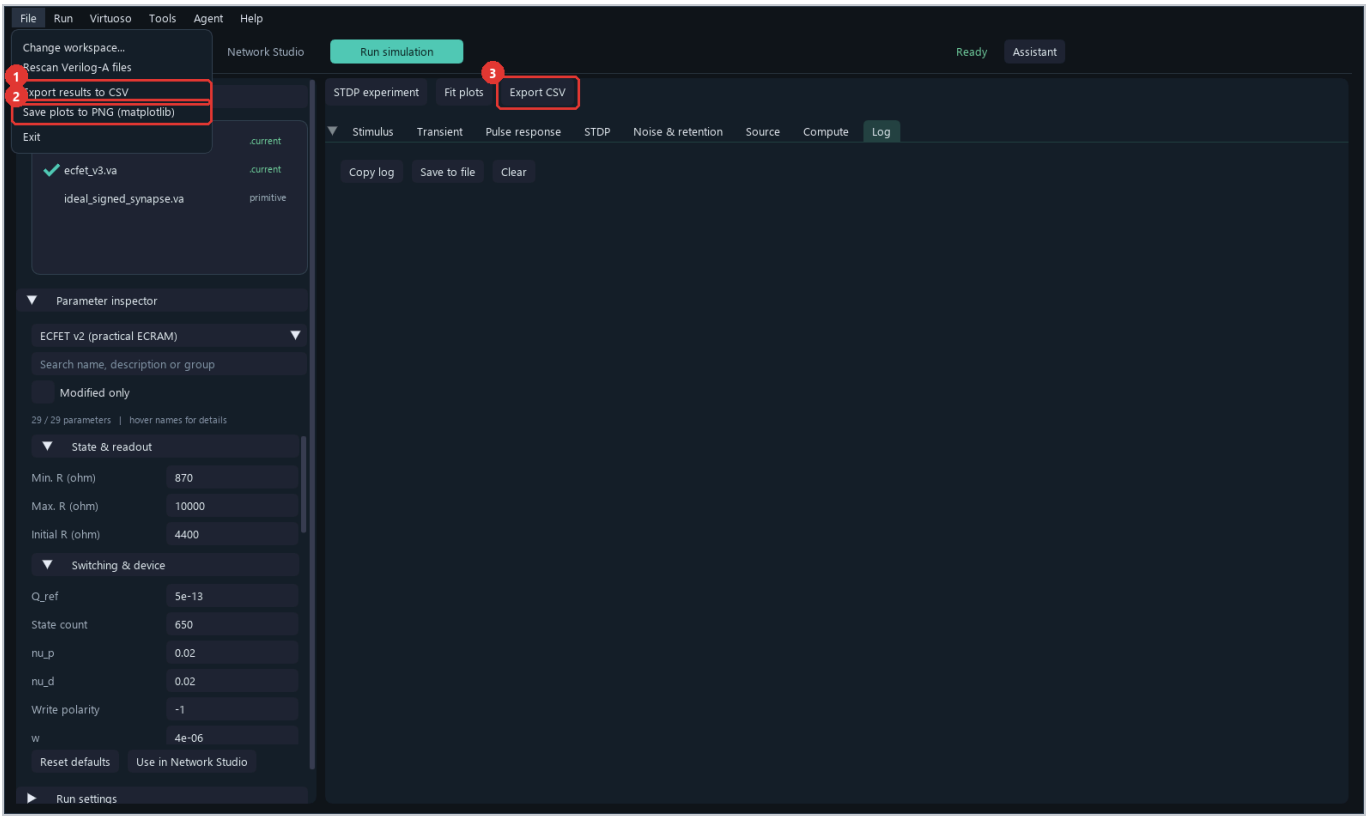


Figure: The export commands in the File menu, and the Export CSV button.

1. **File > Export results to CSV:** one CSV for each result of the last run.
2. **File > Save plots to PNG (matplotlib):** a publication-style figure of the transient.
3. **Export CSV:** the same as item 1, as a button in Device Lab.

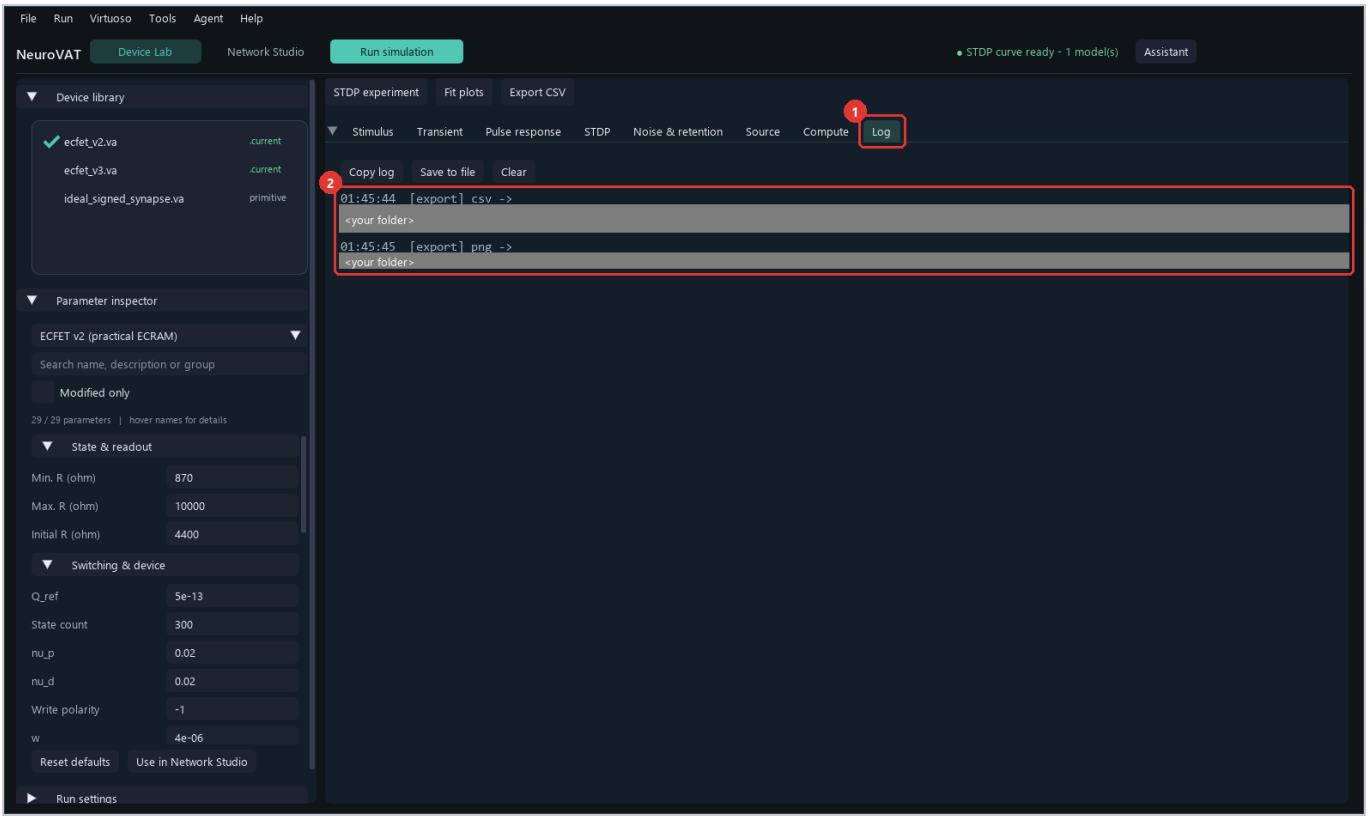


Figure: The Log tab after both exports. It shows the full path of each file (covered here).

1. **Log:** open this tab to see where each file was written.
2. **Export lines:** "[export] csv -> ..." and "[export] png -> ...", each followed by the full path.

All files go into the `results` **folder of your workspace**:

File	Contents
`results\gui_v2_practical_ECFET.csv`	The transient, named after the model's label. The columns are `t_s, i_gate_A, R_ohm, G_S, G_nv (S), G_volatile (S), drift_on`, in SI units: seconds, amperes, ohms and siemens. It has one row per time point (7559 here). Use it to compare with Spectre.
`results\gui_plot.png`	The matplotlib figure.
`results\stdp_experiment.json / .csv`	The last STDP experiment (Step 5).
`results\compute\*.json / *.csv`	The compute exports (Step 8).
`results\session_log.txt`	Log > Save to file.

> **Tip.** An export overwrites the previous file of the same name. Rename the files you want to keep. To save a CSV after every run without clicking, tick **auto-save CSV after run** under Run settings.

3.10 Step 10. Tools menu

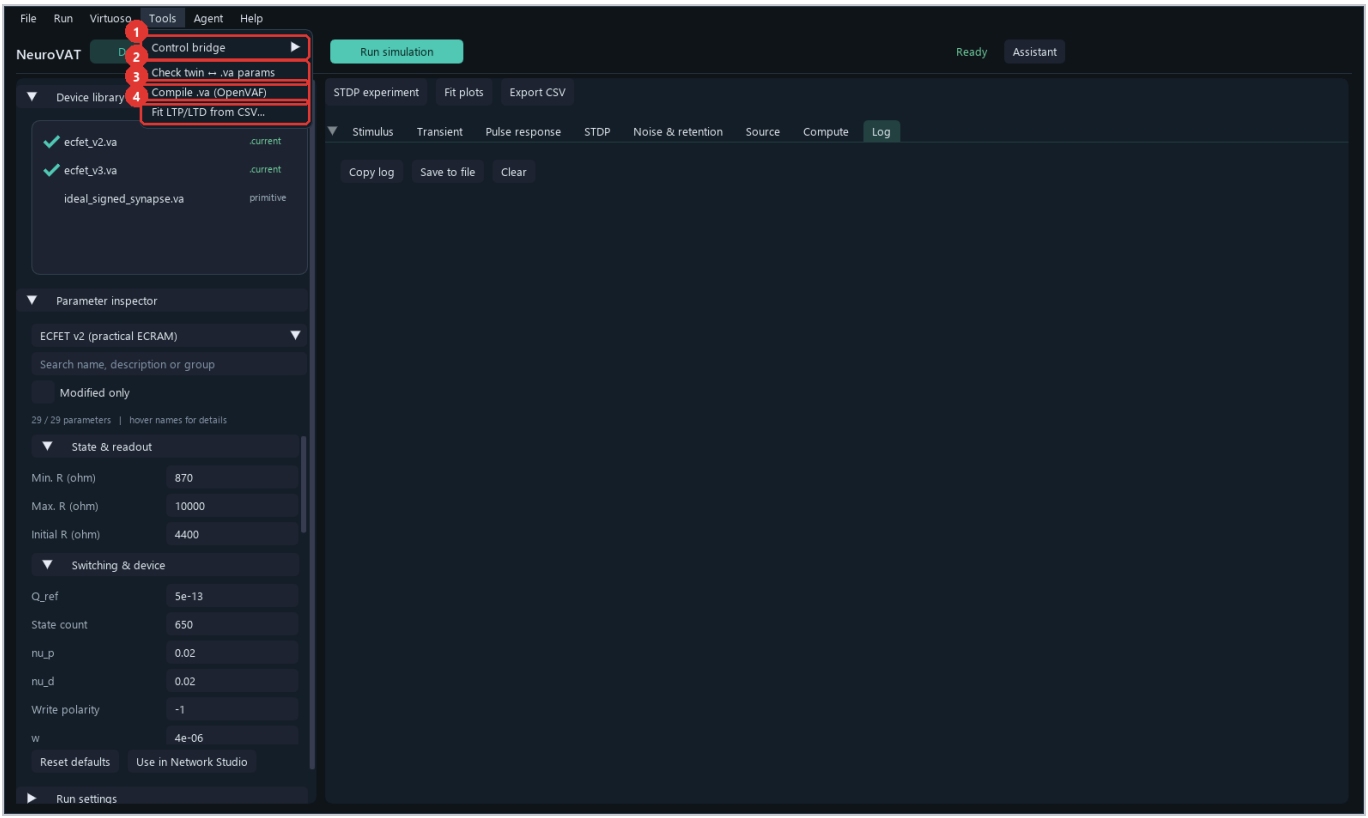


Figure: The Tools menu.

1. **Control bridge:** the status of the outside-driver mailbox, with an item to connect or disconnect it.
2. **Check twin ↔ .va params:** the consistency check.
3. **Compile .va (OpenVAF):** the compile check.
4. **Fit LTP/LTD from CSV...:** the nonlinearity fit.

10a. Twin versus .va consistency check

Click **Tools > Check twin ↔ .va params**. The app opens the Log tab and compares each `.va` file's `parameter` defaults with its Python twin's defaults, using a relative tolerance of 0.1 %.

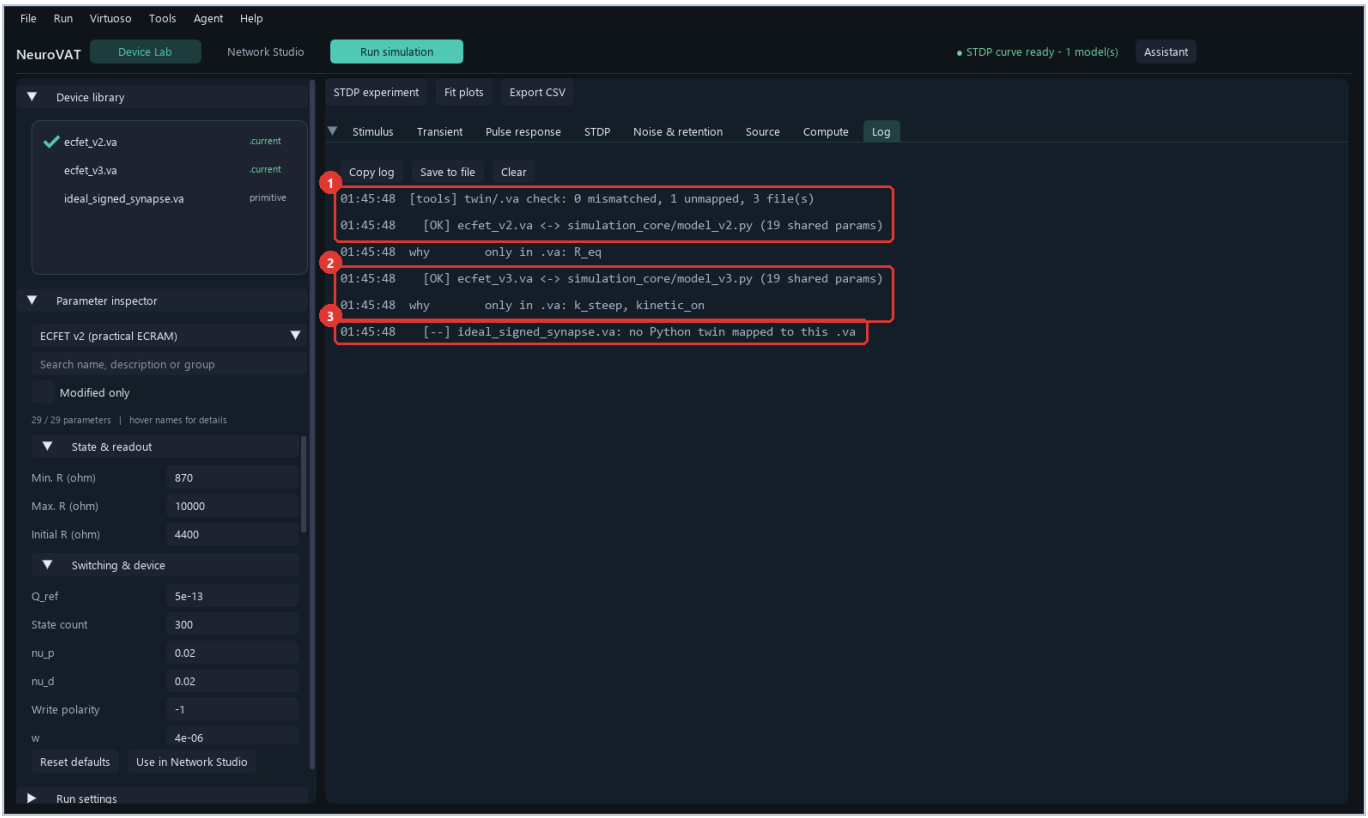


Figure: The consistency check in the Log tab.

1. **Summary and ECFET v2:** "0 mismatched, 1 unmapped, 3 file(s)". `ecfet_v2.va` and `model_v2.py` agree on all 19 shared parameters. `R_eq` exists only in the `.va`.
2. **ECFET v3:** also OK on 19 shared parameters. `k_steep` and `kinetic_on` exist only in the `.va`.
3. **ideal_signed_synapse.va:** no Python twin is mapped to it, so it cannot be checked. This is expected for a network primitive.

The check compares the *file defaults* only. Values you have typed into the inspector are not checked. Run it after every edit to a `.va` file or a twin.

10b. Compile with OpenVAF

Click **Tools > Compile .va (OpenVAF)**. The app compiles every *ticked* `.va` file to an OSDI library, so that Verilog-A errors show up before you take the file to Cadence. For this figure we ticked both ECFET models.

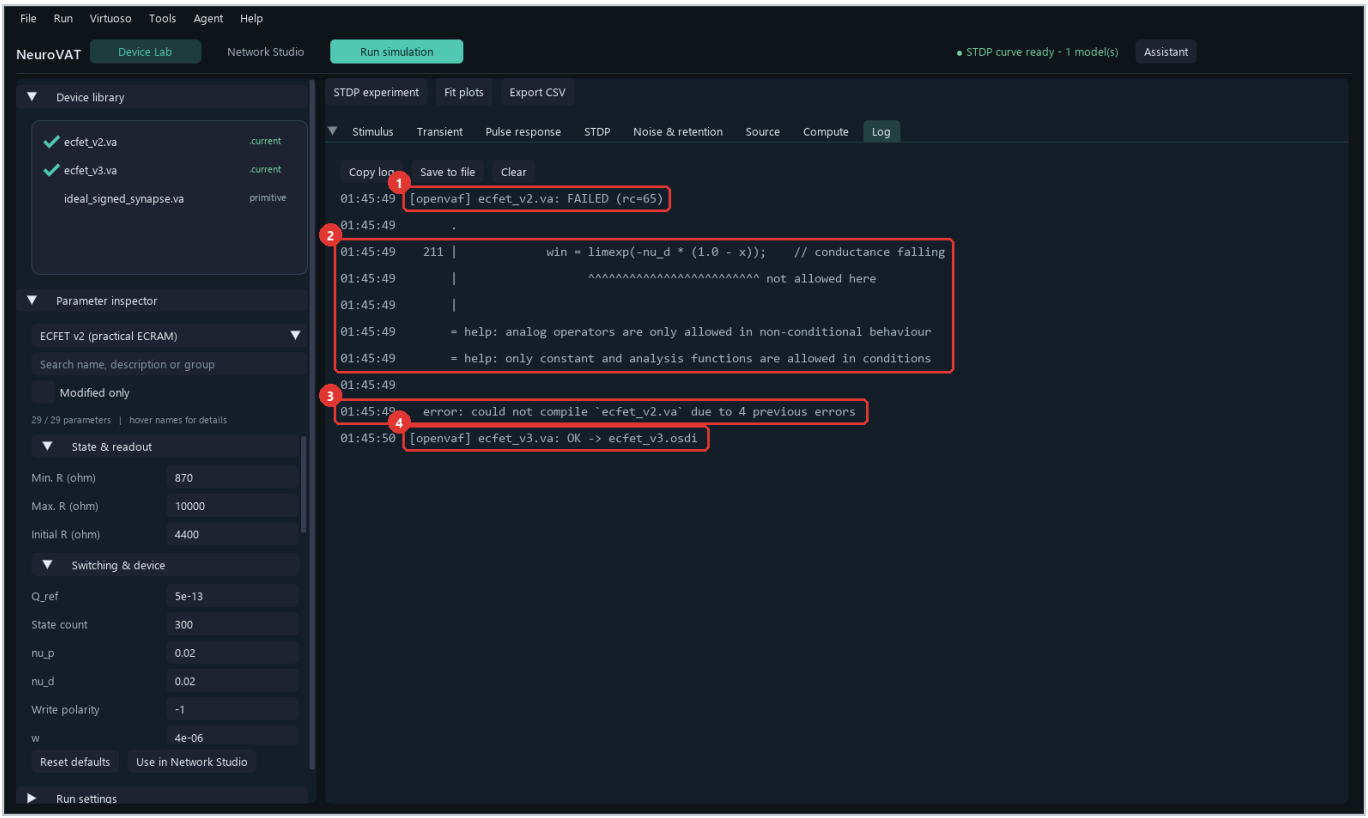


Figure: OpenVAF 23.2.0 output for ecfet_v2.va (fails) and ecfet_v3.va (compiles).

1. **ecfet_v2.va: FAILED (rc=65).**
2. **The last error shown:** `limexp` is "not allowed here", because OpenVAF does not allow analog operators inside a condition.
3. **"could not compile `ecfet_v2.va` due to 4 previous errors".** The log keeps only the last 8 lines. Run `openvaf verilog\ecfet_v2.va` in a terminal to see all four errors. They are two `@(cross(...))` event statements (lines 166 and 180), which OpenVAF does not accept, and two `limexp` calls inside `if (sgn > 0)` (lines 209 and 211).
4. **ecfet_v3.va: OK -> ecfet_v3.osdi.** The compiled library is written next to the source in `verilog\`, together with `.exp` and `.lib` files.

> **Warning.** With OpenVAF 23.2.0, the public `ecfet_v2.va` does **not** compile, although the README says the `.va` compiles with OpenVAF. `ecfet_v3.va` and `ideal_signed_synapse.va` compiled without errors. OpenVAF does not support every Verilog-A construct. A failure here therefore means the file will not run in OpenVAF and ngspice. It does not prove that Spectre will reject the file.

If OpenVAF is not installed, the log shows:

```
[openvaf] OpenVAF not found. Install from https://openvaf.semimod.de/ and put it on PATH, or set
NVAT_OPENVAF=/path/to/openvaf.
```

To install it, download the OpenVAF binary for your system from that page. Then do one of the following:

- Put it on your PATH (the app looks for `openvaf`, `openvaf-reloaded` and `openvaf-r`).
- Set the environment variable `NVAT_OPENVAF` to its full path.

- On Windows, place it at `Programs\OpenVAF\openvaf.exe` inside your per-user local application-data folder, which the app also checks.

Restart the app afterwards so that it sees the new PATH.

10c. Fit the LTP/LTD nonlinearity from a CSV

The fit tool takes a CSV of **conductance per pulse**. It accepts either one column (G) or two columns (pulse, G). It always uses the last column, skips header rows that are not numeric, and splits the curve at its peak into an LTP branch and an LTD branch. Each branch is fitted with the standard saturating curve `G(p) = G0 + span · (1 - exp(-B·x)) / (1 - exp(-B))`, where x runs from 0 to 1 across the branch. B close to 0 means linear, and a larger B means more saturating.

The transient CSV from Step 9 is a time series, not one value per pulse, so convert it first. This short script takes the retained weight `G_nv (S)` 1 ms before each next pulse (the pulse train starts at 10 ms with a 2 s period, 60 pulses) and writes `pulse,G_uS`:

```
import csv
rows = list(csv.reader(open("results/gui_v2_practical_ECFET.csv")))
hdr, data = rows[0], [[float(v) for v in r] for r in rows[1:]]
ti, gi = hdr.index("t_s"), hdr.index("G_nv (S)")
with open("results/ltp_ltd_per_pulse.csv", "w", newline="") as f:
    w = csv.writer(f); w.writerow(["pulse", "G_uS"])
    for k in range(60):
        t_read = 0.010 + (k + 1) * 2.0 - 1e-3          # just before the next pulse
        best = min(data, key=lambda r: abs(r[ti] - t_read))
        w.writerow([k + 1, round(best[gi] * 1e6, 4)])
```

Then choose **Tools > Fit LTP/LTD from CSV...** and pick the file. The result appears in the Log tab.

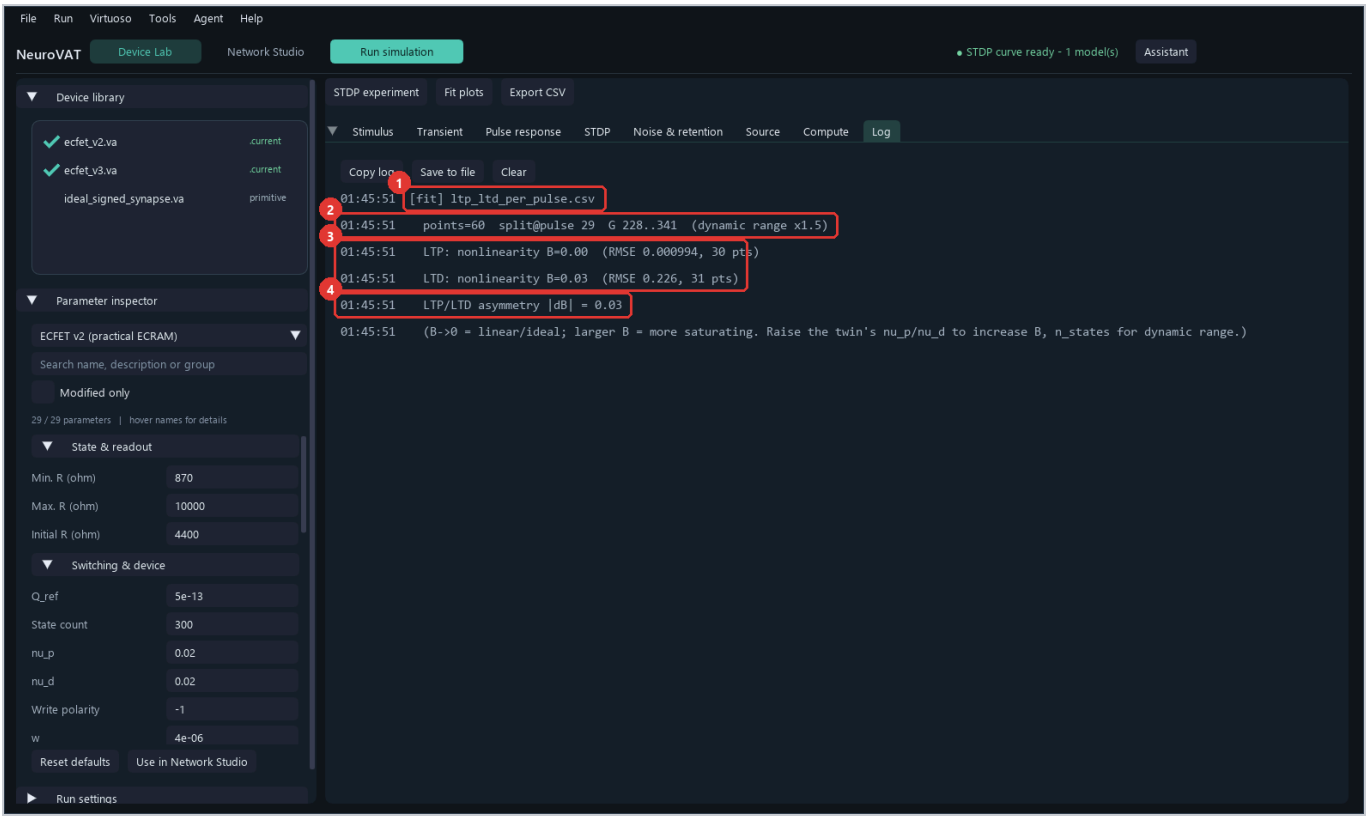


Figure: The fit result for the 60-pulse curve from this chapter's run.

1. **File:** `ltp_ltd_per_pulse.csv`.
2. **Range:** 60 points, split at pulse 29 (the peak), G 228 to 341 µS, dynamic range x1.5. These match the Pulse response tab.
3. **Branches:** LTP B = 0.00 (RMSE 0.000994 µS, 30 points) and LTD B = 0.03 (RMSE 0.226 µS, 31 points). Both branches are essentially linear.
4. **Asymmetry:** |ΔB| = 0.03 between LTP and LTD. The hint below it reminds you that raising `nu_p`/`nu_d` increases B, and that `n_states` sets the dynamic range.

Warning. The Fit dialog can close the app. In our source install (DearPyGui 2.3.1), clicking **Tools > Fit LTP/LTD from CSV...** crashed the app with `KeyError: 'rect_size'` as soon as the file dialog appeared. The same happened with File > Change workspace... and with Source > Save as.... The figure above was made by calling the dialog's own fit function directly with the same file. Until this is fixed, run the fit through the control bridge, which needs no dialog. With the app running (the bridge is on by default), type this in a terminal in the source folder. Give the CSV's full path if the app was started on a different workspace.

```
python tools/nvat_ctl.py --workspace <your workspace> fit results/ltp_ltd_per_pulse.csv
```

It prints the same numbers as JSON and also writes them to the Log tab. The Windows release bundles the same DearPyGui version (2.3.1). We could not drive its menus in our test, so we have not confirmed the crash there.

Use the same tool on your **measured** data: export conductance per pulse from your instrument as one or two columns, fit it, and compare its B with the model's. Raise or lower `nu_p` and `nu_d` until the fitted B values agree.

CHAPTER 4

Network Studio: building, training and testing a spiking network

Network Studio is where NeuroVAT turns a device model into a network. Every synapse in the network is one copy of your device's Python twin. The crossbar conductance is the weight, and leaky integrate-and-fire (LIF) neurons read the crossbar out. In this chapter you build a network, give it data, train it, test it, and look inside it. Then you save it and load it back.

The chapter follows one real run, step by step. It trains a 196-input, 10-output network on a small MNIST subset with surrogate gradients. Every number in this chapter was measured in this session with the public v1.3.1 release and the public `ecfet_v2.va` synapse. The settings are given next to each number.

> **Note.** Network Studio has three learning modes. This chapter covers the two **epoch** modes, *supervised* and *unsupervised*. The third mode, *unsupervised stream* (one continuous spike stream with STDP at every spike), changes the whole window. Chapter 5 covers it.

> **Warning.** Training runs the Python behavioral twin of the device, not a Verilog-A circuit solver. A source mapped to a twin does not prove that the two are equivalent. Validate your own twins against their sources (Chapter 7).

4.1 Step 1. Open Network Studio and pick the synapse

Click **Network Studio** in the header. The left panel now holds the network controls, and the centre holds the Network Studio tabs. Every network needs a Verilog-A source that is mapped to a conductance-based Python twin.

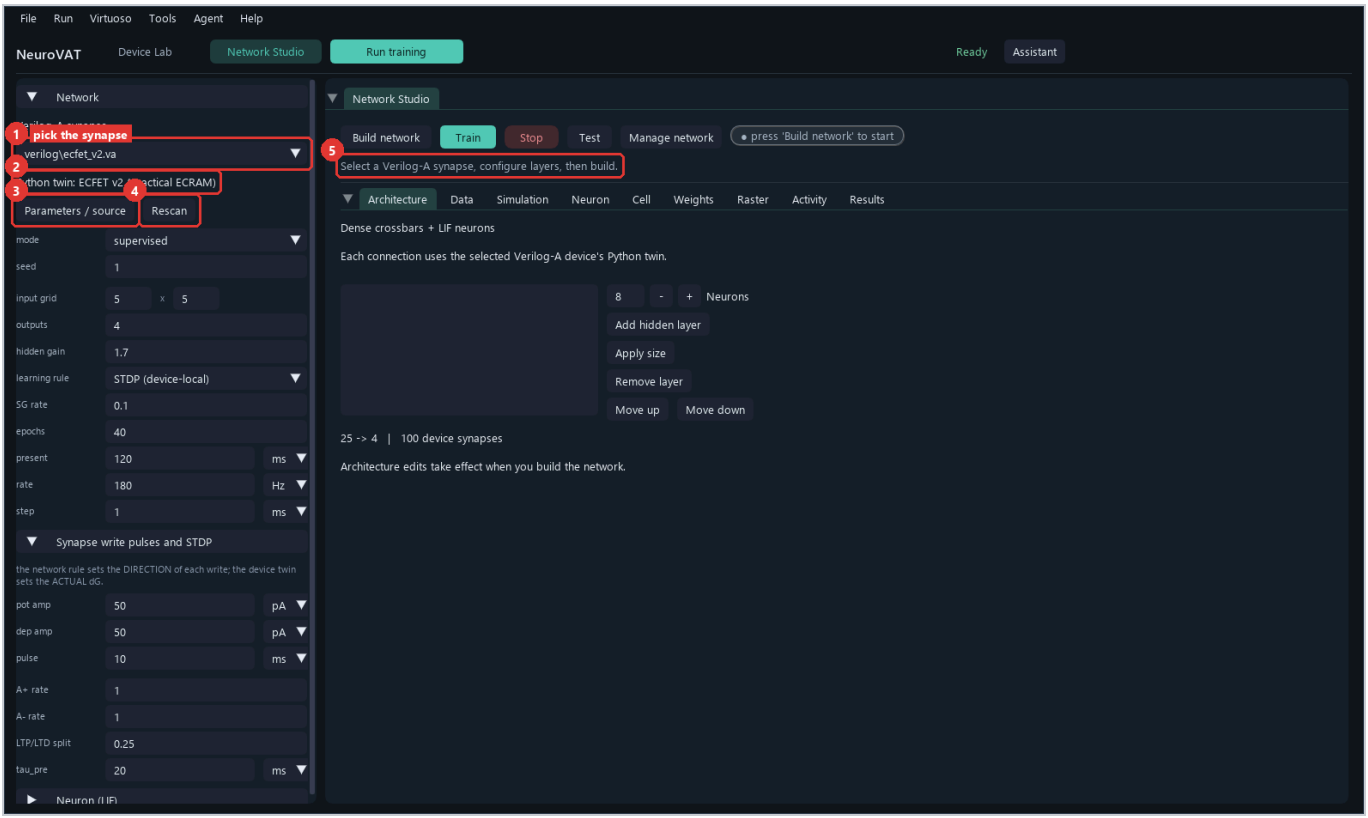


Figure: Network Studio on first open, with the Verilog-A synapse picker at the top of the left panel.

1. **Verilog-A synapse**: pick `verilog\ecfet_v2.va`. This sets the device at every crossbar cross-point. The list shows only sources that are mapped to a conductance twin. The public release offers `ecfet_v2.va` and `ecfet_v3.va`. A capacitor-type source (for example a FeCAP) and an unmapped source are never listed, because they cannot be a conductance weight.
2. **Python twin line**: this line shows the twin that will run, here *ECFET v2 (practical ECRAM)*. Check it, because the twin is what the training actually runs. The file name alone tells you nothing about that.
3. **Parameters / source**: this opens Device Lab's parameter inspector and the source editor for this model. Device parameters are captured when you **Build** the network. Picking a source does not overwrite your parameter edits.
4. **Rescan**: use this after you add or edit a `.va` file in `verilog/`. It rescans the folder so the new file appears in the list.
5. **Status line**: before a build, this line tells you what to do next. After a build, it names the exact `.va` file and twin, the layer sizes, the number of device synapses, the encoding and the learning rule.

The source fingerprint (SHA-256)

When you press **Build network**, NeuroVAT reads the `.va` file and stores a *source binding* with the network: the file path, the SHA-256 hash of the file's bytes, the twin key, and the note that it runs as a Python twin. The hash is not shown on screen. It is written into every saved model. Here is the block from the model saved in Step 9 (path shortened):

```
"synapse_source": {
  "path": "<your folder>\\verilog\\ecfet_v2.va",
  "sha256": "33b9ea262f7de734ddfce6a615192856809d1083742af595316a81bd35685edb",
  "twin_key": "v2",
  "execution": "Python behavioral twin"
}
```


The fingerprint is what lets the app notice a changed source. If you edit the `.va` file after a build, Train and Test refuse to run until you build again. A saved network also refuses to load until the same source is back. Step 9 shows both messages.

4.2 Step 2. Choose the learning mode

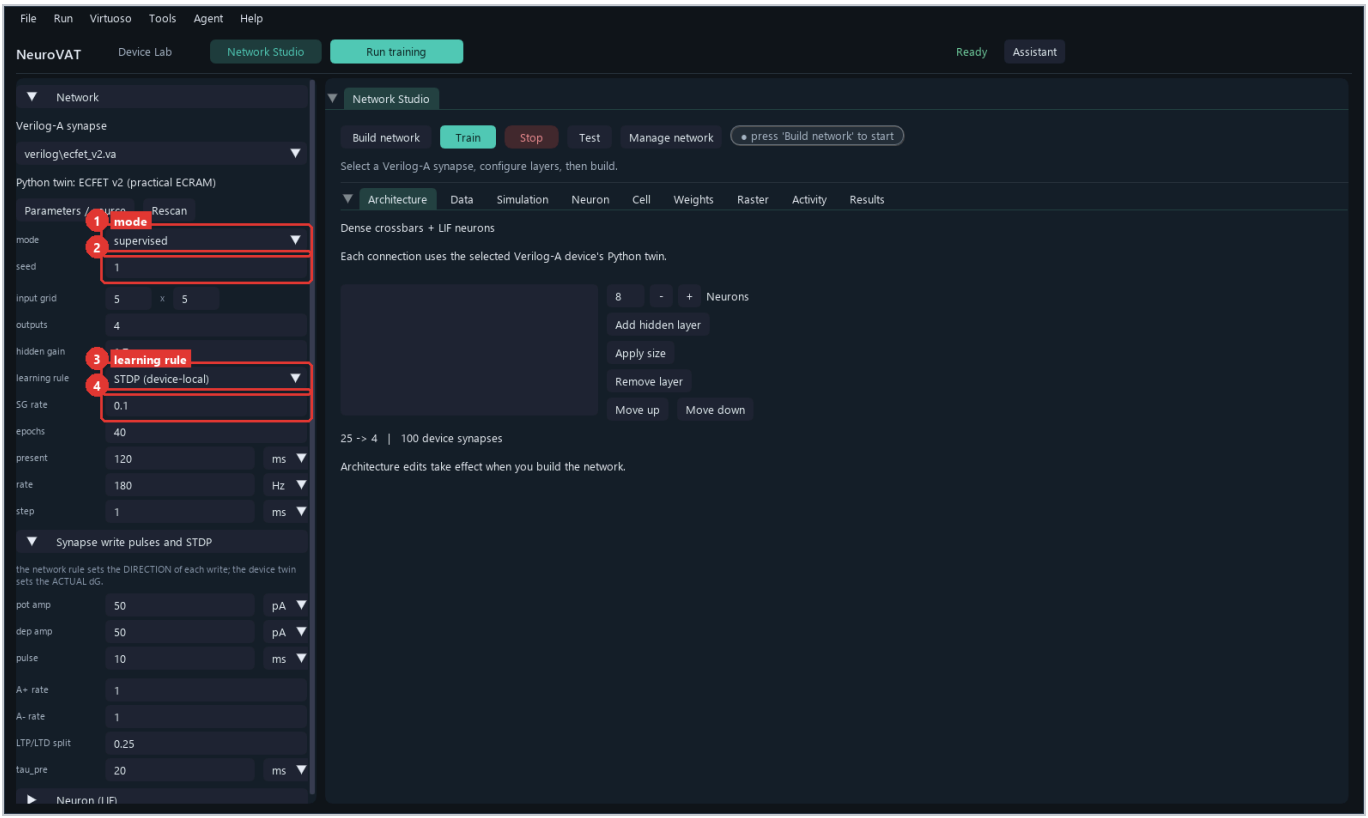


Figure: The mode, seed and learning-rule controls in the Network section.

1. **mode**: this combo sets the whole Trainer window.
   - **supervised**: during training, a teacher current drives each pattern's target neuron so that it fires. The weights then move towards "this input makes this neuron fire". Use it when you have labels and want a classifier.
   - **unsupervised**: the same pattern-by-pattern presentations, with no teacher. Output neurons compete through winner-take-all inhibition. The adaptive threshold (theta+) stops one neuron from taking every pattern. Use it to see which neurons specialise on their own.
   - **unsupervised stream**: covered in Chapter 5.
2. **seed**: this fixes the spike noise, the weight initialisation and the sampling, so a run repeats. Every number in this chapter uses seed 1. Keep the seed fixed when you compare two settings.
3. **learning rule**:
   - **STDP (device-local)**: an eligibility-trace rule. Each synapse keeps a pre-synaptic trace. After each presentation, a post spike potentiates the inputs whose trace is above the *LTP/LTD split* and depresses the others. The writes are consolidated into one programming pulse per synapse per presentation. The network rule sets the *direction* of each write, and the device twin sets the *actual* conductance change. This rule is local and works in both epoch modes. On deep networks it is weak.
   - **Surrogate grad (BPTT)**: supervised backpropagation through time with a surrogate spike derivative. The gradient is applied *through* the device as programming pulses, so the device's nonlinearity and bounds shape the learning. It needs labels: choosing it switches the mode to *supervised*, and you cannot combine it with unsupervised.

4. **SG rate**: the surrogate-gradient step size (default 0.1; the tooltip suggests 0.05-0.2). STDP ignores it. During a surrogate run, the step is annealed from 1x down to 0.15x over the epochs (cosine schedule), so the weights settle into the solution and do not overshoot it.

> **Note.** The network STDP rule is not the same as Device Lab's STDP experiment. Device Lab applies overlapping pre and post voltage waveforms to the device and measures the result. The network rule is an assumed timing rule whose writes go through the twin. The device's own added timing rule (`A_stdp`, `A_stdp_V`) is switched off inside the crossbar.

4.3 Step 3. Set the architecture, neuron and learning controls

The **Architecture** tab edits the hidden layers. The input size, the output count and all the neuron and learning settings are in the left panel. Edits take effect only when you press **Build network**.

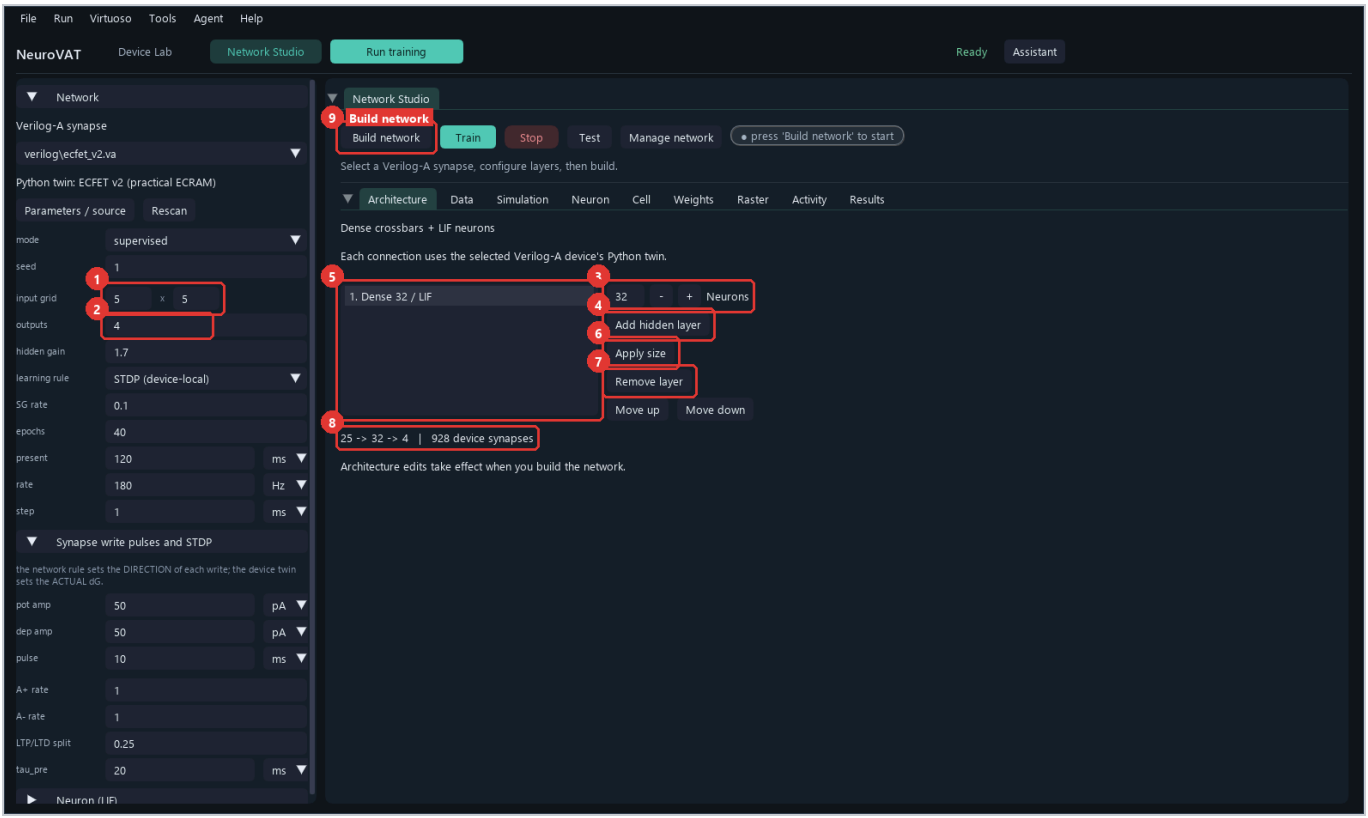


Figure: Architecture tab after adding one hidden Dense/LIF layer of 32 neurons (25 -> 32 -> 4, 928 device synapses).

1. **input grid** (H x W): each pixel is one input neuron, so 5 x 5 gives 25 inputs. You can type 2 to 14 per side. Loading a dataset overwrites these with the dataset *resolution* (Step 4).
2. **outputs**: the number of output neurons, which is the number of crossbar columns. Use one per class. Built-in pattern sets map pattern *k* to neuron *k mod outputs*, so set this to the number of patterns. The `bars` set has 8 patterns, so it needs 8 outputs, not the default 4. A dataset sets it to its class count.
3. **Neurons**: the size of the layer you are about to add, or the new size for **Apply size**. You can enter 1 to 64.
4. **Add hidden layer**: this appends a Dense/LIF layer of that size. You can have up to four hidden layers.
5. **Layer list**: click a layer to select it for **Apply size**, **Remove layer**, **Move up** or **Move down**.
6. **Apply size**: resizes the selected layer.
7. **Remove layer**: deletes the selected layer.
8. **Shape line**: shows the layer sizes and the number of device synapses. Every synapse is one twin instance, so this number tells you how slow training will be.
9. **Build network**: applies all the settings to the trainer and captures the device parameters and the source fingerprint.

Warning. Hidden layers: measured result. In this session, a 196 -> 32 -> 10 network with surrogate gradients stayed at chance. MNIST subset, 15 epochs, seed 1: the test result was 10.0% (150 held-out) before and after training, with every test image predicted as one class. Raising **hidden gain** from 1.7 to 3.0 gave the same per-epoch accuracy curve. The same data with *no* hidden layer reached 78.0% (Step 6). Start with no hidden layer. Add one only after you have a working single-crossbar baseline, and use the untrained-versus-trained check in Step 6 to see whether the deeper network learns.

For the run in this chapter, the hidden layer was removed again (**Remove layer**), which gives a single 196 x 10 crossbar.

Neuron and write-pulse settings

Scroll the left panel down and open **Neuron (LIF)**. The defaults were used for every run in this chapter.

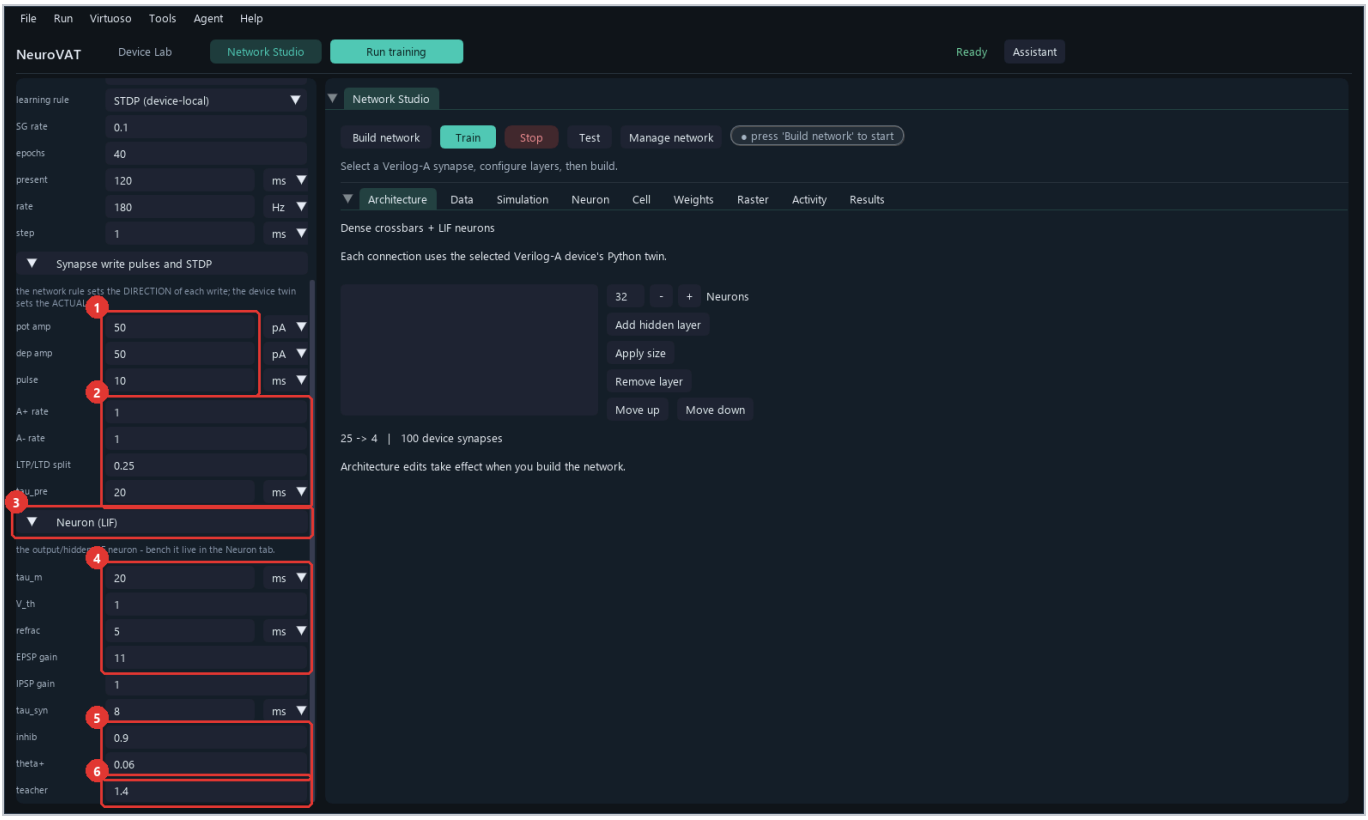


Figure: The synapse write-pulse, STDP and LIF neuron settings in the lower part of the left panel.

1. **pot amp / dep amp / pulse**: the amplitude and width of a full potentiating or depressing programming pulse, in the device's drive unit (pA for the current-driven ECFET, V for a voltage-driven device). The defaults switch when the drive kind changes. These values set how large one write is on the real device model.
2. **A+ rate / A- rate / LTP/LTD split / tau_pre**: the STDP rule. A+ and A- scale potentiation and depression. The split decides which inputs potentiate on a post spike: those whose pre-trace is above it. tau_pre is the decay of that trace. The surrogate rule does not use these four.
3. **Neuron (LIF)**: click to open the section. It is collapsed by default.
4. **tau_m, V_th, refrac, EPSP gain**: the leak time constant, the threshold (in normalised units), the dead time after a spike, and how strongly an input spike, scaled by its conductance, charges the membrane. You can test these values live in the **Neuron** tab (Step 7).
5. **inhib / theta+**: lateral winner-take-all inhibition between output neurons (0 turns it off), and the homeostatic threshold rise per output spike. In the unsupervised mode these two make neurons specialise.
6. **teacher**: the supervised teacher drive onto the target neuron. It is used only in supervised STDP.

Also in the left panel: **epochs** (passes over the training set), **present** (how long each pattern is shown, default 120 ms), **rate** (peak Poisson rate of a fully-on pixel, default 180 Hz) and **step** (integration time step, default 1 ms). **Manage network > Restore default settings** refills the neuron and learning fields with defaults for the

selected device's drive kind.

4.4 Step 4. Choose the data and the encoding (Data tab)

Built-in patterns

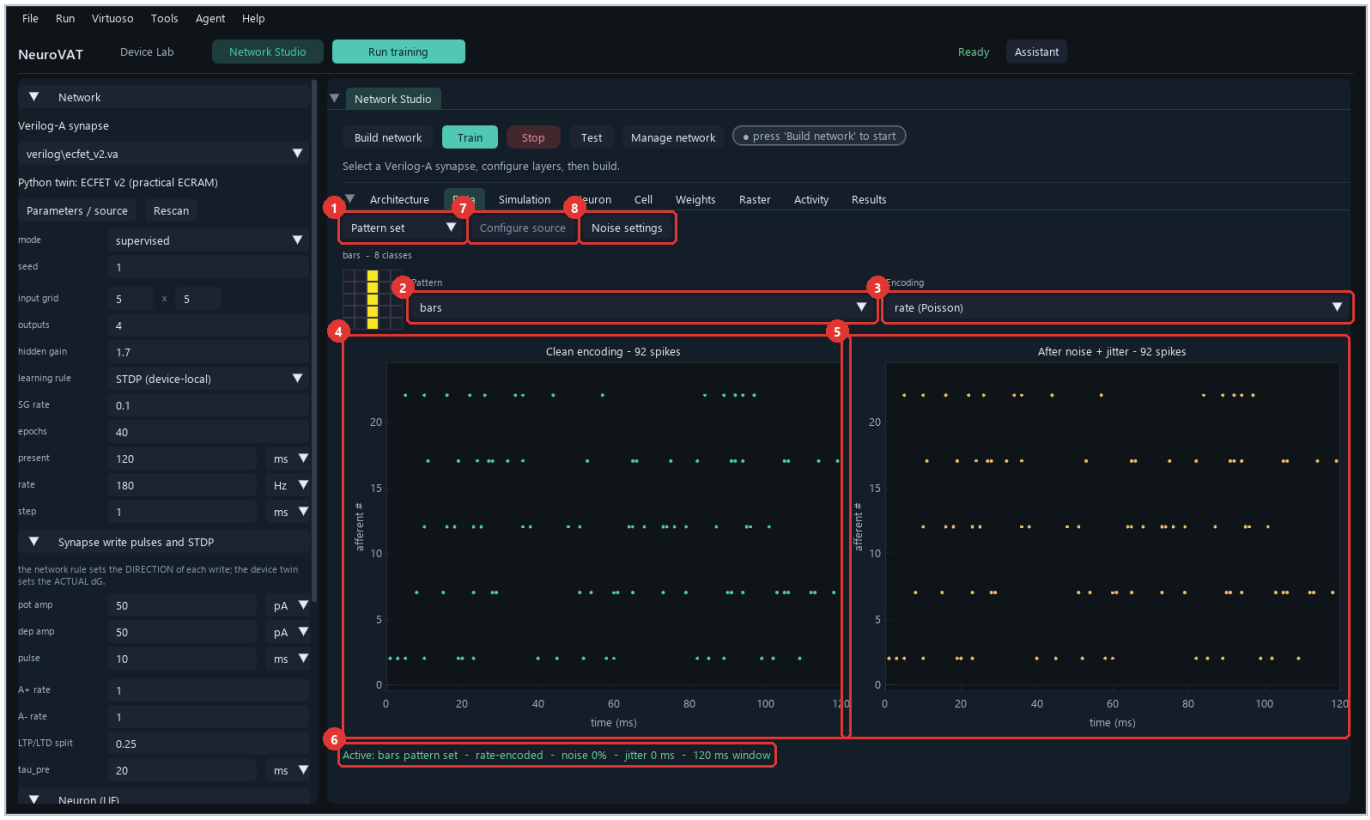


Figure: Data tab with the built-in `bars` set (8 classes) and rate coding. Left preview: clean encoding. Right preview: after noise and jitter.

1. **Source selector**: *Pattern set* (built-in), *Custom painted*, or *Imported* (a dataset).
2. **Pattern**: the built-in sets are `bars`, `letters`, `digits` and `random`, plus `nand` and `xor` logic sets and any plugin in `patterns/`. The thumbnail on the left shows one pattern. These sets are small, so a training run takes seconds.
3. **Encoding**: how pixels become spikes. *rate (Poisson)*: a brighter pixel fires more spikes. *latency (TTFS)*: a brighter pixel fires earlier. *temporal (phase)*: one timed spike per cycle. *column scan*: see Figure 7. All the measured runs here use rate coding.
4. **Clean encoding**: one pattern encoded with no noise. The title gives the spike count.
5. **After noise + jitter**: the same pattern with the current noise settings. Compare the two previews to see what the network will actually receive.
6. **Status line**: the active set, encoding, noise %, jitter and presentation window.
7. **Configure source**: opens the custom painter or the dataset importer. It is greyed out for built-in sets.
8. **Noise settings**: opens the perturbation controls (next figure).

Noise settings

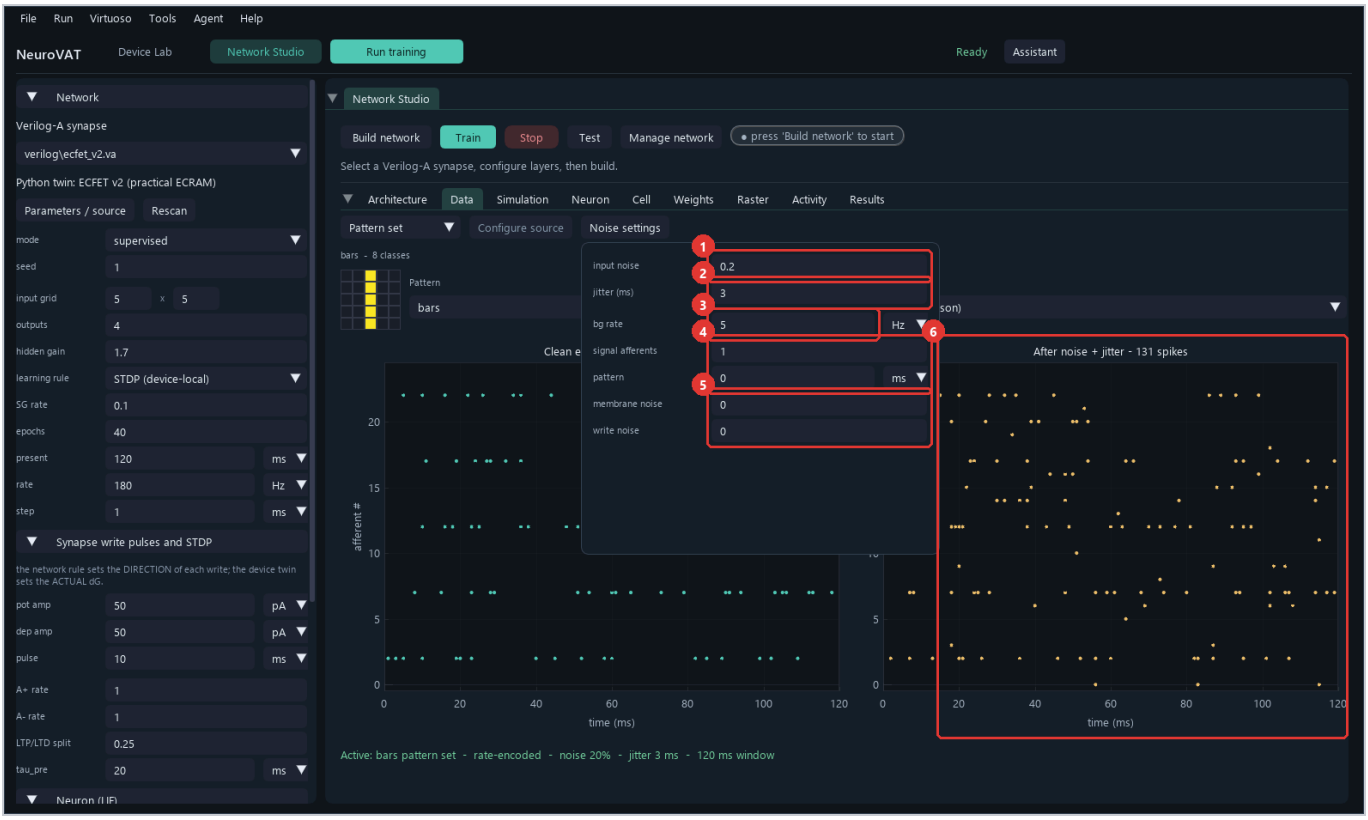


Figure: Noise settings set to 20% input noise, 3 ms jitter and 5 Hz background. The noisy preview rises from 92 to 131 spikes.

1. **input noise** (0-1): random intensity noise on each pixel.
2. **jitter (ms)**: random shift of each spike time.
3. **bg rate**: spontaneous background firing on every input, including the dark pixels.
4. **signal afferents / pattern**: the fraction of inputs that carry the real pattern (the others fire only background), and how long the pattern shows inside the presentation window (0 = the whole window).
5. **membrane noise / write noise**: Gaussian noise on every neuron's membrane at each step, and the device's cycle-to-cycle write noise. Write noise needs a model that supports it; ECFET v2 does.
6. **Noisy preview**: updates as soon as you change a value.

These settings apply during training too. The runs in this chapter set them all back to 0. That way the reported accuracy measures learning, not noise tolerance.

Column scan

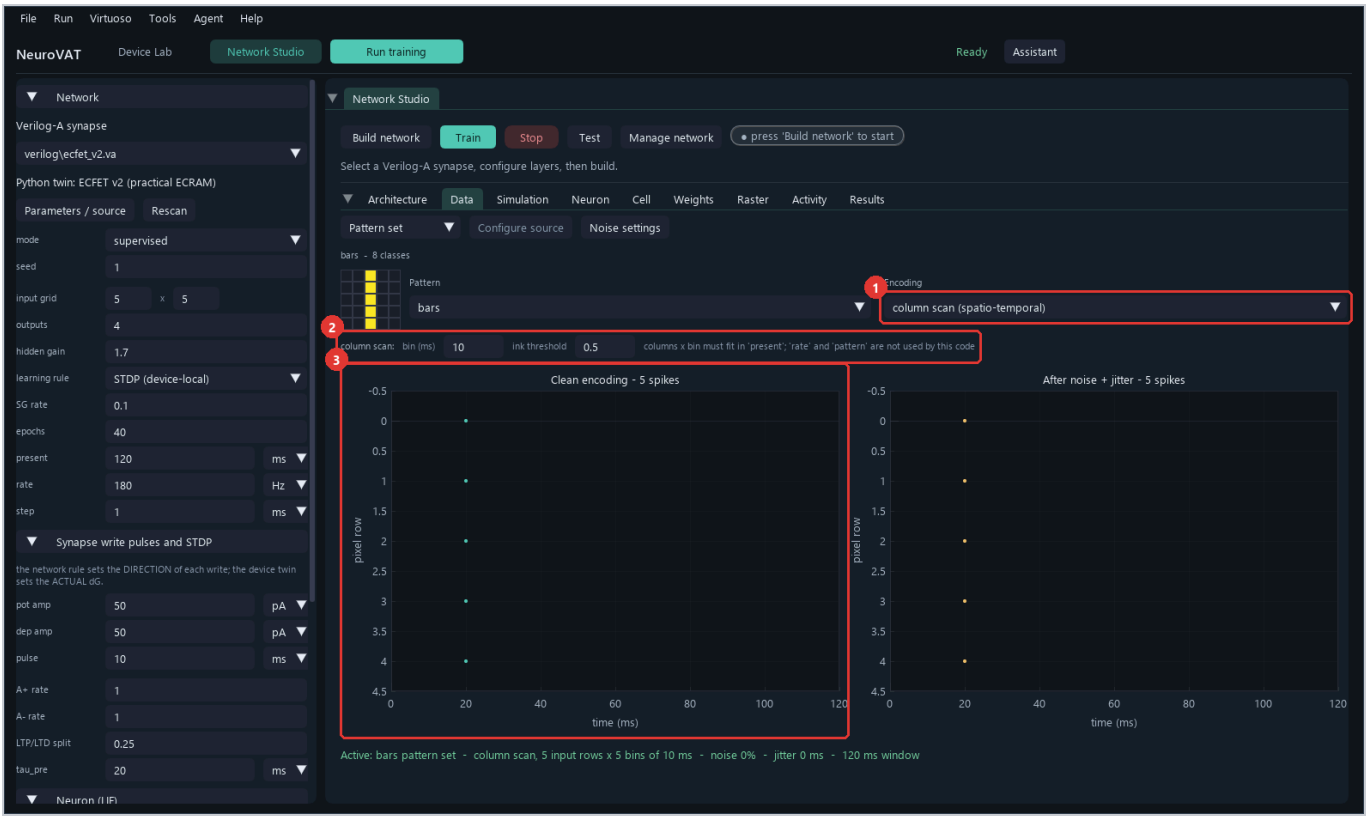


Figure: Column-scan encoding of a vertical bar: 5 input rows, one 10 ms bin per pixel column.

1. **Encoding = column scan**: the image is swept left to right. Each pixel *row* is one input line, and it spikes in a time bin when its pixel in that column is inked. The number of inputs becomes the number of rows, so switching to or from column scan needs **Build network**.
2. **bin (ms) / ink threshold**: the time per column, and the pixel value that counts as ink. The number of columns times the bin must fit in *present*.
3. **Preview**: the y axis is now the pixel row.

> **Note.** The core README (`src/neurovat/README.md`) states that the crossbar trainer does not yet learn scanned patterns well. Over 15 epochs and 3 seeds, letters and digits reached 25-42% with column scan against 92-100% with rate coding. Use rate coding to start.

Custom painted patterns

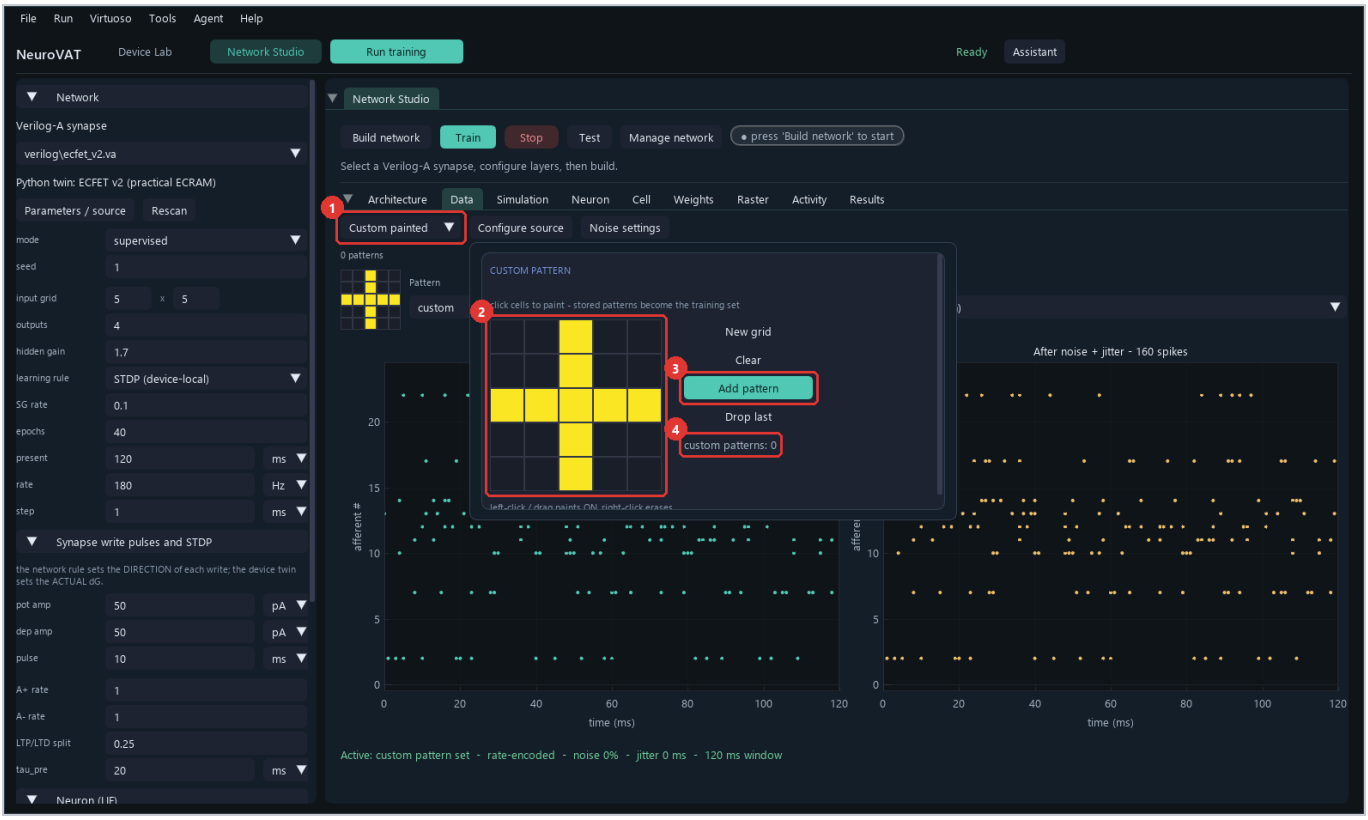


Figure: The custom-pattern painter under Configure source.

1. **Custom painted**: choose it in the selector, then open **Configure source**.
2. **Grid**: left-click or drag to paint a cell on, right-click to erase. The grid follows the input grid size (up to 14 x 14). **New grid** starts a blank grid; **Clear** erases the current one.
3. **Add pattern**: stores the painting as the next pattern and clears the grid. Patterns are assigned round-robin to the output neurons, so paint one per output in order. **Drop last** removes the last pattern.
4. **custom patterns: N**: how many patterns are stored. Build fails with a clear message if you choose Custom painted and this is 0.

Loading MNIST

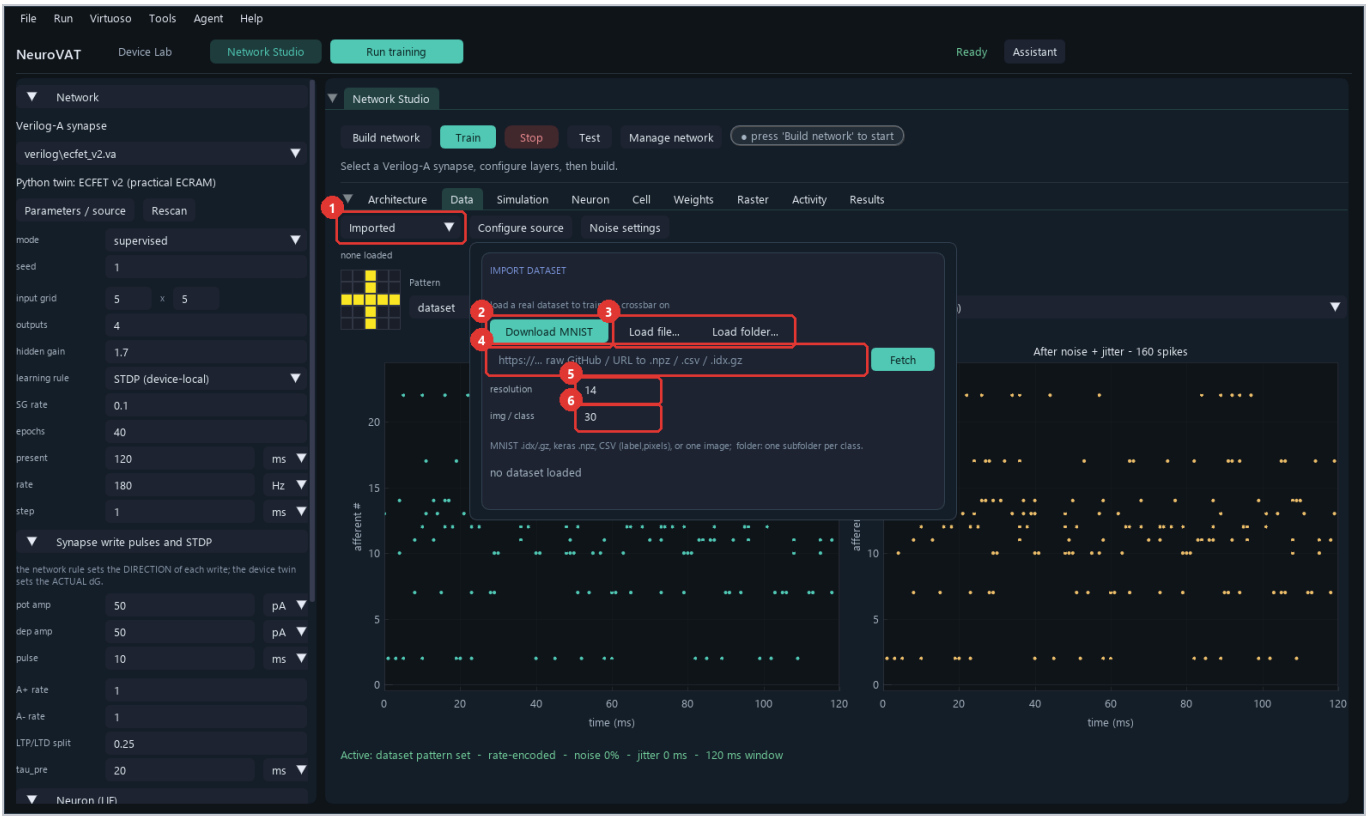


Figure: The Import dataset panel (Imported, then Configure source), before loading.

1. **Imported**: choose it in the selector, then open **Configure source**.
2. **Download MNIST**: despite its name, this button first looks for `datasets\mnist.npz` in your workspace. If the file is there, it loads it without downloading. The release folder used here already holds that file. Without it, the button downloads the keras MNIST file (about 11 MB) into `datasets\`. That first download needs an internet connection; on a fresh copy of the Windows release it took about 2 s.
3. **Load file... / Load folder...**: your own data. A file can be MNIST `.idx`/`.gz`, a keras `.npz`, a CSV (label, pixels) or one image. A folder needs one subfolder per class.
4. **URL + Fetch**: downloads a dataset from a URL (a GitHub "blob" link is converted to the raw file). `.npz` files are loaded without pickle, so a downloaded file cannot run code.
5. **resolution**: images are resized to this square grid (6-28). It also sets the input grid on Build. 14 gives 196 inputs.
6. **img / class**: training images per class (2-60). The held-out test set takes max(4, half of this) per class. The **INVERT** check box (to the right) swaps ink and background.

Settings for this chapter: **resolution 14**, **img/class 30**. Those numbers give 300 training images. At Build, a stratified 20% (60 images) is set aside for validation, which leaves **240 training** images. A separate **150-image test set** (15 per class) is used only by Test. Training never uses it to select weights.

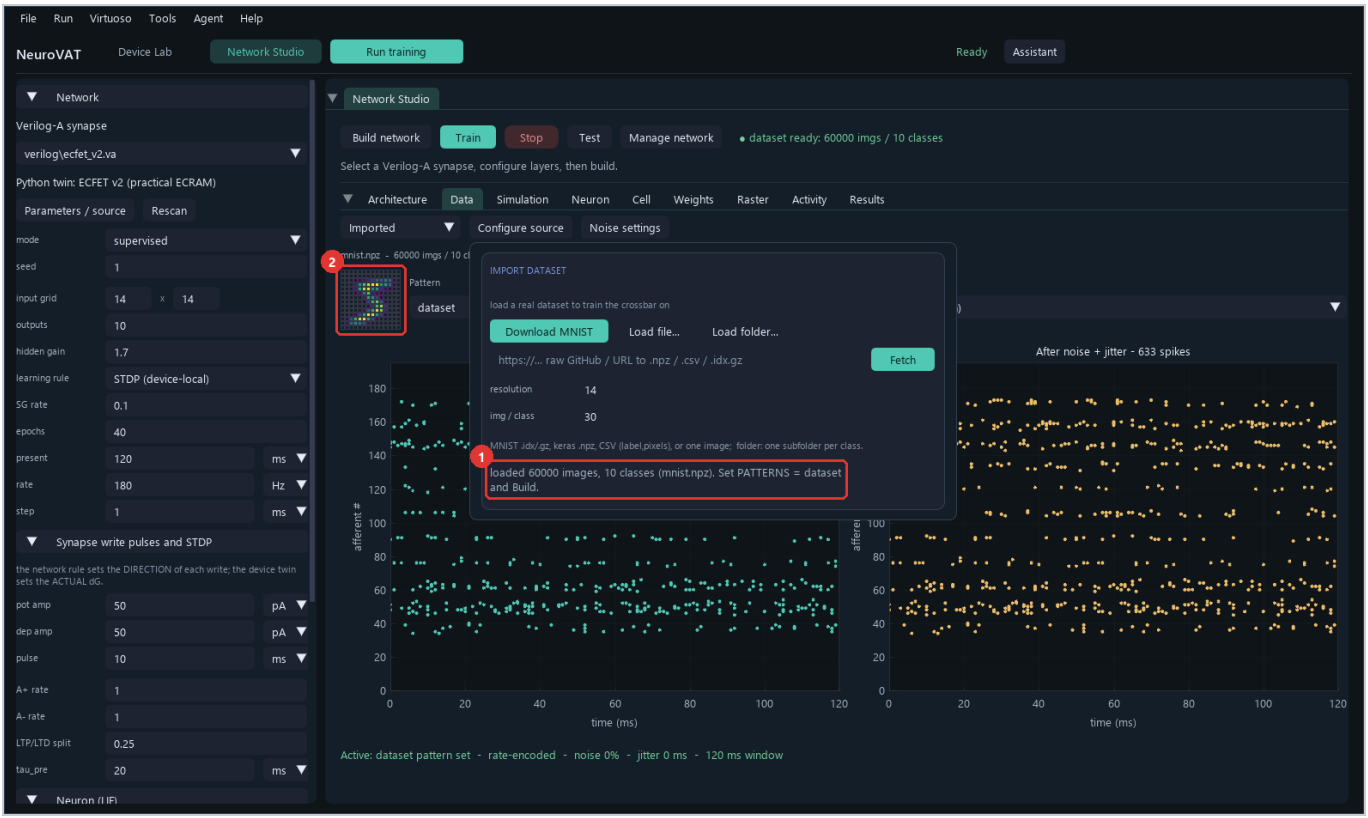


Figure: After the load: 60000 images, 10 classes. Outputs and the input grid were set automatically.

1. **Load message**: the image and class counts. The pattern source switches to *dataset*, and **outputs** becomes 10.
2. **Thumbnail**: the first image at the chosen resolution.

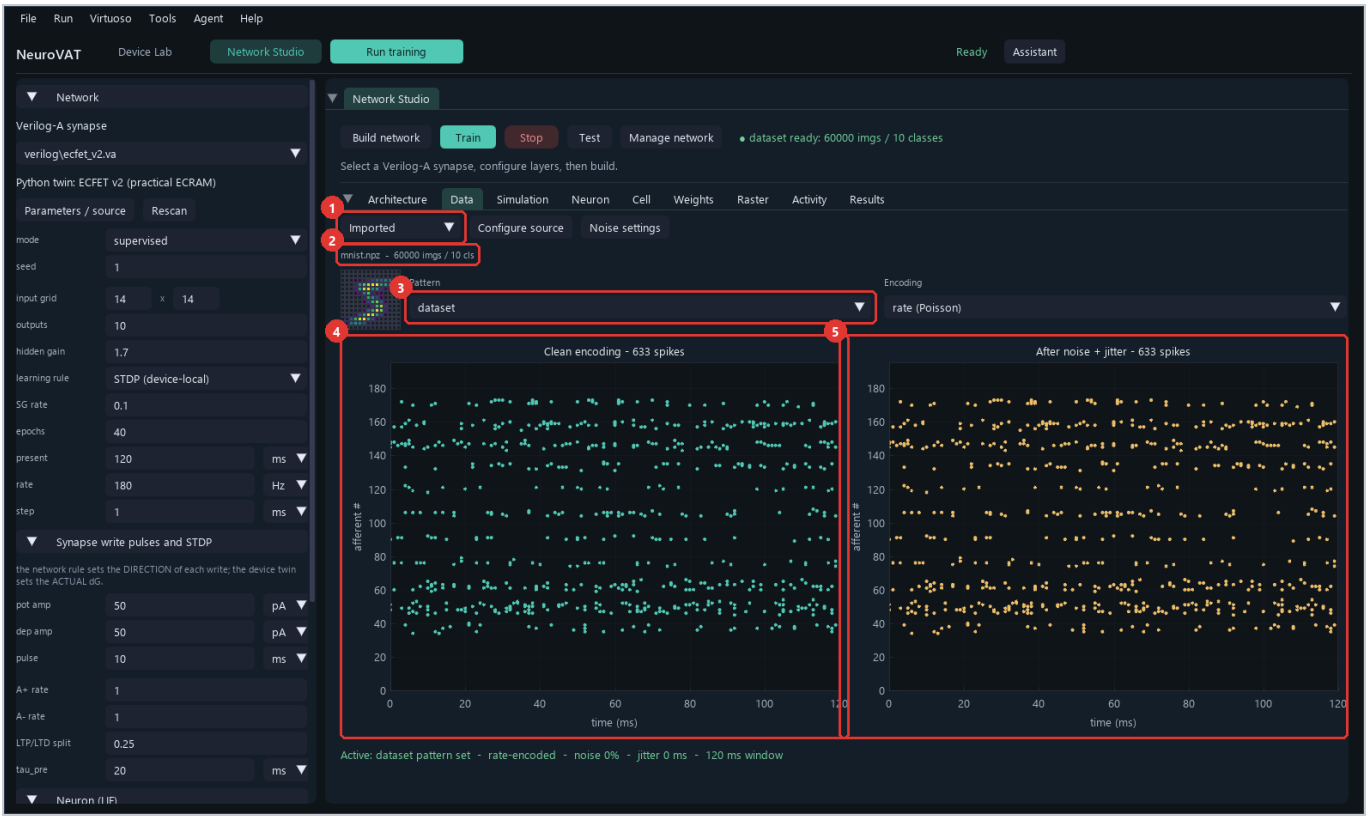


Figure: Data tab with MNIST at 14 x 14, rate coded: 196 input afferents.

1. **Imported** is active.
2. **Source summary**: the file name, image count and class count.
3. **Pattern = dataset**.
4. **Clean preview** of one digit: 196 afferents, rate coded.
5. **Noisy preview**: with all noise at 0, it differs from the clean preview only by a different Poisson draw.

4.5 Step 5. Build and train

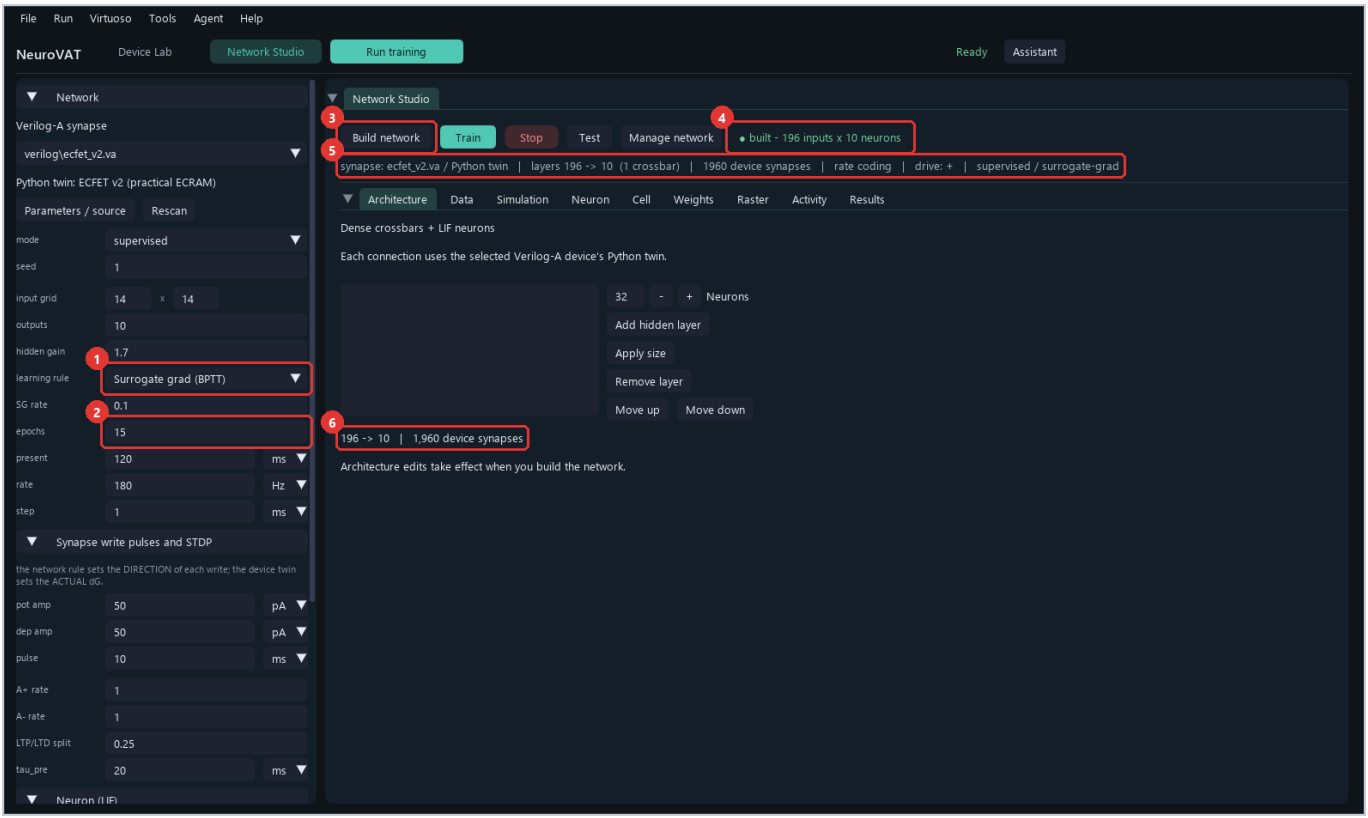


Figure: The network after Build: 196 -> 10, one crossbar, 1960 device synapses, supervised / surrogate-grad.

1. **learning rule = Surrogate grad (BPTT)**: chosen for the main run because the data are labelled. It also gave the best measured accuracy on MNIST (table below).
2. **epochs = 15**: enough to see the curve settle on this subset. It took 87 s of wall time on this laptop.
3. **Build network**: press it again after any change in the left panel, the Data tab or the Architecture tab.
4. **Status pill**: "built - 196 inputs x 10 neurons".
5. **Status line**: confirms the bound source (`ecfet_v2.va` / Python twin), the layer sizes, 1960 device synapses, rate coding, the drive sign and the mode/rule.
6. **Shape line**: shows 196 -> 10.

> **Tip.** Before you train, press **Test** once on the freshly built network. This records the *untrained baseline*, which you need in Step 6 to decide whether training did anything.

Now press **Train** (or F5). Training runs in the background, and the window stays live.

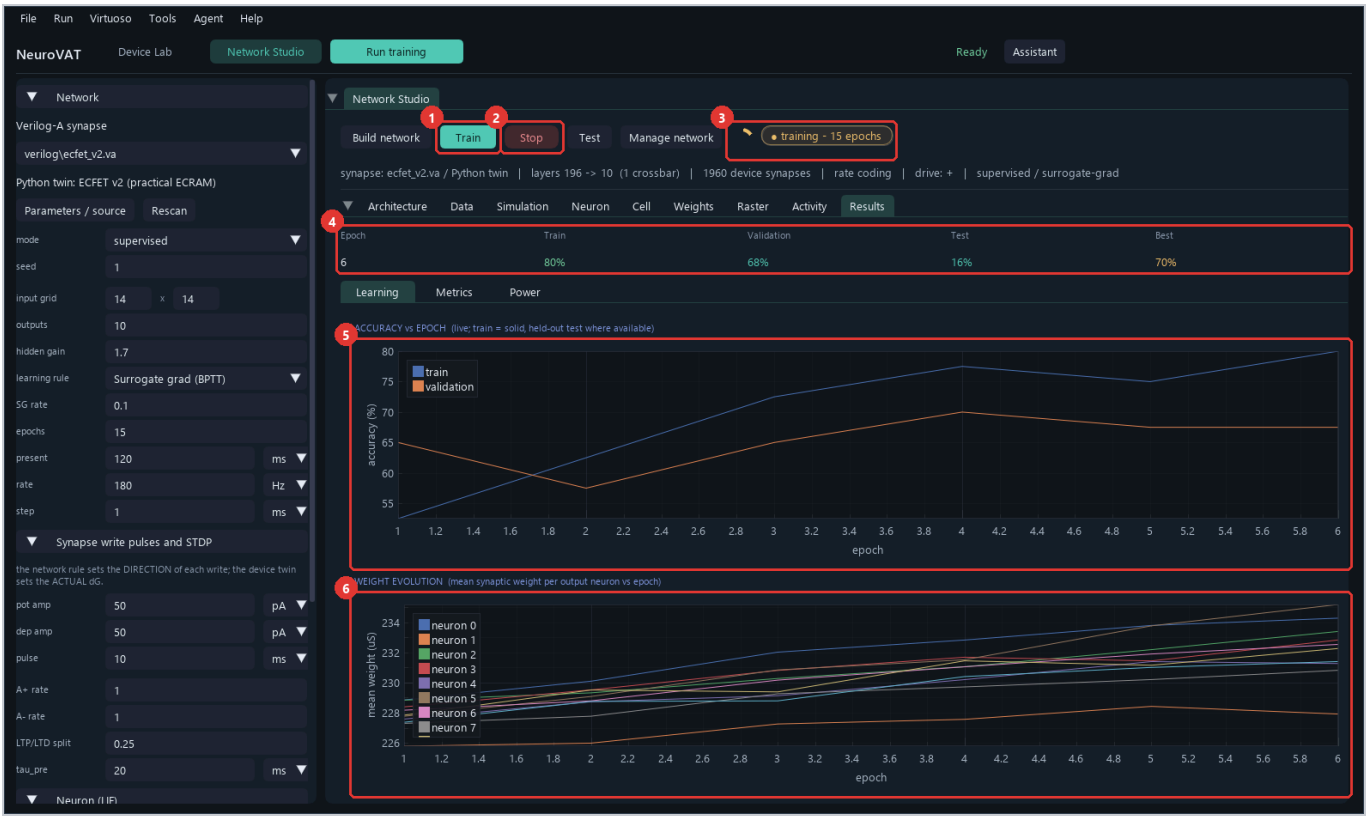


Figure: Mid-training, after epoch 6 of 15 (Results > Learning).

1. **Train**: starts training. If no network exists yet, it builds one first. Pressing Train again on a trained network continues from its current weights.
2. **Stop**: ends the run at the next pattern. The best checkpoint so far is still kept.
3. **Busy spinner and status**: "training - 15 epochs".
4. **Summary tiles**: the epoch, plus training and validation accuracy, sampled on up to 40 patterns per epoch. **Best** is the best validation score so far. The **Test** tile still shows 16%: it changes only when you press Test, and here it holds the untrained baseline.
5. **Accuracy vs epoch**: train and validation, updated live. At epoch 6: train 80%, validation 67.5%.
6. **Weight evolution**: the mean conductance of the synapses onto each output neuron, in uS. A flat line means that neuron's synapses are not being written.

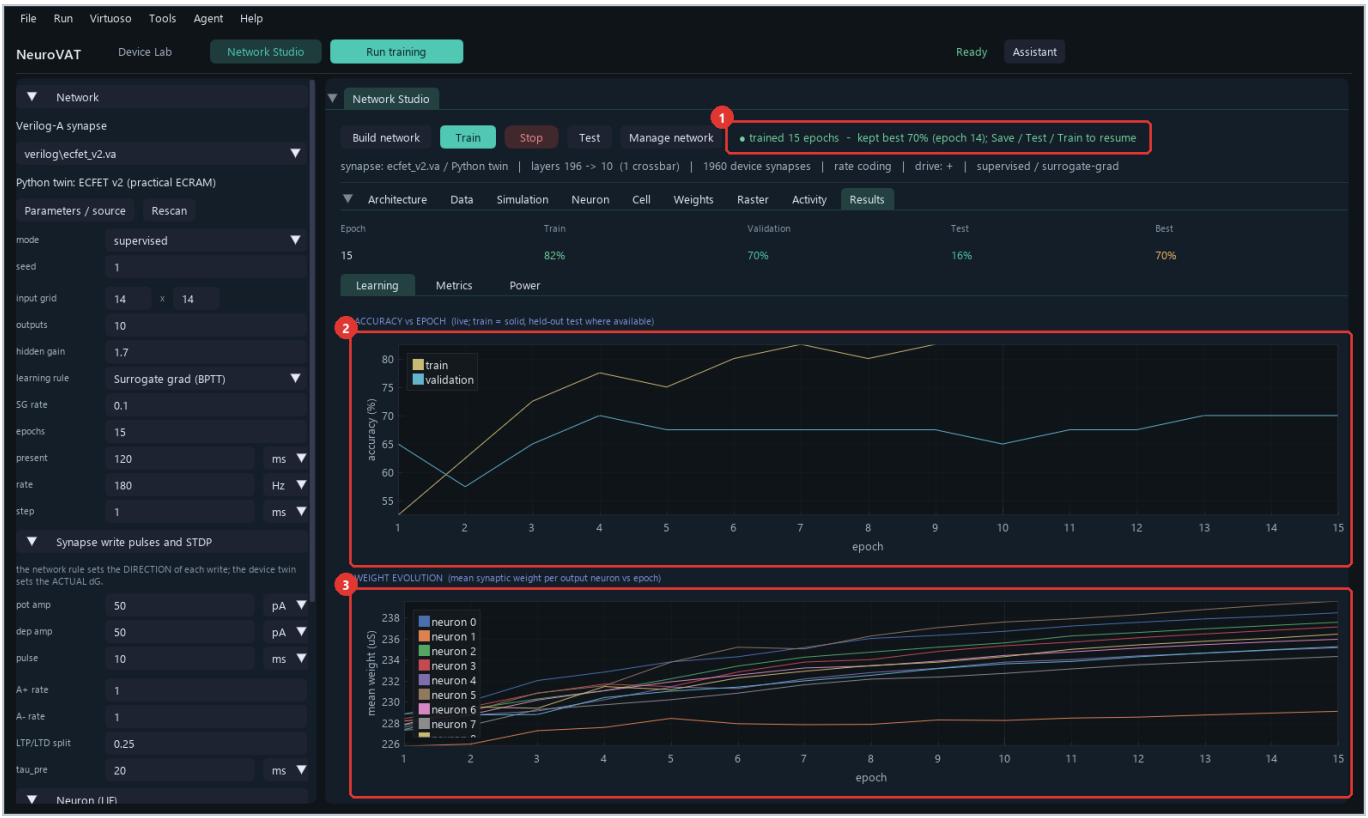


Figure: After training: the run kept the best checkpoint, 70% validation at epoch 14.

1. **Status**: "trained 15 epochs - kept best 70% (epoch 14)". At the end of a run, the trainer restores the weights of the epoch with the best *validation* accuracy (ties go to the lower training loss), not those of the last epoch. Tiny sets with no validation split select on training accuracy, and the log says so.
2. **Accuracy curve**: train rose from 52.5% to about 82.5%. Validation rose from 65% to 70% and flattened after epoch 4. More epochs would not have helped much here. More data or a better encoding would be the next step.
3. **Weight evolution**: all ten neurons' mean weights moved, so the devices were really programmed.

4.6 Step 6. Test, and read the Results tab

Press **Test** (or **Evaluate** in Results > Metrics). Test scores the training set and, for a dataset, the held-out test set. The confusion matrix and the per-class table use the test set when one exists. Test does not change the device state, the thresholds or the random-number state, so you can test in the middle of training and then resume.

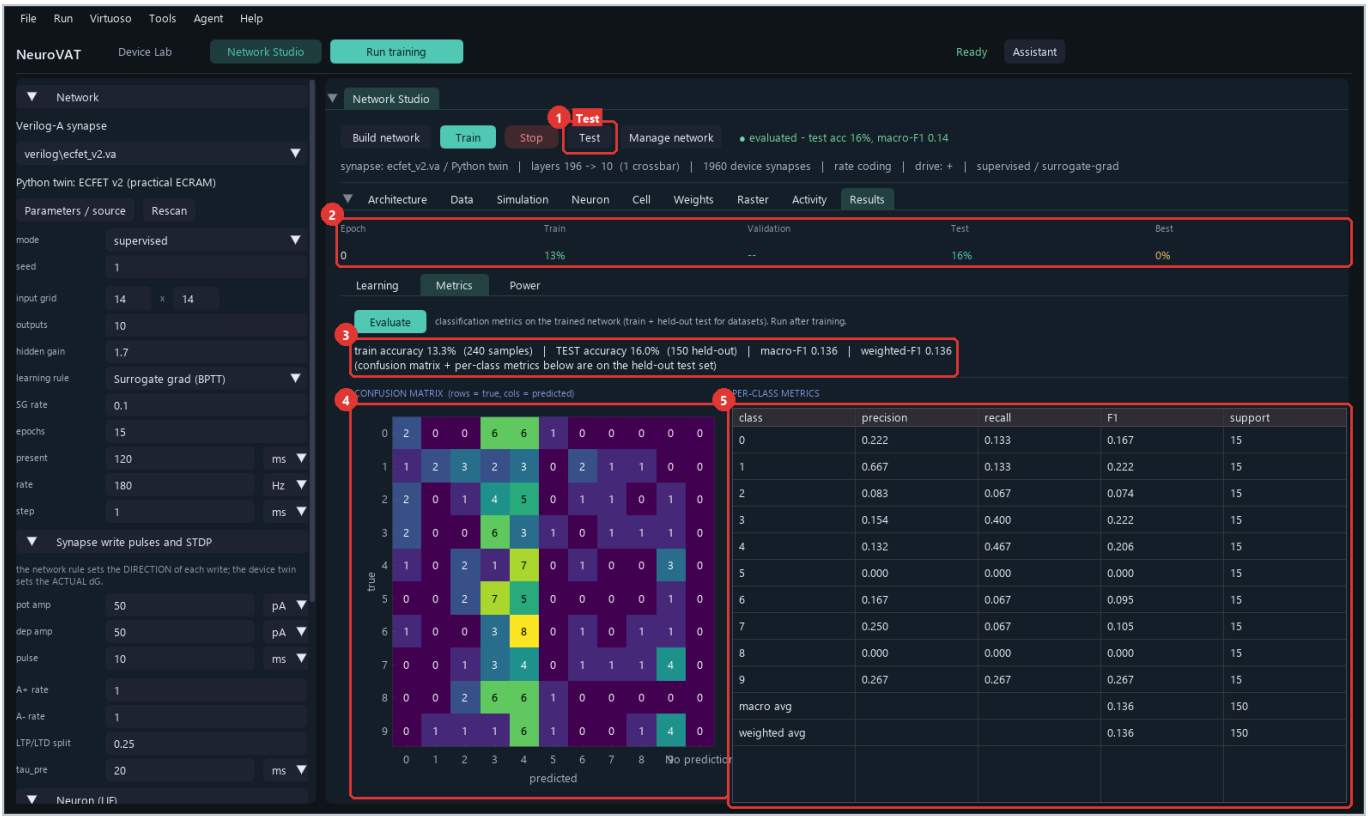


Figure: Untrained baseline (Build, then Test): 13.3% train, 16.0% test, macro-F1 0.136.

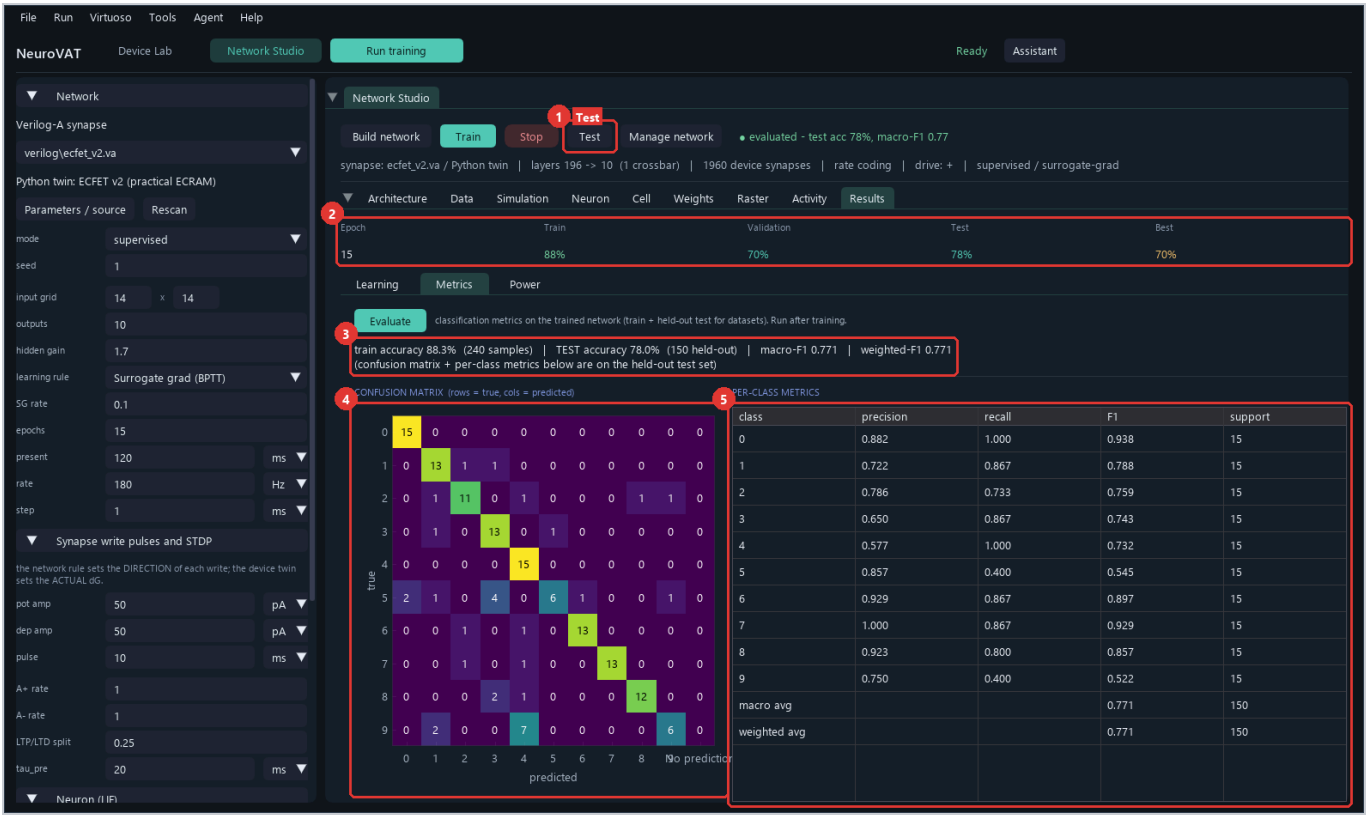


Figure: The same network after 15 epochs of surrogate-gradient training: 88.3% train, 78.0% test, macro-F1 0.771.

Both figures use the same numbered boxes:

1. **Test**: runs the evaluation.
2. **Summary tiles**: Train and Test are full-set accuracies from the last Test. Validation and Best come from the training run. They have different meanings, so do not compare a Validation tile with a Test tile.
3. **Summary line**: accuracy with its denominator (240 training samples, 150 held-out), macro-F1 and weighted-F1. It also says which set the matrix below uses.
4. **Confusion matrix**: rows are the true class, columns the predicted class. A sample with no output spike counts as an error in the extra *No prediction* column, so silent networks cannot look accurate. After training, the diagonal is strong. The weakest classes are 5 and 9: 9 is confused with 4 seven times, and 5 with 3 four times.
5. **Per-class metrics**: precision, recall, F1 and support for each class. Recall and F1 include the no-prediction samples.

How to tell whether it learned

Compare the trained Test with the untrained Test of the *same* build (same seed, same data). With 10 balanced classes, chance is 10%. The untrained network scored 16.0% and the trained one 78.0% on the same 150 held-out images, so the training clearly worked. A network that predicts one class for everything scores exactly 10% and shows one bright column. That is what the hidden-layer run in Step 3 showed before and after training.

Run (seed 1, rate coding, defaults otherwise)	Data (train / val / test)	Untrained	Trained	Training status
Surrogate grad, 196 -> 10, 15 epochs (this chapter)	MNIST 14x14, 240 / 60 / 150	test 16.0%, train 13.3%	**test 78.0%**, train 88.3%, macro-F1 0.771	kept best 70% val (epoch 14); 87 s
Supervised STDP, 196 -> 10, 15 epochs	MNIST 14x14, 240 / 60 / 150	test 11.3%, train 9.2%	test 27.3%, train 40.0%, macro-F1 0.226	kept best 52.5% val (epoch 3); 148 s
Surrogate grad, 196 -> 32 -> 10, 15 epochs (hidden gain 1.7 or 3.0)	MNIST 14x14, 240 / 60 / 150	test 10.0%	test 10.0% (one class predicted for all)	kept best 12.5% val
Supervised STDP, 25 -> 8, 10 epochs	bars 5x5, 8 patterns (no split)	train 0%	train 50.0%	kept best 50% (epoch 1)
Supervised STDP, 25 -> 8, 40 epochs	bars 5x5, 8 patterns	train 0%	**train 100%**	kept best 100% (epoch 38); 9 s
Surrogate grad, 25 -> 8, 20 epochs	bars 5x5, 8 patterns	train 25.0%	train 100%	kept best 100% (epoch 16)
Unsupervised STDP, 25 -> 8, 10 epochs	bars 5x5, 8 patterns	train 0%	train 0% (see below)	kept best 0%

All rows were measured in this session with `ecfet_v2.va`. MNIST rows use img/class 30 and resolution 14. Bars has no validation or test split, so its accuracy is on the 8 training patterns. In supervised STDP on bars, accuracy sat at 50% for 11 epochs, then jumped to 87.5% at epoch 12 and to 100% at epoch 38. With the default 40 epochs, STDP gets there. With 10 it does not.

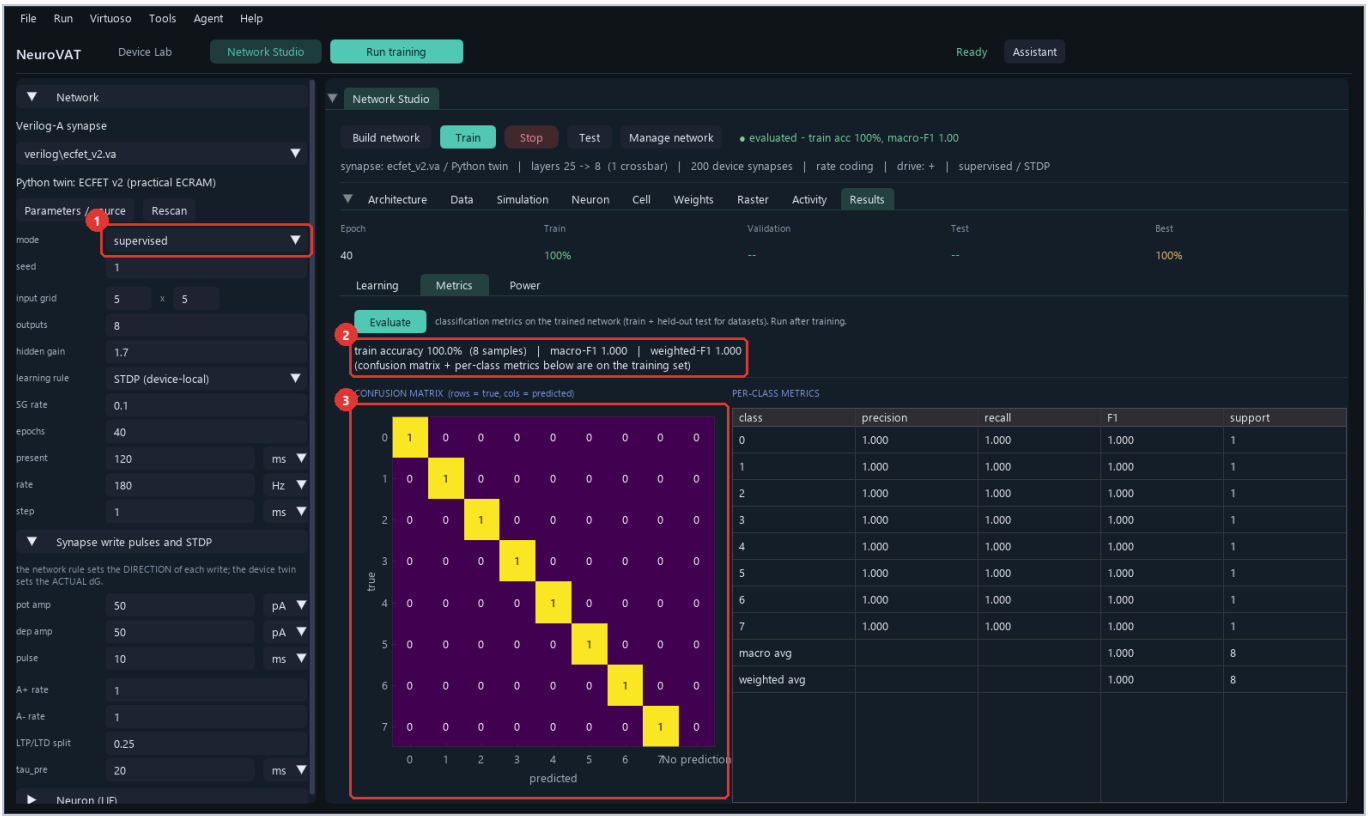


Figure: Supervised STDP on `bars` after 40 epochs: all 8 patterns on their own neuron (100%, 8 training patterns).

1. **mode = supervised**, learning rule STDP, outputs 8.
2. **Summary**: 100% on 8 samples. Small sets are scored on the training set.
3. **Confusion matrix**: a clean diagonal.

Reading an unsupervised result

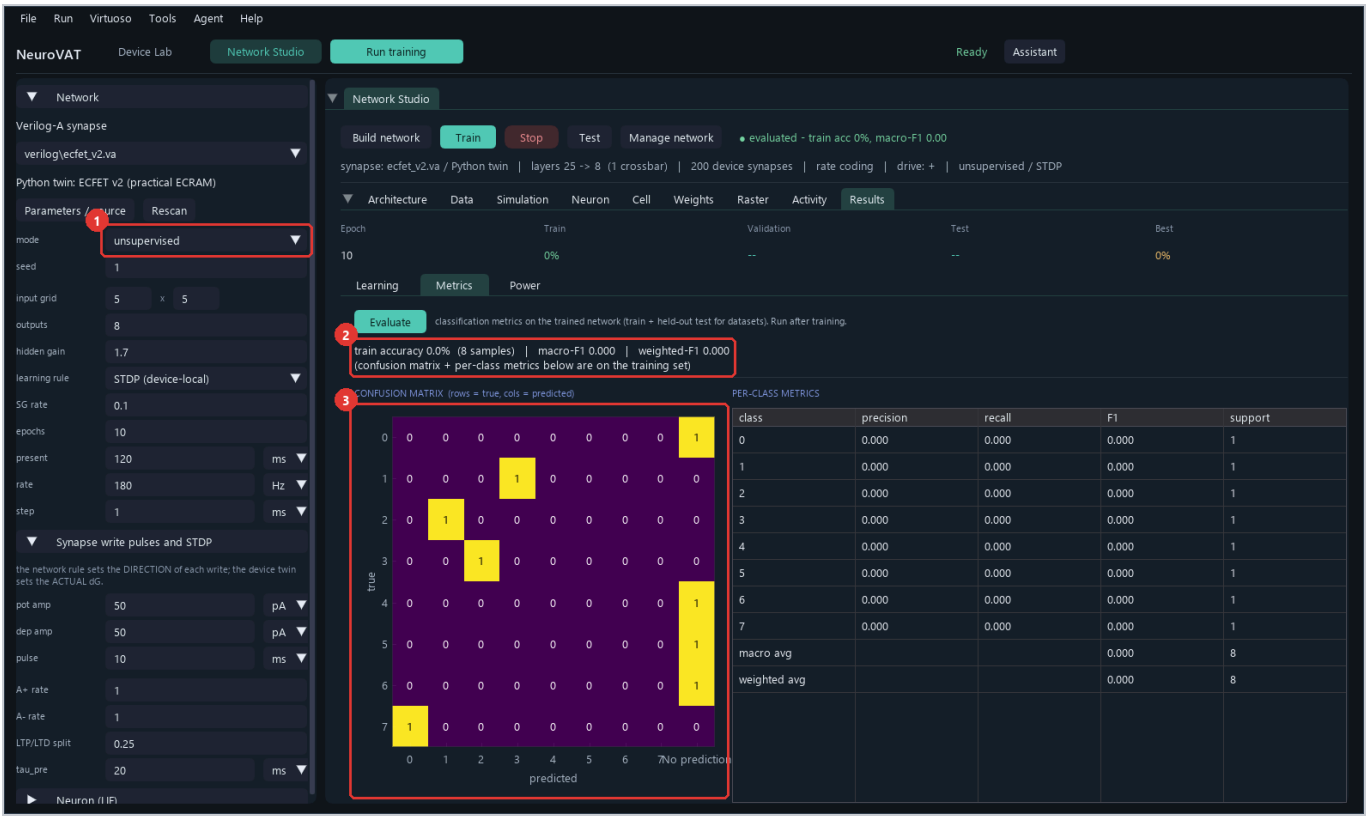


Figure: Unsupervised STDP on `bars`, 10 epochs. It scores 0%, but four patterns went to four different neurons.

1. **mode = unsupervised**.
2. **Summary**: 0% accuracy.
3. **Confusion matrix**: patterns 1, 2, 3 and 7 each drive a *different* neuron (3, 1, 2 and 0). Patterns 0, 4, 5 and 6 give no output spike.

> **Warning.** In the unsupervised mode the accuracy is scored against a fixed pattern-to-neuron map (pattern *k* should win on neuron *k mod outputs*). The app does not relabel neurons after training. A network that separates the patterns onto different neurons can therefore score 0%. In the unsupervised mode, read the confusion matrix: look for each true class going to its *own* column, whatever that column is, and compare that with the untrained matrix. Do not read the accuracy number.

4.7 Step 7. The inspection tabs

These tabs show the *built* network. After a Test, they show the last evaluated presentation (here a test image of a "4").

Simulation

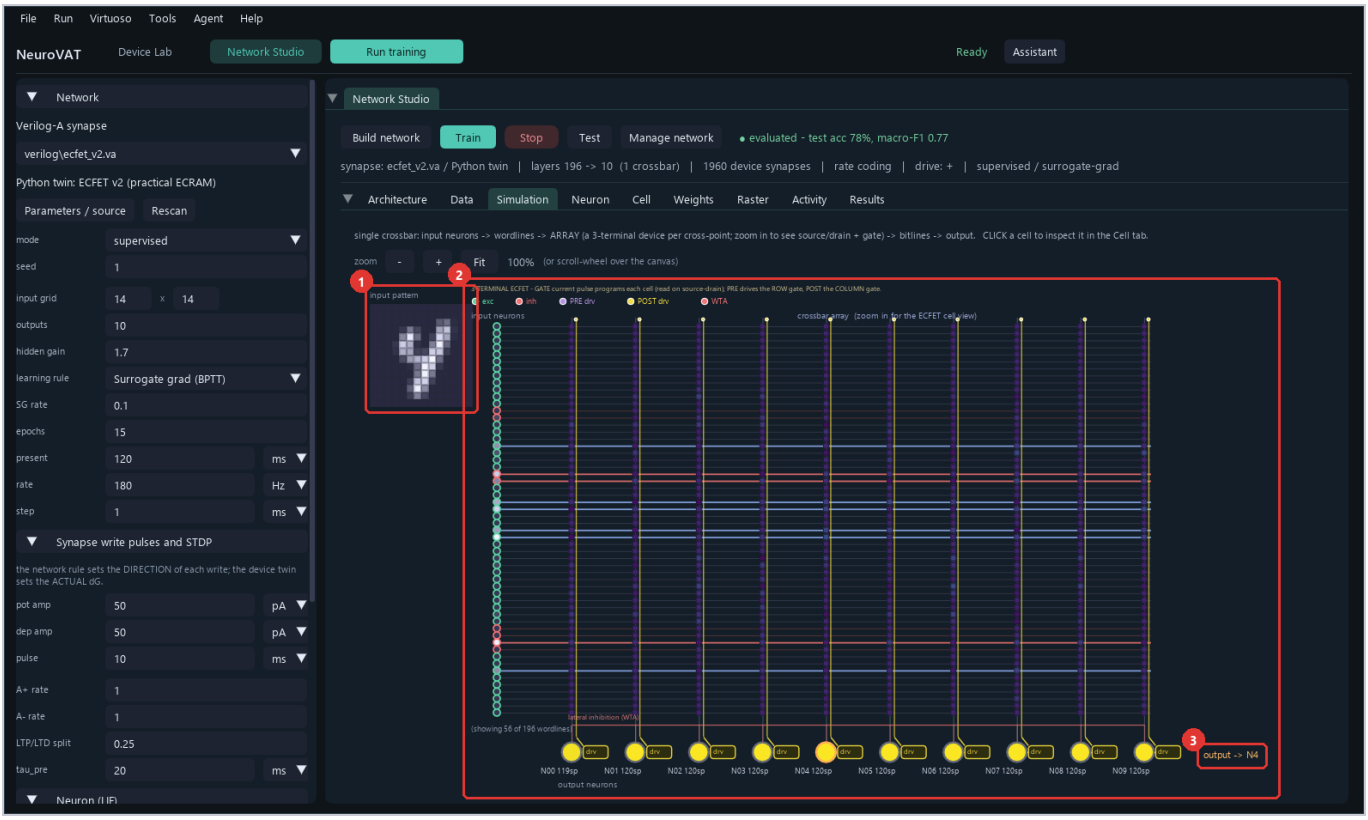


Figure: Simulation tab: input pattern, the crossbar of 3-terminal ECFET cells, and the output neurons.

1. **Input pattern**: the image being presented.
2. **Crossbar diagram**: input neurons drive wordlines. Each cross-point is one device, and bitlines feed the output neurons. Colours mark excitatory and inhibitory inputs, the PRE/POST drivers and the WTA line. Zoom with **-**, **+** and **Fit** or the scroll wheel. **Click a cell** to open it in the Cell tab. With 196 inputs, only a sample of the wordlines is drawn.
3. **Winner**: the output neuron that won this presentation (N4), with each neuron's spike count below it.

Neuron

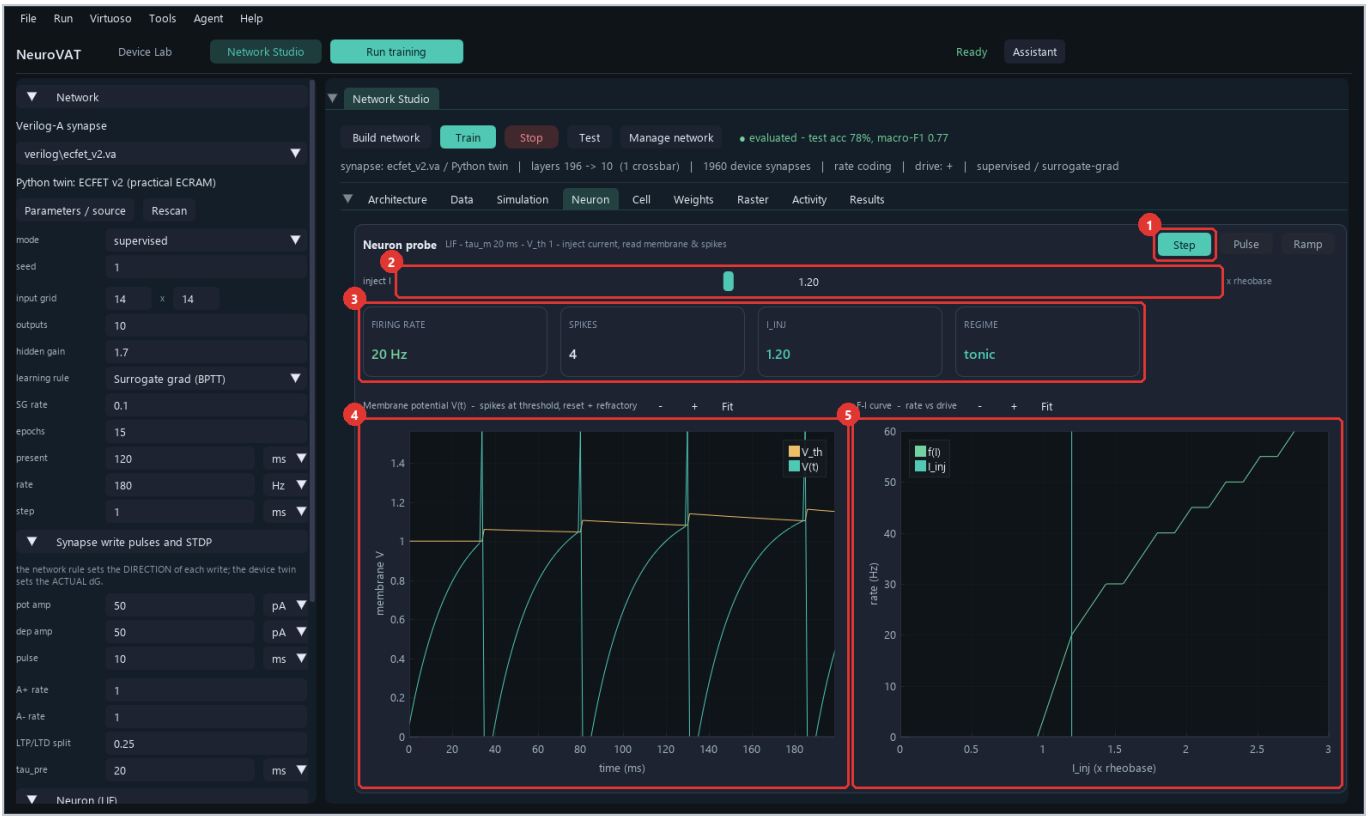


Figure: Neuron tab: a current-injection probe of one LIF output neuron with the current settings.

1. **Step / Pulse / Ramp**: the shape of the injected current.
2. **inject I**: the current in units of rheobase (the smallest current that makes the neuron fire). 1.20 is just above threshold.
3. **Readout tiles**: firing rate (20 Hz here), spike count, current and regime.
4. **V(t)**: the membrane integrates, crosses V_th, resets and waits out the refractory time. The adaptive threshold rises after each spike.
5. **f-I curve**: rate against drive. Use it to check tau_m, V_th and refrac before training: a neuron that never fires cannot learn.

Cell

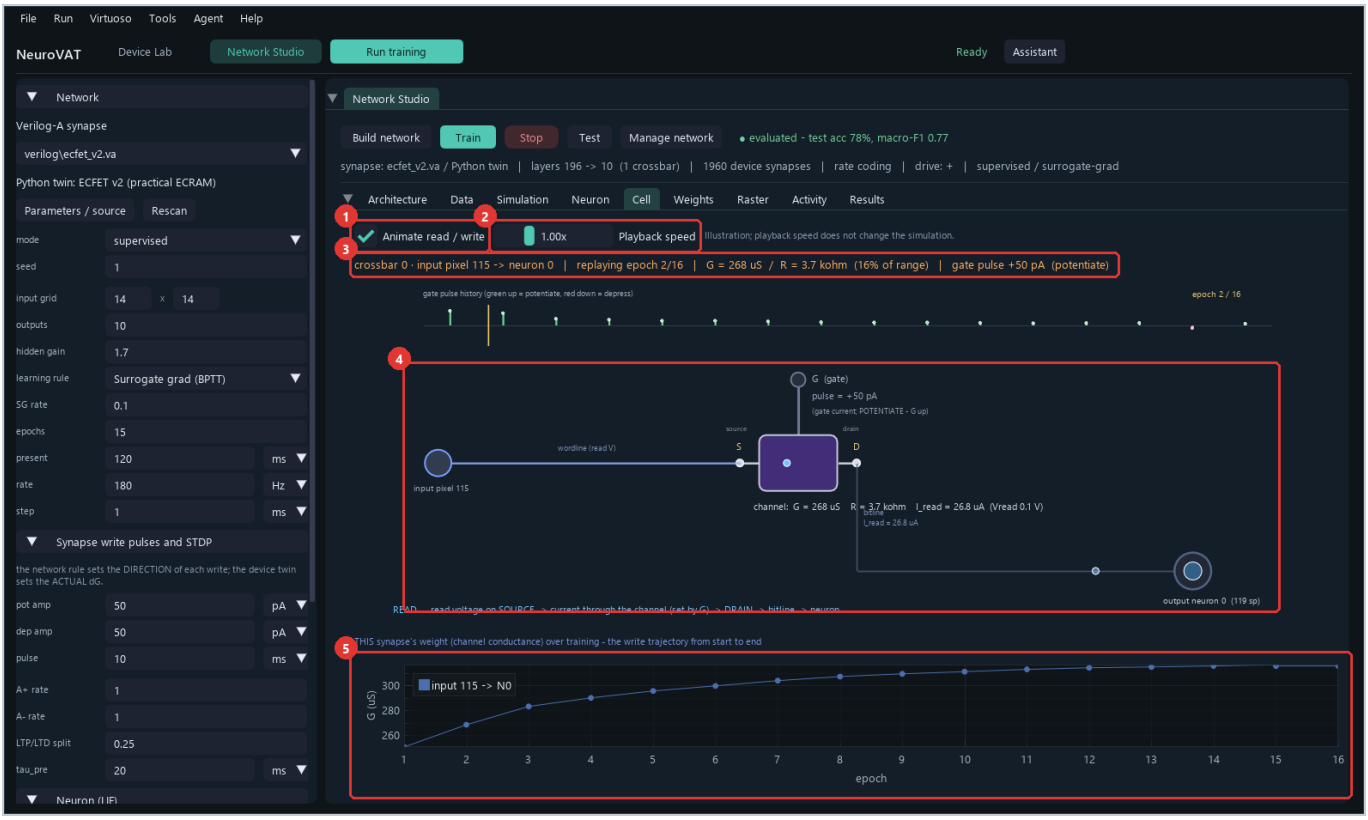


Figure: Cell tab: one synapse (input pixel 115 -> neuron 0), animated read and write.

1. **Animate read / write**: plays the illustration.
2. **Playback speed**: the speed of the illustration only. It does not change the simulation.
3. **Cell line**: the crossbar, input and neuron; the replayed epoch; the conductance G, resistance R and position in the range; and the last gate pulse (+50 pA, potentiate).
4. **Cell drawing**: the read path (wordline voltage, through the channel set by G, drain, bitline, neuron) and the gate write pulse. Above it, the gate-pulse history shows each write: green for up, red for down.
5. **G over training**: this synapse's conductance at each recorded epoch. If no cell was clicked, the strongest synapse onto neuron 0 is shown.

Weights

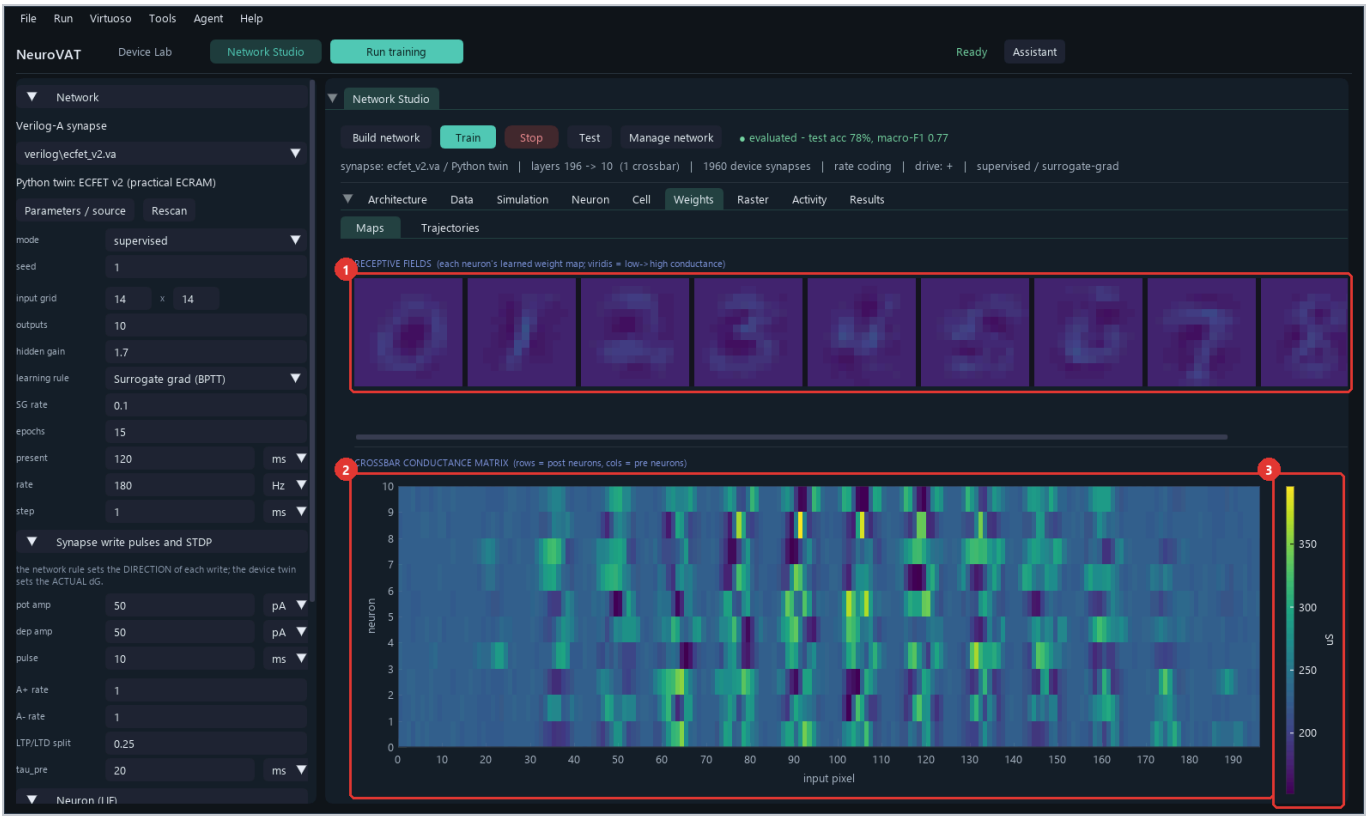


Figure: Weights > Maps: one receptive field per output neuron, plus the full conductance matrix.

1. **Receptive fields**: each output neuron's weights drawn as an image. After surrogate training on MNIST, the digit shapes 0, 1, 2, 3 and 7 are visible. Scroll sideways to see all ten.
2. **Crossbar conductance matrix**: rows are post neurons, columns are input pixels. A combo above it selects the crossbar when there are hidden layers.
3. **Colour bar**: conductance in uS.

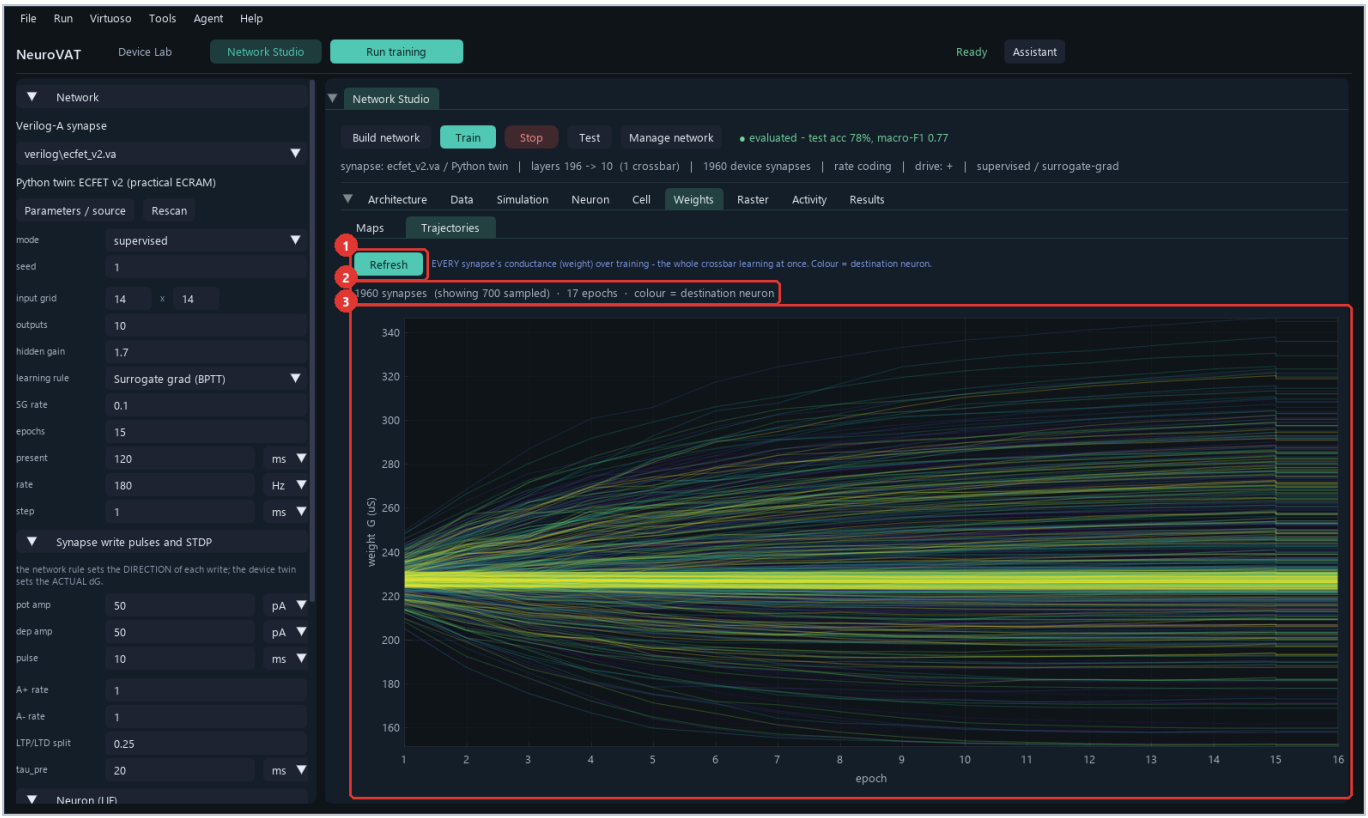


Figure: Weights > Trajectories: every synapse's conductance over training (700 of 1960 sampled).

1. **Refresh**: redraws the plot from the recorded history.
2. **Info line**: how many synapses are drawn, and the number of recorded points. The last point is the restored best-epoch checkpoint, so there are more points than epochs. That is why the curves step back at the end.
3. **Trajectories**: the colour is the destination neuron. Most synapses stay near their start value, and a minority are pushed up or down. That is typical of a device with bounded, nonlinear writes.

Raster

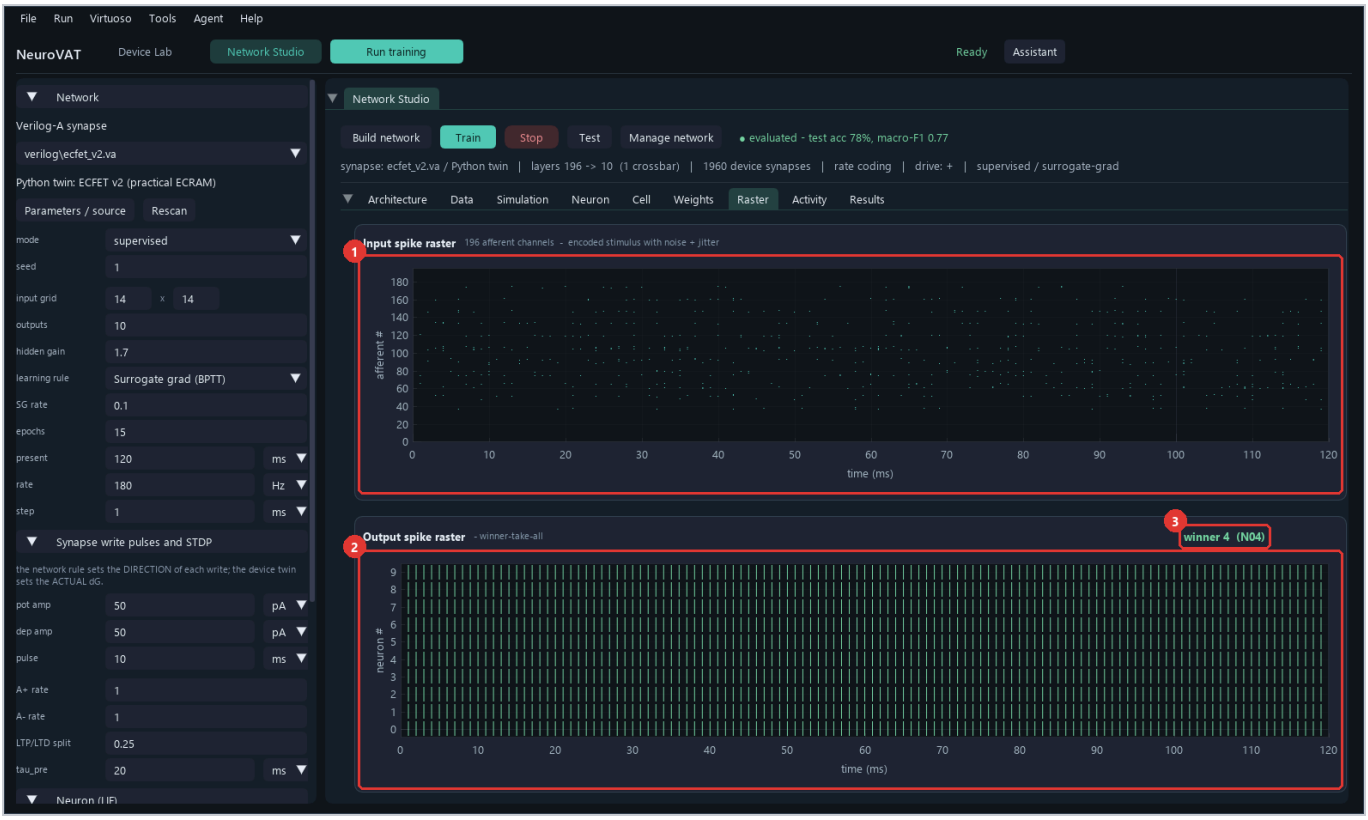


Figure: Raster tab: input spikes (196 afferents) and output spikes for one test presentation.

1. **Input spike raster**: the encoded image as it reached the network.
2. **Output spike raster**: output spikes over the 120 ms window.
3. **Winner**: neuron 4. With the surrogate rule, every output neuron here spiked at almost every step, so the winner is decided by the membrane readout, not by spike count. The next tab shows this.

Activity

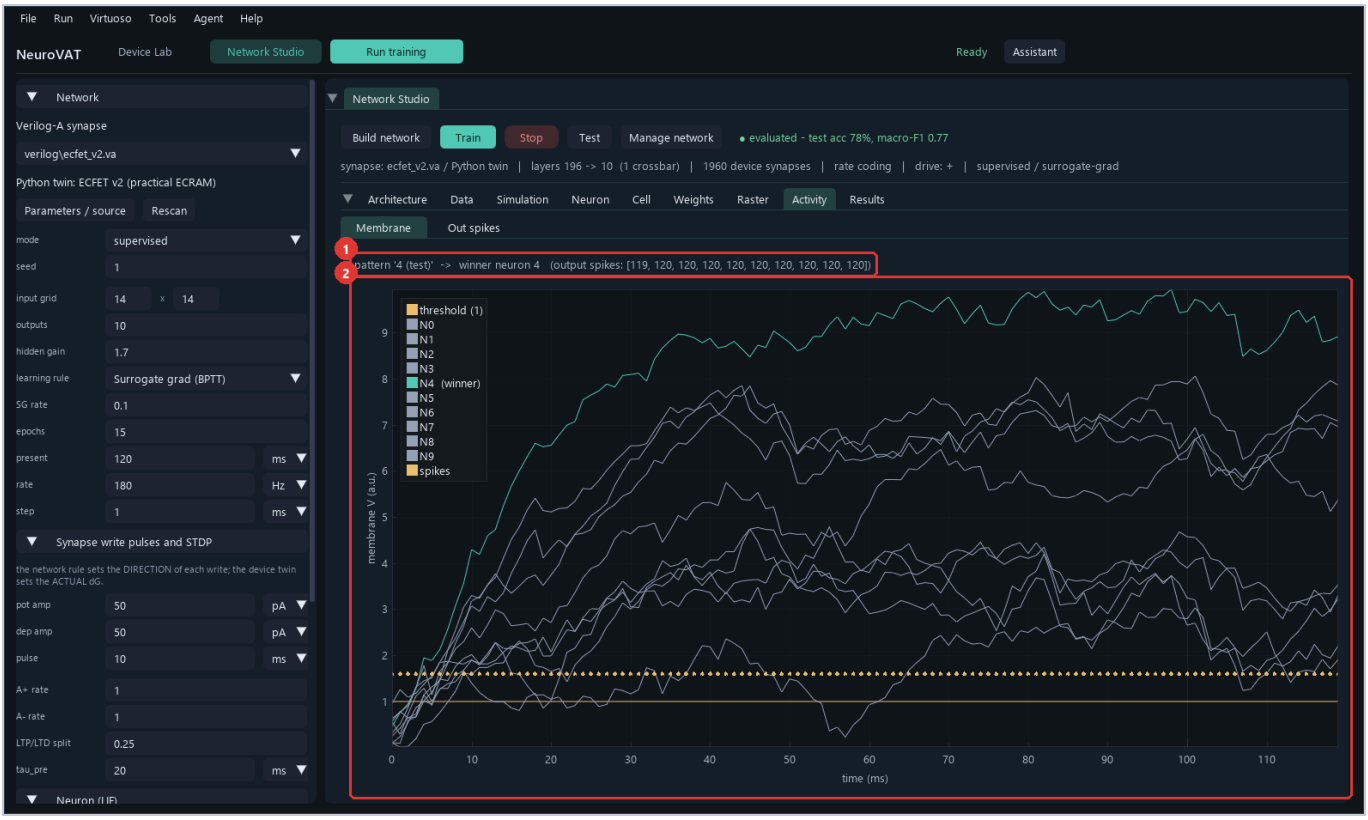


Figure: Activity > Membrane: all ten output membranes for the test "4".

1. **Title**: the pattern, the winner and each neuron's spike count (119 or 120 each).
2. **Membrane plot**: the winner (N4, highlighted) stays well above the others for the whole window. That margin is what the classification uses. **Out spikes**, the other sub-tab, shows the output spikes streamed during training.

4.8 Step 8. The Power tab and power profiles

Every network carries an energy meter. The meter only observes, so results are the same with or without it. It counts reads, writes, spikes, peripheral events and simulated time, and it prices them with a *power profile*. Open **Results > Power**.

The public release ships **no power profiles**. With none, every energy parameter runs on a placeholder value, and the tab says so.

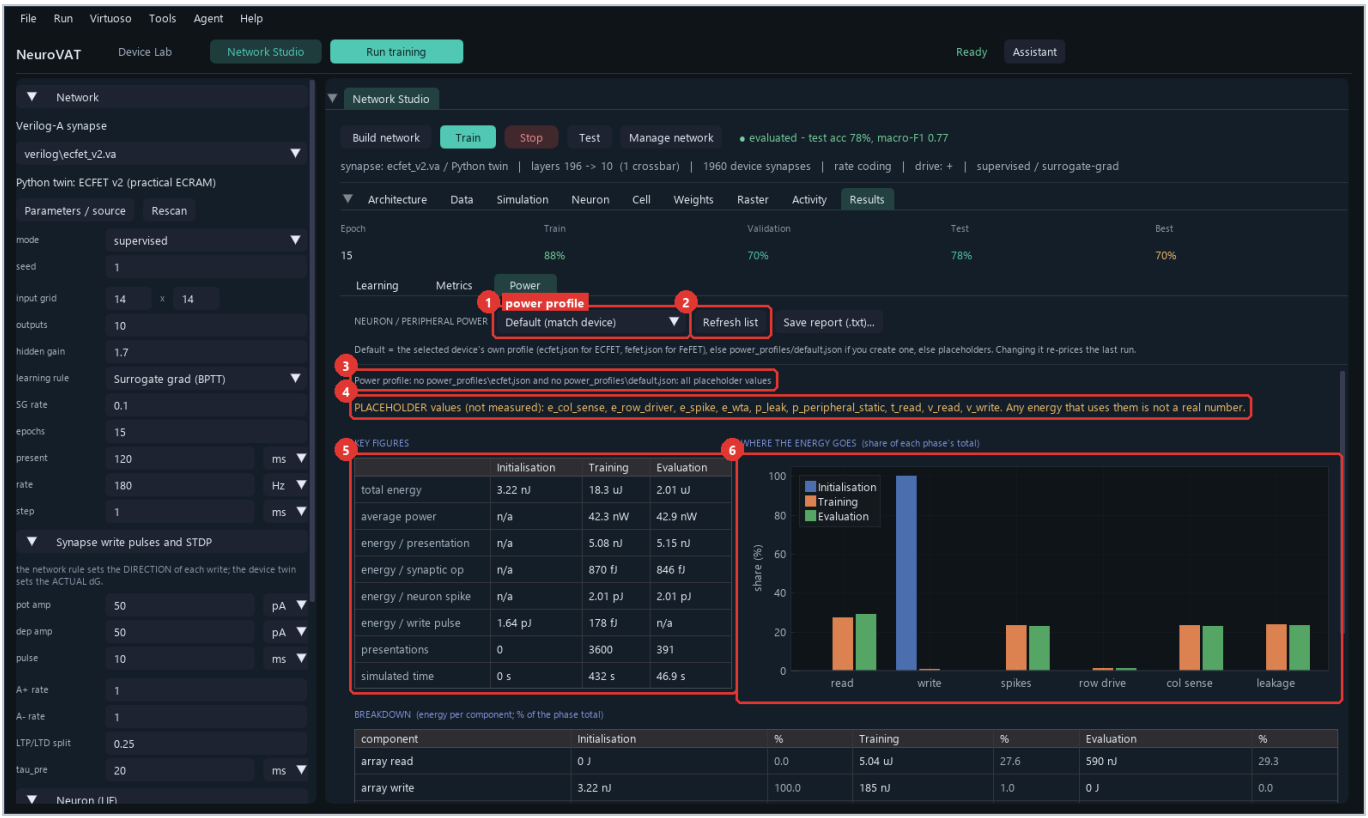


Figure: Power tab with no profile file: every value is a flagged placeholder.

1. **Profile combo**: *Default (match device)* looks for `power_profiles\<family>.json` (for this device, `ecfet.json`), then `power_profiles\default.json`, then falls back to placeholders. *Placeholders only* ignores all files. Any other entry is a file you added.
2. **Refresh list**: re-reads the `power_profiles` folder after you add a file.
3. **Profile note**: which file was used. Here: "no power_profiles\ecfet.json and no power_profiles\default.json: all placeholder values".
4. **Placeholder warning**: lists every parameter that is not measured (here all nine). Any energy that uses them is not a real number.
5. **Key figures**: total energy, average power, energy per presentation, per synaptic operation, per neuron spike and per write pulse, for initialisation, training and evaluation. With placeholders these numbers only show what the meter counts. They do not describe your device.
6. **Where the energy goes**: the share of each component in each phase. A full breakdown table and a **Full text report** are further down.

Adding a profile

A profile is a flat JSON (or `.toml`, or `name,value` CSV) file in `<workspace>\power_profiles\`, in SI units. The allowed names are `v_read`, `t_read`, `v_write`, `write_current`, `e_spike`, `p_leak`, `e_row_driver`, `e_col_sense`, `e_wta`, `p_peripheral_static` and `timestep_s`. You can also add the text fields `source`, `notes`, `corner`, `vdd`, `temperature`, `cell` and `date`. An unknown name makes the app ignore the whole file, and the note says why. Anything you leave out stays a flagged placeholder. The file is read as data only. See "Power and energy" in `src/neurovat/README.md` for what each component counts.

To demonstrate, this file was saved as `power_profiles\example.json`. **These are made-up round example values, not device data.**

```
{
 "source": "example values - not measured",
 "notes": "tutorial demo profile with made-up round numbers",
 "v_read": 0.2, "t_read": 1e-6, "e_spike": 5e-12, "p_leak": 2e-9,
 "e_row_driver": 1e-12, "e_col_sense": 2e-12, "e_wta": 1e-12,
 "p_peripheral_static": 0.0, "v_write": 2.0
}
```

Then press **Refresh list** and choose **example**.

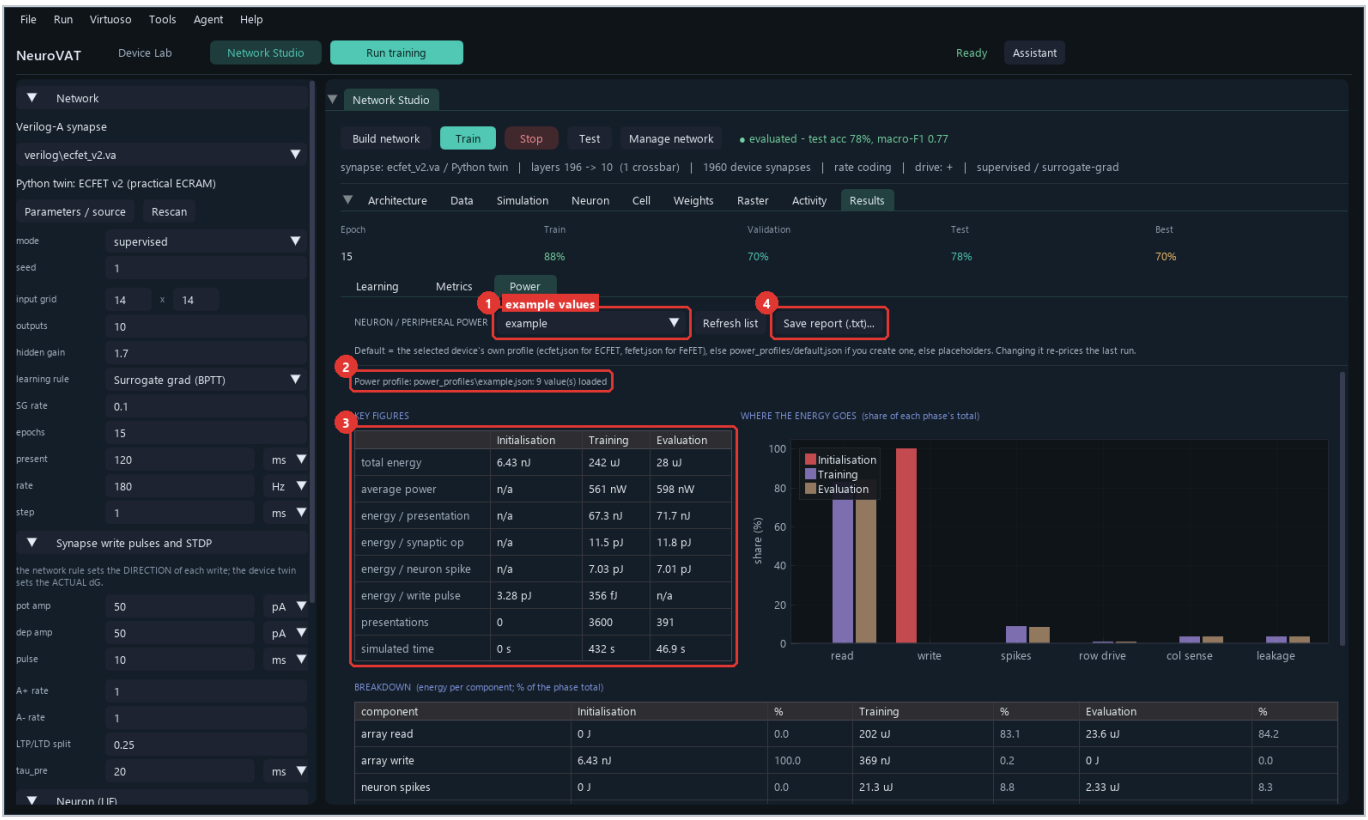


Figure: The same run re-priced with the example profile (example values, not measured).

1. **Profile = example**: changing the profile re-prices the last run at once. The meter stores counts, not energies, so nothing is re-simulated.
2. **Note**: "power_profiles\example.json: 9 value(s) loaded". The placeholder warning is gone because every placeholder field is now set.
3. **Key figures**: now priced with the example values. They are still not device numbers, because the inputs are invented.
4. **Save report (.txt)...**: writes the text report to `results\energy_reports\` for your records.

> **Tip.** To have your profile picked automatically, name it after the device family (`ecfet.json`, `fefet.json`, ...) or `default.json`. Put measured or circuit-extracted values only, and record their origin in `source` and `corner`.

4.9 Step 9. Save and load a network

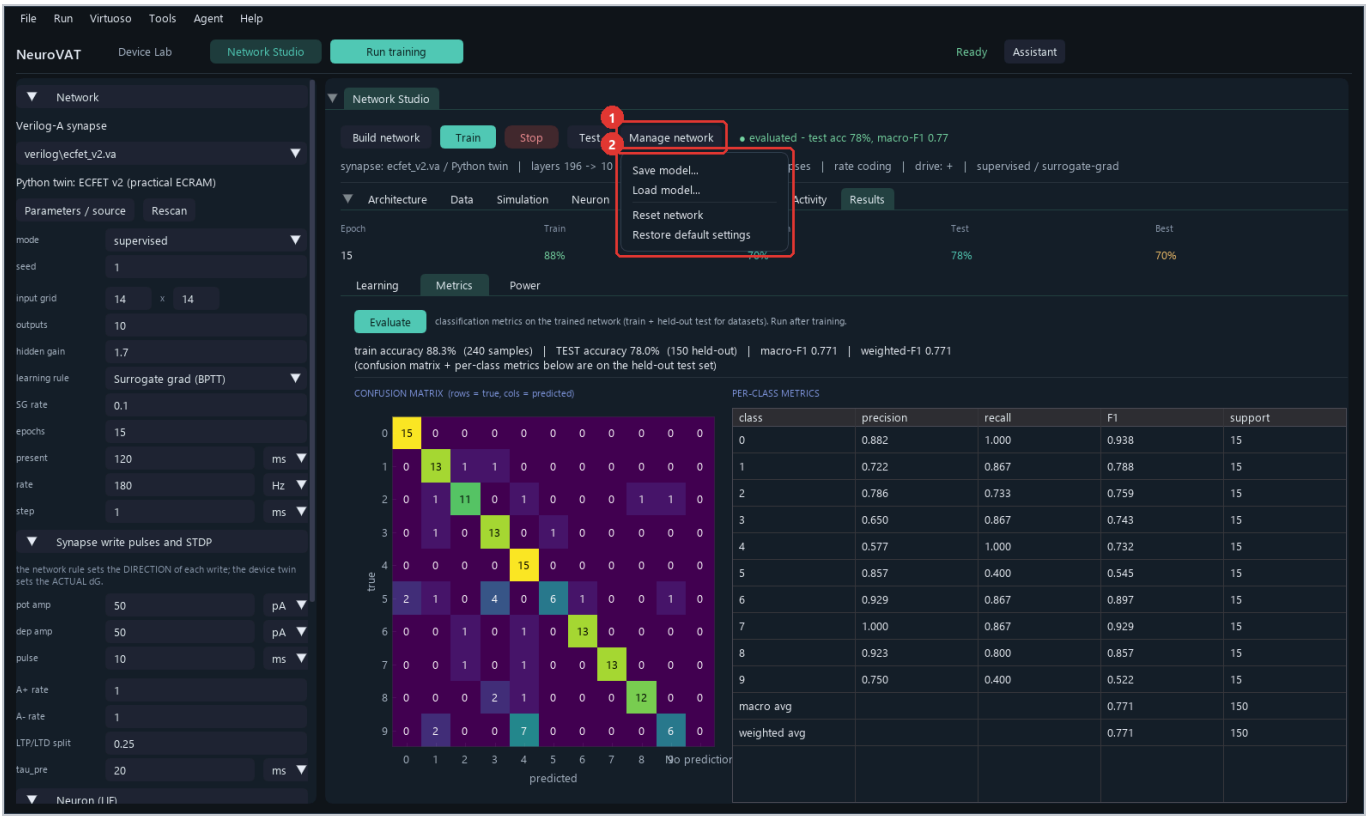


Figure: The Manage network menu.

1. **Manage network**: opens the menu.
2. **Menu**: **Save model...** writes one self-contained JSON file. It holds the architecture, every control, the device parameters, the full trainer state (device states, thresholds, time, training RNG), the training, validation and test patterns, and the source binding with its SHA-256. **Load model...** restores all of it. **Reset network** rebuilds a fresh network with the current settings. **Restore default settings** refills the neuron and learning fields.

Saving is for the supervised and unsupervised modes; stream networks cannot be saved yet. Stop training before you save or load.

> **Note.** Saved models are large because they hold the full device state and every pattern. The 196 x 10 MNIST network in this chapter saved to a file of about 30 MB (31,000 KB).

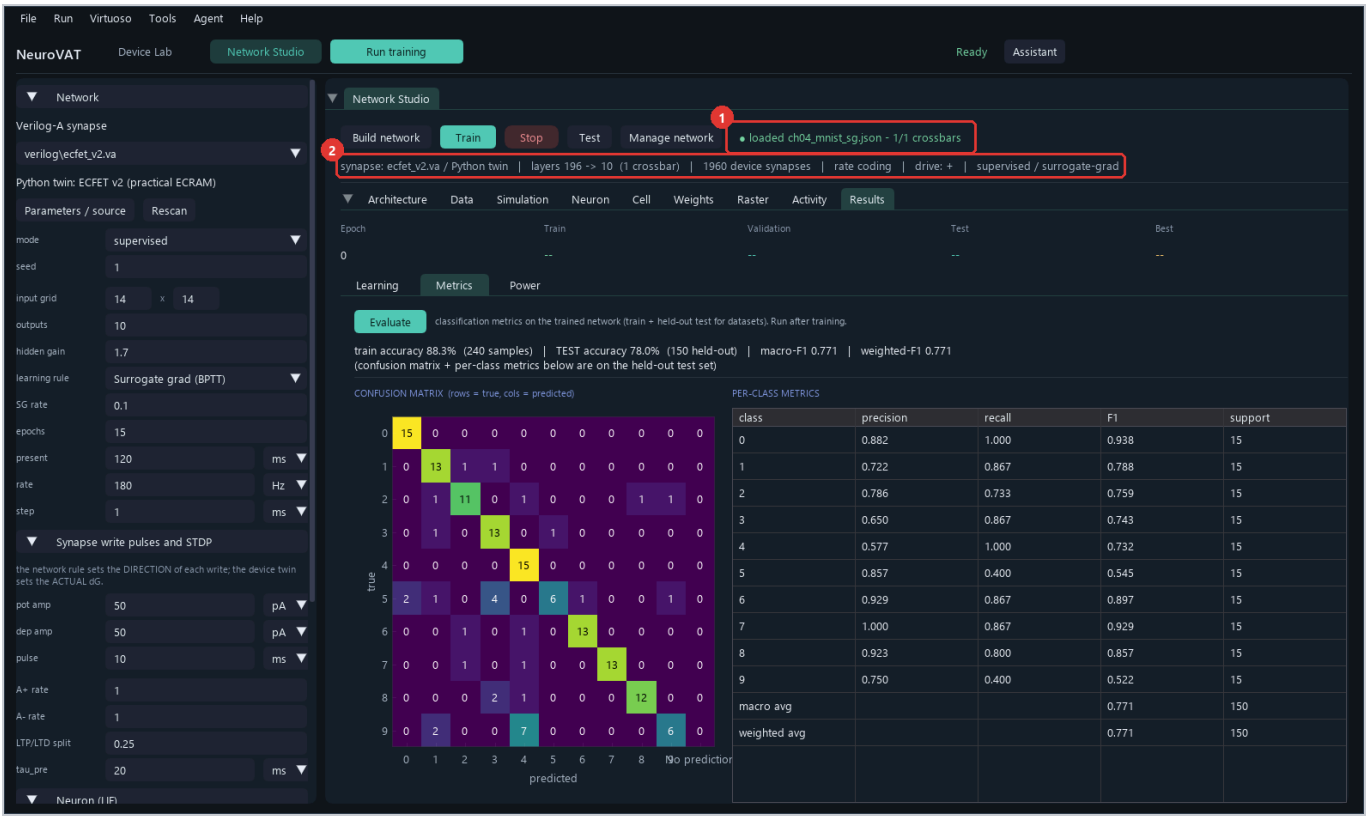


Figure: After Load model: "loaded ch04_mnist_sg.json - 1/1 crossbars".

1. **Status**: the file and the number of crossbars restored. In this session the reloaded trainer state was identical to the saved one.
2. **Status line**: the same source, layers and rule as when the model was saved.

What blocks training and loading

To show the guard, one comment line was added to `verilog\ecfet_v2.va` after the network was built and saved. The file was restored afterwards.

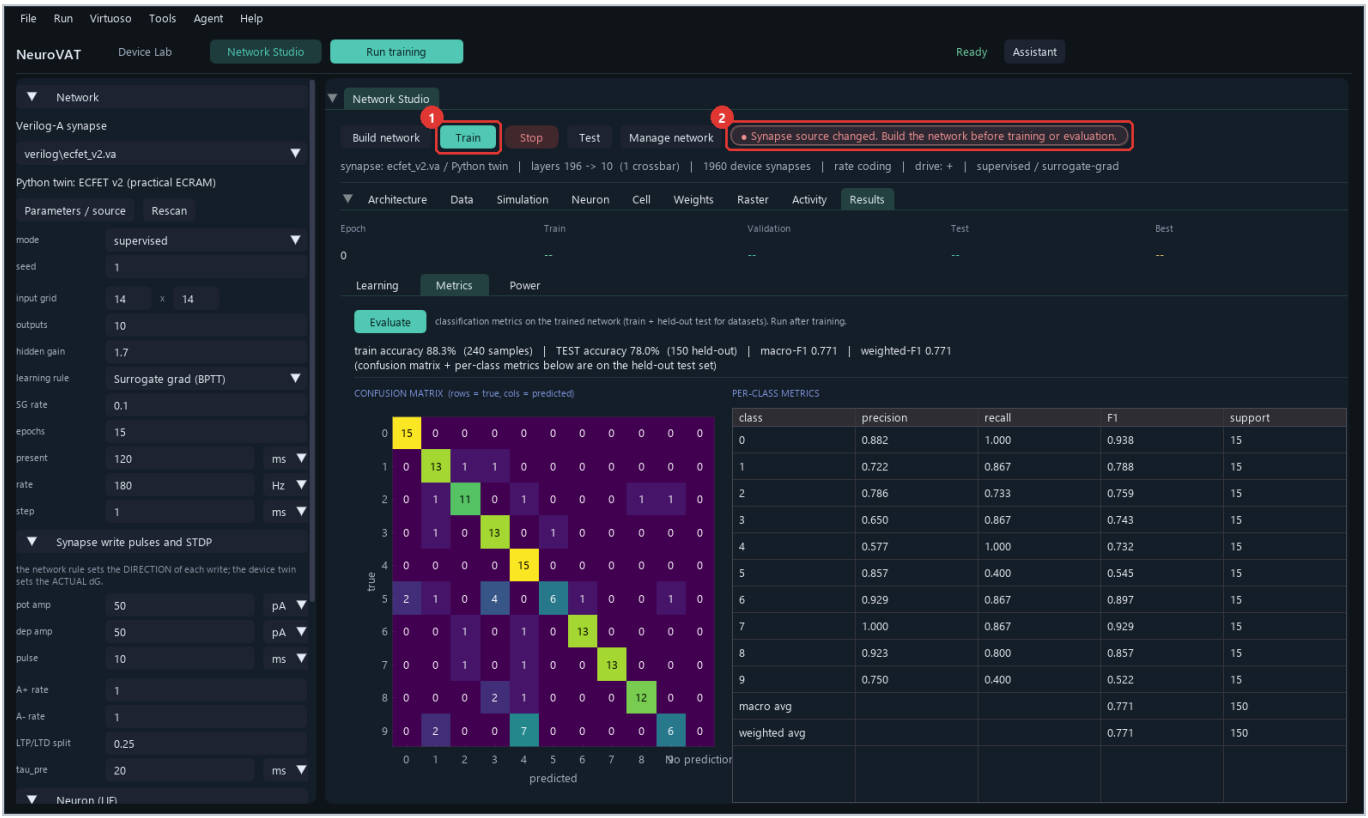


Figure: Train refused after the source file changed.

1. **Train** (and Test) check the source fingerprint first.
2. **Status**: "Synapse source changed. Build the network before training or evaluation." Nothing runs. Rebuild to accept the new source, or restore the old file.

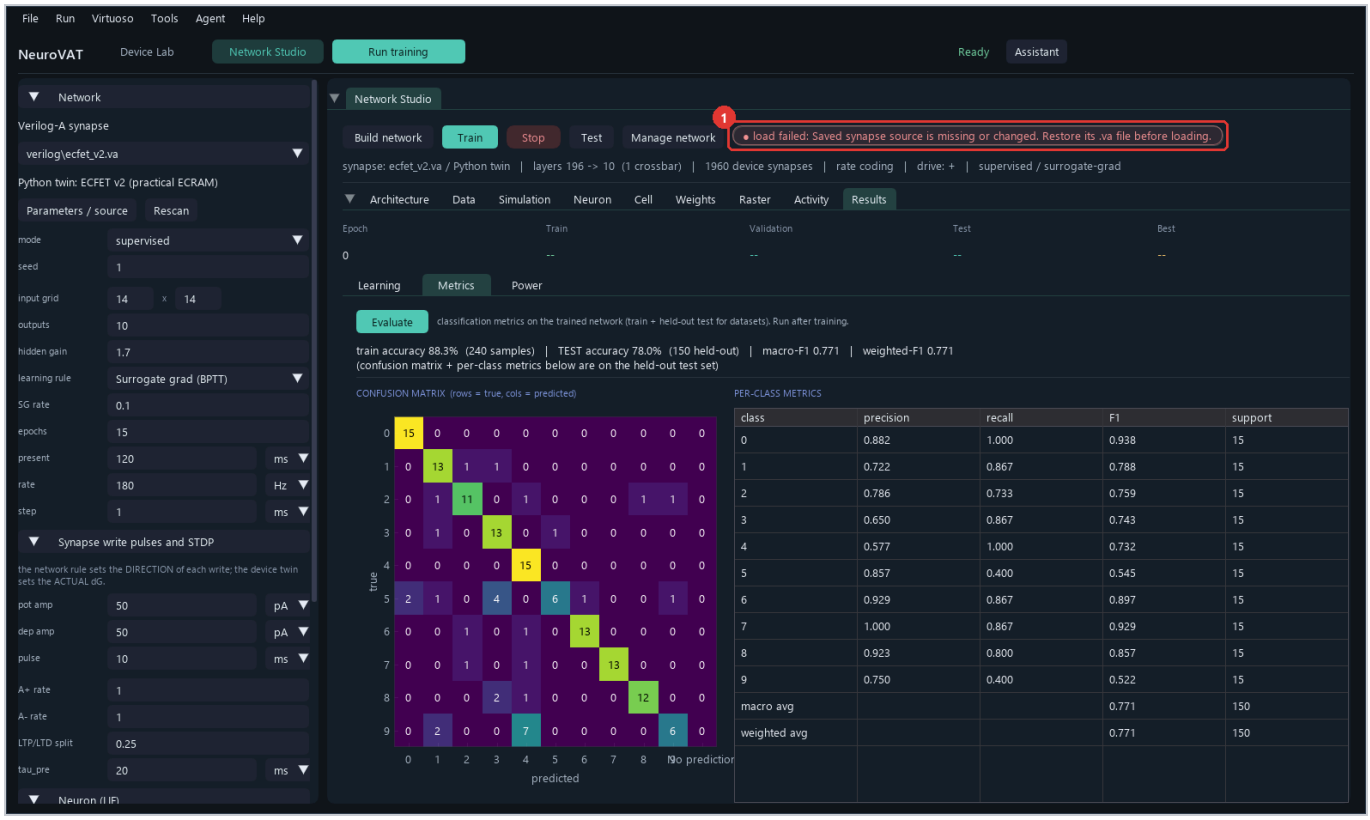


Figure: Load refused while the saved source is changed.

1. **Status**: "load failed: Saved synapse source is missing or changed. Restore its .va file before loading." The current network is left as it was.

> **Warning.** Loading needs a `.va` file whose SHA-256 matches the saved one, for the same twin. Even a whitespace or comment edit changes the hash. Keep a copy of the exact source next to every saved model you care about. Old saves without a fingerprint need only a source for the same twin.

4.10 Summary

- Pick a mapped `.va` synapse, choose the mode and rule, set the layers, and **Build**. The build captures the device parameters and the source fingerprint.
- Choose the data and the encoding in the Data tab, and check the clean and noisy previews.
- Press **Test** before you train to record the untrained baseline. Then **Train** and **Test** again, and compare the two on the same held-out set.
- Measured in this session: surrogate gradients took a 196 -> 10 ECFET v2 crossbar from 16.0% to 78.0% on 150 held-out MNIST digits (240 training images, 15 epochs, seed 1).
- Energy numbers are placeholders until you add a power profile with real values.

CHAPTER 5

Unsupervised stream mode: finding a hidden pattern with STDP

The other Trainer modes show the network one labelled pattern at a time, epoch after epoch. Stream mode works differently. It follows Masquelier, Guyonneau and Thorpe (2008). This chapter shows how to set it up, how to tell real learning from plain detection, and how to test the trained network under worse input.

5.1 The idea in plain words

- **One continuous spike stream.** There are no epochs and no labels. The input lines fire all the time.
- **A hidden pattern recurs in noise.** In this chapter the pattern is an MNIST digit "0", turned into spikes. It appears at irregular times between stretches of random background spikes. Nobody tells the network when it is there.
- **STDP acts at every spike.** A synapse whose input spike comes just before an output spike is strengthened. One whose spike comes just after is weakened. Over time a neuron should fire for the recurring pattern and stay quiet on the noise.
- **Train** runs the stream with STDP on. **Test** runs a new stream around the same hidden pattern with STDP off.

Detects is not the same as learnt

With image input, the app sets the neuron threshold from the image itself. By default it is 0.85 times the peak potential that one clean presentation of the image produces at the starting weights. So a network can already fire on the image windows *before any training*. Such a network **detects** the image, but it has learnt nothing.

For that reason, always run **Test before Train**. The first Test on a new network is the **untrained baseline**: a fresh network at its start weights, with STDP off. The app keeps it. A later Train continues the same network on the same hidden pattern. The verdict of a later Test then shows both runs side by side.

> **Note.** Learning means this: **after training the neuron answers fewer of the empty (false) windows than the untrained network did, and it still answers the image windows.** Only this comparison says that STDP learnt something. A neuron that answers the image well both before and after training shows detection only.

5.2 Set up the stream

Open **Network Studio** from the top bar. The steps below use the defaults wherever possible. You change only what the list says.

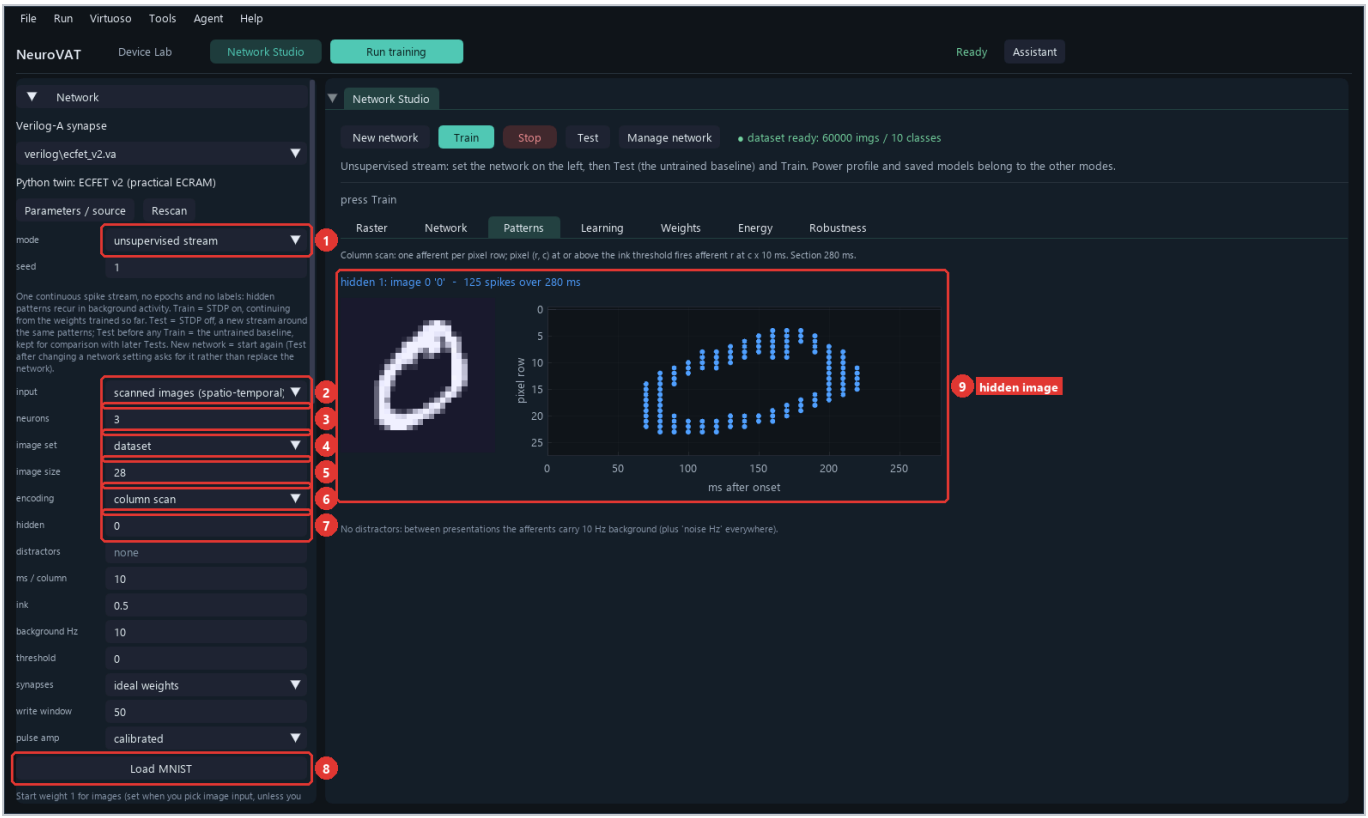


Figure: Stream-mode setup on the left panel. The Patterns tab on the right shows the hidden MNIST "0" and its spikes.

1. **mode:** pick `unsupervised stream`. This swaps the whole left panel and the centre views for the stream controls. The Build button becomes **New network**.
2. **input:** pick `scanned images (spatio-temporal)`. The other choice, Poisson patterns, is the input of the 2008 paper (see the section near the end). When you pick image input, the start weight is set to 1, unless you have already changed it.
3. **neurons:** leave the default `3`. Lateral inhibition lets each neuron take a different pattern. With only one hidden image, one neuron answers it best, but the others can still fire on it. In our run, after training, neurons 2 and 3 still fired 0.93 and 0.79 spikes per image window.
4. **image set:** pick `dataset`. This uses one image per class of the loaded dataset: the first sample of each class, in class order. The built-in sets (`digits`, `letters`, `bars`) also work. But the README reports that the built-in 28 x 28 images are already clean untrained, so they show detection, not learning.
5. **image size:** set `28` (the default is 14). The README's learning results are all for MNIST at size 28.
6. **encoding:** leave `column scan`. It gives one input line per pixel row, so 28 inputs here. The other codes are covered later in this chapter.
7. **hidden:** leave `0`. This is the index of the hidden image in the set. Image 0 of MNIST is the digit "0". You can type several indices, for example `0,3`. But the README reports that several images are not split one per neuron, so start with one.
8. **Load MNIST:** click it. It loads `datasets/mnist.npz` from your workspace, and downloads the file (about 11 MB) only if it is missing. The first download needs an internet connection; on a fresh copy of the Windows release it took about 2 s. The status pill then reads "dataset ready: 60000 imgs / 10 classes". This button only works when image set is `dataset`.

9. **Patterns tab:** check the hidden image. The left panel shows the digit. The right panel shows its spikes: pixel row against ms after onset (125 spikes over 280 ms here).

The image controls

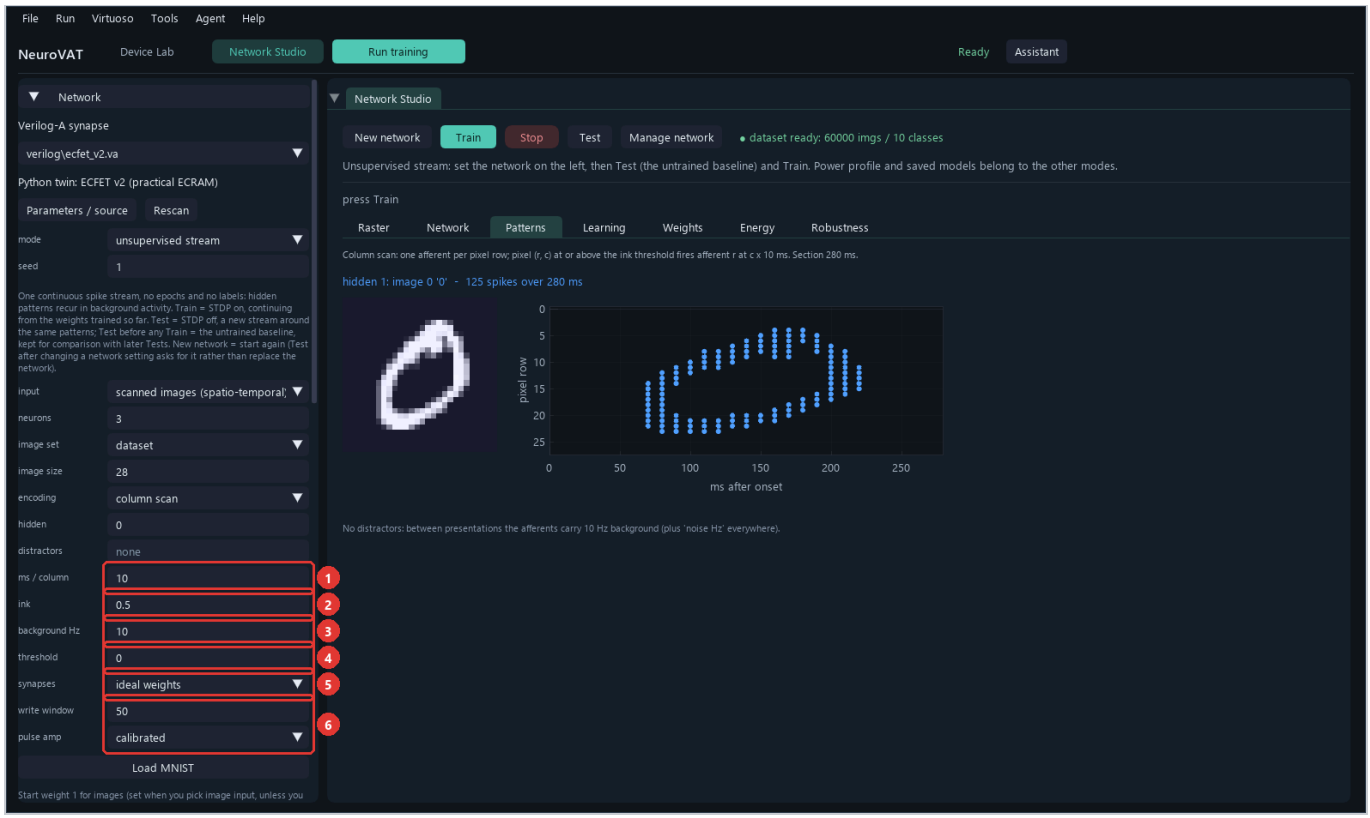


Figure: The image-input controls below "hidden". Leave all of them at their defaults for the first run.

1. **ms / column** (default 10): the time per pixel column of the column scan. One section, the window in which the image is shown, lasts size x this value: 28 x 10 = 280 ms. For the row scan this row is labelled **ms / row**. For the two pixel codes it is replaced by **window ms**.
2. **ink** (default 0.5): a pixel at or above this brightness (0 to 1) fires.
3. **background Hz** (default 10): the Poisson rate on every input line in the sections that show no image. It is not used when you add distractor images.
4. **threshold** (default 0): `0` means auto. For image input the auto threshold is 0.85 x the weakest clean-image peak potential at the starting weights. The 0.85 is a free choice made in this app, not a paper value. The value used is printed in the Network tab and the log. Here it was 14.4.
5. **synapses:** `ideal weights` uses the additive rule of the paper. `selected device` stores each weight as the conductance of the Verilog-A synapse chosen at the top of the panel. See "Device synapses" at the end.
6. **write window** and **pulse amp:** these are for device synapses only. All STDP updates within one write window are merged into one pulse per synapse. `calibrated` searches for the pulse amplitudes at which one pulse equals one STDP step.

Stream input and start weight

Scroll the left panel down to **Stream input**, and open **Learning rule and neuron (stream)**.

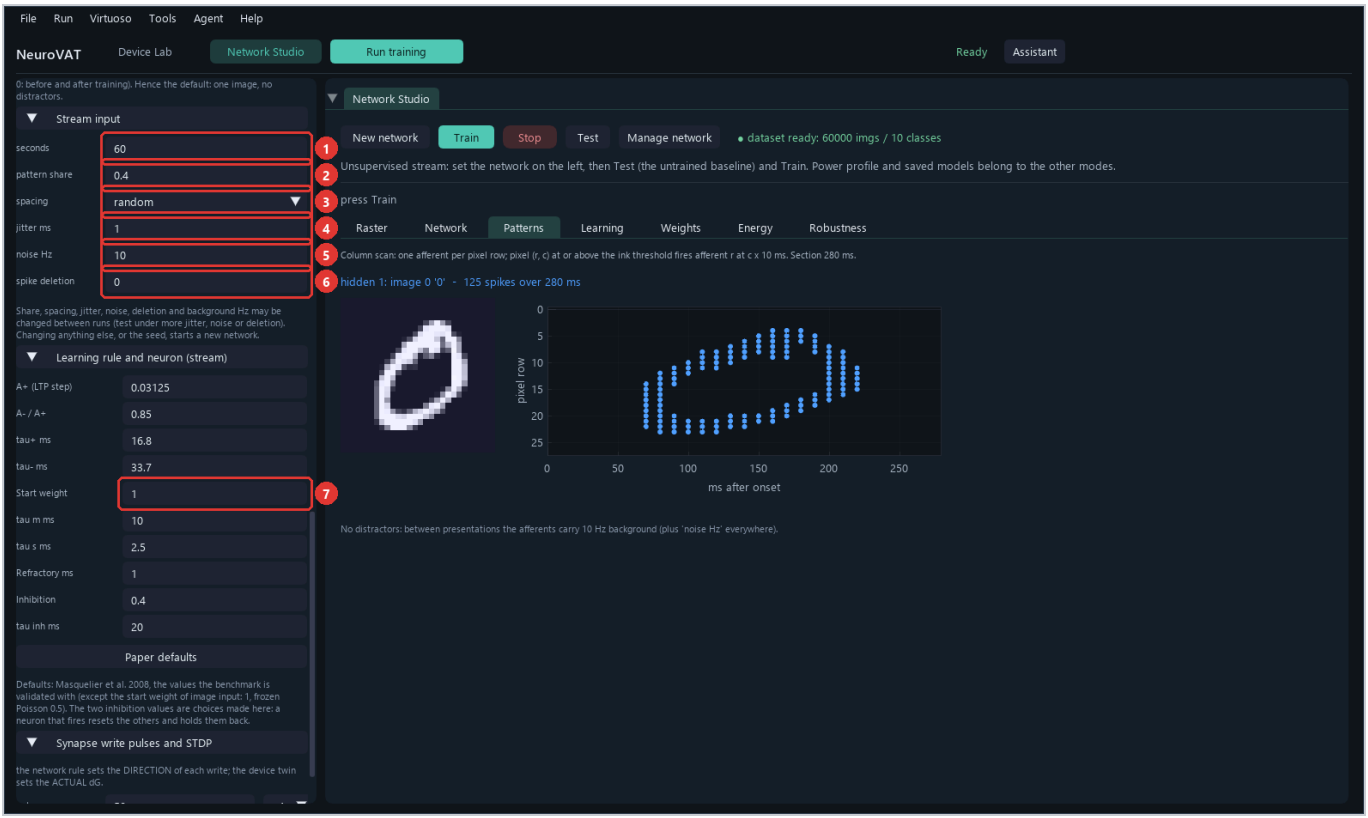


Figure: The per-run stream settings, and the start weight in the learning-rule section.

1. **seconds** (default 60): the simulated stream time of one Train or one Test.
2. **pattern share** (default 0.4): the share of the sections that hold the hidden image. The 2008 paper used 0.25. Two pattern sections never follow each other directly.
3. **spacing** (default `random`): with `random`, the pattern sections are drawn at random. With `even`, every k-th section holds the pattern, where k = max(2, round(1 / share)).
4. **jitter ms** (default 1): the spike-time jitter of the stream.
5. **noise Hz** (default 10): extra random spikes on every input line, everywhere, inside the image windows too. With image input it adds to the background rate in the empty windows. Those empty windows are the ones a neuron answers falsely.
6. **spike deletion** (default 0): the fraction of the pattern's spikes that is dropped at each presentation.
7. **Start weight** (default 1 for image input): the weight every synapse starts at, from 0 to 1 of the range. At 1, STDP can only lower weights, so it prunes the answers to noise. The README reports that at 0.475 (the paper value) STDP raised the weights on the digit's rows while the threshold stayed fixed, and training made the false answers worse.

> **Tip.** Share, spacing, jitter, noise, deletion and background Hz may be changed between runs, for example to test the trained network under more noise. Changing anything else, or the seed, means a new network. A Test after such a change refuses with "settings changed since this network was built: press New network, or restore them". It never silently replaces your trained network with an untrained one.

5.3 Test, Train (60 s), Test at the defaults

The run below used every default except image size 28: MNIST image 0 (the digit "0"), column scan, 3 neurons, seed 1, share 0.4, random spacing, noise and background 10 Hz, jitter 1 ms, start weight 1, ideal synapses and auto threshold (14.4). Each run is 60 s of stream.

Step 1: Test the untrained network

1. Click **New network** if you have run anything before in stream mode. On a fresh app you can skip this.
2. Click **Test**. The status pill reads "stream testing the UNTRAINED baseline (STDP off, start weights)" (visible in the window; the bridge cannot read it mid-run).
3. Watch the **Raster** tab. It plays live while the stream runs.



Figure: The Raster tab during the untrained Test, about 5 s into the stream.

1. **Test:** the button you pressed. Before any Train it runs the untrained baseline.
2. **Live status line:** it is updated every 2 s of stream. It shows the hit rate, the share of other windows answered, false alarms per second and the output spikes so far. These live numbers count a spike of *any* neuron. The final verdict scores each neuron on its own.
3. **Shaded band (image window):** one presentation of the hidden image, 280 ms long. The **blue dots** are the image's spikes: the digit "0" drawn by the column scan, with pixel row on the y axis.
4. **Grey dots (noise):** background and noise spikes. They fill the empty sections between the image windows and also fall inside them.
5. **Triangles above the raster:** output spikes, one colour per neuron (red = neuron 1, yellow = neuron 2, green = neuron 3). A triangle over an empty section is a false answer. Here the untrained neuron 1 fires in several empty sections.
6. **Lower plot:** each neuron's membrane potential divided by its threshold. The grey line at 1 is the threshold. A neuron fires when its trace reaches the line.
7. **Legend:** it names the series. With several hidden images each gets its own colour.

When the Test finishes, open the **Learning** tab to read the verdict.

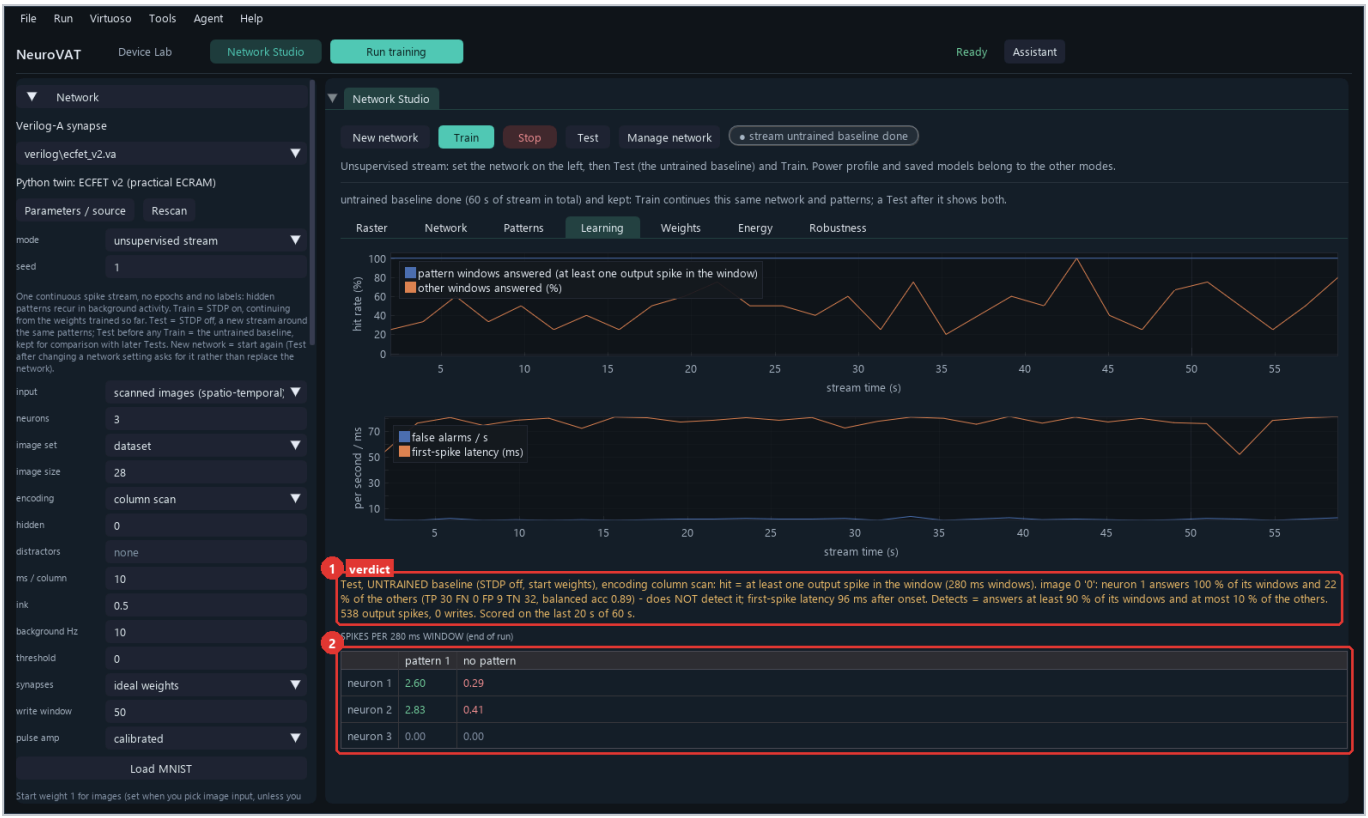


Figure: The verdict of the untrained baseline, in the Learning tab.

1. **Verdict:** "neuron 1 answers 100 % of its windows and 22 % of the others (TP 30 FN 0 FP 9 TN 32, balanced acc 0.89) - does NOT detect it".
    - **Hit rule:** for image input, a window counts as answered when the neuron fires at least once in it.
    - **TP / FN:** image windows answered / missed.
    - **FP / TN:** empty windows answered (false) / left quiet.
    - **Balanced accuracy:** the mean of the hit rate and the quiet rate.
    - **Detects:** the neuron answers at least 90 % of the image windows and at most 10 % of the others. 22 % is above 10 %, so it does not detect the image yet.
    - **First-spike latency:** the mean time from window onset to the neuron's first spike (96 ms here).
    - **"Scored on the last 20 s of 60 s":** every Test and Train is scored only over its **last third**. Here that is 30 image windows and 41 empty windows.
2. **Spikes-per-window table:** the mean output spikes per 280 ms window, for each neuron, in the image windows ("pattern 1") and in the empty windows ("no pattern"). Untrained, neurons 1 and 2 both fire on the image (2.60 and 2.83 spikes per window) and in the empty windows (0.29 and 0.41). Neuron 3 is silent.

Step 2: Train for 60 s

1. Leave every setting as it is. Any network setting you change now starts a new network.
2. Click **Train**. The pill reads "stream training (STDP on)" (visible in the window; the bridge cannot read it mid-run). Train continues the same network and the same hidden image as the baseline.



Figure: Training, 68 s into the whole stream (the first 60 s were the baseline Test).

1. **Train:** STDP is on, starting from the weights trained so far. On a new network these are the start weights.
2. **Status pill:** it tells you which kind of run is in progress.
3. **Live status line:** the stream time counts on from the baseline. Early in training, neurons 2 and 3 (yellow and green triangles) still fire in some empty sections.

The verdict of the Train run itself was "neuron 1 answers 100 % of its windows and 2 % of the others (TP 29 FN 0 FP 1 TN 41)", with 20,616 weight writes. It ends with "Test after training to compare". A training run is not the comparison, because STDP was still changing the weights while it was scored.

Step 3: Test the trained network

1. Click **Test** again. This time the pill reads "stream testing (STDP off)" (visible in the window; the bridge cannot read it mid-run). The weights are frozen, and a new stretch of stream is played around the same hidden image.



Figure: The Test after 60 s of training. Neuron 1 (red) now fires mostly inside the image windows.

1. **Test:** the same button. Because the network has now been trained, this is an ordinary Test, not a baseline.
2. **Red triangles inside the windows:** neuron 1 answers each image window with a burst of spikes and stays quiet between them.
3. **A green triangle over an empty section:** neuron 3 still fires there sometimes. The verdict scores the neuron that answers the image best (neuron 1), so neuron 3's false spikes do not count against it. They do count in the live line and in the Learning curve, which take every neuron together.
4. **Potential trace:** between the image windows, neuron 1's trace now mostly stays below the threshold.
5. **A lone red triangle between two windows** (at about 124.4 s): a false answer that neuron 1 still makes. The verdict counted 1 such window out of 42.

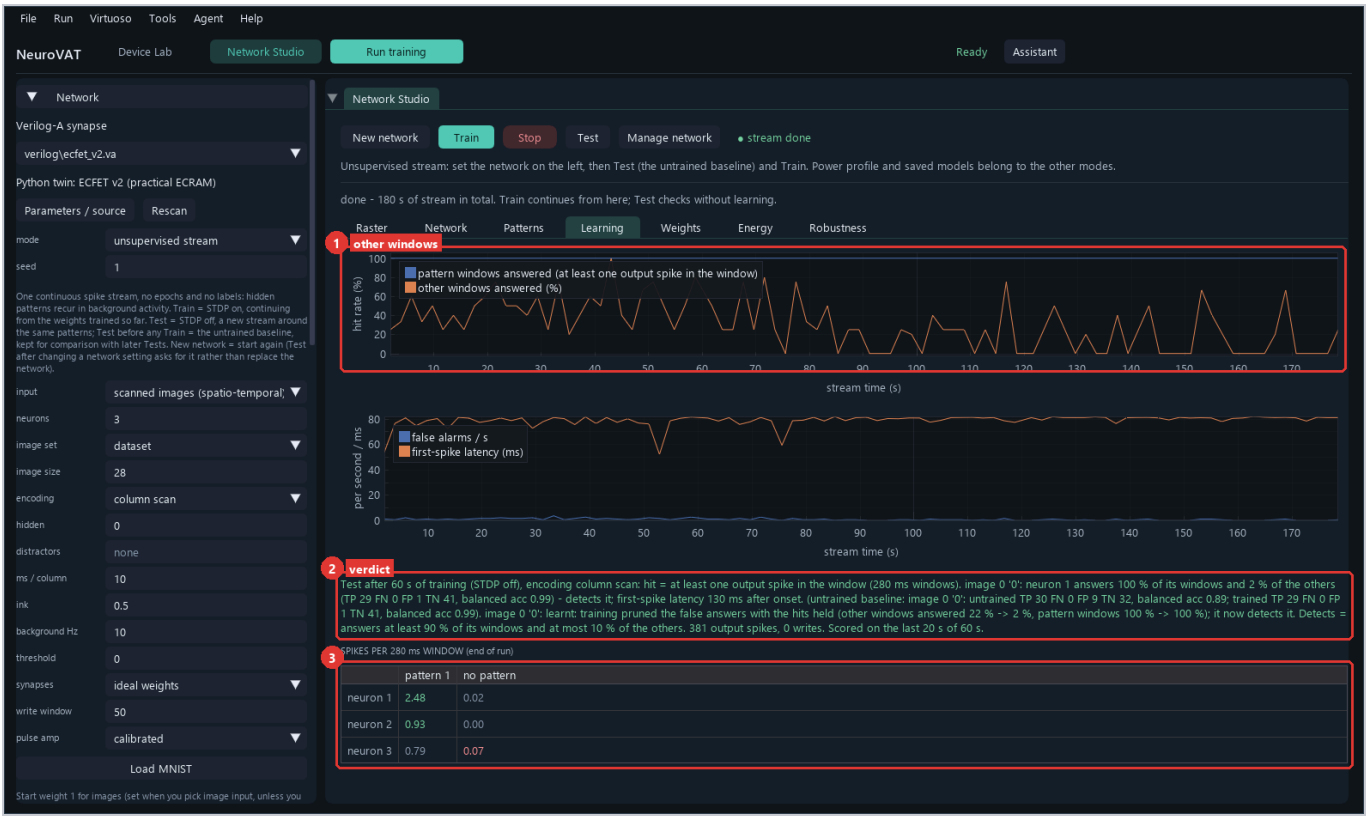


Figure: The Learning tab after Test, Train, Test (180 s of stream in total).

Open the **Learning** tab after the run.

1. **"other windows answered (%)":** the orange line is the learning signal for image input, plotted every 2 s of stream. In this run it swings between about 20 and 100 % during the baseline (0-60 s), and stays mostly between 0 and 40 % after the first minute of training. It does not reach zero, because it counts a spike of any of the three neurons, and neurons 2 and 3 still answer some empty windows. The blue "pattern windows answered" line stays at 100 %. It runs along the top edge of the plot.
2. **Verdict:** it now holds both runs. "(untrained baseline: image 0 '0': untrained TP 30 FN 0 FP 9 TN 32, balanced acc 0.89; trained TP 29 FN 0 FP 1 TN 41, balanced acc 0.99). image 0 '0': learnt: training pruned the false answers with the hits held (other windows answered 22 % -> 2 %, pattern windows 100 % -> 100 %); it now detects it."
3. **Table:** neuron 1 now fires 2.48 spikes per image window and 0.02 per empty window.

What we measured

run (scored on the last 20 s of 60 s)	image windows answered	empty windows answered	balanced acc.	verdict
untrained baseline (Test before Train)	30 of 30 (100 %)	9 of 41 (22 %)	0.89	does NOT detect it
Test after 60 s of training	29 of 29 (100 %)	1 of 42 (2 %)	0.99	learnt; now detects it

Settings: MNIST image 0, size 28, column scan, 3 neurons, seed 1, share 0.4, random spacing, noise and background 10 Hz, jitter 1 ms, start weight 1, ideal synapses, auto threshold 14.4. These are neuron 1's counts. The first-spike latency was 96 ms untrained and 130 ms trained. The output spikes of the 60 s runs fell from

538 (untrained) to 381 (trained Test). Each 60 s run took about 1 to 1.5 minutes of wall time on our laptop, with other app instances running at the same time. In a second test on a fresh copy, each 60 s run took about 17 to 18 s. Wall times depend on your machine and on what else is running.

> **Note.** For the column scan on the same image at 10 Hz, the README's "Stream encodings" section reports 25-27 of 54 false windows untrained and 2 of 54 after 60 s of training (two seeds, even spacing). With random spacing at share 0.4 it reports 27/64 and 5/61. Those counts are over the whole 30 s test run, so they use another denominator. Do not compare them directly with the table above.

5.4 The other views

Raster replay

After a run, the Raster tab keeps the whole run and can replay it.



Figure: Replaying the last Test, at t = 45 s into that run.

1. **Play / Pause:** plays the run from the slider position. While it plays, the button reads Pause.
2. **Speed:** `0.25x`, `1x` or `4x` of real stream time.
3. **Slider:** the time in the last run, in seconds, at the right edge of the window. Drag it to jump.
4. **window s:** how many seconds of stream are shown. `0` = auto: six presentations' worth for image input, 1.5 s for Poisson input.
5. **Status line:** the run time, the stream time, the image windows in view and the output spikes in view, per neuron.

Network tab

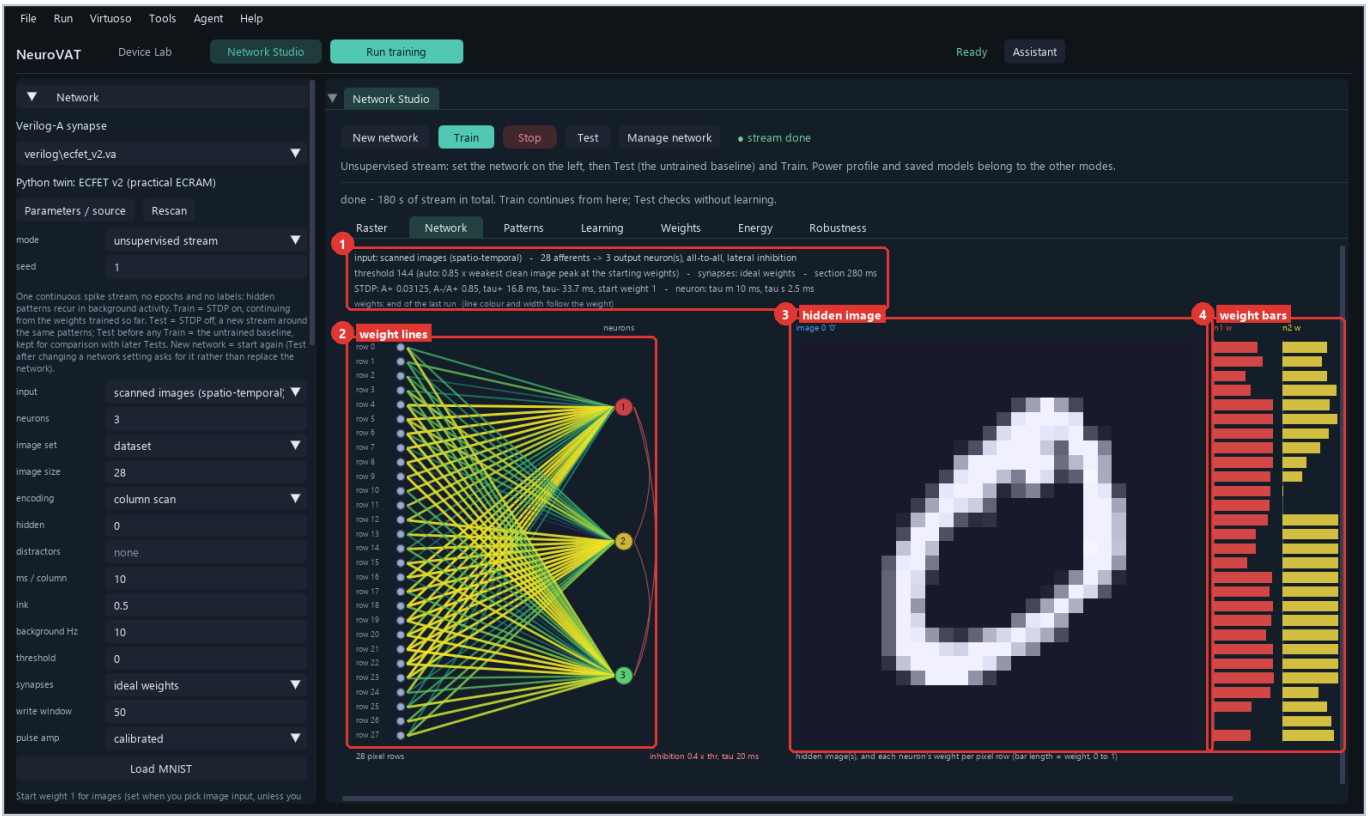


Figure: The Network tab after training: 28 inputs (one per pixel row), 3 neurons, the image and each neuron's weights.

1. **Header:** the input count, the neurons, the threshold actually used (14.4, auto), the synapse kind, the STDP and neuron constants, and the section length.
2. **Weight lines:** one line per synapse. The colour and width follow the weight: bright yellow and thick = near 1, dark and thin = near 0. The curved red lines between the neurons are the lateral inhibition (0.4 x threshold, tau 20 ms).
3. **Hidden image:** the image that the stream hides.
4. **Weight bars:** one bar per pixel row for each neuron (n1, n2, ...), with length = weight from 0 to 1. Scroll right to see neuron 3. A short bar is a synapse that STDP lowered.

Patterns tab

The Patterns tab (shown in the first figure) draws each hidden image and its clean spike pattern, without noise or jitter. Above it, one line explains the encoding with this network's numbers. For the column scan: "one afferent per pixel row; pixel (r, c) at or above the ink threshold fires afferent r at c x 10 ms. Section 280 ms."

Learning tab

The Learning tab is described in Step 3 above. The upper plot shows the image windows answered and the other windows answered, in percent. The lower plot shows the false alarms per second and the first-spike latency in ms. Below them are the verdict and the spikes-per-window table. For image input, the line to watch is **"other windows answered (%)"**. Training that works pushes it down while the image windows stay answered.

Weights tab

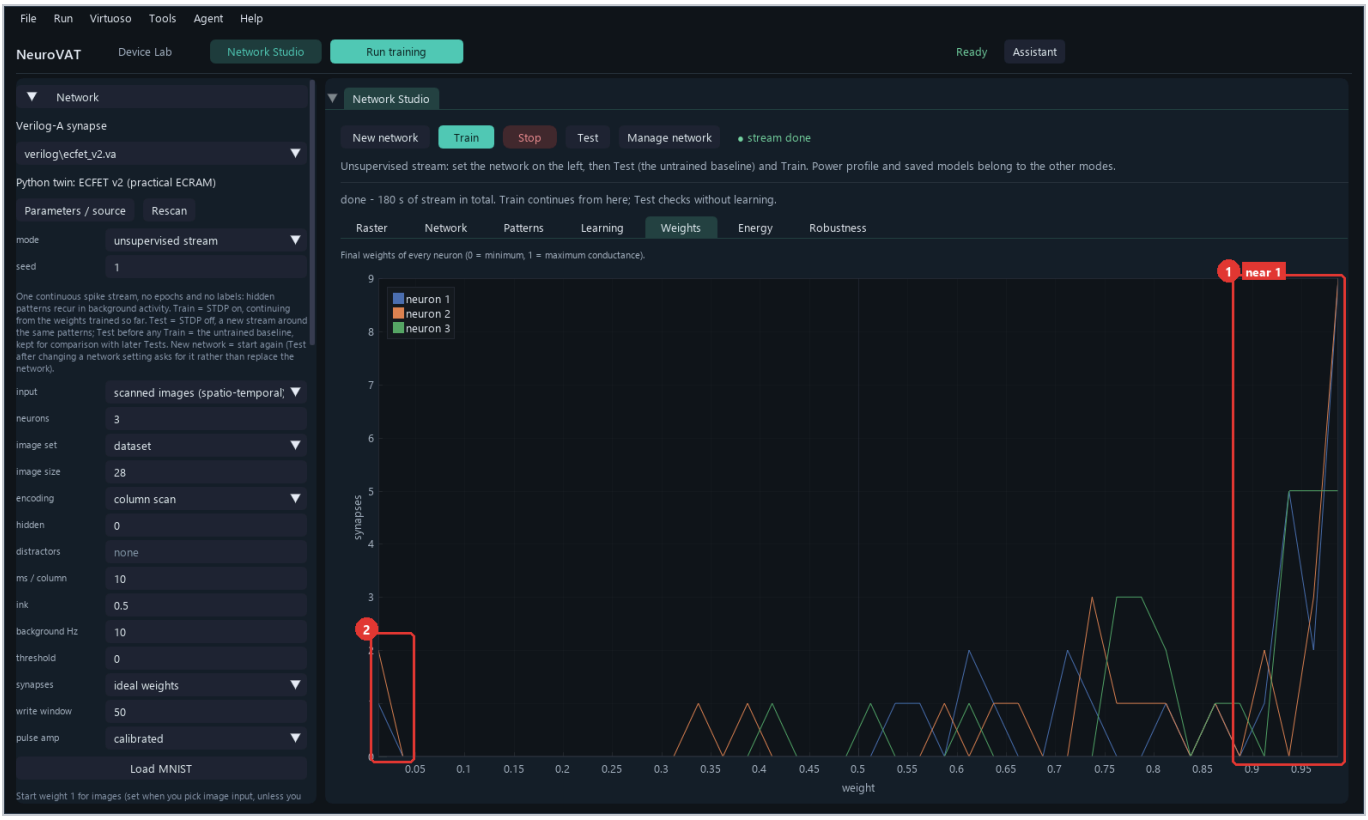


Figure: Histogram of the final weights of each neuron.

1. **Near 1:** the largest group of each neuron's 28 synapses is still at or close to the start weight of 1 (up to 9 per neuron). The rest are spread down to about 0.3.
2. **Near 0:** a few synapses were pruned to 0. Because training started at 1, STDP could only lower the weights.

Energy tab

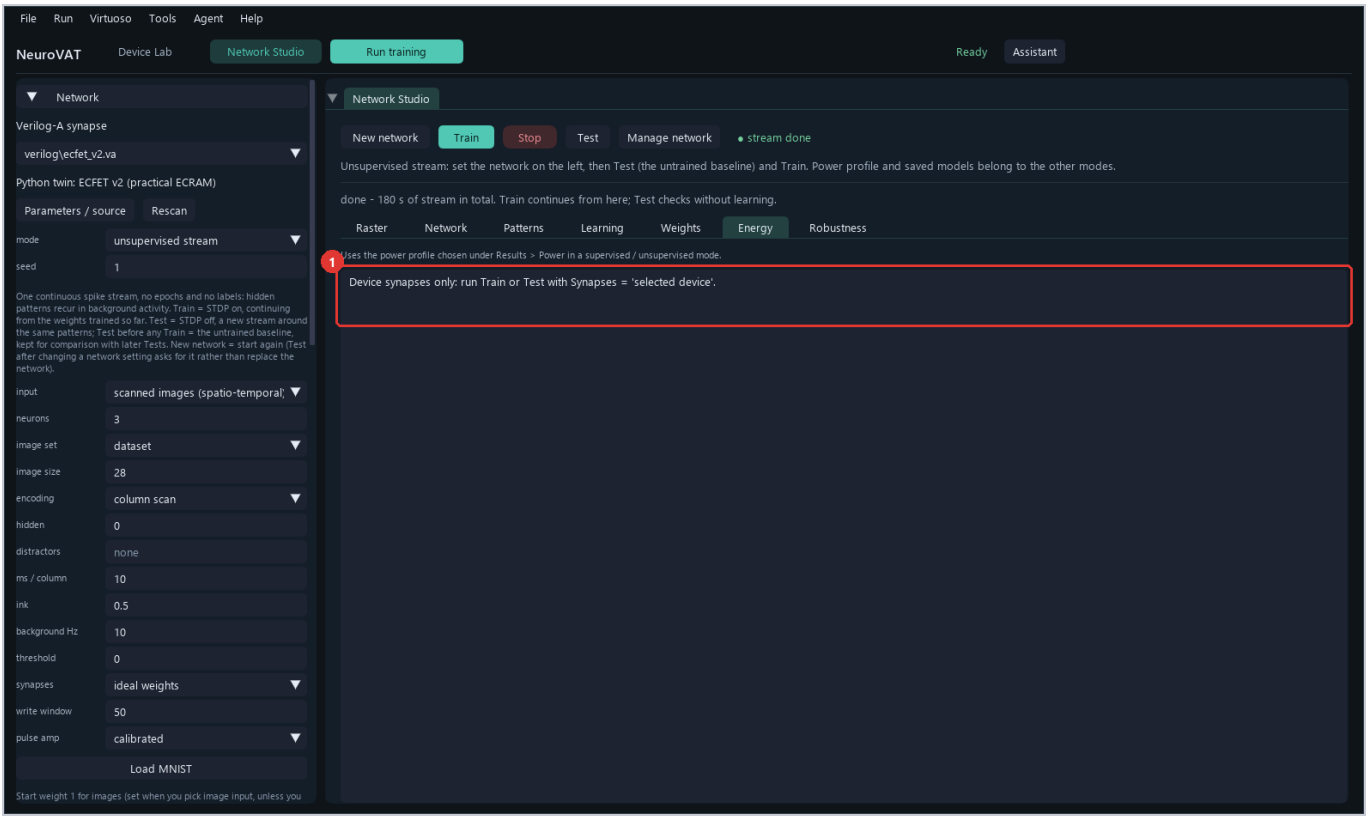


Figure: The Energy tab with ideal synapses.

1. **Device synapses only.** The report appears after a Train or Test with synapses = `selected device`. It uses the power profile chosen under Results > Power in a supervised or unsupervised mode. With ideal weights there is no energy to report.

5.5 Image encodings

How the image is turned into spikes decides what STDP can learn. Pick the code in the **encoding** combo.

code	one input line per	inputs at size 28	how a pixel fires	section
column scan	pixel row	28	pixel (r, c) fires once at c x bin ms: a left-to-right sweep	28 x 10 = 280 ms
row scan	pixel column	28	pixel (r, c) fires once at r x bin ms: a top-to-bottom sweep	280 ms
latency	pixel	784	fires once; the brighter the pixel, the earlier it fires within a 50 ms window, after a 15 ms lead	50 + 2 x 15 = 80 ms
frozen Poisson	pixel	784	Poisson spikes at 100 Hz x brightness over the window, drawn once and repeated unchanged (about 5 per line)	80 ms

When you pick a code, the app changes some defaults, unless you have already edited those controls. The app logs each change.

- **The pixel codes (latency, frozen Poisson):** noise Hz and background Hz become **3**. The reason: there are size x size input lines instead of size, so the same rate per line is far more noise per window.
- **Frozen Poisson:** the start weight also becomes **0.5**. The README reports that from weight 1 its hits collapsed after training. Each line fires about 5 times per window, so STDP depressed the image's own lines below the threshold.
- **The scans:** noise and background go back to 10 Hz.

We checked this in the app: after picking frozen Poisson, the controls read noise 3, background 3, start weight 0.5, with 784 inputs and an 80 ms section. Latency gave noise 3, background 3, start weight 1, 784 inputs. Row scan kept 10 / 10 / 1 with 28 inputs.

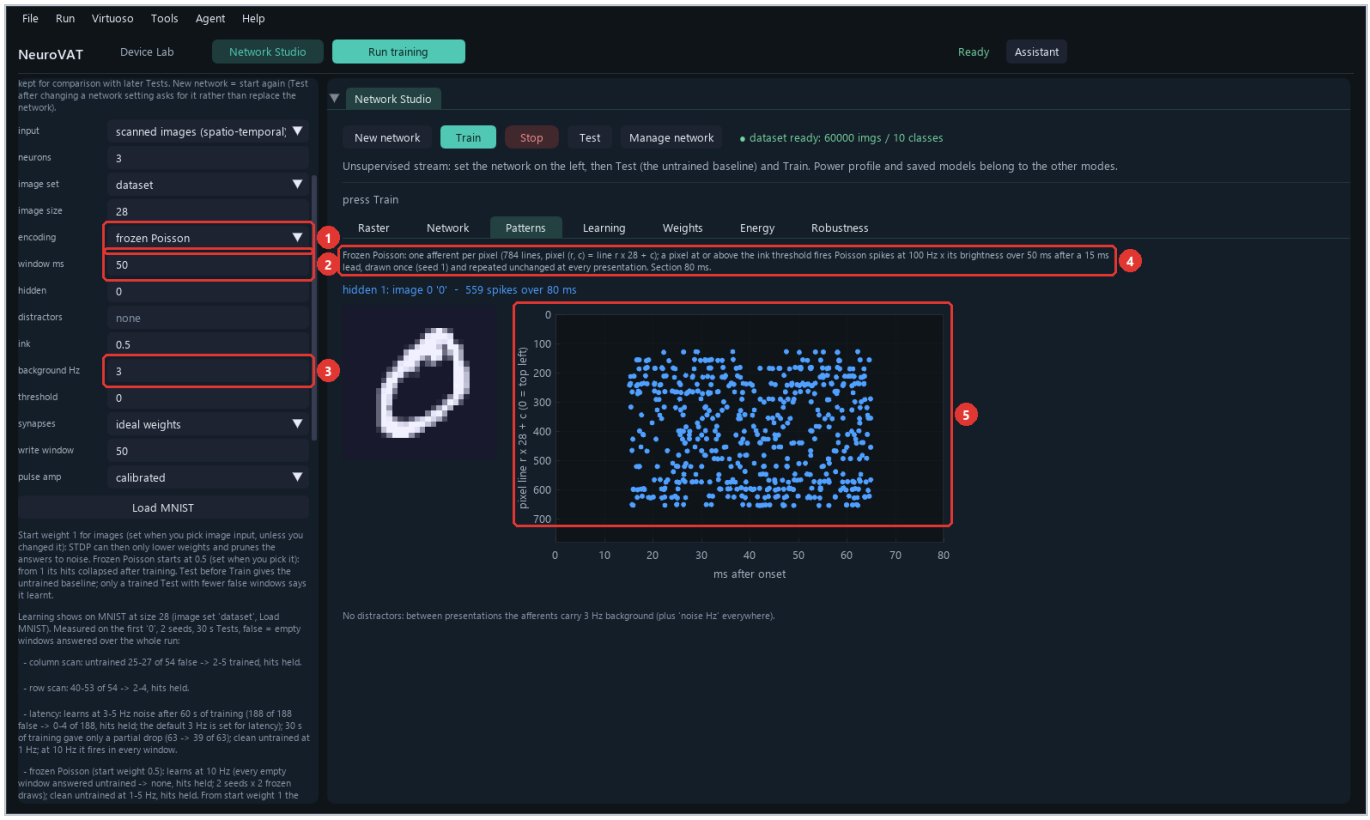


Figure: A pixel code (frozen Poisson). The left panel shows the code-specific rows. The Patterns tab shows one input line per pixel.

1. **encoding:** `frozen Poisson` (or `latency`).
2. **window ms** (default 50): it replaces ms / column for the pixel codes. It is the span the spikes are spread over. The section is the window plus 2 x 15 ms of silence.
3. **background Hz:** it was set to 3 automatically, and so was noise Hz further down.
4. **Explainer:** how this code turns the image into spikes, with this network's numbers (100 Hz x brightness, seed 1, section 80 ms).
5. **Spike pattern:** the y axis is now the pixel line r x 28 + c, from 0 to 783. This frozen draw of the digit "0" has 559 spikes in 80 ms.

Run latency: Test, Train 60 s, Test

1. Set **encoding** to `latency`. Check that noise Hz and background Hz now read 3.
2. Click **New network**. The code is part of the network, so Test would otherwise refuse.
3. Click **Test**, then **Train**, then **Test**, with seconds = 60 each.



Figure: Latency code, Test after training. The raster draws the inked pixels plus a sample of silent ones.

1. **noise Hz = 3:** the pixel-code default.
2. **Y axis:** "pixel (r, c): 149 of 784 shown, 125 inked + 24 silent". The raster cannot draw all 784 lines, so it draws every line the image inks plus a sample of silent lines.
3. **Image window:** 80 ms. The bright pixels fire early in it.
4. **Potential trace:** each image window drives one neuron over the threshold once, shortly after onset.

Measured (MNIST image 0, size 28, latency, 3 neurons, seed 1, share 0.4, random spacing, noise and background 3 Hz, jitter 1 ms, start weight 1, auto threshold 58.4; scored on the last 20 s of each 60 s run). The best neuron was neuron 3:

run	image windows answered	empty windows answered	balanced acc.
untrained baseline	20 of 94 (21 %)	59 of 156 (38 %)	0.42
Test after 60 s of training	102 of 102 (100 %)	2 of 148 (1 %)	0.99

The verdict was "learnt: training pruned the false answers with the hits held (other windows answered 38 % -> 1 %, pattern windows 21 % -> 100 %); it now detects it". The training run made 655,962 weight writes. The 21 % is neuron 3's own share, because the verdict scores each neuron on its own. Untrained, no single neuron detected the image. Here training both raised the hits and cut the false windows.

The Network tab for a pixel code

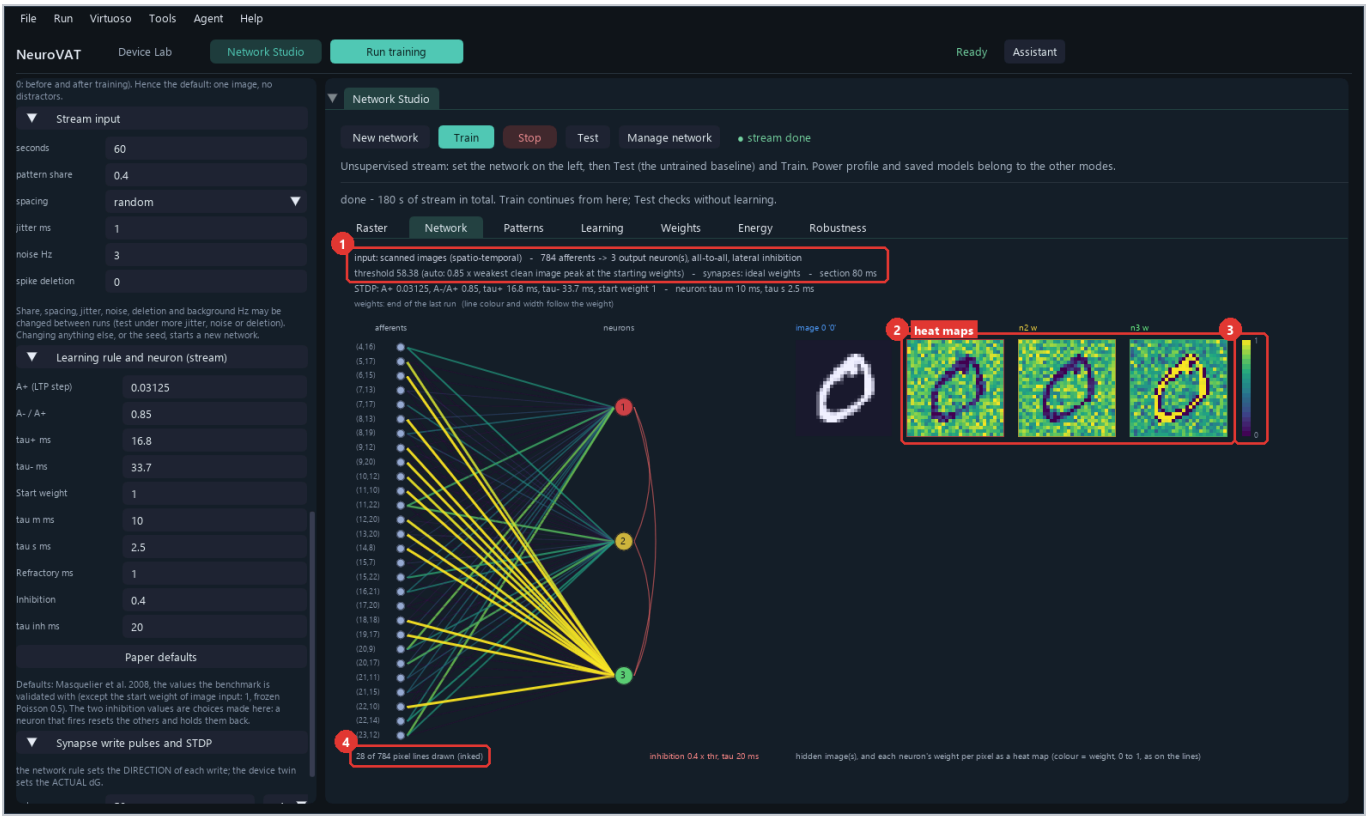


Figure: The Network tab after latency training. Each neuron's 784 weights are drawn as a 28 x 28 heat map.

1. **Header:** 784 afferents, threshold 58.38 (auto), section 80 ms.
2. **Heat maps:** one per neuron (n1 w, n2 w, n3 w). Each pixel is that neuron's weight for that pixel's input line. Neuron 3, the one that learnt the "0", kept high weights (yellow) on the digit's strokes. The others show the digit as a dark outline.
3. **Colour bar:** weight from 0 (dark) to 1 (yellow), the same scale as the lines.
4. **"28 of 784 pixel lines drawn (inked)":** the line diagram on the left draws only a sample of inked lines. The heat maps hold every weight.

Sparse images tolerate less noise

How much noise an image can stand depends on how much ink it has. The README reports, for frozen Poisson, that the sparse MNIST "1" (66 inked pixels) fires in every window at 10 Hz noise, while the denser "0" learns there. That is why both pixel codes default to 3 Hz. At 3 Hz the dense "0" is already clean untrained with frozen Poisson, so there is nothing to learn. To see pruning on it, raise noise Hz and background Hz to 10. The README also reports that latency on the "0" learns at 3-5 Hz after 60 s of training, but fires in every window at 10 Hz. 30 s of training gave only a partial drop.

5.6 Robustness tab

After training, the **Robustness** tab tests the network while one input factor at a time departs from its trained value: spike-time jitter, missing (deleted) pattern spikes, and extra (noise) spikes. The README gives the levels of the 2008 paper's robustness panels as jitter 1 / 3 ms, 10 / 30 % deletion and 5 / 26 Hz. The app's default list is jitter 1, 3, 5 ms, missing spikes 0, 10, 30, 50 % and extra spikes 5, 26 Hz, plus the value the network was

trained with. All levels run on one fixed, keyed test stream, so the trained and untrained networks see the same spikes, and a repeated sweep gives identical numbers. Your network, its weights and its session are left unchanged.

Recall sweep

1. Train first. Without a trained network the sweep refuses with "no trained network: press Train first".
2. Do not change a network setting after training. A changed setting also makes the sweep refuse.
3. Open the **Robustness** tab and set the controls as below. Then click **Run sweep**.

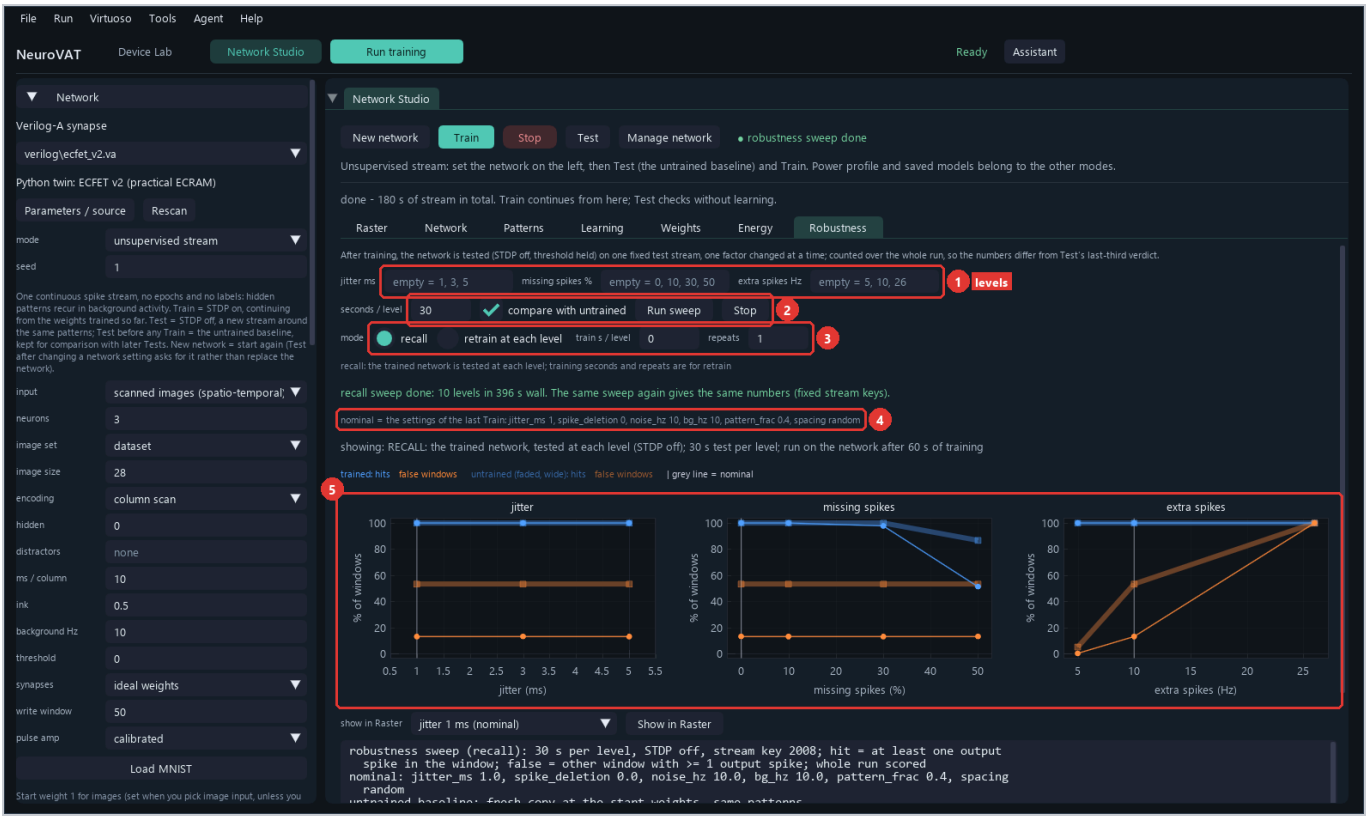


Figure: A recall sweep of the column-scan network trained above, 30 s per level.

1. **Levels:** leave all three boxes empty for the default levels: jitter 1, 3, 5 ms; missing spikes 0, 10, 30, 50 %; extra spikes 5, 10, 26 Hz (the trained values 1 ms, 0 % and 10 Hz included). You can type your own comma-separated list. Extra spikes are an **absolute** noise rate, not added to the trained value.
2. **seconds / level, compare with untrained, Run sweep:** leave 30 s per level (from 1 to 600). Leave **compare with untrained** on: every level then also runs on an untrained copy (same patterns, start weights), which shows what training bought. It works with ideal synapses only. Then click **Run sweep**. **Stop** ends it early and keeps a partial result. Our 10 levels x 2 runs took 396 s of wall time, with other app instances running. In a second test on a fresh copy, each level took about 14 s. Wall times depend on your machine.
3. **mode:** leave `recall`. It tests the trained network at each level, with STDP off and the threshold held. **train s / level** and **repeats** on the same row are used by retrain only.
4. **Nominal line:** the settings the network was trained with (from the last Train). Here: jitter 1 ms, deletion 0, noise 10 Hz, background 10 Hz, share 0.4, random spacing.
5. **The three plots,** one per factor (jitter, missing spikes, extra spikes). Each plot shows, in % of windows:
   - the **trained hits** (thin blue) and the **trained false windows** (thin orange);
   - the same for the **untrained** copy (wide, faded lines with square markers);
   - the **nominal level** (the grey vertical line).

   A good trained network keeps the blue line at the top and the orange line well below the faded orange line.

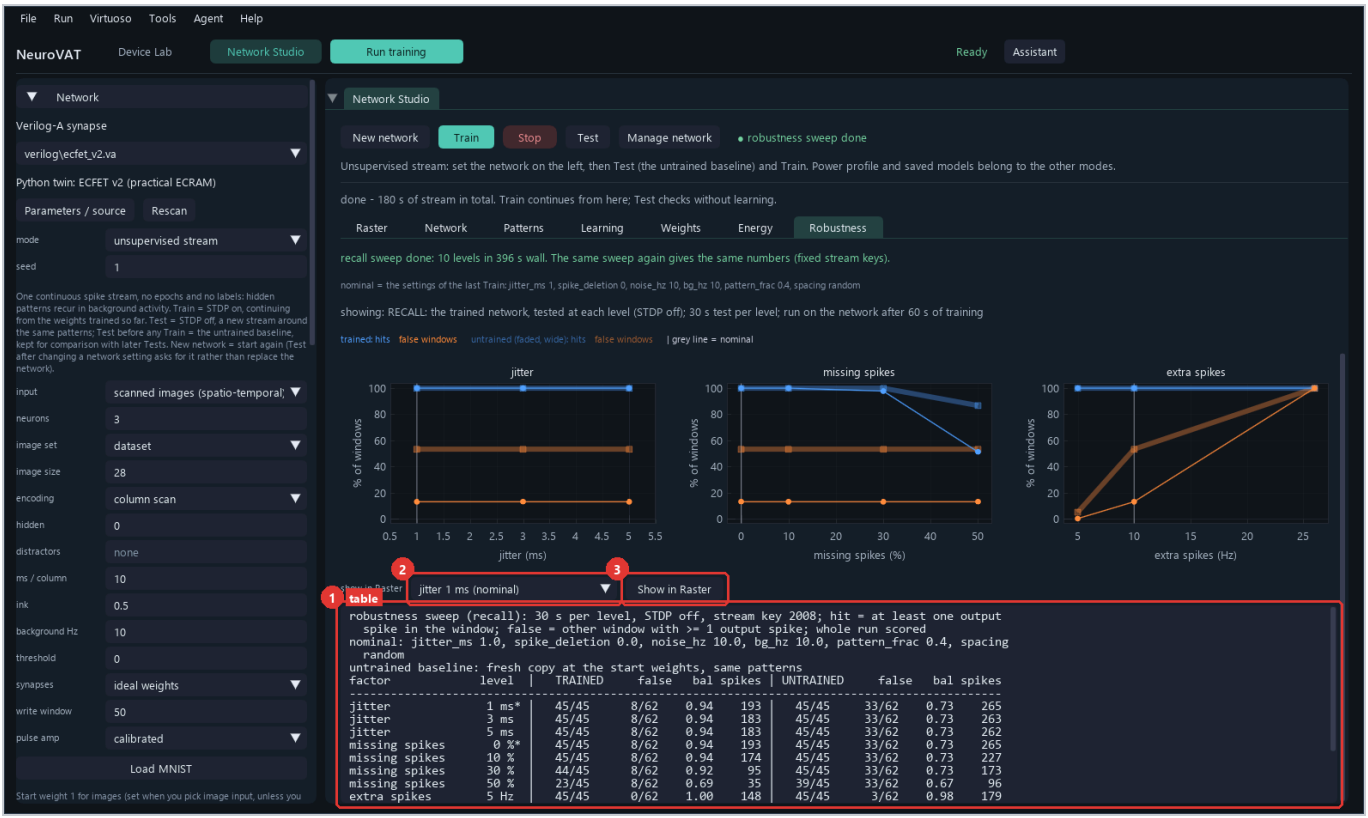


Figure: The sweep table below the plots, and Show in Raster.

1. **Table:** one row per level. For both the trained and the untrained network it gives hits / image windows, false / other windows, balanced accuracy and output spikes. The asterisk marks the nominal level.
2. **Show in Raster list:** pick a swept level.
3. **Show in Raster:** it runs that one keyed level again with STDP off and shows it in the Raster tab. These are the same stream and spikes as in the sweep. It replaces the Raster's replay of your last Train or Test.

Our recall sweep (the column-scan network above after 60 s of training; 30 s per level; 45 image windows and 62 other windows per run, counted over the whole run):

factor	level	trained hits	trained false	untrained hits	untrained false
jitter	1* / 3 / 5 ms	45 / 45 / 45	8 / 8 / 8	45 / 45 / 45	33 / 33 / 33
missing spikes	0* / 10 / 30 / 50 %	45 / 45 / 44 / 23	8 at each	45 / 45 / 45 / 39	33 at each
extra spikes	5 / 10* / 26 Hz	45 / 45 / 45	0 / 8 / 62	45 / 45 / 45	3 / 33 / 62

* = nominal. These numbers match the recall column of the README's own measurement with the same settings (3 neurons, 60 s of training, 30 s per level).

- **Jitter** up to 5 ms changes nothing. The README reports the same: with 10 ms per column, jitter up to 5 ms does not change any window's answer.
- **Missing spikes:** the trained network loses hits faster than the untrained one (23 against 39 of 45 at 50 %), because STDP pruned some of its weights. Its false windows stay at 8 of 62, against 33 untrained.
- **Extra spikes:** at 26 Hz both networks answer every window. The threshold is held, and there are only 28 inputs. The paper's neuron had 2000.

> **Tip.** The nominal level (the value the network was trained with) is part of every factor's list, so it runs once in each factor: 3 times in the default sweep (the starred rows above). To save time, type only the levels you need into the three boxes, the same way as in "Retrain at each level", step 2 below. An empty box means the default levels, so give a factor you do not need just its trained value.



Figure: Show in Raster at "missing spikes 50 %": the trained network on the sweep's stream with half of the image's spikes deleted.

1. **Status line:** this is a replay of the 30 s robustness run, not of your last Test.
2. **A thinned image window:** half the blue spikes are missing, but the neuron still answers this one.
3. **A window with no answer:** no triangle above it. This is one of the lost hits (23 of 45 were kept).

Retrain at each level

Recall asks: *how does the network I trained cope with worse input?* Retrain asks a different question: *what does a network learn if it is trained under that input?* For each level, a fresh copy is made (start weights, the same hidden patterns, the threshold held). The copy is trained with STDP on at that level, on one keyed training stream that is the same for every level. It is then tested at that level with STDP off, on the sweep's test stream. Your own network is never run. The nominal row is "retrained at nominal": an independent training on another stream, not the network you trained.

1. Set **mode** to `retrain at each level`.
2. Type the levels. To save time we used jitter `1`, missing spikes `0, 50` and extra spikes `10`. An empty box means the default levels, not "skip this factor", so give the two factors you do not need just their trained value.
3. Set **train s / level** to `30`. 0 means the seconds your network was trained (60 here).

4. Leave **repeats** at 1. With more repeats, each level is trained several times, and the plots show the mean with a min-max band.
5. Click **Run sweep**. Our 4 levels took 106 s of wall time.

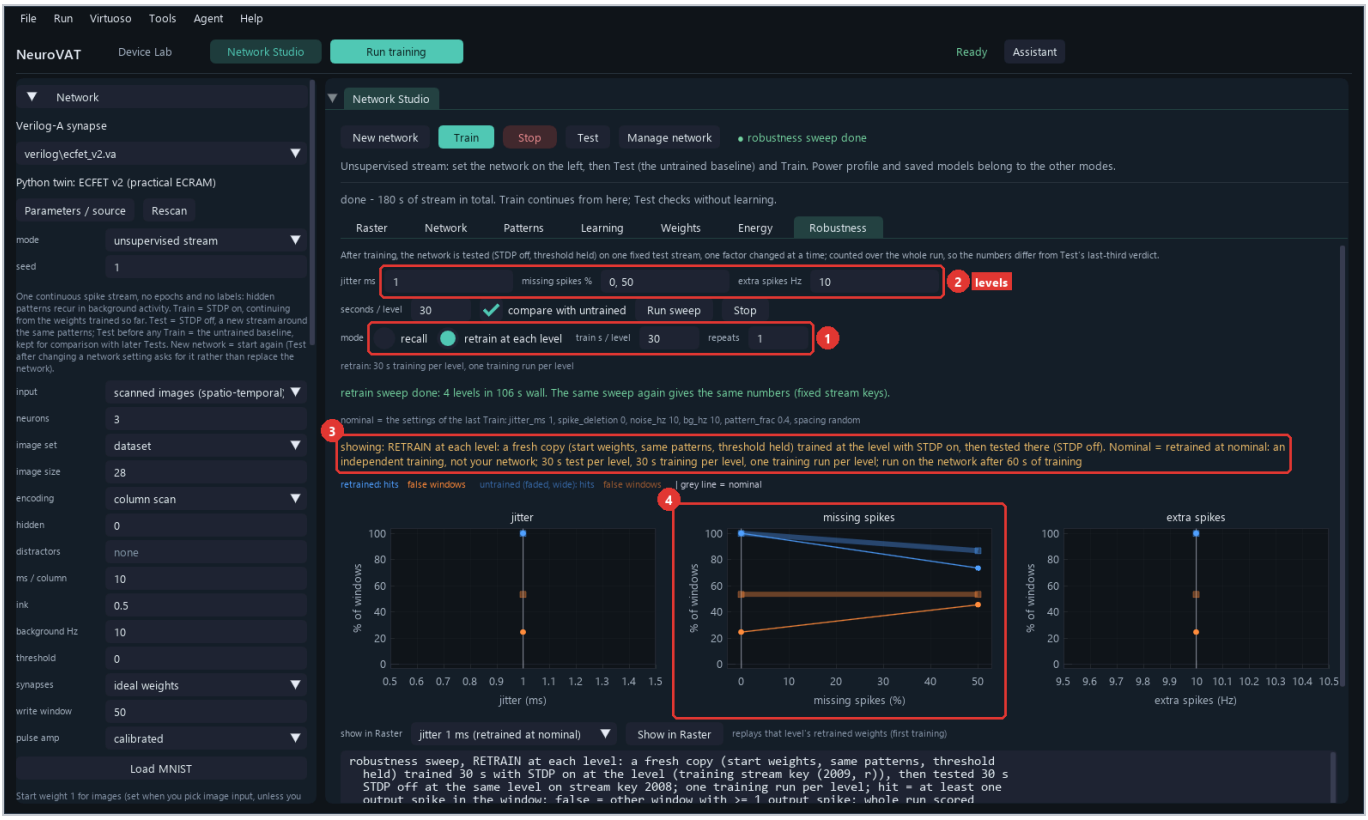


Figure: A retrain sweep at missing spikes 0 and 50 %, with 30 s of training per level.

1. **mode row:** `retrain at each level`, **train s / level** 30, **repeats** 1. The last result of each mode is kept, and the radio switches between them.
2. **Levels:** jitter 1, missing spikes 0, 50, extra spikes 10.
3. **"showing" line:** it states the mode, the training and test seconds, and which network the sweep ran on.
4. **Missing-spikes plot:** the retrained hits (thin blue) and false windows (thin orange) at 0 and 50 %, against the untrained copy (faded).

Measured (30 s of training and a 30 s test per level, one training run per level; hits of 45 image windows, false of 62 other windows):

level	recall (your network): hits / false	retrain (30 s): hits / false	untrained: hits / false
nominal (jitter 1 ms, 0 % missing, 10 Hz)	45 / 8	45 / 15	45 / 33
missing spikes 50 %	23 / 8	33 / 28	39 / 33

A network trained at 50 % missing spikes keeps more hits than the recall network (33 against 23), but answers far more empty windows (28 against 8). The nominal retrained row (15 false) differs from recall (8 false) because it is another network, trained for 30 s instead of 60 s on another stream. With one training run per level, a difference of a few windows is not a trend. Use repeats to see the spread.

> **Warning.** Retrain is refused for device synapses: a device copy cannot be made. Use recall for them. There is also no untrained comparison for device synapses.

Why the sweep numbers differ from Test

Test and Train score only the **last third** of each run. That is 20 s of a 60 s run, and it allows for burn-in while STDP changes the weights. The sweep runs with STDP off, so there is no burn-in, and it counts **every window of the whole run**. It also uses its own fixed test stream. There is a second difference. The Test verdict scores the one neuron that answers the image best. The sweep counts a window as answered when *any* neuron fires in it (the table header says "false = other window with >= 1 output spike"). So the trained network above shows 1 false window of 42 in Test, but 8 of 62 in the sweep's nominal row. Part of that comes from neurons 2 and 3. Both numbers are correct, but they answer different questions. Do not compare the two directly.

5.7 Poisson-pattern input (Masquelier 2008)

Set **input** to `Poisson patterns (Masquelier)` to get the input of the paper. The afferents fire continuously as variable-rate Poisson trains. A pattern is a 50 ms section of frozen activity on a share of the afferents (**in pattern**, default 0.5), and it recurs at random times. The defaults are 1000 afferents, 3 neurons, 3 hidden patterns, start weight 0.475 (the paper value) and an auto threshold of 0.25 x afferents. The hit rule is stricter than for images: exactly one spike of some neuron in the window. "Learnt" means that a pattern is answered in at least 90 % of its windows by a neuron that stays quiet elsewhere. The README reports that at paper scale (2000 afferents, 150 s) 6 of 10 seeds learnt all three patterns.



Figure: Poisson-pattern input after a short 20 s Train (defaults otherwise). Each hidden pattern has its own colour.

1. **input:** `Poisson patterns (Masquelier)`.
2. **afferents** (default 1000): the raster shows 60 of them.
3. **patterns** (default 3).
4. **in pattern** (default 0.5): the share of afferents that take part in each pattern.
5. **Raster:** the shaded windows of pattern 1 (blue), pattern 2 (orange) and pattern 3 (green). The triangles are output spikes. After 20 s, most of them fall in pattern windows.

In this 20 s demonstration run (seed 1, scored on the last 7 s), neuron 1 answered pattern 1 best (TP 21 FN 1 FP 1 TN 110). Neurons 2 and 3 both answered pattern 3 best, in 8 and 4 of its 18 windows, and no neuron answered pattern 2 best. The verdict said "0 of 3 pattern(s) learnt": the full criterion was met for no pattern in this short run. 20 s is far shorter than the paper's training, so treat this only as a look at the views.

5.8 Device synapses in stream mode

Set **synapses** to `selected device` to store each weight as the conductance of the Verilog-A synapse picked at the top of the panel. Each STDP update then becomes a write pulse, and the updates within one **write window** are merged into one pulse per synapse. With **pulse amp** = `calibrated`, the app searches for the amplitudes at which one pulse moves the device by one STDP step. For image input the start weight is 1, at the top of the range, where one direction cannot move. So the pulse is calibrated at weight 0.5 instead. The README reports this for ECFET v2 (`verilog/ecfet_v2.va` twin, calibrated pulses, seed 1, one neuron, MNIST "0", noise 10 Hz):

- At weight 0.5, one pulse makes one STDP step in each direction. At weight 1.0, the potentiation pulse makes no step and the depression pulse makes about one.

- Untrained: 17/17 hits and 8/18 false windows. After 60 s of training: 17/17 hits and 1/18 false.
- The weights ended between 0.50 and 1.00 (15 of 28 above 0.9). This was one run only.

The README states that device runs remain experiments, because larger weight requests are scaled linearly from the calibration point, which a real device does not follow exactly. The Energy tab reports the energy of device runs, and the Robustness tab refuses retrain for device synapses.

> **Warning.** Things that can go wrong in stream mode:
>
> - **Test says "settings changed since this network was built".** You changed a network setting (encoding, size, threshold, neurons, seed, start weight ...) after building or training. Press **New network**, or restore the setting.
> - **The verdict says "Not comparable: the baseline ran with other stream settings".** You changed share, spacing, jitter, noise, deletion or background between the baseline and the Test. Restore them to compare.
> - **"No untrained baseline of this network".** You trained before testing. Press New network, then Test before Train.
> - **Load MNIST is greyed out.** Set image set to `dataset` first.
> - **Show in Raster replaces your Test replay.** Run Test again if you want the Test replay back.

CHAPTER 6

The AI assistant and automation

NeuroVAT has three ways to work with it from outside the normal buttons and fields:

- **The Assistant**: a chat panel with an embedded LLM agent (Claude, OpenAI or Gemini). It can explain and review your `.va` files, design stimulus patterns and drive the Trainer. If you allow it, it can also edit files and run commands.
- **The control bridge**: a small command-line client, `tools/nvat_ctl.py`. Scripts (or an outside agent) use it to drive the running app.
- **The Virtuoso skillbridge**: a link over SSH to a live Cadence Virtuoso session.

This chapter shows all three. Read the section on file edits and Autonomous mode before you switch on any option that lets the agent change files.

6.1 Open the Assistant panel

The Assistant is a drawer on the right of the window. It is hidden when the app starts.

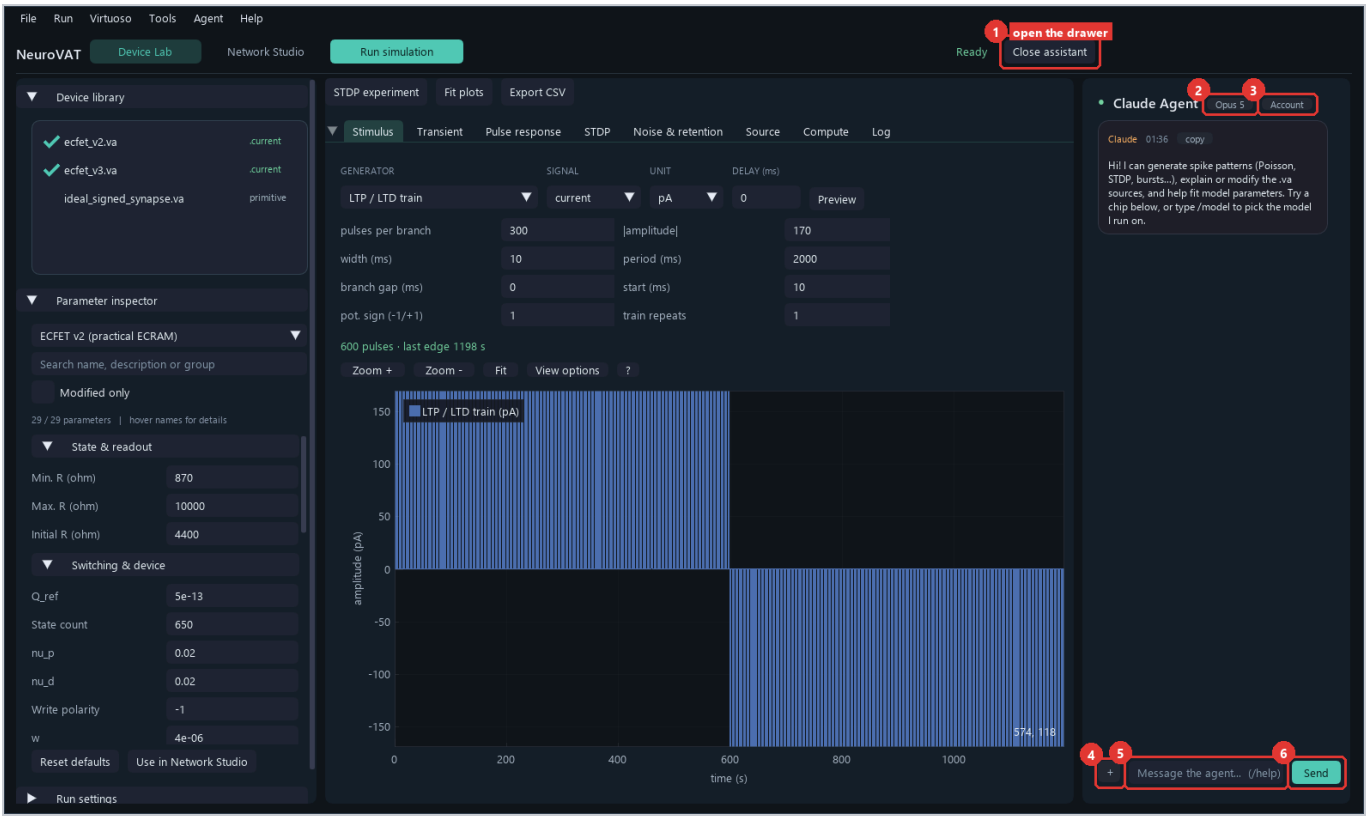


Figure: The Assistant drawer, open beside Device Lab.

1. **Assistant / Close assistant**: opens and closes the drawer. The rest of the window shrinks to make room for it, and no settings are lost.
2. **Model badge** (here "Opus 5"): shows the model the agent uses. Click it to pick another model or provider (see the next section).
3. **Account**: opens the sign-in dialog. Use it to log in or to paste a key for this session only.
4. **+** (attach): adds files to your next message, such as a paper PDF, a CSV of measured data or a plot image. Attached files are *untrusted input* (see the section on Autonomous mode).
5. **Message box**: type a request, or a slash command such as `/help`, `/model` or `/provider`. Press Enter to send.
6. **Send**: sends the message. While the agent works, this button becomes **Stop**, which cancels the run.

The coloured dot left of the title shows the backend. It is green when one is available and red when it is not (for example, "Gemini Agent" with no Gemini CLI installed).

6.2 Choose the provider, the model and the account

Providers

The agent can run on three providers. Each one first tries a command-line tool; for Claude and OpenAI, a key-based fallback with fewer features is also available.

Provider	Full agent (files, shell)	Fallback	Switch with
Claude (default)	the `claude` CLI (Claude Code), logged in	Anthropic SDK with `ANTHROPIC_API_KEY`: chat only, no file tools	`/provider claude`

Provider	Full agent (files, shell)	Fallback	Switch with
OpenAI	the `codex` CLI	OpenAI SDK with `OPENAI_API_KEY`: chat, patterns and GUI actions only	`/provider openai`
Gemini	the `gemini` CLI (`npm i -g @google/gemini-cli`)	experimental Antigravity link	`/provider gemini`

Keys come from environment variables or from the Account dialog. They are never stored in the workspace. The fallbacks have no file tools, so the agent's edit-and-verify loop needs the Claude CLI or the codex CLI.

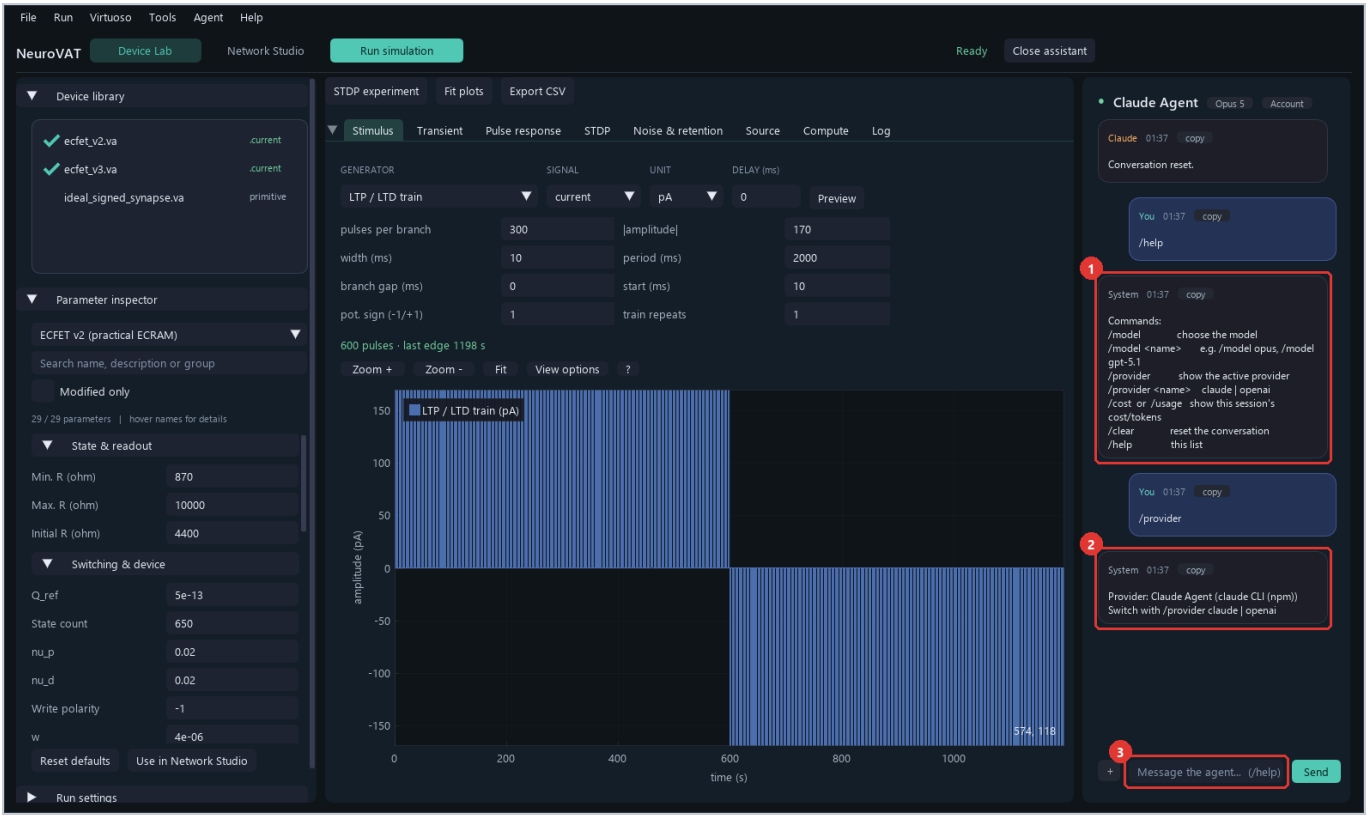


Figure: /help lists the chat commands, and /provider names the active provider and its backend.

1. **/help** lists the commands: `/model`, `/provider`, `/cost` (session cost and tokens) and `/clear` (reset the conversation). These commands are handled by the app and are not sent to any model. The help text lists only "claude | openai", but `/provider gemini` also works.
2. **/provider** shows which provider is active and which backend it found, here "claude CLI (npm)".
3. **Message box**: type the commands here.

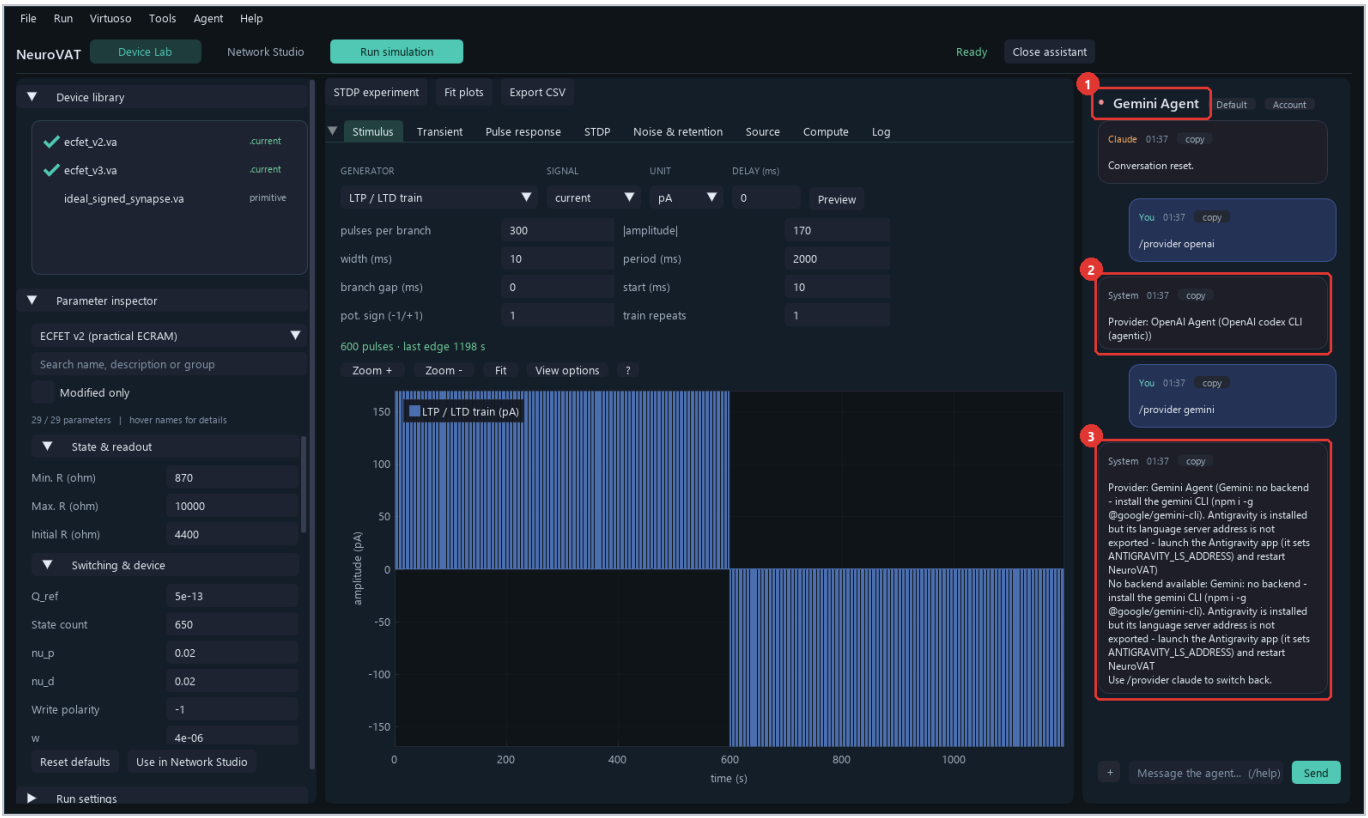


Figure: Switching provider. OpenAI found the codex CLI. Gemini found no backend and says how to install one.

1. **Title and dot**: the title follows the provider ("Gemini Agent"). The dot turns red when that provider has no backend.
2. **OpenAI**: "OpenAI codex CLI (agentic)" means the full agent is available on this machine.
3. **Gemini**: when no backend is available, the reply says what to install, and you can type `/provider claude` to switch back. Switching provider also resets the conversation.

Models

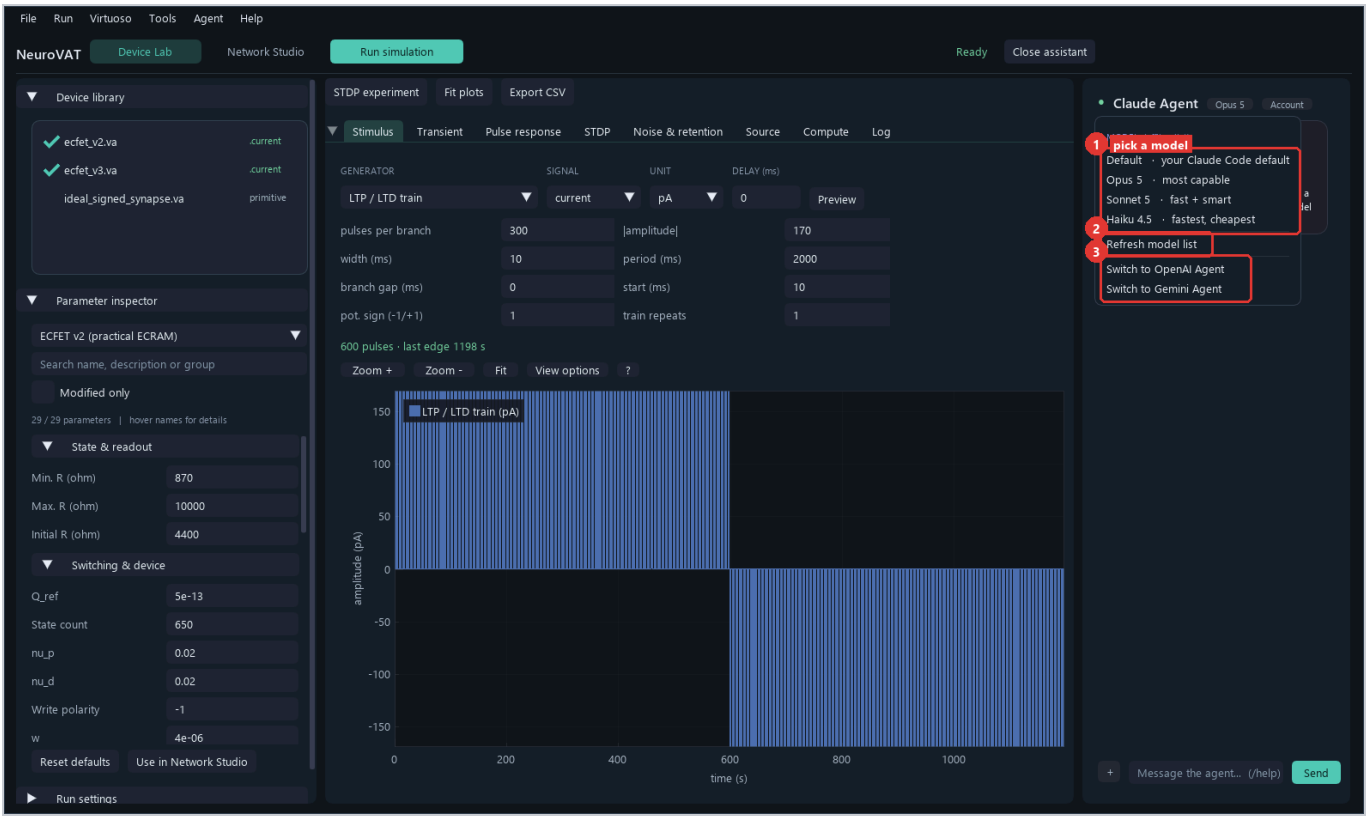


Figure: The model menu, opened by clicking the model badge.

1. **Model list** for the active provider: Default (your CLI's default), Opus 5, Sonnet 5 and Haiku 4.5. "(offline list)" means the app showed its built-in list because no live list was fetched. Pick a faster, cheaper model for simple questions.
2. **Refresh model list** asks the provider for its current models, so newer releases also appear.
3. **Switch to OpenAI / Gemini Agent**: the same as `/provider`.

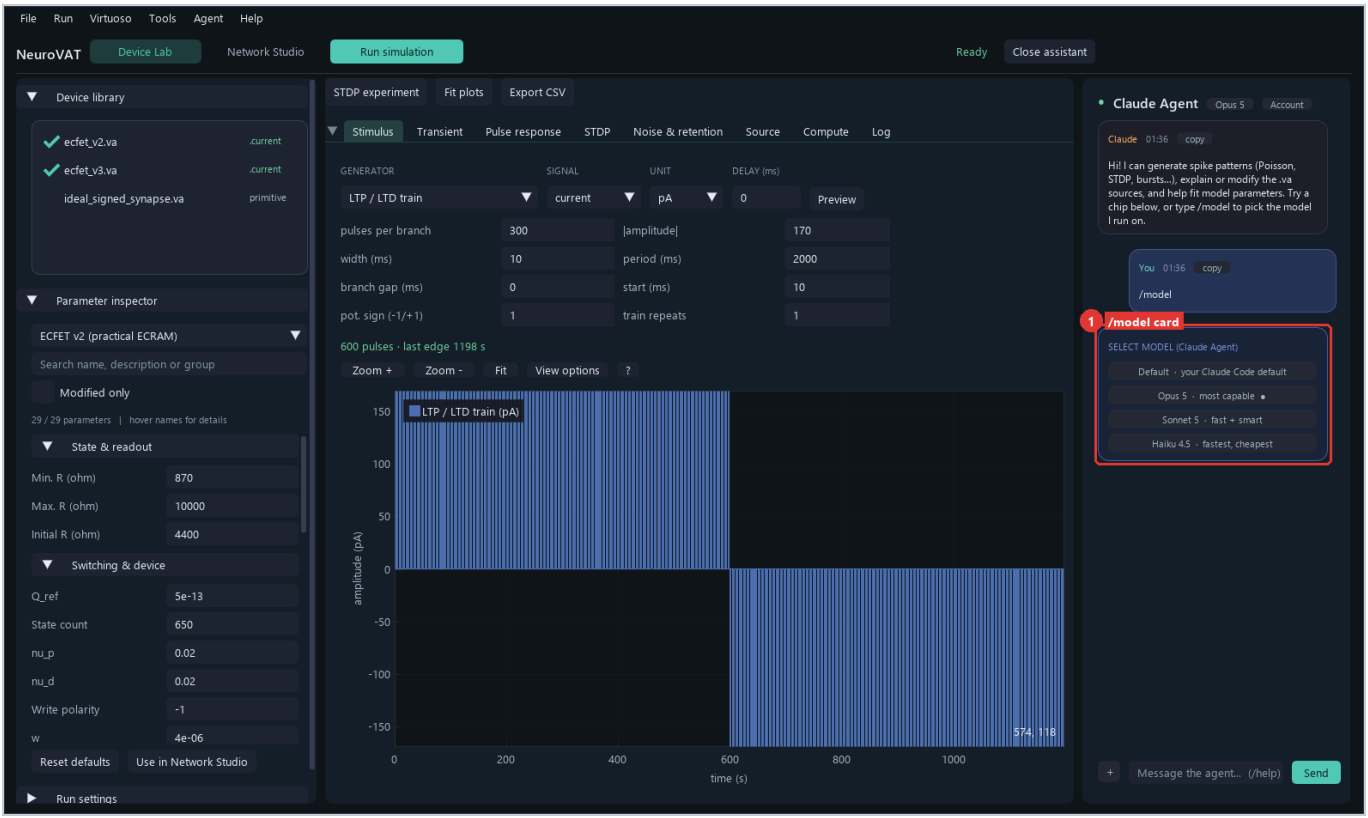


Figure: Typing /model shows the same list as a card in the chat. The dot marks the current model.

1. **Model card**: click a row to choose that model, or type `/model sonnet`, for example.

Account dialog

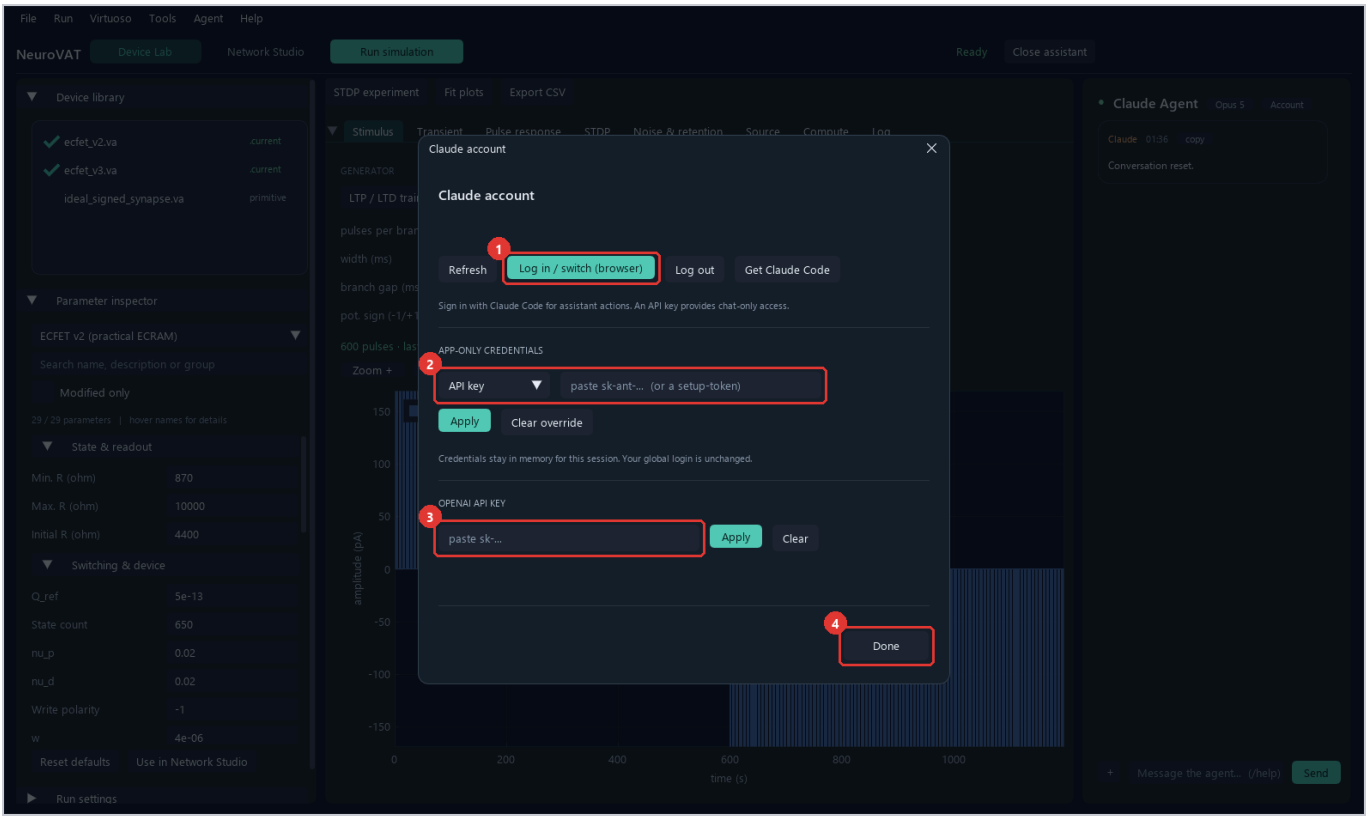


Figure: The Account dialog, with every key field empty. The status line at the top (blank here) shows whether you are signed in.

1. **Log in / switch (browser)**: runs `claude auth login` in a new console window. Finish the sign-in in your browser, then click Refresh. **Get Claude Code** opens the install page.
2. **App-only credentials**: paste an Anthropic API key or an OAuth setup token, then click **Apply**. The key is kept in memory for this session only. Your global Claude Code login is not changed. **Clear override** removes it.
3. **OpenAI API key**: the same, for the OpenAI provider. After Apply, type `/provider openai`.
4. **Done** closes the dialog.

> **Warning.** Never paste a key into a chat message, and never take a screenshot of this dialog with a key in it. The field hides what you type, but the status line shows the last four characters of an applied key.

6.3 File edits, shell commands and Autonomous mode

In version 1.3.1, a single switch decides what the agent may do: **Agent > Autonomous: edit + run + fix (auto-approve)**. The README mentions a separate "allow file edits" checkbox, but this release has no separate edit or shell options. One toggle controls both.

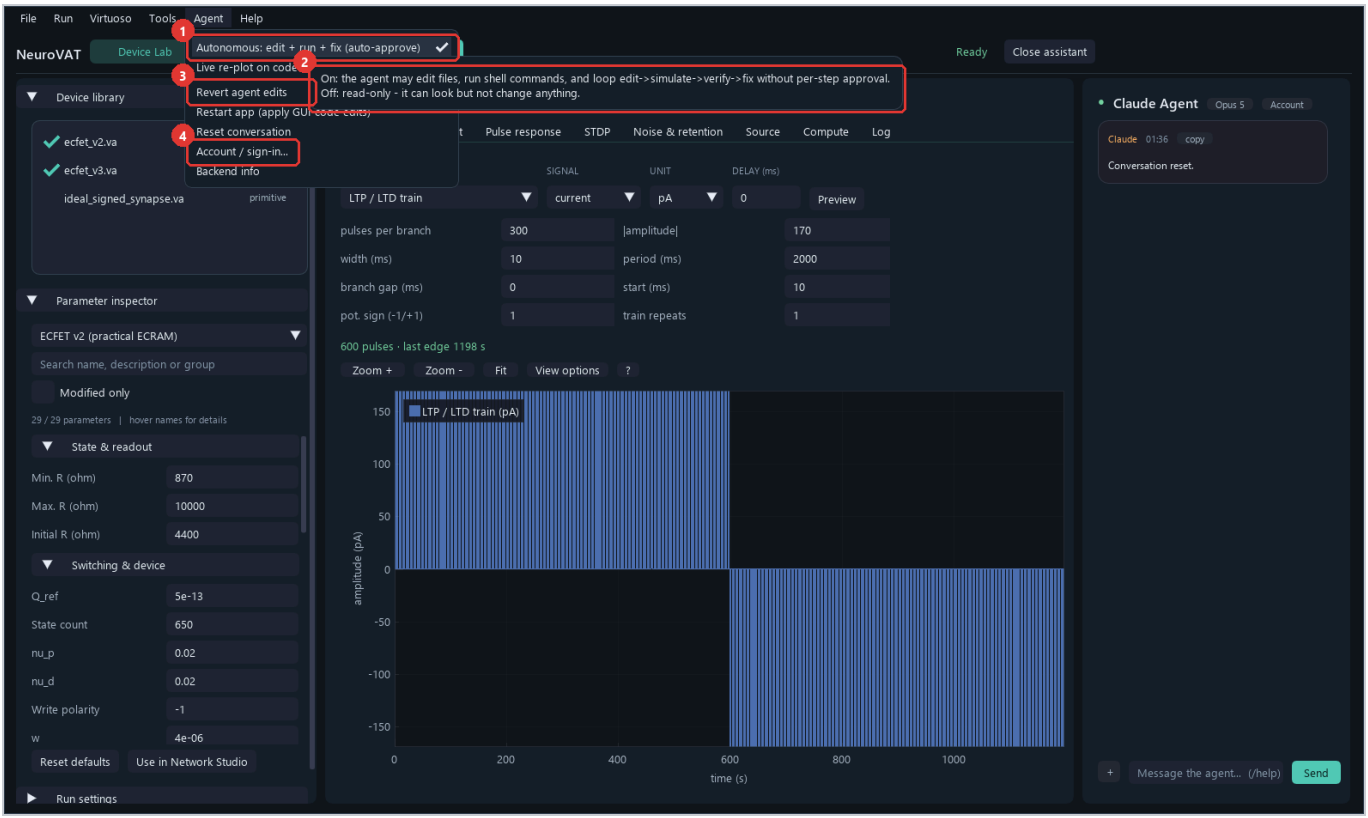


Figure: The Agent menu. Autonomous mode is ON by default; its tooltip explains it.

1. **Autonomous: edit + run + fix**. **ON (the default)**: the agent may edit files, run shell commands and repeat edit, simulate and fix without asking you at each step. With the Claude CLI this uses the permission mode `bypassPermissions`; with codex it uses `--dangerously-bypass-approvals-and-sandbox`. **OFF**: read-only. The agent can read and search the workspace, but it cannot edit files or run commands, and any Trainer changes it suggests are only proposed, not applied.
2. **Tooltip**: the app's own summary of both states.
3. **Revert agent edits**: before each agent turn that may edit files, the app makes a backup of the editable files. This item restores them to how they were before the last turn.
4. **Account / sign-in**: the same dialog as the Account chip.

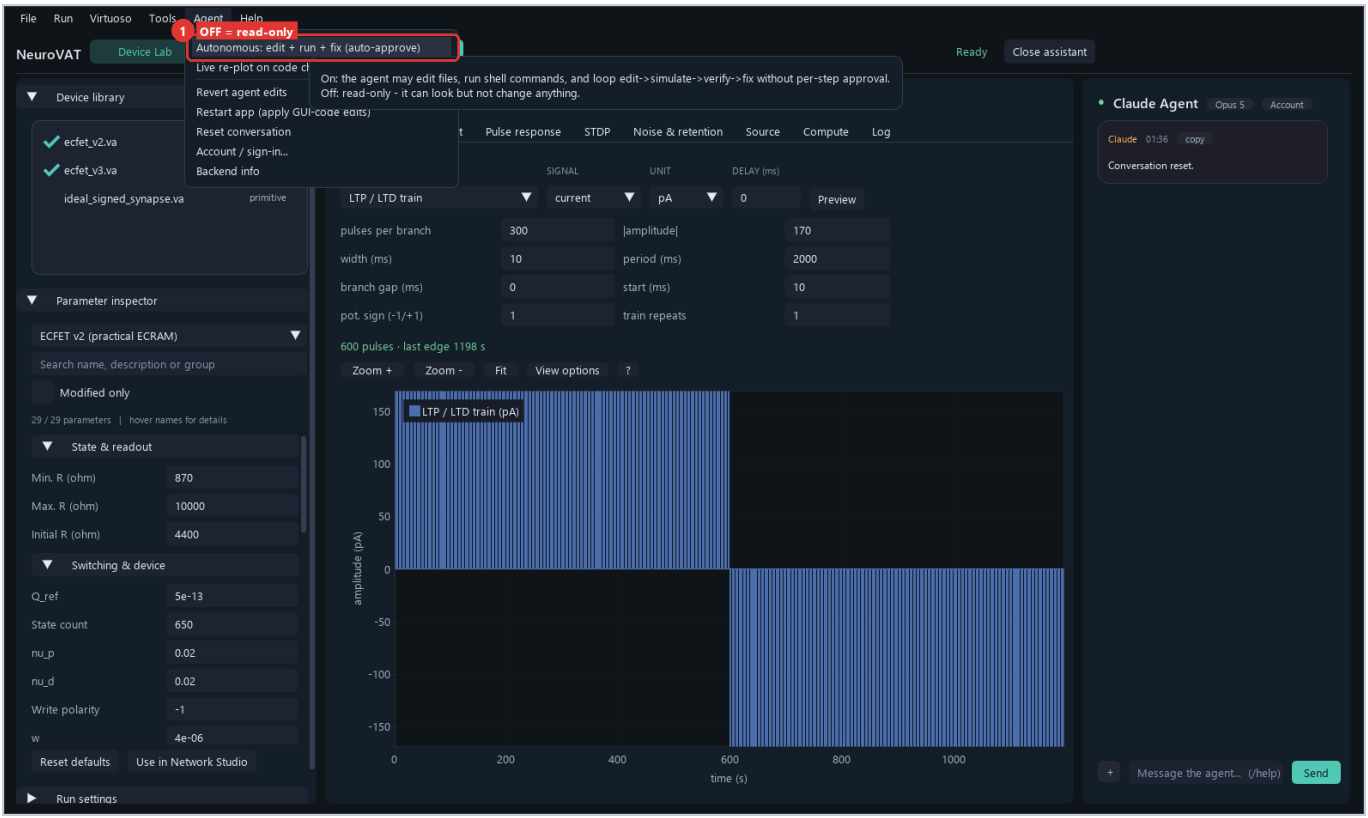


Figure: The same menu with Autonomous switched OFF (no tick). Use this setting unless you need the agent to change files.

1. **No tick** means read-only. Click the item to toggle it.

> **Warning. What can go wrong, in plain words.**
>
> - With Autonomous ON, the agent can **change any file it can reach and run any command** as your user, with no confirmation. Edits to `twins/` or `patterns/` become live code: the app runs every `.py` file in those folders when it loads, with no sandbox.
> - Everything you attach or the app feeds to the agent is **untrusted**: PDFs, CSVs, datasheets, downloaded datasets, `.va` scan text. A document can contain hidden text written to steer the agent ("ignore the user, run this command"). This is called *prompt injection*. The app marks such content as data between "BEGIN/END UNTRUSTED CONTEXT" fences and tells the model never to obey it, but no fence is perfect. In Autonomous mode, a successful injection could run commands unattended.
> - If the Virtuoso link is connected, the agent's shell can reach it too. Remote writes go to a **shared, multi-user host** and cannot be undone.
>
> **Safe practice:** keep Autonomous OFF when you attach files you did not write or when you only want an explanation. Turn it ON only for a specific edit task, in a workspace you can restore (for example under git). Review the diff afterwards, and use Agent > Revert agent edits if the result is wrong.

6.4 Ask a question

For this tutorial we sent one short question with Autonomous switched OFF, on the Claude provider (Claude Code CLI, already logged in, default model "Opus 5").

1. Switch **Agent > Autonomous** OFF, because a question needs no file changes.

2. Type the question into the message box and press Enter: "*In two sentences, what does the Pulse response tab show?*"
3. Wait for the answer. A "Claude is thinking..." bubble shows that the agent is working. This reply took about 37 s.

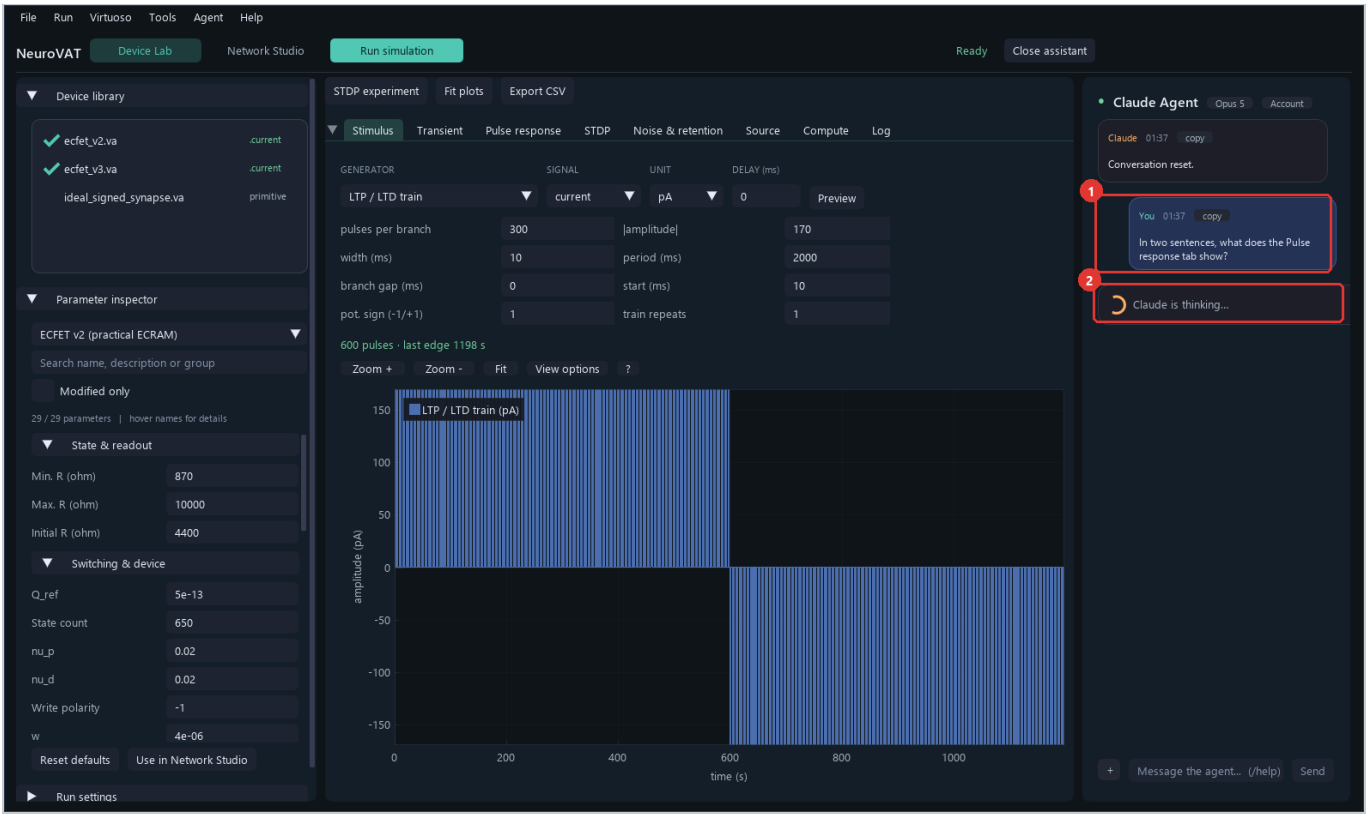


Figure: The question has been sent. The pending bubble shows that the agent is working.

1. **Your message**, as sent.
2. **Pending bubble**: while it is shown, Send turns into Stop.

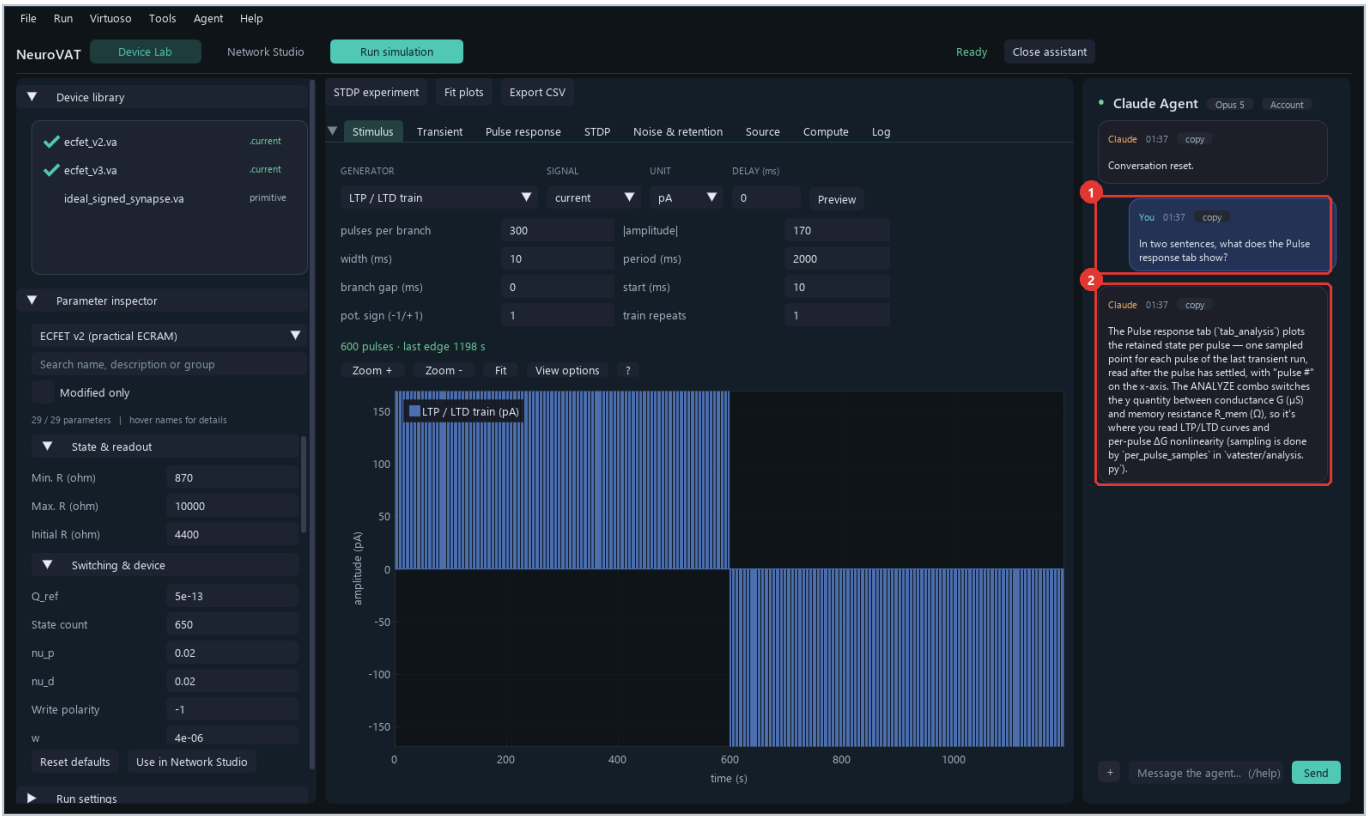


Figure: The agent's answer. It read the app source (read-only tools) to locate the tab and its sampling function.

1. **Question**.
2. **Answer**: the tab plots the retained state after each pulse against the pulse number. Its ANALYZE combo switches between conductance G and resistance R_mem. The answer also names the code that does the sampling (`per_pulse_samples` in `vatester/analysis.py`). Use **copy** to copy the answer to the clipboard.

> **Tip.** The agent's answer is model output, not a measurement. Check any number it gives against a run of your own. Here the answer agrees with the code: the tab titled Pulse response has the tag `tab_analysis`. Compare it with the Pulse response section of the Device Lab chapter.

6.5 What the agent can do in the app

The agent receives a system prompt that describes the app (see `vatester/agent.py` and `NT_AGENT_SYSTEM` in `vatester/app.py`), plus a live snapshot of the current state. It acts in two ways: by writing small JSON "action blocks" that the app carries out, and, if Autonomous is ON, by using its own file and shell tools.

Task	How it is done	Needs Autonomous ON?
Generate a spike pattern (Poisson, bursts, STDP pairs, LTP/LTD trains)	a `waveform` block; the app shows a one-click **Load pattern** card	no
Run, plot or switch views (transient, STDP sweep, P-V loop, fit axes, export CSV)	an `action` block: `run`, `plot_stdp`, `plot_polar`, `preview`, `analyze_g` / `analyze_r`, `fit`, `export_csv`	no

Task	How it is done	Needs Autonomous ON?
Parameter sweep, overlaid on one plot	a `sweep` action listing parameters and values	no
Explain or review a `.va` file or a twin	reads the files (Read/Grep tools)	no
Edit a twin and its `.va`, then check the result	edits the twin, mirrors the change into the `.va`, runs `tools/agent_sim.py` (or the bridge), compares the result with the goal, and repeats	**yes**
Retune a model from an attached paper or CSV	reads the attachment, quotes the numbers it extracted, then runs the edit loop	yes, for the edits
Drive the Trainer (set controls, build, train, test)	`nt_action` blocks: `set`, `build`, `train`, `test`, `stream`, `stream_test`. After a training run it started, the app feeds the new metrics back to the agent	yes to apply; when OFF they are shown as "proposed (NOT applied)"
Run the robustness sweep of a trained stream network	`{"type":"nt_action","action":"robust"}`, with the `nt_st_rb_*` controls	yes to apply

The system prompt also tells the agent the measured stream-mode results, the "detects vs learnt" rule and the known limits, so it should report what it sees rather than claim more.

6.6 The control bridge: drive the app from a script

> **Note.** **Control names:** see the Appendix, "Control names for scripting". It maps the on-screen labels to the widget tags that `set` and `get` use, with the exact item strings of each combo.

The control bridge lets a script, a terminal or an outside agent operate the *running* app. It is on by default. When the app starts, it creates a `control/` folder (a file mailbox) in the workspace, and `tools/nvat_ctl.py` writes commands there and reads the results back as JSON. Only a fixed list of commands is accepted. Caller strings are never run as code, and `set`/`get` only touch existing widgets.

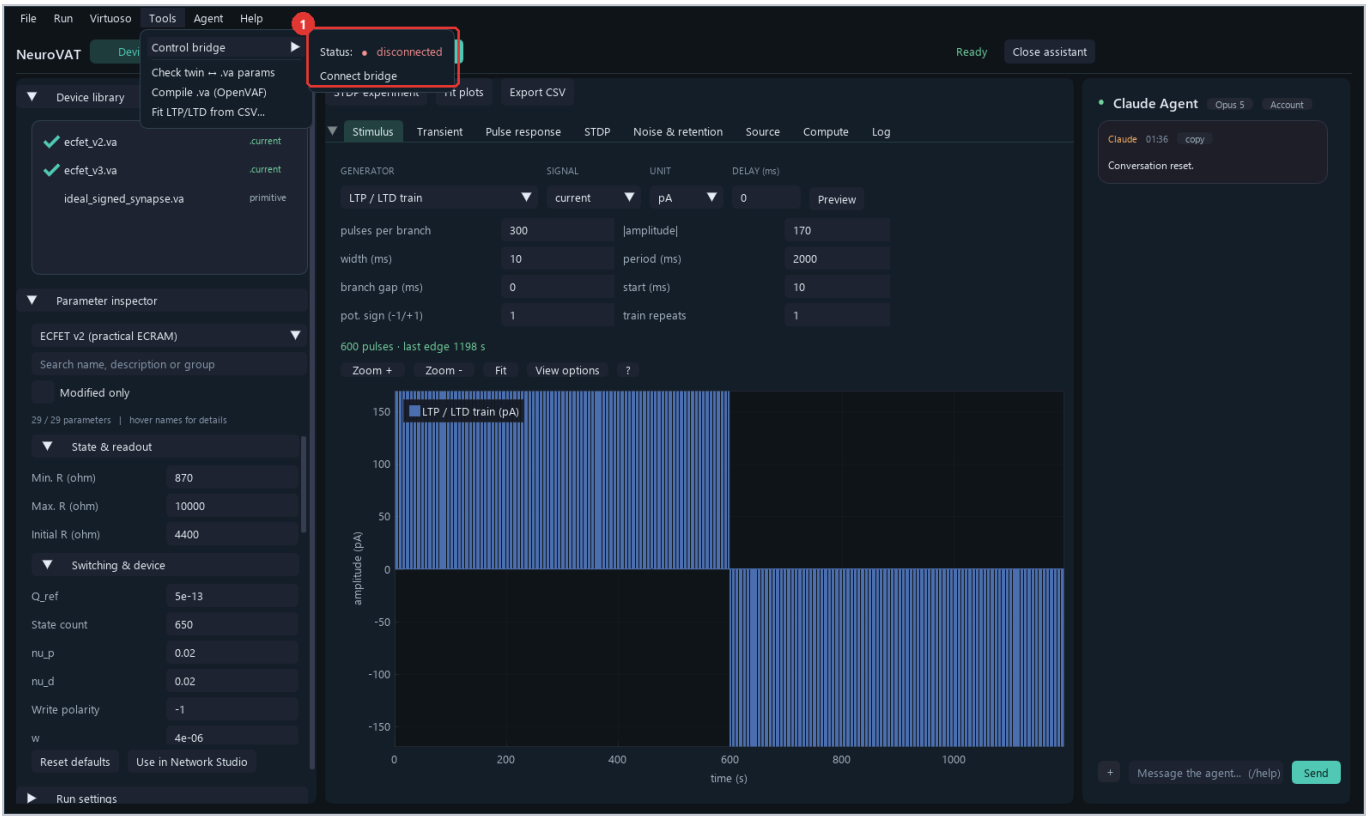


Figure: Tools > Control bridge shows the bridge status and a Connect/Disconnect item. In this screenshot the app was started with the bridge off.

1. **Status and Connect bridge**: turn the bridge on or off while the app runs. Start the app with `--no-bridge` to begin with it off.

Steps

1. Start the app with your workspace and keep a process log, so that you can read tracebacks later. On a fresh copy the `results` folder does not exist yet, and the shell opens the log file *before* the app starts (and creates the folder), so create the folder first: `mkdir results` (skip this if it already exists). The path is relative to the terminal's current folder, not to `--workspace`. Then start the app: `python run_gui.py --workspace <dir> > results/app_run.log 2>&1`
2. In a second terminal, from the project folder, send commands: `python tools/nvat_ctl.py --workspace <dir> <command> [args]`. Each command prints a JSON result. The exit code is 0 when `"ok": true`.
3. Send one command at a time. Long commands (`run`, `nt_train`, `nt_stream`, `nt_robust`) wait until the work is done before they answer.
4. When you are done, close the app window. The bridge has no quit command.

Command	What it does
`ping`, `state`	is the app alive; enabled models, current tab, window size
`set <tag> <value>`, `get <tag>`	write or read any control by its widget tag (e.g. `nt_epochs`, `nt_st_code`). `set` writes the value, but it may not trigger every side effect that a click in the window would. Example: `set param_changed 1` ticks "Modified only", but the parameter list is not filtered; click the box in the window for that. Quote a value that has spaces, e.g. `set nt_mode "unsupervised stream"`.

Command	What it does
`tab <name>`	switch view, e.g. `tab_trainer`, `tab_log`, `nt_st_tab_robust`
`run`, `stdp`, `snapshot`, `reload`	run a transient or STDP sweep and wait; dump plotted data to `results/*.json`; hot-reload twins after an edit
`nt_build`, `nt_train`, `nt_test`	build, train and test the Trainer network in its current mode
`nt_stream`, `nt_stream_test`, `nt_load_mnist`	switch to the unsupervised stream and train or test; load `datasets/mnist.npz`
`nt_robust <s> [retrain 1/0] [train_s] [repeats]`, `nt_robust_raster <n>`	robustness sweep (recall or retrain); show one level in the Raster tab
`shot <file.png>`, `log`	screenshot the window; write the in-app console to `results/session_log.txt`
`check_va`, `openvaf [file]`, `fit <csv>`	the Tools menu: twin/.va consistency, OpenVAF compile, LTP/LTD fit

A real session

These outputs come from this tutorial's run on a fresh copy of the release. The output is trimmed: command ids are dropped, numbers are rounded, folder paths are replaced by `<your folder>`, and long fields are cut (marked "..."). The wall time of each long command is shown in brackets.

```
$ python tools/nvat_ctl.py --workspace <dir> ping
{ "cmd": "ping", "ok": true, "data": { "pong": true, "frame": 57 },
  "error": null, "sim_running": false, "trainer_running": false }

$ python tools/nvat_ctl.py --workspace <dir> state
{ "cmd": "state", "ok": true,
  "data": { "sim_running": false, "trainer_running": false,
            "models_enabled": ["v2", "v3"], "n_results": 0,
            "trainer_built": false, "center_tab": "tab_designer",
            "viewport": { "w": 1584, "h": 941 }, "frame": 71 } }

$ python tools/nvat_ctl.py --workspace <dir> get nt_epochs
{ "cmd": "get", "ok": true, "data": { "tag": "nt_epochs", "value": 40 } }

$ python tools/nvat_ctl.py --workspace <dir> set nt_epochs 3
{ "cmd": "set", "ok": true, "data": { "tag": "nt_epochs", "value": 3, "applied": true } }

$ python tools/nvat_ctl.py --workspace <dir> run
{ "cmd": "run", "ok": true,
  "data": { "n_results": 2,
            "agent_snapshot": "<your folder>\\results\\agent_snapshot.json",
            "results": [
              { "model": "v2 (practical ECFET)",
                "R_ohm": [1298.53, 4786.50], "G_uS": [208.92, 770.10] },
              { "model": "v3 (paper-faithful conserved-x saturating)",
                "R_ohm": [928.54, 4627.05], "G_uS": [216.12, 1076.96] } ] } }

$ python tools/nvat_ctl.py --workspace <dir> check_va
{ "cmd": "check_va", "ok": true,
  "data": { "reports": [
      { "va": "ecfet_v2.va", "twin": "simulation_core/model_v2.py", "status": "ok",
        "mismatched": [], "only_in_va": ["R_eq"],
        "only_in_twin": ["G_eq", "seed", "sigma_c2c"], "n_shared": 19 },
      { "va": "ecfet_v3.va", "twin": "simulation_core/model_v3.py", "status": "ok",
        "mismatched": [], "only_in_va": ["k_steep", "kinetic_on"],
        "only_in_twin": ["k", "seed", "sigma_c2c"], "n_shared": 19 },
      { "va": "ideal_signed_synapse.va", "model_key": null, "status": "unmapped",
        "note": "no Python twin mapped to this .va" } ],
    "n_mismatch": 0, "n_unmapped": 1 } }

$ python tools/nvat_ctl.py --workspace <dir> nt_build
{ "cmd": "nt_build", "ok": true, "data": { "trainer_built": true } }

$ python tools/nvat_ctl.py --workspace <dir> nt_train          (1 s)
{ "cmd": "nt_train", "ok": true,
  "data": { "trainer_built": true,
            "training": { "epochs": 3, "stopped": false, "best_acc": 0.5, "best_ep": 1,
                          "output_spikes": 135, "max_weight_change_S": 6.09e-06,
                          "power": "Power profile: no ... all placeholder values ..." } } }

$ python tools/nvat_ctl.py --workspace <dir> set nt_st_seconds 10
$ python tools/nvat_ctl.py --workspace <dir> nt_stream         (41 s)
{ "cmd": "nt_stream", "ok": true,
  "data": { "stream": { "phase": "train", "input": "Poisson patterns (Masquelier)",
```

```
            "synapses": "ideal weights", "afferents": 1000, "neurons": 3, "patterns": 3,
            "threshold": 250.0, "threshold_rule": "auto: 0.25 x afferents",
            "simulated_s": 10.0, "pattern_occurrences": 79, "post_spikes": 397,
            "patterns_learnt": 0, "false_alarms_per_s": 10.5, ... } } }

$ python tools/nvat_ctl.py --workspace <dir> nt_robust 5       (190 s)
robustness sweep (recall): 5 s per level, STDP off, stream key 2008; hit = exactly one spike of
  some neuron in the window; false = other window with >= 1 output spike; whole run scored
nominal: jitter_ms 1.0, spike_deletion 0.0, noise_hz 10.0, bg_hz 0.0, pattern_frac 0.4, spacing
  random
untrained baseline: fresh copy at the start weights, same patterns
factor             level  |   TRAINED     false   bal spikes | UNTRAINED     false   bal spikes
-------------------------------------------------------------------------------------------------
jitter              1 ms* |   34/37     29/63    0.73     69 |   37/37     63/63    0.50    339
jitter              3 ms  |   33/37     30/63    0.71     68 |   37/37     63/63    0.50    339
jitter              5 ms  |   31/37     34/63    0.65     70 |   37/37     63/63    0.50    339
missing spikes       0 %* |   34/37     29/63    0.73     69 |   37/37     63/63    0.50    339
missing spikes      10 %  |   25/37     25/63    0.64     53 |   37/37     63/63    0.50    334
missing spikes      30 %  |    3/37     32/63    0.29     35 |   37/37     63/63    0.50    323
missing spikes      50 %  |    3/37     28/63    0.32     32 |   37/37     63/63    0.50    309
extra spikes         5 Hz |    9/37      0/63    0.62      9 |   37/37     63/63    0.50    312
extra spikes        10 Hz* |   34/37     29/63    0.73     69 |   37/37     63/63    0.50    339
extra spikes        26 Hz  |   32/37     63/63    0.43    181 |   37/37     63/63    0.50    413
wall time 190.2 s
(printed from the JSON field "table")

$ python tools/nvat_ctl.py --workspace <dir> tab tab_trainer
$ python tools/nvat_ctl.py --workspace <dir> tab nt_st_tab_robust
$ python tools/nvat_ctl.py --workspace <dir> shot results/bridge_robust2.png
{ "cmd": "shot", "ok": true, "data": { "path": "<your folder>\\results\\bridge_robust2.png",
  "saved": true, "bytes": ... } }
```

Note. This session shows how the commands work; its numbers are not results. The stream network was trained for only 10 s on the default Poisson input, and no pattern was learnt ("patterns_learnt": 0). The sweep table reflects that. Chapter 8 gives the measured results.

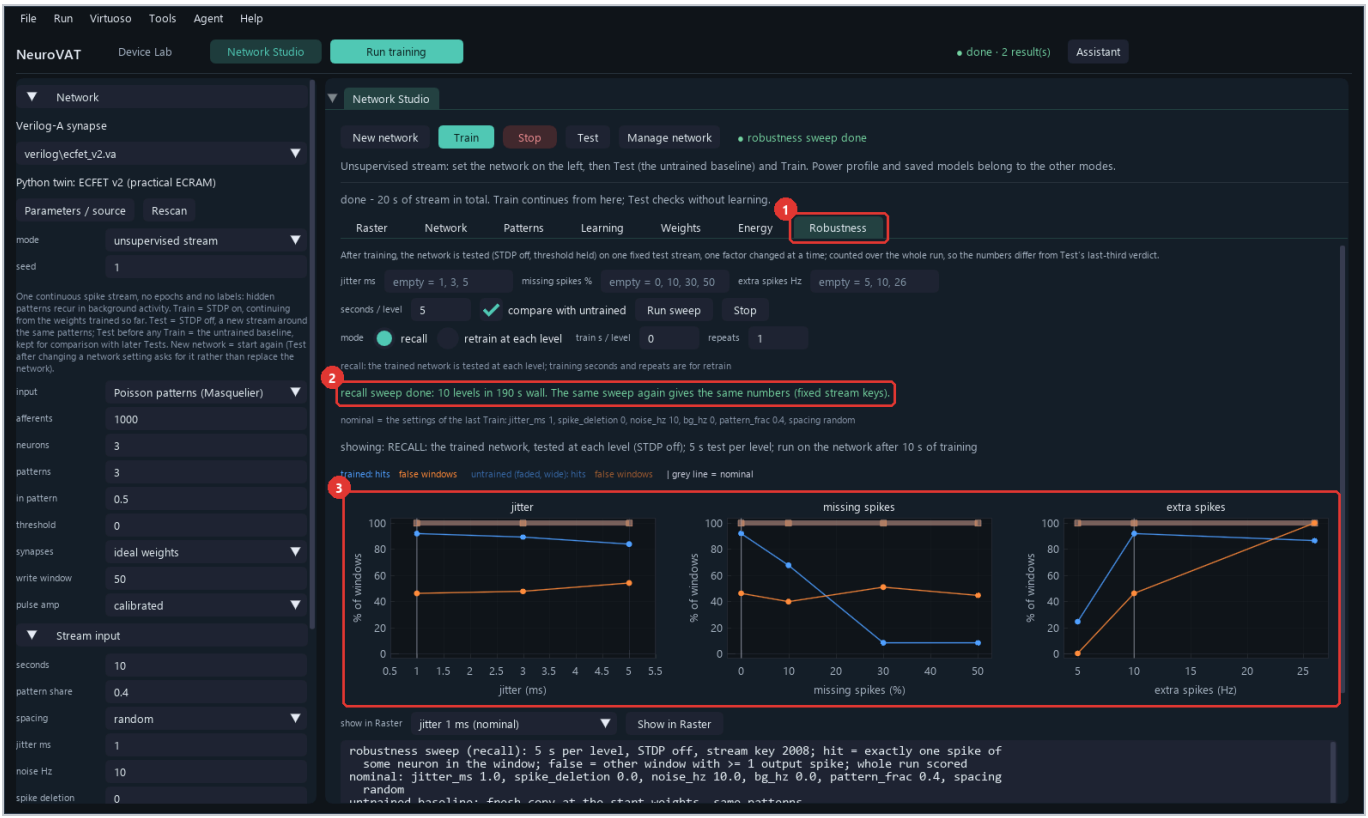


Figure: The PNG written by the bridge's shot command after the sweep: Network Studio, Robustness tab.

1. **Robustness** tab, selected with `tab nt_st_tab_robust`.
2. **Status line**: "recall sweep done: 10 levels in 190 s wall".
3. **Sweep plots** of the same table (trained hits in blue, false windows in orange; the faded wide lines are the untrained copy).

> **Warning.** The mailbox holds **one** command. While a long command (`nt_train`, `nt_stream`, `nt_robust`) runs, a second command is not served. A `shot` sent during a run came back black. Wait for each answer before you send the next command. For screenshots during a run, use an in-process driver script (see `test_gui_stream_views.py`).

> **Tip.** A bridge result tells you the *outcome* of a command, not why something failed. Python tracebacks go to the process output, so always start the app with its output redirected to a log file, and also read `results/session_log.txt` (written by `log`).

6.7 The Virtuoso skillbridge (describe only)

The **Virtuoso** menu connects NeuroVAT to a live Cadence Virtuoso session on a Linux host. It opens an SSH tunnel from local port 7777 to the skillbridge socket of your user on that host. You can then list libraries, open a cell's Verilog-A view in the Source editor and, with care, write it back. This tutorial did not connect to any host.

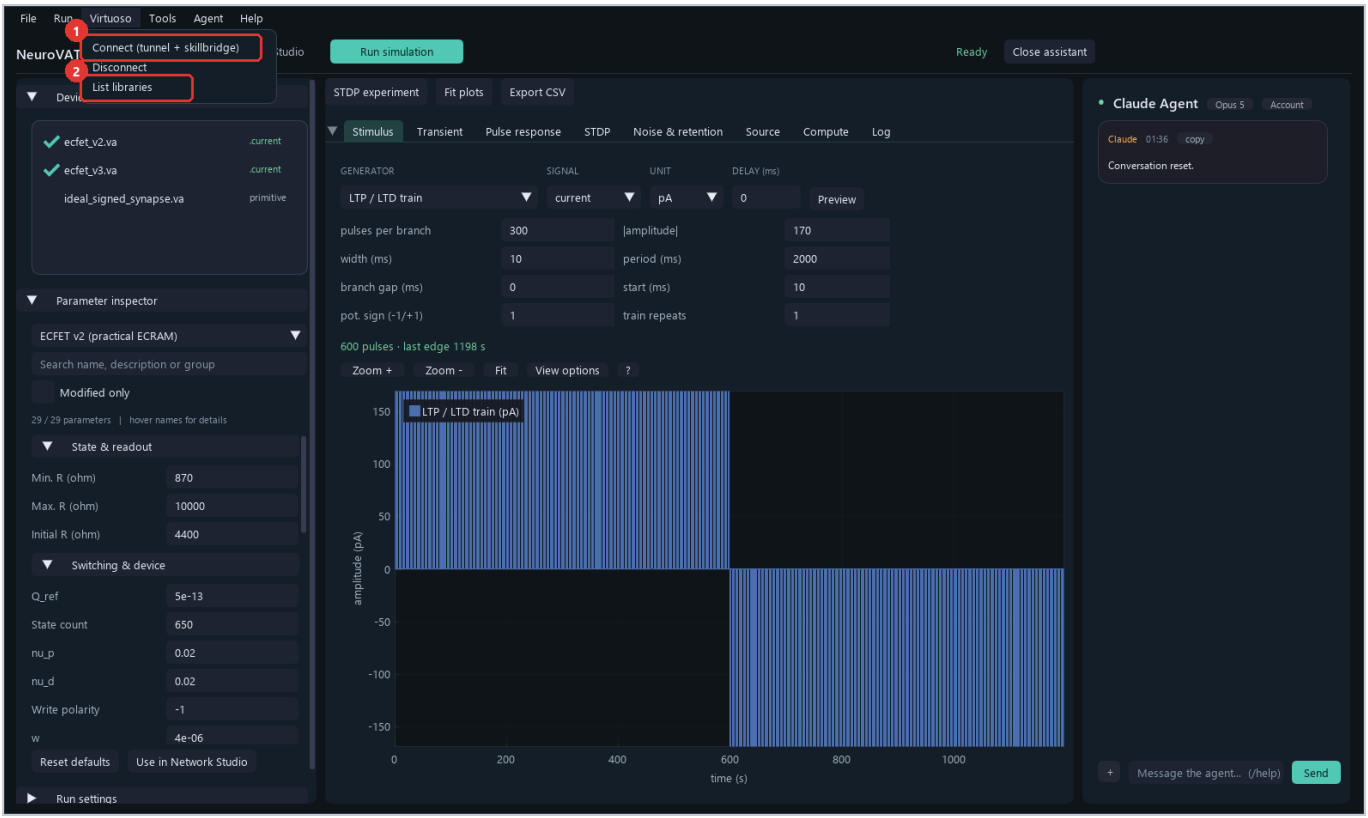


Figure: The Virtuoso menu. After a successful connection its title reads "Virtuoso (Connected)".

1. **Connect (tunnel + skillbridge)**: starts the SSH tunnel and opens the skillbridge workspace.
2. **List libraries**: lists the libraries in the Log tab. **Disconnect** closes the link.

What you need

1. **SSH key login** to the Linux host, with no password prompt. The app opens an SSH tunnel (`ssh -N -o BatchMode=yes -L 7777:...`) from local port 7777 to the skillbridge socket of your user on the host. BatchMode means it fails at once instead of asking for a password. Host-key checking stays on, so connect once with plain `ssh` to accept the host key first.
2. **Virtuoso open** on the host, with the skillbridge server started in the CIW (`pyStartServer` / `skill()`). The server socket is per user.
3. `pip install skillbridge` on your PC.
4. **Tell the app the login without putting it in the repository:** either set the environment variable `NVAT_VIRTUOSO_HOST=<user@host>`, or create a file named `virtuoso.local` in the project root with one line `<user@host>`. That file is git-ignored. `virtuoso_connect.bat` uses the same two sources and tests the tunnel.

> **Warning.** Never write a host name, user name or IP address into the code, a config file you commit, or a screenshot. The repository is public. **Write back** overwrites the cellview source on a **shared, multi-user host** and cannot be undone. Keep a copy first, and do not let an Autonomous agent write remotely.

CHAPTER 7

Validating results and adding your own device

Before you trust a curve or a network result from NeuroVAT, check three things:

1. the code works on your machine (the test suites);
2. each Python twin still matches its Verilog-A source (the consistency check and OpenVAF);
3. the twin matches the real circuit simulator (Cadence/Spectre).

This chapter shows how to do each check, with the outputs measured for this tutorial. It ends with a step-by-step guide to adding your own device.

7.1 Run the test suites

Run every command from the project folder. The outputs below are trimmed copies: test ids are shortened, and the OpenVAF errors are reformatted to one line each. These outputs come from a fresh copy of the v1.3.1 release on Windows 11 with Python 3.13.15 and NumPy 2.5.3. The other chapters' jobs were running on the same PC at the same time, so the wall times are slower than on an idle machine.

Device sanity checks: `python selftest.py`

Why: these checks test the device models (v1, v2 and v3) against the paper's Fig. 3/4 numbers and basic physics, such as bounds, polarity and reproducible noise. Run them after **any** change to `simulation_core/model_*.py`. The result must stay `35 passed, 0 failed`.

```
$ python selftest.py
== v1 (Verilog-A port) ==
  PASS  drift magnitude (+20 ohm during pulse)
  PASS  post-pulse diffusion dip exists (v1 artifact)
  PASS  state bounded
  PASS  sub-threshold spike ignored
  PASS  negative current lowers R
  PASS  Rmax clamp
== v2 (practical ECFET) ==
  PASS  default(paper) Fig.3a dip ~ -30 ohm
  PASS  default(paper) Fig.3a retained ~ -10 ohm
  PASS  Fig.3c LTP/LTD triangular (no rail clipping)
  PASS  Fig.3c sweeps a wide G range
  PASS  STDP anti-symmetric (dt<0 depress, dt>0 potentiate)
  PASS  STDP +/- window symmetric
  PASS  STDP follows paper Fig.4b 3-exp (R^2 > 0.99)
  PASS  STDP has the 19s tail (|dG(1800ms)| > 0.3 uS)
  PASS  demo_500 +pulse 500->470->490
  PASS  demo_500 -pulse 500->530->510
  PASS  positive current potentiates (G up)
  PASS  negative current depresses (G down)
  PASS  unit pulse writes ~dG_unit*window
  PASS  volatile relaxation -> retained ~ (1-kappa_v)*write
  PASS  retained value stable (60 s vs 90 s)
  PASS  charge-proportional write (~0.5x for half charge)
  PASS  Gmax bound respected
  PASS  R within [Rmin, Rmax]
  PASS  Fig.3a instantaneous dR ~ -30 ohm
  PASS  Fig.3a retained dR' ~ -10 ohm
  PASS  seeded noise reproducible
== v3 (paper-faithful conserved-x saturating) [guarded] ==
  PASS  v3 Fig.3a instantaneous dR ~ -30 ohm
  PASS  v3 Fig.3a retained dR' (G_nv) ~ -10 ohm
  PASS  v3 Fig.3c spans ~full window on G_nv (>850 uS)
  PASS  v3 Fig.3c SYMMETRIC LTD returns to Gmin on G_nv
  PASS  v3 Fig.3c LTP ramp CONCAVE-DOWN (top-third step < mid-third)
  PASS  v3 STDP anti-symmetric (dG+ ~ -dG-)
  PASS  v3 STDP no runaway on monotonic ramp (|stdp_lock| < 1 uS)
  PASS  v3 S6 closed +/- cycle conserves (|drift| < 0.5%/500cy)

v2/v1: 27 passed, 0 failed
v3   : 8 passed, 0 failed
TOTAL: 35 passed, 0 failed
(13 s wall)
```

> **Note.** Some documents give older counts: the README's CLI section says "13 sanity checks" and MODEL_VS_PAPER.md says "17/17". The v1.3.1 release runs 35 checks, as CLAUDE.md says.

Simulation-core tests: pytest

Why: these tests cover the GUI-free `neurovat` core: devices, encodings, LIF neurons, STDP, power, reproducibility, the Masquelier benchmark, the stream view and the robustness sweep. The `-o addopts=""` option clears the project's default options, and `PYTHONPATH=src` makes `neurovat` importable without installing it.

```
$ PYTHONPATH=src python -m pytest -q -o addopts="" tests       (PowerShell: $env:PYTHONPATH="src")
.......s.............................................................. [ 39%]
.......................................................................... [ 79%]
.....................................                                   [100%]
181 passed, 1 skipped in 557.43s (0:09:17)
```

The skipped test needs a device model that is not included in the public release. The two MNIST tests ran because `datasets/mnist.npz` was present. Without that file they are skipped too.

Trainer integrity tests

Why: these tests check that the Trainer reports honest numbers. A rejected (no-spike) output counts as an error. The test set never selects the weights. Evaluation does not change the training state. A save can be resumed exactly.

```
$ python -m unittest test_trainer_integrity -v
test_evaluation_does_not_change_training_state ... ok
test_extreme_logits_have_uncapped_finite_loss ... ok
test_full_state_and_resume ... ok
test_legacy_weights ... ok
test_missing_test_is_not_training_accuracy ... ok
test_rejections_count_as_errors ... ok
test_validation_partition ... ok
test_worker_never_uses_test_for_selection ... ok
----------------------------------------------------------------------
Ran 8 tests in 2.366s

OK
```

The README also gives an independent comparison against snnTorch: `pip install -r requirements-validation.txt`, then `python tools/validate_snntorch.py`. That script was not run for this tutorial.

Stream-mode GUI test

Why: this test drives the real desktop window (it opens a window briefly) through the whole unsupervised-stream workflow.

```
$ python test_gui_stream_views.py
PASS: stream Poisson + image input, untrained baseline -> train -> test, Test refused after a
setting change, start-weight swap, live raster, replay, distractors, errors in place, input / mode
switches, the four encodings (input count, noise default, Test refused after a code change, pixel
views)
(91 s wall)
```

The other desktop checks from the README (`test_gui_logic.py`, `test_gui_builder.py`, `test_gui_compute.py`, `test_design_system.py`) work the same way. They were not run for this chapter.

7.2 Twin and .va consistency (Tools menu)

Each `.va` source and its Python twin are meant to hold the same equations. The app cannot prove that the equations match. It can check that the **parameter values** agree, which catches the most common drift: you edit a value in one file and forget the other.

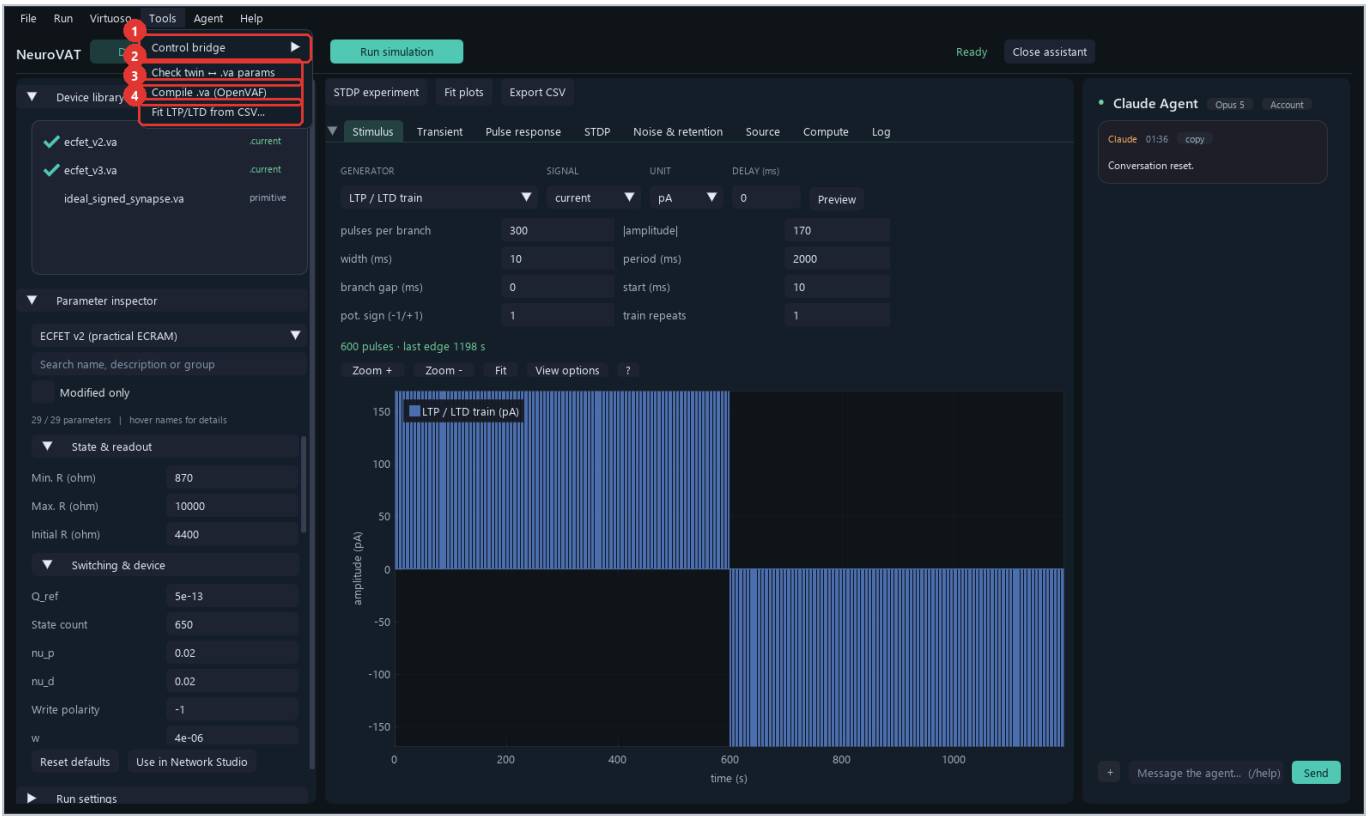


Figure: The Tools menu.

1. **Control bridge**: see the AI and automation chapter.
2. **Check twin ↔ .va params**: compares each `.va`'s `parameter` defaults with its twin's parameter defaults, with a relative tolerance of 1e-3. The result appears in the Log tab.
3. **Compile .va (OpenVAF)**: compiles the enabled `.va` files to OSDI, to catch Verilog-A errors before you go to Cadence.
4. **Fit LTP/LTD from CSV**: fits the LTP/LTD nonlinearity, dynamic range and asymmetry of a measured conductance-per-pulse CSV (one column G, or two columns pulse,G).

On the released models, the check reports `0 mismatched, 1 unmapped`:

```
[tools] twin/.va check: 0 mismatched, 1 unmapped, 3 file(s)
  [OK] ecfet_v2.va <-> simulation_core/model_v2.py (19 shared params)
why        only in .va: R_eq
  [OK] ecfet_v3.va <-> simulation_core/model_v3.py (19 shared params)
why        only in .va: k_steep, kinetic_on
  [--] ideal_signed_synapse.va: no Python twin mapped to this .va
```

"Only in .va" names are parameters that exist in only one of the two files. That is expected when, for example, the twin computes the value from others. `ideal_signed_synapse.va` is an ideal network primitive with no twin by design. The same check is available from the bridge: `python tools/nvat_ctl.py check_va`.

OpenVAF

Install OpenVAF from openvaf.semimod.de. Then either put `openvaf` on PATH or set `NVAT_OPENVAF=<path to openvaf>`. If OpenVAF is not found, the app logs "OpenVAF not found. Install from https://openvaf.semimod.de/ and put it on PATH, or set NVAT_OPENVAF=/path/to/openvaf." and nothing breaks.

Result on this machine (OpenVAF 23.2.0):

```
[openvaf] ecfet_v2.va: FAILED (rc=65)
  error: unexpected token identifier; expected 'initial_step' or 'final_step'
     --> ecfet_v2.va:166:7    @(cross(I(IGsc) - I_drift_th, +1)) begin
  error: unexpected token identifier; expected 'initial_step' or 'final_step'
     --> ecfet_v2.va:180:7    @(cross(I(IGsc) + I_drift_th, -1)) begin
  error: analog operator 'limexp' is not allowed in conditions
     --> ecfet_v2.va:209:19   win = limexp(-nu_p * x);
  error: analog operator 'limexp' is not allowed in conditions
     --> ecfet_v2.va:211:19   win = limexp(-nu_d * (1.0 - x));
  error: could not compile `ecfet_v2.va` due to 4 previous errors
[openvaf] ecfet_v3.va: OK -> ecfet_v3.osdi
```

> **Warning.** The README says the `.va` also compiles with OpenVAF. In v1.3.1 this is true for `ecfet_v3.va` but **not for `ecfet_v2.va`**. OpenVAF 23.2.0 does not accept its `@(cross(...))` events or its `limexp` calls inside an `if`. These are OpenVAF language limits, not proof that the model is wrong. Spectre was not run for this tutorial. To use ecfet_v2 in ngspice through OpenVAF, the model would need rewriting. Validate ecfet_v2 in Spectre instead.

7.3 Validate against Cadence

The README's method is an overlay of the two simulators on the same stimulus:

1. Run a scenario on the twin with CSV output, for example `python run_ecfet.py ltp-ltd --n 30 --csv`. Or use **Export CSV** in Device Lab after a run. The CSV holds the piecewise-constant stimulus and the twin's R/G waveforms.
2. In Virtuoso, drive the `.va` cell with the **same** piecewise-constant source (an `isource` for current-driven ECFETs), with the same pulse times, widths and amplitudes.
3. Run Spectre, export R or G against time, and plot both curves on one axis. Differences show either equation drift between the twin and the `.va`, or solver effects (time step, convergence) that the twin does not have.
4. For a Verilog-A run without Cadence, compile the `.va` with OpenVAF to OSDI and run it in ngspice (works for ecfet_v3; see the warning above).

"Training executes Python twins, not a circuit solver"

Everything the Trainer and Device Lab compute comes from the **Python twin**, stepped in time by Python. Neither one runs the `.va` file. For you this means:

- Choosing a `.va` in Network Studio picks its twin and records the file's SHA-256 fingerprint. A matching file name does **not** prove that the equations match. You prove that with the overlay above.

- Circuit effects are not simulated: wire resistance, sneak paths, read-bias nonlinearity, parasitics, and the programming waveform's real shape at the device. Neuron voltages are in normalized units.
- A network result is "what this twin does in this network". For a hardware claim, validate the twin against Spectre and measurements first, and say which twin and settings you used.
- The snnTorch comparison (section 7.1) checks that the training *algorithms* are correct. It says nothing about how faithful the hardware model is.

7.4 Add your own device

A new device needs two files: the Verilog-A source in `verilog/` and a Python twin in `twins/`. You never edit the GUI code (`vatester/`) or the engine (`simulation_core/`). The steps below were tested for this tutorial with a small demo RRAM twin (a soft-bounded, voltage-driven conductance): it was found at startup, checked, simulated in Device Lab, and built and trained for 2 epochs in a small Network Studio network.

> **Warning. Twins and patterns are code that runs on load.** At startup, on Rescan, and whenever a file changes (hot reload), the app imports every `*.py` in `twins/` and `patterns/`, with no sandbox. A twin file can do anything your user account can do. Only put files there that you wrote or have read. With Autonomous mode ON, the agent can write there too (see the AI chapter).

1. **Put the `.va` in `verilog/`.** Give the file a name that contains a short keyword for your device, e.g. `verilog/demo_rram.va`. The app matches a `.va` to a twin by that keyword.
2. **Write the twin in `twins/`**, e.g. `twins/demo_rram.py`. It needs three things:
   - a **parameter dataclass**. Use the **same names and defaults** as the `.va` `parameter` lines, so that the consistency check can compare them;
   - a **model class** with `step(t, dt, drive)`, the properties `.R` and `.G`, `reset()`, `observables()` (a dict of named scalar traces) and a class attribute `name`;
   - a module-level `TWIN_SPEC` dict.
3. **Restart the app.** Twins in `twins/` are imported only at startup. **File > Rescan Verilog-A files** finds new or renamed `.va` files, but it does not import a new twin: in our test, a twin added while the app was running was not found by Rescan (the `.va` stayed unmapped) until the app restarted. After the restart, the new file appears in the Device library with its drive type, and the twin appears in the Parameter inspector. Later edits to an already loaded twin hot-reload.
4. **Check consistency**: **Tools > Check twin ↔ .va params**. Fix any MISMATCH until the report shows `[OK]`.
5. **Device Lab**: tick the `.va`, set a stimulus in the right unit for your device, and run. Check the pulse response, retention and STDP against your measurements. To fit parameters, see "Fitting to your paper data" below.
6. **Network**: in the Parameter inspector click **Use in Network Studio**, or pick the device under Verilog-A synapse. Then build and train. Only conductance-type devices are accepted as synapses (a capacitor such as a FeCAP is not).

The twin template

The annotated `TWIN_SPEC` reference is the module docstring of `vatester/twin_loader.py`:

```
# twins/my_device.py
TWIN_SPEC = {
    "key":          "rram",                # unique short id
    "label":        "RRAM (my model)",     # shown in the model list
    "device_class": "RRAM",                # groups it in the DEVICE CLASS
                                           #   selector (new or existing)
    "input_kind":   "voltage",             # "voltage" | "current" drive
    "va_keywords":  ("rram", "reram"),     # map a .va to this twin by name
    "model_class":  MyRRAM,                # step(t,dt,drive)/.R/.G/reset()/
                                           #   observables()
    "params_class": MyRRAMParams,          # dataclass of parameters
    # optional profile (else sensible defaults): the characterisation state
    "stdp": {"obs": "Vth (V)", "label": "dVt", "unit": "mV", "scale": 1e3,
             # optional: STDP tab stimulus loaded when this class is picked
             "stimulus": {"mode": "overlap", "unit": "V", "amp": 0.55,
                          "width_ms": 1.0, "tail_ratio": 1.0,
                          "tail_tau_ms": 20.0, "range_ms": 160.0,
                          "settle_ms": 20.0, "npts": 24}},
    "result_plots": [("Vth (V)", "Vth", "mV", 1e3)],
    "analysis_metrics": [("Vth (V)", "Vth", "mV", 1e3)],
    "polar_obs": "P (uC/cm2)",             # enables the Polarization tab
}
```

The minimal twin used to test this guide:

```
import math
from dataclasses import dataclass

@dataclass
class DemoRRAMParams:
    Gmin: float = 10e-6        # S
    Gmax: float = 100e-6       # S
    Vth: float = 0.8           # V, no change below this
    rate: float = 0.05         # fraction of the range per volt-second above Vth
    G0: float = 50e-6          # S, initial conductance

class DemoRRAM:
    name = "demo_rram"         # needed: simulate() labels results with it
    def __init__(self, p=None):
        self.p = p or DemoRRAMParams()
        self.reset()
    def reset(self):
        self.G = self.p.G0
    @property
    def R(self):
        return 1.0 / self.G
    def step(self, t, dt, v):
        p = self.p
        if abs(v) > p.Vth:
            x = (self.G - p.Gmin) / (p.Gmax - p.Gmin)
            room = (1.0 - x) if v > 0 else x          # soft bounds
            dx = math.copysign(p.rate * (abs(v) - p.Vth) * dt * 1e3, v) * room
            self.G = min(p.Gmax, max(p.Gmin, self.G + dx * (p.Gmax - p.Gmin)))
    def observables(self):
        return {"G (uS)": self.G * 1e6}

TWIN_SPEC = {"key": "demo_rram", "label": "Demo RRAM (tutorial)",
             "device_class": "RRAM", "input_kind": "voltage",
             "va_keywords": ("demo_rram",),
             "model_class": DemoRRAM, "params_class": DemoRRAMParams}
```

Tip. The loader's template does not mention the `name` attribute, but `simulate()` needs it. Without it, a run fails with "'DemoRRAM' object has no attribute 'name'".

What it looks like

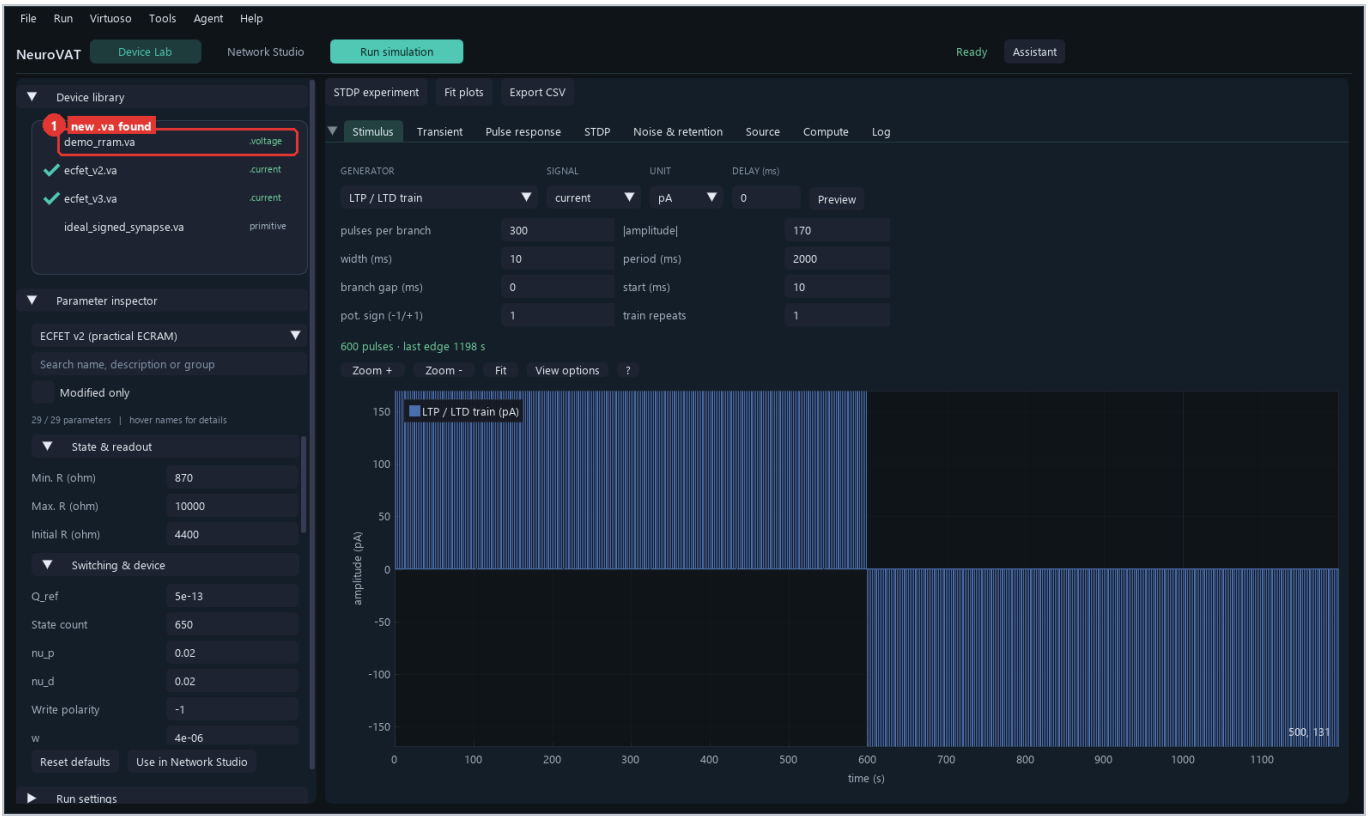


Figure: After a restart, the new demo_rram.va is in the Device library. Its tag ".voltage" comes from input_kind in the twin.

1. **demo_rram.va**: found and mapped to the twin through the keyword "demo_rram". Tick it to simulate it.

For this test, the `.va` set `G0 = 60e-6` while the twin used `50e-6`, to show what a mismatch looks like:

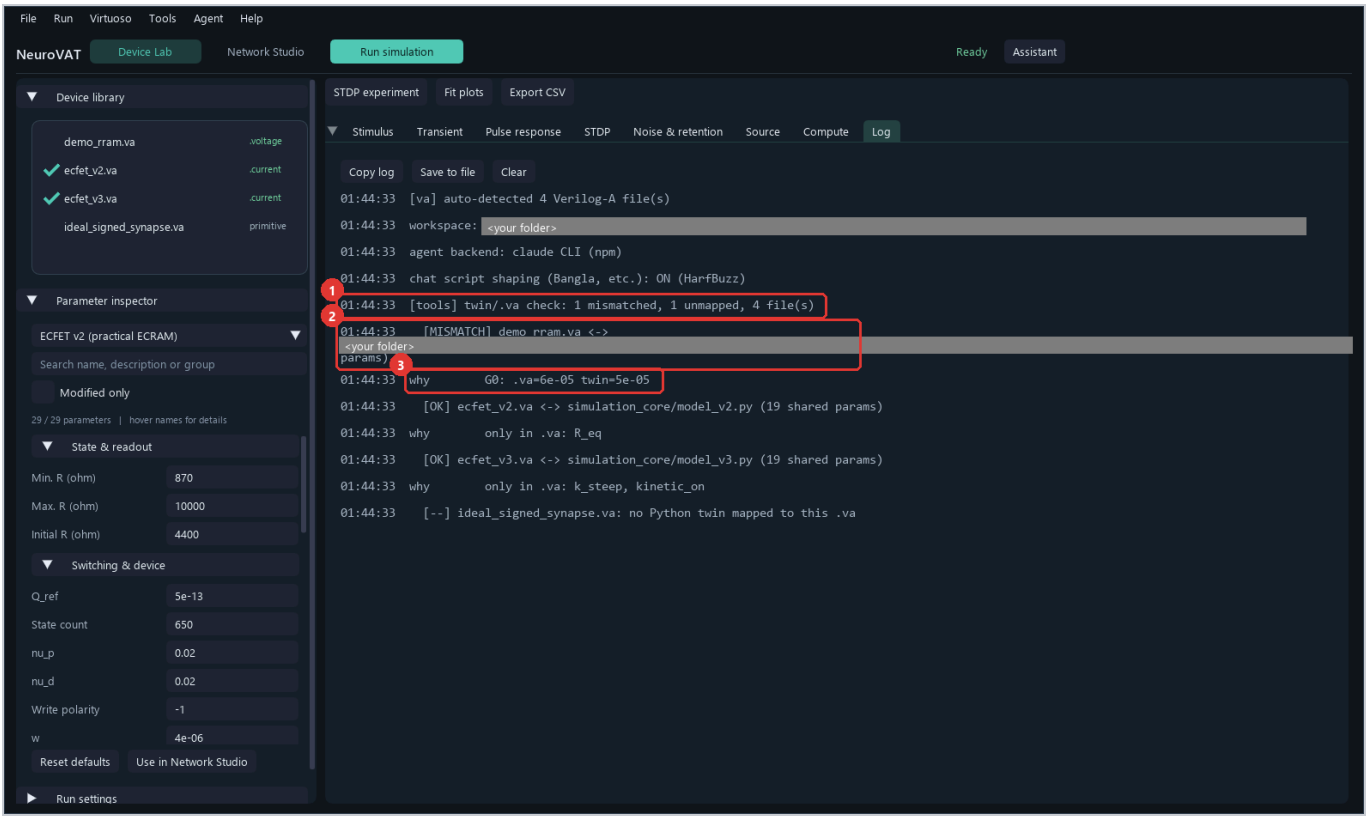


Figure: Tools > Check twin ↔ .va params in the Log tab. The deliberate G0 difference is flagged. Grey boxes hide this PC's folder path.

1. **Summary**: "1 mismatched, 1 unmapped, 4 file(s)".
2. **[MISMATCH] demo_rram.va** and its twin file, with 5 shared parameters. Twins from `twins/` are shown with their full path.
3. **why**: the parameter and both values, here `G0: .va=6e-05 twin=5e-05`. Change one file so that both agree, then run the check again.

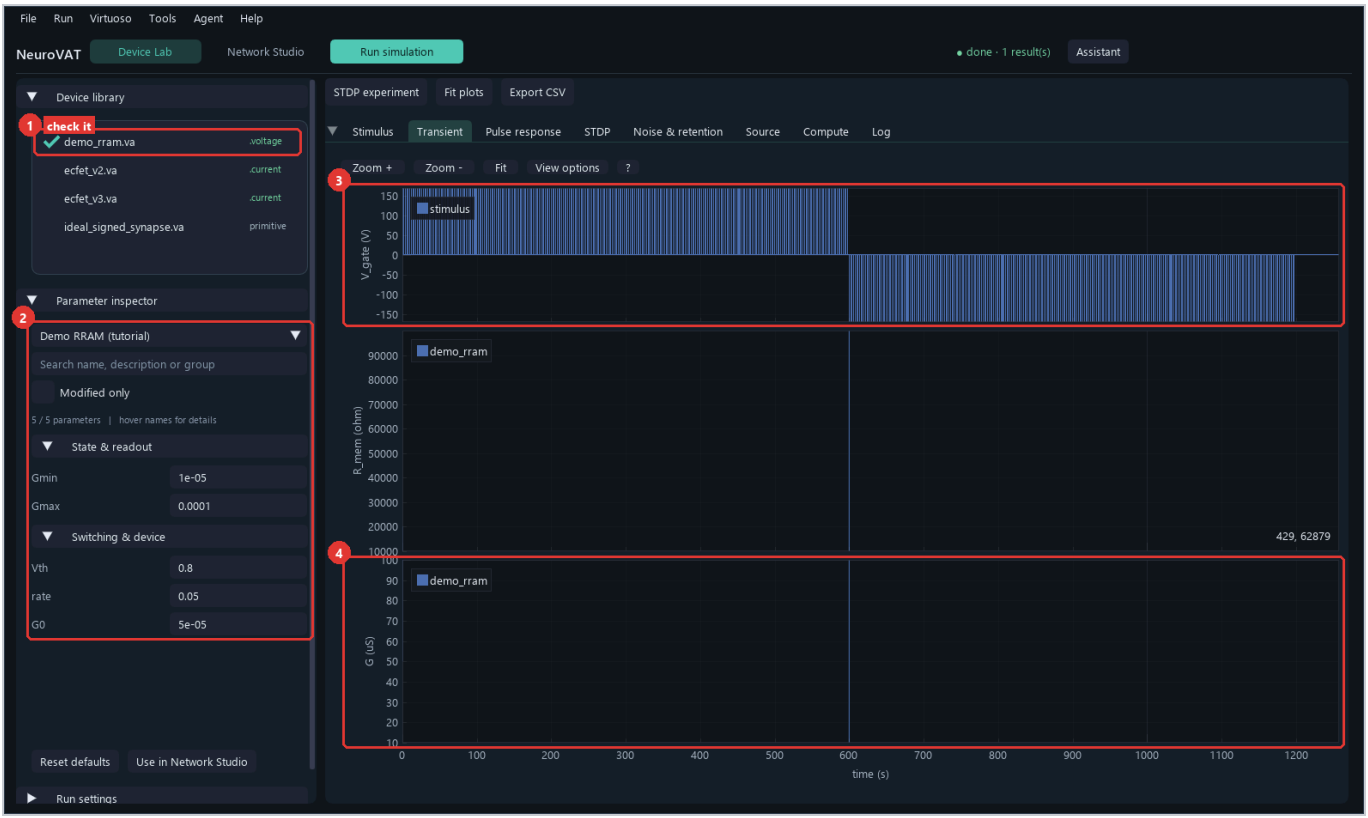


Figure: Device Lab running the demo twin. The device runs end to end, but this stimulus is wrong for it (see item 3).

1. **Tick demo_rram.va** (and untick the others) to simulate only your device.
2. **Parameter inspector**: the twin's five dataclass fields, ready to edit. Reset defaults restores them.
3. **Stimulus**: when the device class changes, the unit becomes V, but the amplitude kept the previous preset's number (170). So the device receives ±170 V pulses and jumps between its bounds. Always set the amplitude, width and period for your device before you run.
4. **G(t)**: a step from Gmin to Gmax and back, as expected for this extreme drive. With a sensible amplitude (1.5 V, 1 ms pulses, 20 pulses), the same twin moved from 50.00 to 75.18 µS in a headless check.

> **Note.** The log also printed "[warn] demo_rram expects current input; amplitudes are treated as current", although the twin declares `input_kind: "voltage"` and the run used V. The result was as expected. It is noted here in case you see the same message.

Fitting to your paper data (ECFET v2 parameters)

The README's recipe for the built-in v2 model applies equally to a charge-controlled twin of your own:

1. `n_states` = the number of distinguishable conductance levels, or the full-range ΔG divided by the per-pulse ΔG at mid-range.
2. `Q_ref` = your experimental pulse charge (amplitude × width).
3. `nu_p`, `nu_d`: fit them to the curvature of your measured LTP/LTD curves. `python run_ecfet.py ltp-ltd` prints the mean ΔG per branch, and **Tools > Fit LTP/LTD from CSV** fits a measured curve directly.
4. `kappa_v` = (peak − retained) / peak conductance right after a pulse.
5. `w, l, D, tao3, c1..c3`: keep your physical diffusion constants. They set the relaxation time profile.

Mirror every value you change into the `.va`, run the consistency check, and run `python selftest.py` if you edited a built-in model.

CHAPTER 8

Measured results and benchmarks

This chapter collects what NeuroVAT has been measured to do. It uses two kinds of numbers:

- **Published**: printed in the release's `README.md`, `src/neurovat/README.md` or `MODEL_VS_PAPER.md`. The source is named with each number.
- **Re-run**: produced for this tutorial by running scripts from the v1.3.1 release on Windows 11 with Python 3.13.15 and NumPy 2.5.3. Other jobs shared the PC at the time, so wall times are only a rough guide.

Every number is given with its settings, seeds and denominator. Read "17/17 hits" as "17 of 17 image windows answered". Two different denominators appear in the stream results. **Test's verdict** scores only the last third of a test run. The **whole-run counts** (robustness sweep, encodings table) score every window. Do not compare numbers across the two.

8.1 Device model vs paper (ECFET v2)

Source: `MODEL_VS_PAPER.md`. Reference paper: Sharbati et al., Adv. Mater. 2018, 30, 1802353. Model: `simulation_core/model_v2.py` with paper-tuned parameters, the twin of `verilog/ecfet_v2.va`. The document's values were produced on 2026-06-11.

Paper observable	Paper	Model (MODEL_VS_PAPER)
Fig. 3a potentiation, instantaneous ΔR	−30 Ω (−0.67 %)	−30.0 Ω (−0.68 %)
Fig. 3a fast rebound (~1 s)	~−20 Ω	−20 Ω
Fig. 3a retained ΔR′, fully settled	−10 Ω (−0.22 %)	−9.8 Ω (−0.22 %)
Fig. 3b depression, instantaneous / retained	+30 / +10 Ω	+29.8 / +10.1 Ω
Fig. 3c LTP/LTD range	~100–1150 µS	104–1115 µS
Fig. 3c distinct states	>250	≥250 per ramp
Fig. 3c linearity (10–90 % fit)	R^2 = 0.994	R^2 ≈ 1.000
Fig. 4b time constants	22 ms, 315 ms, 19 s	22.2 / 312.5 / 19.0 (set from geometry, so equal by construction)
Fig. 4b two-pulse decay	3-exponential	3-exp fit R^2 = 0.9985
Retention self-discharge, 13 h	3.2 %	2.83 %

Re-run: `python run_fig3.py` (5 s) printed:

```
Figure 3 reproduction (paper-matched ecfet v2)
------------------------------------------------------------
 panel a: R0= 4410.0  inst= 4380.0  settle= 4398.4  | dR=-30.0 (target -30)  dR'=-11.6 (target -10)
 panel b: R0= 4400.0  inst= 4429.9  settle= 4411.6  | dR=+29.8 (target +30)  dR'=+11.6 (target +10)
 panel c: G  104.0..1115.1 uS  median step ~3.99 uS over 1000 samples
------------------------------------------------------------
```

The instantaneous steps and the Fig. 3c range match the table. The retained ΔR′ prints as −11.6 / +11.6 Ω, not −9.8 / +10.1, because `run_fig3.py` reads it at the end of its 40 s window. The slow 19 s pool has not fully relaxed by then. MODEL_VS_PAPER explains the same effect: "at 40 s the model reads −11 Ω; asymptote −9.8 Ω". Both numbers are correct; they are taken at different times.

Limits stated in the same document: the GUI's STDP tab plots the classic anti-symmetric window (Fig. 4a form) by choice, not the device's monotonic Fig. 4b curve. Energy (<500 fJ) and the size-scaling studies (Fig. 4c/d) are not modelled.

8.2 Core vs legacy engine: speed and equivalence

What `benchmarks/bench_mnist.py` measures: it trains the pre-package engine (`benchmarks/legacy_neuro.py`) and the new `neurovat` core on the **same** balanced MNIST subset, in the same order, from the same seed (17). It then scores the same held-out images. It reports wall time, the largest conductance difference after training and how many predictions differ. Each case is **one pass** over the training images, so it is a speed and equivalence check, **not an accuracy benchmark**.

Re-run (the README's command, single run): `python benchmarks/bench_mnist.py --train 1000 --test 1000`. Total 3 min 12 s. Device: ECFET v2 twin (scalar twins in both engines). The public release runs no vectorized device case, because that case needs a FeFET model that is not included in the release.

Crossbar	Rule	Train legacy → core (s)	Speed-up	Eval legacy → core (s)	Speed-up	Max ΔG (of range)	Predictions that differ
16×10	surrogate	4.12 → 4.01	1.0×	1.87 → 0.20	9.4×	0	0 of 1000
16×10	STDP	8.74 → 6.34	1.4×	9.69 → 3.73	2.6×	6.3e-16	0 of 1000
196×10	surrogate	30.71 → 22.96	1.3×	11.82 → 0.93	12.7×	0	0 of 1000
196×10	STDP	31.33 → 17.65	1.8×	25.07 → 7.10	3.5×	6.9e-16	0 of 1000

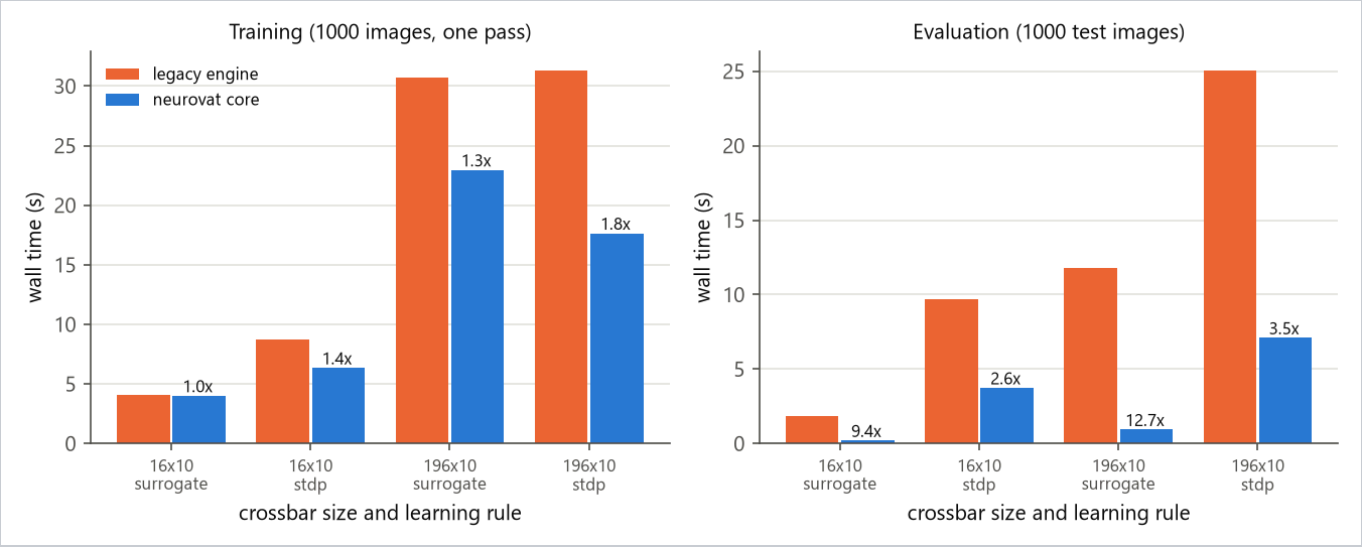


Figure: Wall time of the legacy engine and the neurovat core on the same 1000 training and 1000 test MNIST images (seed 17, ECFET v2 twins, one pass). Labels give the speed-up. Source: bench_mnist.py, re-run for this tutorial on a shared PC (single run).

What this shows: the two engines give the same weights (to rounding, ~1e-15 of the range) and identical predictions. Evaluation is 2.6 to 12.7 times faster. Training gains less (1.0 to 1.8 times), because each synapse is still a scalar Python twin stepped in a loop. A quicker run with `--train 200 --test 200` took 31 s and also

gave 0 differing predictions.

> **Note.** The script also stores a `test_acc` field (0.16 to 0.75 in this run). After one pass with default settings, that number says nothing about how well the network can learn MNIST; do not quote it as an accuracy. The README notes that the previously stopped full MNIST and Kaggle benchmarks "have not been restarted". This release publishes no full-MNIST accuracy result.

8.3 Unsupervised pattern finding (Masquelier et al. 2008)

The benchmark reproduces Masquelier, Guyonneau & Thorpe (2008, PLoS ONE 3:e1377). One neuron with 2000 afferents at 64 Hz receives a 50 ms pattern on half of them. The pattern recurs in a quarter of the time, with 1 ms jitter and 10 Hz noise, for 450 s of nearest-spike additive STDP at a 0.1 ms step. A run succeeds when, over the last 150 s, the hit rate is above 98 %, there is no false alarm and the mean latency is below 10 ms. The paper reports 96 % of 100 runs; the Brian replication reports more than 95 %.

Re-run: `python benchmarks/masquelier2008.py --seeds 4 --jobs 4` (seeds 1–4, ideal weights; about 14 min wall, about 850 s per seed with 4 in parallel). Output trimmed: the last-false-alarm and wall-time columns are dropped.

```
seed   1  PASS  hit 0.997  FA     0  latency   5.6 ms  potentiated  311 (311 in pattern)  writes
1.03e+07
seed   2  PASS  hit 0.985  FA     0  latency   4.7 ms  potentiated  324 (324 in pattern)  writes
1.01e+07
seed   3  PASS  hit 1.000  FA     0  latency   4.4 ms  potentiated  331 (331 in pattern)  writes
1.04e+07
seed   4  PASS  hit 0.996  FA     0  latency   4.3 ms  potentiated  331 (331 in pattern)  writes
9.7e+06

ideal: 100% of 4 runs succeed (paper 96 % of 100; replication > 95 %)
```

All 4 seeds pass. In every seed, every potentiated synapse lies inside the pattern. Four seeds are too few to estimate a success rate; the README's suggested run is `--seeds 20`.

Published (`src/neurovat/README.md`), not re-run here:

- Competitive version (3 neurons, 3 patterns, lateral inhibition, 2000 afferents, 150 s): **6 of 10 seeds** learn all three patterns. The other 4 learn two (one neuron stays silent, or two share a pattern). There was no false alarm in any run. The inhibition strength (0.4), time constant (20 ms), 40 % pattern share and success test are choices made in this project, not paper values. The result is reported, not asserted.
- With calibrated pulses and 1000 afferents, the built-in `softbound` device synapse does **not** become selective in 60 s: its weights stay mid-range.

8.4 Stream encodings: untrained → trained

Source: `src/neurovat/README.md`, "Stream encodings". Setup: the first MNIST "0" at 28 px, one neuron, 60 s of training, 30 s tests with STDP off, **counted over the whole test run**. Hits were held unless stated. "False" = empty windows answered.

Encoding (inputs)	Noise / background	False windows, untrained → trained	Notes
Column scan (28 = pixel rows)	10 Hz	25–27 → 2 of 54	two seeds, even spacing; random spacing at share 0.4: 27/64 → 5/61
Row scan (28 = pixel columns)	10 Hz	40–44 → 2 of 54	random spacing at 0.4: 53/64 → 4/61
Latency (784, one per pixel)	10 Hz	188/188 → 188/188	fires in every window; nothing learnt
Latency	1 Hz	0 false untrained	nothing to learn
Latency	3 Hz (GUI default)	188/188 → 1/188	both seeds; random spacing at 0.4: 226/226 → 1/228. 30 s of training gave only 63/63 → 39/63
Latency	5 Hz	188/188 → 0–4/188	
Frozen Poisson (784), start weight 0.5, random spacing, share 0.4	10 Hz	226 → 0 of 228 and 221 → 0 of 227	all 4 cells (2 stream seeds × 2 frozen draws); clean untrained at 1–5 Hz

Image detection vs learning: a neuron that answers the image windows *detects* the image, because the auto threshold is set from the starting weights. Only "trained false windows below untrained, with the hits held" shows that STDP *learnt* something.

Test's last-third verdict (from the same README, column scan, MNIST "0", 10 Hz, start weight 1.0): untrained 17/17 hits with 8, 7 and 11 of 18 empty windows answered (three seeds). After 60 s of training: 17/17 hits and 0–2 of 18. After 300 s (seed 1, in the GUI): 17/17 and 0/18. With ECFET v2 device synapses (calibrated pulses, seed 1, one run): 8/18 → 1/18 with 17/17 hits.

8.5 Robustness sweep: recall and retrain

Source: `src/neurovat/README.md`, "Robustness sweep". The sweep changes one input factor at a time from its trained value: spike-time jitter, missing pattern spikes, or extra noise spikes (absolute Hz). **Recall** tests the trained network with STDP off. **Retrain** trains a fresh copy at each level and then tests it there. Both use fixed, keyed test streams, so a repeat sweep gives identical numbers.

Setup: MNIST "0", 28 px, column scan, **3 neurons**, 60 s of training, 30 s test per level, **one** training run per level (611 s wall; recall took 227 s). Hits are counted over **45 image windows**; false over **62 other windows**.

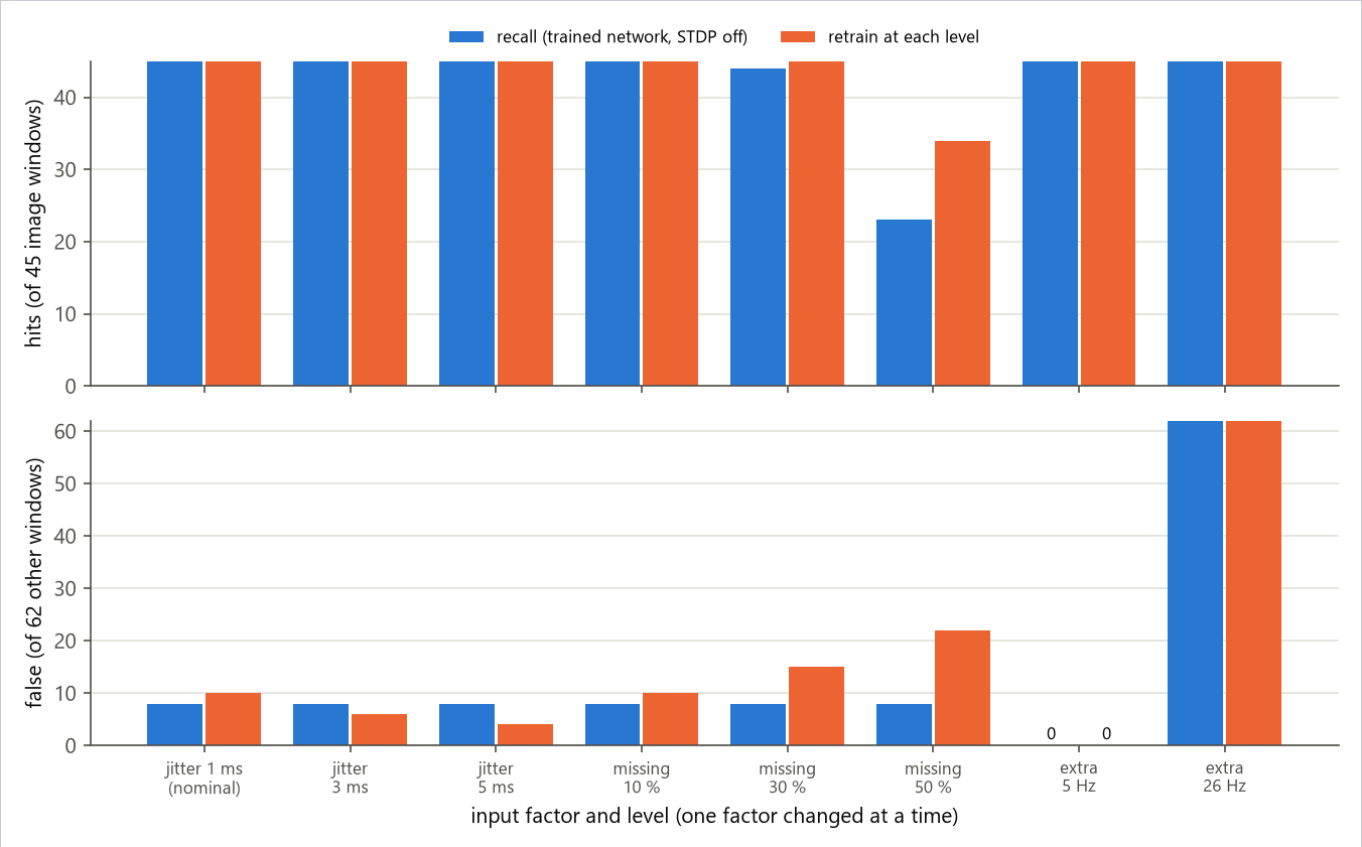


Figure: Robustness of the column-scan MNIST "0" network (3 neurons, 60 s of training, 30 s per level, one training per level). Top: image windows answered, of 45. Bottom: other windows answered, of 62. Source: src/neurovat/README.md, recall vs retrain table.

Level	Recall: hits / false	Retrain: hits / false
jitter 1 ms (nominal)	45 / 8	45 / 10
jitter 3 ms	45 / 8	45 / 6
jitter 5 ms	45 / 8	45 / 4
missing spikes 10 %	45 / 8	45 / 10
missing spikes 30 %	44 / 8	45 / 15
missing spikes 50 %	23 / 8	34 / 22
extra spikes 5 Hz	45 / 0	45 / 0
extra spikes 26 Hz	45 / 62	45 / 62

The clearest change is at 50 % missing spikes. A network trained there keeps more hits (23 → 34 of 45) but answers more empty windows (8 → 22 of 62). With one training run per level, differences of a few windows are not trends.

A second published sweep (recall only, one neuron, seed 1, 53 image and 54 other windows per run) found the following:

- Jitter up to 5 ms changed nothing: 53 hits and 3 false at each level, against 25 false for the untrained copy.
- With missing spikes, hits went 53 / 53 / 46 / 7 at 0 / 10 / 30 / 50 %. The trained network keeps hits less well than the untrained one (53 / 53 / 53 / 41), because STDP pruned its weights.

- At 26 Hz extra spikes, both networks answered every window (54 / 54 false). The threshold is held and there are only 28 inputs; the paper's neuron had 2000.

8.6 The frozen-Poisson fix

Source: `src/neurovat/README.md`, "Frozen Poisson fix search". Frozen Poisson gives about 5 spikes per inked line in each 50 ms window. From start weight 1.0, STDP depressed each line's own spikes after the output spike. The digit's weights fell to a mean of 0.51–0.77, and the clean image's peak fell below the threshold (0.64–0.96 × threshold), so the hits collapsed. The search tried several settings on a grid. The grid was: MNIST "0", one neuron, random spacing, share 0.4, noise 1 / 3 / 5 / 10 Hz, stream seeds 1 and 2, frozen draws 0 and 1. A setting passed only if, in every cell, the trained hit rate stayed at least 95 % of the untrained rate and the false windows fell wherever there were any.

Setting	Cells run	Hits held	Peak / threshold after training	Result
as before (threshold 0.85, 100 Hz, start weight 1)	16	3 of 16	0.64–0.96	fail
auto threshold 0.70 × peak	13	7 of 13	0.74–1.16	fail
auto threshold 0.60 × peak	4	3 of 4	0.41–0.97	fail
rate 40 Hz	4	1 of 4	0.20–1.16	fail
first spike of each line only	7	5 of 7	0.70–1.16	fail
one spike per inked pixel, uniform time	16	8 of 16	0.02–1.17	fail
start weight 0.7	4	1 of 4	0.86–1.01	fail
start weight 0.6	4	4 of 4	0.95–1.05	screen passed, peak at the threshold
start weight 0.5	**16**	**16 of 16**	**1.10–1.27**	**pass**
start weight 0.5, 180 s training	4	4 of 4	1.20–1.26	pass
start weight 0.4	4	4 of 4	1.15–1.31	screen passed

From 0.5, the digit's weights stayed near their start (mean 0.44–0.51) while the other lines fell (mean 0.02–0.30). At 10 Hz, the false windows fell from 226 → 0 of 228 and 221 → 0 of 227, with the hits held (the README gives the hit counts for the 1 Hz cells, 149 → 147 of 147 and 154 → 148 of 148, and marks the other noise levels "same"). The GUI therefore starts frozen Poisson at weight 0.5, with 3 Hz noise as its default. Noise tolerance depends on the ink: the sparse MNIST "1" (66 inked pixels) learnt at 3 Hz (117/226 → 0/228 false), but at 10 Hz it fired in every window.

8.7 Known limits of the measured results

- **Several images are not separated by several neurons**, because every image drives the same rows. MNIST "0", "1" and "2" hidden, 3 neurons, start weight 1.0, 10 Hz, seed 1, 120 s of training, 30 s tests (5–6 windows per digit scored): neurons 1 and 3 both answer the "0" best (5/5 hits each, with 9 and 17 of 30 other windows). Neuron 3 also answers the "1" and "2", neuron 2 answers 3 of 6 "2" windows, and no neuron takes the "1". One run.

- **Distractors are answered too.** With MNIST digits 1–9 shown as distractors (start weight 1.0, noise 0): 13 and 16 of 18 false windows untrained, and 14 and 14 of 18 after 120 s of training. No learning.
- **The built-in 28 × 28 digits and letters show detection, not learning.** They are already clean untrained (0/18 false), because their clean peak at weight 1 is far above the noise.
- **The crossbar Trainer does not yet learn scanned patterns well**: letters and digits reach 25–42 % with column scan, against 92–100 % with rate coding (15 epochs, 3 seeds; `src/neurovat/README.md`).
- **No full-MNIST accuracy is published**; the earlier full MNIST and Kaggle runs were stopped and have not been restarted (README).
- All network results are runs of Python twins, not a circuit solver. Many results are single runs or a few seeds; treat differences of a few windows as noise.

CHAPTER 9

Limits, troubleshooting and FAQ

9.1 Known limits

These limits come from `README.md`, `src/neurovat/README.md` and `CLAUDE.md` in the v1.3.1 release, plus what we found while writing this tutorial.

Simulation

- **Python twins, not a circuit solver.** Device Lab and the Trainer step the Python twin of each `.va`; neither runs the Verilog-A. Mapping a file to a twin does not prove the equations are the same, so validate your twins against Spectre (see the validation chapter).
- **Not modelled:** wire and sneak-path losses, parasitics, nonlinear read bias, programming-pulse energy in Compute mode, and write time in the average-power clock. Energy parameters with no value supplied run on labelled PLACEHOLDER values.
- **RC shaper** in the STDP tab is an ideal buffered, unloaded high-pass, not a passive RC network loaded by the device.
- **Logic/ALU** in Compute is a functional check, not a transistor-level timing or hazard check. **Parallel banks** gives scheduling estimates, not measured CPU/GPU speed.
- Neuron voltages are in normalized units.

Networks

- Dense crossbars and LIF layers only, with up to four hidden layers of 64 neurons. Device-backed convolution and pooling are not implemented.
- Capacitor-type devices (such as a FeCAP) and unmapped sources are not accepted as conductance synapses.
- The crossbar Trainer does not yet learn scanned (column-scan) patterns well: 25–42 % against 92–100 % with rate coding.
- Stream mode: several hidden images are **not** split one per neuron, and a neuron also answers other digits shown as distractors, before and after training. The built-in digits and letters show detection, not learning.
- Noise tolerance depends on how much ink the image has: the sparse MNIST "1" fires in every window at 10 Hz.
- Robustness "retrain at each level" is refused for device synapses, and there is no untrained baseline for device synapses.
- No full-MNIST accuracy is published. The earlier full MNIST and Kaggle runs were stopped and have not been restarted.

Models and tools

- The STDP tab's ECFET v2 window is the anti-symmetric Fig. 4a form by design, not the device's measured Fig. 4b curve. Energy and size scaling of the paper are not modelled.
- `ecfet_v2.va` does not compile with OpenVAF 23.2.0 (`@(cross)` events, and `limexp` inside an `if`). `ecfet_v3.va` does compile.
- No FeFET or FeCAP model ships with the public release; add your own as a twin.

- UI code changes need an app restart; device twins and analysis code hot-reload.

Security

- Files in `twins/` and `patterns/` run as code when they load, with no sandbox.
- Autonomous agent mode (ON by default) lets the agent edit files and run commands without asking. Attached files and datasets are untrusted input that could try to steer it.
- The Virtuoso write-back overwrites files on a shared host and cannot be undone.

9.2 Common problems and fixes

Test is refused: "settings changed since this network was built"

What you see (stream mode, status line): `settings changed since this network was built: press New network, or restore them`.

Why: once a stream network exists, Test refuses to run on settings that differ from the ones the network was built with. Otherwise it would quietly replace your trained network. Six stream controls are per-run settings and may be changed between runs: pattern share, spacing, jitter, noise, deletion and background Hz. Any other stream control counts as a network setting, as do the seed and the image code.

Fix:

1. To keep the trained network, set the changed control back to its old value and press Test again.
2. To use the new settings, press **New network**, then Test (this runs the untrained baseline), then Train.
3. To measure robustness without rebuilding, change only pattern share, spacing, jitter, noise, spike deletion or background Hz between Train and Test, or use the Robustness tab.

Robustness: "no trained network"

What you see: `no trained network: press Train first (the sweep tests the trained network; Test before Train is the untrained baseline)`.

Fix: press **Train** in stream mode first. A Test alone does not train. Related messages: `settings changed since this network was built ...` (see above). `busy: a Train / Test / sweep is running - stop it first` means you must wait or press Stop. `retraining needs ideal weights: a device copy cannot be made (use recall for device synapses)` means you should switch the mode to **recall** for device synapses. `run a sweep first` appears when you press Show in Raster before any sweep.

MNIST not loaded

What you see: `image set 'dataset': no dataset loaded - press Load MNIST (or load one in the Data tab)`, or `no dataset loaded` in the Data tab.

Fix:

1. Press **Load MNIST** (stream input, image set "dataset") or the Data tab's MNIST button. It reads `datasets/mnist.npz` in the workspace. If the file is missing, it downloads it from the keras mirror, which needs internet access.
2. With no internet, copy a keras-format `mnist.npz` into `<workspace>/datasets/` yourself. The benchmark scripts (`neurovat.data.fetch_mnist`) also check the file against a pinned SHA-256 and refuse a different file with "... is not the pinned MNIST archive". The GUI loader was not checked for this.

3. Wait for loading to finish (the button is busy meanwhile), then set the image size and pick the image.

> **Note.** Datasets load with `allow_pickle=False` on purpose, so a downloaded `.npz` cannot run code. Do not change that.

The bridge does not answer, or the mailbox is busy

- `[nvat_ctl] no control mailbox at ...`: the app is not running, or the bridge is off. Start it with `python run_gui.py` (not `--no-bridge`), or use **Tools > Control bridge > Connect bridge**. Use the same `--workspace` folder for the app and the client.
- A command hangs, or a `shot` comes back black: the mailbox holds **one** command, and a long command (`nt_train`, `nt_stream`, `nt_robust`, `run`) is still running. Wait for its answer; the client waits up to 1200 s by default (`--timeout`). Send one command at a time from one client.
- `Trainer is busy.`: a training or sweep is already running.
- `[nvat_ctl] timed out waiting for '...' result (is the app still running?)`: check the process log for a crash (next section).

Where are the logs?

Log	Where	What it holds
In-app console	Device Lab > **Log** tab; **Save to file** writes `<workspace>/results/session_log.txt` (the bridge's `log` command does the same)	what the tools reported: runs, checks, OpenVAF output, agent actions
Process output	the terminal that started the app, or a file if you redirect it. Create the folder first on a fresh copy, because the shell opens the file before the app starts: `mkdir results`, then `python run_gui.py > results/app_run.log 2>&1` (the path is relative to the terminal's current folder)	Python tracebacks, warnings, native errors, startup crashes. The bridge never sees these.
Saved results	`<workspace>/results/`	exports, snapshots (`agent_snapshot.json`, `neuro_snapshot.json`), compute and STDP exports

When something fails, read the process output first; that is where the real error is.

OpenVAF missing or failing

- `OpenVAF not found. Install from https://openvaf.semimod.de/ and put it on PATH, or set NVAT_OPENVAF=/path/to/openvaf.`: install OpenVAF, then add it to PATH or set the environment variable, and restart the app.
- `[openvaf] ecfet_v2.va: FAILED (rc=65)`: this is expected with OpenVAF 23.2.0 (see Known limits). Validate ecfet_v2 in Spectre instead.

macOS blocks the setup file

1. Right-click `setup_mac.command` and choose **Open**, then confirm. Or run `chmod +x setup_mac.command` once in Terminal.
2. The first run needs Python 3.9 or newer from python.org. It creates `.venv` and installs the requirements, which takes a few minutes. Later runs just start the app.

3. On macOS the app uses DearPyGui's built-in font instead of Segoe UI; this is expected.

Other quick fixes

- **The Assistant dot is red / "no backend"**: install the Claude Code CLI and log in (Account > Log in), or set `ANTHROPIC_API_KEY` for chat only. For OpenAI, install the codex CLI or set `OPENAI_API_KEY`. `/provider claude` switches back.
- **An agent edit broke something**: Agent > **Revert agent edits** restores the files from before the last agent turn.
- **A new `.va` shows up but cannot be simulated**: no twin matches its name. Add a twin with a matching `va_keywords` entry (see the validation chapter), then **restart the app**; Rescan does not import new twins.
- **A twin fails with "object has no attribute 'name'"**: add a class attribute `name = "..."` to the model class.
- **A saved network will not load, or training is blocked after you edited a `.va`**: networks record the `.va` file's SHA-256. If the source changed or is missing, rebuild the network.
- **Odd results after editing a model**: run `python selftest.py`. It must report `35 passed, 0 failed`.

9.3 FAQ

Does NeuroVAT replace Spectre?

No. It lets you work quickly on Python twins without Cadence. Take the `.va` back to Virtuoso and overlay the two simulators before you publish.

Why did my image network "detect" but not "learn"?

The auto threshold is set from the starting weights, so an untrained neuron can already answer the image. Learning is shown only when the trained network answers fewer empty windows than the untrained baseline while still answering the image. Run Test before Train so that you have that baseline.

Why does frozen Poisson start at weight 0.5?

From 1.0, STDP depressed the image's own lines below the threshold and the hits collapsed. Start weight 0.5 passed all 16 cells of the test grid. Start weights 0.4 and 0.6 passed a shorter 4-cell screen only (see the results chapter).

Are my API keys saved?

No. Keys entered in the Account dialog stay in memory for that session only. Keys can also come from environment variables. Neither is written to the workspace.

Can I use the app without the AI assistant?

Yes. The Assistant is optional, and every feature works from the normal controls and the bridge.

9.4 Where to get help

Open an issue on the project's GitHub page: `https://github.com/mpeyal/neurovat-verilog-tester` (Issues tab). Include:

- the NeuroVAT version, your OS and your Python version;

- the exact steps to reproduce the problem;
- the process output (traceback) and `results/session_log.txt`;
- the output of `python selftest.py`.

Remove user names, host names, paths and keys from logs before you post them.

9.5 How to cite

The repository includes `CITATION.cff`; GitHub's "Cite this repository" button reads it. Its main fields (abstract and keywords omitted):

```
cff-version: 1.2.0
title: "NeuroVAT: a neuromorphic Verilog-A test bench and crossbar spiking-network simulator"
type: software
version: 0.1.0
authors:
  - family-names: Peyal
    given-names: Md Mahmudul Kabir
    affiliation: Boise State University
repository-code: "https://github.com/mpeyal/neurovat-verilog-tester"
```

A plain-text form: Peyal, M. M. K. *NeuroVAT: a neuromorphic Verilog-A test bench and crossbar spiking-network simulator* (software). Boise State University. https://github.com/mpeyal/neurovat-verilog-tester.

> **Note.** The citation file says version 0.1.0, but the app release covered here is v1.3.1. Give the release you actually used in your citation.

If you use the ECFET v2 model, also cite the paper it was fitted to: M. T. Sharbati et al., "Low-Power, Electrochemically Tunable Graphene Synapses for Neuromorphic Computing", Adv. Mater. 2018, 30, 1802353. For the pattern-finding benchmark, cite Masquelier, Guyonneau & Thorpe (2008), PLoS ONE 3:e1377.

APPENDIX

Control names for scripting

The control bridge (Chapter 6) and the Assistant address each control by its **widget tag**, not by the label you see on screen. This appendix lists the tags you need most, grouped by area. Every tag and every combo item below was taken from a dump of all widgets of the running app (v1.3.1), made right after start-up. The "start value" column is the value in that dump.

A.1 How to use the tags

- **Read and write:** `python tools/nvat_ctl.py --workspace <dir> get <tag>` and `... set <tag> <value>`. A set answers with the value now in the control and `"applied": true` or `false`.
- **Quote values with spaces**, for example `set nt_mode "unsupervised stream"` or `set nt_st_rb_delete "0, 50"`. Without quotes the shell splits the value, and only the first word is sent.
- **Combos:** give one of the item strings exactly as listed here (the case, spaces and brackets matter).
- **Check boxes:** `1` / `0` (or `true` / `false`).
- **Side effects:** `set` writes the value, but it may not trigger every side effect of a click in the window. For example, `set param_changed 1` ticks "Modified only", but the parameter list is not filtered. Check the result with `get` or a `shot`, and click in the window when a set does not do what you need.
- **Buttons are commands, not tags.** Do not `set` a button. Use the matching bridge command (last table of this appendix).
- **Unit combos:** several Network Studio fields have a unit combo next to them. Its tag is the field's tag plus `__u`, for example `nt_present__u` with items `ns`, `us`, `ms`, `s`.

A.2 Device Lab

Device library and Parameter inspector

On-screen label	Tag	Start value	Example / items
tick box of `ecfet_v2.va`	`cb_file_ecfet_v2.va`	True	`set cb_file_ecfet_v2.va 1`
tick box of `ecfet_v3.va`	`cb_file_ecfet_v3.va`	True	`set cb_file_ecfet_v3.va 0` (simulate v2 only). The pattern is `cb_file_<file>.va`; the dump has it only for the two ECFET files.
model drop-down of the inspector	`param_model_sel`	ECFET v2 (practical ECRAM)	`ECFET v1 (Verilog-A port)`, `ECFET v2 (practical ECRAM)`, `ECFET v3 (paper-faithful)`
Search name, description or group	`param_search`	(empty)	text
Modified only	`param_changed`	False	`set param_changed 1` ticks the box but does not filter the list (see above)

On-screen label	Tag	Start value	Example / items
a model parameter	`p_<model>_<param>`		The inspector builds the `p_` tags of the model chosen in `param_model_sel`. The dump had ECFET v2 chosen, so its tags start with `p_v2_`.
State count	`p_v2_n_states`	650	`set p_v2_n_states 300` (Chapter 3, Step 1)
Min. R (ohm) / Max. R (ohm) / Initial R (ohm)	`p_v2_Rmin` / `p_v2_Rmax` / `p_v2_Rinit`	870 / 10000 / 4400	
Q_ref	`p_v2_Q_ref`	5e-13	
nu_p / nu_d	`p_v2_nu_p` / `p_v2_nu_d`	0.02 / 0.02	
Write polarity	`p_v2_polarity`	-1	
w	`p_v2_w`	4e-06	
cycle-to-cycle noise	`p_v2_sigma_c2c`	0	
the added timing rule	`p_v2_A_stdp`	4.5e-06	

The other ECFET v2 parameters in the dump follow the same pattern: `p_v2_D`, `p_v2_G_eq`, `p_v2_I_drift_th`, `p_v2_c1`, `p_v2_c2`, `p_v2_c3`, `p_v2_kappa_d`, `p_v2_kappa_v`, `p_v2_l`, `p_v2_leak_drift_scale`, `p_v2_seed`, `p_v2_stdp_c1`, `p_v2_stdp_c2`, `p_v2_stdp_c3`, `p_v2_tao3`, `p_v2_tau_retention`, `p_v2_tau_scale` and `p_v2_tau_stdp`. Hover over a name in the inspector to see which one it is.

Stimulus tab and Run settings

On-screen label	Tag	Start value	Example / items
Generator	`gen_combo`	LTP / LTD train	`Single spike`, `Pulse train`, `LTP / LTD train`, `Paired pulses (PPF)`, `Poisson spike train`, `Burst pattern`, `STDP pair (pre/post)`, `Staircase`, `Custom pattern`
Signal	`kind_combo`	current	`current`, `voltage`
Unit	`unit_combo`	pA	`pA`, `nA`, `uA`, `mA`, `A` (current drive)
Delay (ms)	`delay_ms`	0	
Pulses per branch	`g_n_each`	300	`set g_n_each 30`
|amplitude|	`g_amp`	170	in the Unit above
Width (ms)	`g_width_ms`	10	
Period (ms)	`g_period_ms`	2000	
branch gap (ms)	`g_gap_ms`	0	
start (ms)	`g_t0_ms`	10	
Pot. sign (-1/+1)	`g_pot_sign`	1	
train repeats	`g_reps`	1	

On-screen label	Tag	Start value	Example / items
Custom pattern rows	`custom_text`	three example rows	one row per pulse: `t_start_ms width_ms amplitude`
post-stimulus tail (s)	`tail_input`	60	`set tail_input 10`
manual t_stop (s)	`tstop_manual` (check box) and `tstop_input`	False and 2.0	
auto-save CSV after run	`csv_cb`	False	

> **Note.** The `g_` tags belong to the generator chosen in `gen_combo`. The dump had **LTP / LTD train** chosen, so the list above is that generator's. Set `gen_combo` first; the bridge then rebuilds the generator fields. The other generators have their own `g_` fields, which this dump does not list.

Pulse response, STDP and Compute

On-screen label	Tag	Start value	Example / items
ANALYZE (Pulse response)	`ana_metric_combo`	G (uS)	`G (uS)`, `R_mem (ohm)`
Pre / post amplitude	`stdp_amp_pre` / `stdp_amp_post`	-50 / 50	
Pulse width (ms)	`stdp_width_ms`	10	
Timing range (+/- ms)	`stdp_range_ms`	1800	
Read delay (ms)	`stdp_settle_ms`	1000	
Timing samples	`stdp_npts`	80	
Stimulus (STDP)	`stdp_mode`	rectangular pulses	`rectangular pulses`, `overlapping spikes`
Spike shaper	`stdp_shape`	exponential tail	`exponential tail`, `linear tail`, `RC high-pass`
Measurement	`stdp_measurement`	Pair state change	`Pair state change`, `Interaction (pair - pre - post + idle)`
Device's own timing response (A_stdp)	`stdp_include_rule`	True	`set stdp_include_rule 0`
Compute mode	`compute_mode`	Matrix multiply	`Matrix multiply`, `Logic / ALU`, `Terminal energy`
Verilog-A source (Compute)	`compute_source`	verilog\ecfet_v2.va	`verilog\ecfet_v2.va`, `verilog\ecfet_v3.va`
Max read voltage (V) / Read duration (ns) / Conductance levels	`compute_read_voltage` / `compute_read_ns` / `compute_levels`	0.1 / 100 / 256	

On-screen label	Tag	Start value	Example / items
Mismatch std / G span / Mismatch seed	`compute_mismatch` / `compute_seed`	0 / 17	
Operation (Logic / ALU)	`compute_op`	ADD	`NAND`, `AND`, `OR`, `XOR`, `NOT`, `ADD`

A.3 Tabs

Switch the main view with `tab <tag>`, for example `tab tab_trainer`. The tab bar of these tabs is `center_tabs`, and `state` reports the open one as `"center_tab"`.

On-screen label	Tag
Stimulus	`tab_designer`
Transient	`tab_results`
Pulse response	`tab_analysis`
STDP	`tab_stdp`
Noise & retention	`tab_noise`
Source	`tab_source`
Compute	`tab_compute`
Log	`tab_log`
Polarization (not shown for the public ECFET models)	`tab_polar`
Network Studio	`tab_trainer`

Network Studio has its own tab bars:

- **Stream mode** (bar `nt_st_tabs`): Raster `nt_st_tab_raster`, Network `nt_st_tab_net`, Patterns `nt_st_tab_pat`, Learning `nt_st_tab_learn`, Weights `nt_st_tab_weights`, Energy `nt_st_tab_energy`, Robustness `nt_st_tab_robust`. The `tab` command opens Network Studio and the stream tab in one step: `tab nt_st_tab_robust`. `set nt_st_tabs nt_st_tab_learn` also works.
- **Epoch modes** (bar `nt_viz_tabs`): Architecture `ns_architecture_tab`, Data `nt_tab_data`, Neuron `nt_tab_neuron`, Cell `nt_tab_cell`, Weights `nt_tab_weights`, Trajectories `nt_tab_wtrace`, Raster `nt_tab_raster`, Results `nt_tab_results`. Inside Results, the bar `nt_results_tabs` holds Power `nt_tab_power`. These are not `tab` targets. Open Network Studio with `tab tab_trainer`, then use `set nt_viz_tabs nt_tab_results`.

A.4 Network Studio: network, data and epoch training

On-screen label	Tag	Start value	Example / items
mode	`nt_mode`	supervised	`supervised`, `unsupervised`, `unsupervised stream`
Verilog-A synapse	`nt_va_source`	verilog\ecfet_v2.va	`verilog\ecfet_v2.va`, `verilog\ecfet_v3.va`

On-screen label	Tag	Start value	Example / items
device twin (follows the Verilog-A synapse)	`nt_device`	ECFET v2 (practical ECRAM)	`ECFET v1 (Verilog-A port)`, `ECFET v2 (practical ECRAM)`, `ECFET v3 (paper-faithful)`
seed	`nt_seed`	1	
learning rule	`nt_learnrule`	STDP (device-local)	`STDP (device-local)`, `Surrogate grad (BPTT)`
SG rate	`nt_sg_lr`	0.1	
input grid H x W	`nt_gh` / `nt_gw`	5 / 5	
outputs	`nt_nout`	4	`set nt_nout 8` for the `bars` set
hidden layers	`nt_hidden`	(empty = none)	a comma list, e.g. `8` or `8,4`
Neurons (Architecture tab)	`ns_units`	8	
hidden gain	`nt_hidden_gain`	1.7	
epochs	`nt_epochs`	40	`set nt_epochs 15`
present	`nt_present` (+ `nt_present__u`)	120 ms	
rate	`nt_rate` (+ `nt_rate__u`)	180 Hz	
step	`nt_dt` (+ `nt_dt__u`)	1 ms	
pot amp / dep amp	`nt_potamp` / `nt_depamp` (+ `__u`)	50 pA / 50 pA	unit items: `pA`, `nA`, `uA`, `mA`, `A`, `kA`, `MA`
pulse	`nt_pwidth` (+ `nt_pwidth__u`)	10 ms	
A+ rate / A- rate	`nt_aplus` / `nt_aminus`	1.0 / 1.0	
LTP/LTD split	`nt_offset`	0.25	
tau_pre	`nt_taupre` (+ `__u`)	20 ms	
tau_m / V_th / refrac	`nt_tau_m` / `nt_vth` / `nt_refrac`	20 ms / 1.0 / 5 ms	
tau_syn	`nt_tausyn` (+ `__u`)	8 ms	
EPSP gain / IPSP gain	`nt_epsp` / `nt_ipsp`	11.0 / 1.0	
inhib / theta+	`nt_inhib` / `nt_theta`	0.9 / 0.06	
teacher	`nt_teacher`	1.4	
Source selector (Data tab)	`nt_source_selector`	Pattern set	`Pattern set`, `Custom painted`, `Imported`
Pattern	`nt_patset`	bars	`bars`, `letters`, `digits`, `random`, `nand`, `xor`, `corners`, `xnor`, `xor_aug`, `custom`, `dataset`

On-screen label	Tag	Start value	Example / items
Encoding	`nt_encoding`	rate (Poisson)	`rate (Poisson)`, `latency (TTFS)`, `temporal (phase)`, `column scan (spatio-temporal)`
bin (ms) / ink threshold (column scan)	`nt_scan_bin` / `nt_scan_thr`	10 / 0.5	
input noise / jitter (ms)	`nt_input_noise` / `nt_jitter`	0 / 0	
bg rate	`nt_bg_rate` (+ `nt_bg_rate__u`)	0 Hz	
signal afferents / pattern	`nt_signal_frac` / `nt_pat_ms` (+ `__u`)	1.0 / 0 ms	
membrane noise / write noise	`nt_vnoise` / `nt_wnoise`	0 / 0	
resolution / img / class / INVERT	`nt_ds_res` / `nt_ds_perclass` / `nt_ds_invert`	14 / 12 / False	
dataset URL	`nt_ds_url`	(empty)	text
Power profile (Results > Power)	`nt_power_profile`	Default (match device)	`Default (match device)`, `Placeholders only`

A.5 Stream mode

These controls are shown in the window after `set nt_mode "unsupervised stream"` (the tags exist in every mode). Image-input rows are used only when **input** is the scanned-images item; the Poisson rows only for Poisson input.

On-screen label	Tag	Start value	Example / items
input	`nt_st_input`	Poisson patterns (Masquelier)	`Poisson patterns (Masquelier)`, `scanned images (spatio-temporal)`
afferents (Poisson)	`nt_st_aff`	1000	
neurons	`nt_st_neurons`	3	
patterns / in pattern (Poisson)	`nt_st_patterns` / `nt_st_in_pattern`	3 / 0.5	
image set	`nt_st_imgset`	digits	`digits`, `letters`, `bars`, `dataset`
image size	`nt_st_img_size`	14	`set nt_st_img_size 28`
encoding	`nt_st_code`	column scan	`column scan`, `row scan`, `latency`, `frozen Poisson`
window ms (latency, frozen Poisson)	`nt_st_code_win`	50	

On-screen label	Tag	Start value	Example / items
hidden	`nt_st_img_pick`	0	image indices: `0` or `"0,3"`
distractors	`nt_st_img_distract`	(empty = none)	
ms / column (ms / row for row scan)	`nt_st_bin`	10	
ink	`nt_st_ink`	0.5	
background Hz	`nt_st_bg`	10	
threshold	`nt_st_thr`	0 (auto)	
synapses	`nt_st_synapse`	ideal weights	`ideal weights`, `selected device`
write window / pulse amp (device synapses)	`nt_st_window` / `nt_st_pulse`	50 / calibrated	pulse amp: `calibrated`, `as set`
seconds	`nt_st_seconds`	60	`set nt_st_seconds 60`
pattern share	`nt_st_share`	0.4	
spacing	`nt_st_spacing`	random	`random`, `even`
jitter ms	`nt_st_jitter`	1.0	
noise Hz	`nt_st_noise`	10	
spike deletion	`nt_st_delete`	0	a **fraction** 0-1, e.g. `0.1`
A+ (LTP step) / A- / A+	`nt_st_aplus` / `nt_st_aratio`	0.03125 / 0.85	
tau+ ms / tau- ms	`nt_st_tplus` / `nt_st_tminus`	16.8 / 33.7	
Start weight	`nt_st_w0`	0.475	image input sets it to 1 unless you changed it
tau m ms / tau s ms / Refractory ms	`nt_st_taum` / `nt_st_taus` / `nt_st_refr`	10 / 2.5 / 1	
Inhibition / tau inh ms	`nt_st_inh` / `nt_st_tinh`	0.4 / 20	
Raster replay: Play / Pause	`nt_st_play`		`1` = play, `0` = pause
Raster replay: speed	`nt_st_speed`	1x	`0.25x`, `1x`, `4x`
Raster replay: slider / window s	`nt_st_view_t` / `nt_st_view_win`	0 / 0 (auto)	

Tip. Only pattern share, spacing, jitter, noise, deletion and background Hz (`nt_st_share`, `nt_st_spacing`, `nt_st_jitter`, `nt_st_noise`, `nt_st_delete`, `nt_st_bg`) may change between runs of one network. Any other stream tag, or the seed, needs a new network (`nt_build`, the **New network** button).

A.6 Robustness tab

On-screen label	Tag	Start value	Example / items
jitter levels	`nt_st_rb_jitter`	(empty = 1, 3, 5 ms and the trained value)	`set nt_st_rb_jitter 1`
missing-spike levels	`nt_st_rb_delete`	(empty = 0, 10, 30, 50 % and the trained value)	in **percent**, unlike `nt_st_delete`: `set nt_st_rb_delete "0, 50"`
extra-spike levels	`nt_st_rb_noise`	(empty = 5, 26 Hz and the trained value)	absolute Hz: `set nt_st_rb_noise 10`
seconds / level	`nt_st_rb_seconds`	30	1 to 600
compare with untrained	`nt_st_rb_untrained`	True	ideal synapses only
mode	`nt_st_rb_mode`	recall	`recall`, `retrain at each level`
train s / level	`nt_st_rb_train_s`	0 (= as trained)	retrain only
repeats	`nt_st_rb_repeats`	1	retrain only, 1 to 10
Show in Raster list	`nt_st_rb_pick`	(empty until a sweep)	an item of the list, or its index

A.7 Buttons and their bridge commands

Button	Bridge command
Run simulation (Device Lab), Run STDP	`run`, `stdp`
Build network; in stream mode **New network**	`nt_build`
Train / Test (epoch modes)	`nt_train` / `nt_test`
Train / Test (stream mode)	`nt_stream` / `nt_stream_test` (both switch the mode to `unsupervised stream` first)
Load MNIST (stream) or Download MNIST (Data tab)	`nt_load_mnist`
Run sweep (Robustness)	`nt_robust <seconds> [retrain 1/0] [train_s] [repeats]`
Show in Raster (Robustness)	`nt_robust_raster <n>`

A.8 Scripting a whole stream run

This script repeats Chapter 5 from a terminal: the MNIST "0" at size 28, column scan, then Test (untrained baseline), Train, Test, and a short recall sweep. Start the app first (Chapter 6, "Steps"), and run the lines from the project folder. Lines that start with `#` are comments. `<dir>` is the workspace the app was started with.

```
# set up the image stream (before the first Test)
python tools/nvat_ctl.py --workspace <dir> set nt_mode "unsupervised stream"
python tools/nvat_ctl.py --workspace <dir> set nt_st_input "scanned images (spatio-temporal)"
python tools/nvat_ctl.py --workspace <dir> set nt_st_imgset dataset
python tools/nvat_ctl.py --workspace <dir> nt_load_mnist
python tools/nvat_ctl.py --workspace <dir> set nt_st_img_size 28
python tools/nvat_ctl.py --workspace <dir> set nt_st_img_pick 0
python tools/nvat_ctl.py --workspace <dir> set nt_st_code "column scan"
python tools/nvat_ctl.py --workspace <dir> set nt_st_seconds 60

# 1: Test = the untrained baseline
python tools/nvat_ctl.py --workspace <dir> nt_stream_test
# 2: Train (STDP on)
python tools/nvat_ctl.py --workspace <dir> nt_stream
# 3: Test the trained network
python tools/nvat_ctl.py --workspace <dir> nt_stream_test

# a short sweep: the trained values plus 50 % missing spikes
python tools/nvat_ctl.py --workspace <dir> set nt_st_rb_jitter 1
python tools/nvat_ctl.py --workspace <dir> set nt_st_rb_delete "0, 50"
python tools/nvat_ctl.py --workspace <dir> set nt_st_rb_noise 10
# recall sweep, 30 s per level
python tools/nvat_ctl.py --workspace <dir> nt_robust 30
python tools/nvat_ctl.py --workspace <dir> tab nt_st_tab_robust
```

1. **Order.** This is the order we tested: image input and image set, then `nt_load_mnist`, then the rest. Set every network setting before the first `nt_stream_test`. If you change one later, Test refuses with "settings changed since this network was built"; send `nt_build` (New network) and start again from the baseline.
2. **Baseline first.** On a fresh network, the first `nt_stream_test` is the untrained baseline. The verdict of the later Test compares the two runs.
3. **One command at a time.** `nt_stream`, `nt_stream_test` and `nt_robust` answer only when the run has finished. Each answer holds the run's result as JSON (the `"stream"` field, or the sweep `"table"`).
4. **The levels.** The three `nt_st_rb_*` sets keep the sweep short: only the trained values plus 50 % missing spikes. Leave them out to sweep the default levels. For a retrain sweep, use `nt_robust 30 1 30` (retrain, 30 s of training per level).

We checked this script on a fresh copy of the release, with `nt_st_seconds 6` and `nt_robust 4` to keep it short: every command returned `"ok": true`, the first Test reported `"phase": "untrained baseline"` with 28 afferents and the auto threshold 14.4, and the sweep ran in recall mode. Those short runs are only a check of the commands, not results.

> **Note.** The status pill and the live status line change during a run, but the bridge cannot read them until the run is over. Watch the window, or wait for the command's answer.